

PROYEK AKHIR

RANCANG BANGUN APLIKASI MANAJEMEN HOTEL KAPSUL MULTI-CABANG BERBASIS WEBSITE (TAMU.ID)



Oleh:

ALIIF ARIEF MAULANA

21/479029/SV/19418

**PROGRAM STUDI SARJANA TERAPAN
TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
2025**

PROYEK AKHIR

RANCANG BANGUN APLIKASI MANAJEMEN HOTEL KAPSUL MULTI-CABANG BERBASIS WEBSITE (TAMU.ID)

Proyek Akhir
Program Studi Teknologi Rekayasa Perangkat Lunak

Diajukan untuk memenuhi salah satu syarat memperoleh gelar Sarjana Terapan Teknik pada
Program Studi Teknologi Rekayasa Perangkat Lunak Departemen Teknik Elektro dan
Informatika Sekolah Vokasi Universitas Gadjah Mada

Oleh:
ALIIF ARIEF MAULANA
21/479029/SV/19418

**PROGRAM STUDI SARJANA TERAPAN
TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
2025**

HALAMAN PENGESAHAN

Judul : RANCANG BANGUN APLIKASI MANAJEMEN HOTEL KAPSUL
MULTI-CABANG BERBASIS WEBSITE (TAMU.ID)
Nama : Aliif Arief Maulana
Program Studi : Teknologi Rekayasa Perangkat Lunak
Pembimbing : Dr.Eng. Ganjar Alfian, S.T., M.Eng.
Waktu Ujian : Kamis, 4 Juni 2025, pukul 13.00 s.d. 15.00 ruang HS209 Gedung
Herman Yohannes

Telah dipertanggungjawabkan dan diuji oleh Tim Penguji serta disetujui dan disahkan
Sebagai syarat kelengkapan studi jenjang Sarjana Terapan Program Studi Teknologi
Rekayasa Perangkat Lunak Sekolah Vokasi Universitas Gadjah Mada

Yogyakarta, 4 Agustus 2025

Diterima dan disetujui oleh,

Ketua Penguji

Pembimbing/Sekretaris Penguji

Nama Ketua Penguji
NIP. XXXXXXXXXXXXXXXXXXXX

Dr.Eng. Ganjar Alfian, S.T., M.Eng.
NIP. 111198701202201101

Anggota Penguji

Nama Anggota Penguji
NIP. XXXXXXXXXXXXXXXXXXXX

Mengetahui,

Ketua Departemen
Teknik Elektro dan Informatika

Ketua Program Studi
Teknologi Rekayasa Perangkat Lunak

Ir. Nur Rohman Rosyid, S.T., M.T., D.Eng., IPM
NIP. 111197510201206101

Dr. Umar Taufiq, S.Kom., M.Cs.
NIP. 111198212201610101

SURAT PERNYATAAN

Saya yang bertandatangan di bawah ini saya:

Nama : Aliif Arief Maulana
NIM : 21/479029/SV/19418
Tahun Terdaftar : 2021
Program Studi : Teknologi Rekayasa Perangkat Lunak
Fakultas : Sekolah Vokasi

Menyatakan bahwa dalam dokumen ilmiah Proyek Akhir ini tidak terdapat bagian dari karya ilmiah lain yang telah diajukan untuk memperoleh gelar akademik di suatu lembaga Pendidikan Tinggi, dan juga tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang/lembaga lain, kecuali yang secara tertulis disitasi dalam dokumen ini dan disebutkan secara lengkap dalam daftar pustaka.

Dengan demikian saya menyatakan bahwa dokumen ilmiah ini bebas dari unsur-unsur plagiasi dan apabila dokumen ilmiah Proyek Akhir ini di kemudian hari terbukti merupakan plagiasi dari hasil karya penulis lain dan/atau dengan sengaja mengajukan karya atau pendapat yang merupakan hasil karya penulis lain, maka penulis bersedia menerima sanksi akademik dan/atau sanksi hukum yang berlaku.

Yogyakarta, 27 Juni 2025

Yang menyatakan,

Aliif Arief Maulana
21/479029/SV/19418

HALAMAN MOTTO

"Dunia ini ibarat bayang-bayang, ketika engkau kejar, maka engkau tak akan dapat menangkapnya. Tapi, ketika engkau balikkan badanmu darinya, maka bayang-bayang tak punya pilihan lain kecuali mengikutimu."

(Ibnu Qayyim al-Jauziyah)

KATA PENGANTAR

Puji syukur kehadiran Allah SWT atas segala nikmat dan karunia-Nya yang tiada tara sehingga penulis dapat menyelesaikan laporan Proyek akhir ini dengan judul "Rancang bangun Aplikasi Manajemen Hotel Kapsul multi-Cabang berbasis Website (Tamu.id)" yang merupakan salah satu syarat untuk menyelesaikan pendidikan di Program Studi Sarjana Terapan Teknologi Rekayasa Perangkat Lunak, Departemen Teknik Elektro dan Informatika, Sekolah Vokasi, Universitas Gadjah Mada.

Proyek Akhir ini dapat diselesaikan tidak lepas dari bantuan dan kerjasama dari berbagai pihak. Berkenaan dengan hal tersebut, penulis juga ingin menyampaikan ucapan terima kasih kepada:

1. Ir. Nur Rohman Rosyid, S.T., M.T., D.Eng., IPM, selaku Ketua Departemen Teknik Elektro dan Informatika, Sekolah Vokasi, Universitas Gadjah Mada
2. Dr. Umar Taufiq, S.Kom., M.Cs., selaku Kepala Program Studi Teknologi Rekayasa Perangkat Lunak, Departemen Teknik Elektro dan Informatika, Sekolah Vokasi, Universitas Gadjah Mada
3. Dr.Eng. Ganjar Alfian, S.T., M.Eng., selaku Dosen Pembimbing, yang telah memberikan bimbingan, arahan, serta masukan yang sangat berharga dalam penyusunan laporan ini dengan penuh kesabaran dan ketelitian sehingga penulis dapat menyelesaikan Proyek Akhir ini.
4. Dr.Eng. Ganjar Alfian, S.T., M.Eng., selaku Dosen Penguji, yang telah memberikan bimbingan, arahan, serta masukan yang sangat berharga dalam penyusunan laporan ini dengan penuh kesabaran dan ketelitian sehingga penulis dapat menyelesaikan Proyek Akhir ini.
5. Kedua Orang Tua dan Adik - Adikku tercinta, yang selalu memberikan dukungan apapun, semangat, serta doa yang tak henti-hentinya selama perjalanan hidup penulis hingga saat ini.

Alhamdulillah selesai juga dan semoga segala bantuan yang telah diberikan oleh semua pihak dapat menjadi amalan yang bermanfaat dan mendapatkan buah karma baik di masa kini maupun di masa mendatang. Semoga Proyek Akhir ini menjadi informasi bermanfaat bagi pembaca atau pihak lain yang membutuhkannya.

Yogyakarta, 27 Juni 2025

Aliif Arief Maulana
21/479029/SV/19418

DAFTAR ISI

HALAMAN PENGESAHAN	i
HALAMAN PERNYATAAN BEBAS PLAGIASI	ii
HALAMAN MOTTO	iii
KATA PENGANTAR	iv
DAFTAR ISI	vi
DAFTAR GAMBAR	ix
DAFTAR TABEL	xii
DAFTAR KODE.....	xiii
INTISARI	xiv
<i>ABSTRACT</i>	xv
BAB 1 Pendahuluan	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah.....	3
1.3 Tujuan dan Manfaat Proyek Akhir	3
1.3.1 Tujuan Proyek Akhir	3
1.3.2 Manfaat Proyek Akhir.....	4
1.4 Batasan Masalah	5
1.5 Sistematika Penulisan	5
BAB 2 Tinjauan Pustaka	8
2.1 Studi Pustaka	8
2.2 Dasar Teori	12
2.2.1 Hotel Kapsul	12
2.2.2 Sistem Informasi.....	13
2.2.3 Website.....	13
2.2.4 OAuth 2.0	13
2.2.5 Webhook	14
2.2.6 Rest API (Representational State Transfer Application Programming Interface).....	15

2.2.7	<i>Software Development Life Cycle</i> dengan Metode <i>Kanban Board</i>	16
2.2.8	Unified Modeling Language (UML)	16
2.2.9	<i>Use Case Diagram</i>	16
2.2.10	<i>Activity Diagram</i>	17
2.2.11	<i>Entities Relationship Diagram (ERD)</i>	18
2.2.12	Teknologi dan Kakas Pengembangan	19
2.2.13	Pengujian Sistem	20
BAB 3	Metode Proyek Akhir	22
3.1	Bahan	22
3.2	Perlatan dan Kakas	22
3.2.1	Perangkat Lunak	22
3.2.2	Perangkat Keras	23
3.3	Tahapan Proyek Akhir	24
3.4	Analisis Kebutuhan Sistem	26
3.4.1	Analisis Pengguna Sistem	26
3.4.2	Analisis Kebutuhan Fungsional	26
3.4.3	Analisis Kebutuhan Non-Fungsional	29
3.4.4	Pengembangan Sistem	29
3.5	Rancangan Sistem	30
3.5.1	Use Case Diagram	30
3.5.2	Activity Diagram	35
3.5.3	Entity Relationship Diagram	49
3.5.4	Ilustrasi Implementasi <i>Offline Fault Tolerance</i> pada rotasi Kunci Pod	55
3.5.5	Ilustrasi Integrasi <i>WebHook Payment Gateway</i>	56
3.5.6	Ilustrasi Integrasi <i>Passwordless</i> dengan OAuth2 Google	57
3.5.7	Peluncuran Aplikasi (<i>Deployment</i>)	58
3.5.8	Pengujian Sistem	60
BAB 4	Hasil dan Pembahasan	81
4.1	Hasil dan Pembahasan	81
4.1.1	Halaman User	81
4.1.2	Halaman Admin	99
4.1.3	Halaman Superadmin	119
4.1.4	Halaman Simulasi Hardware	130
4.1.5	Analisis Implementasi OAuth2 Google dengan One Time Token (OTT) ..	132
4.1.6	Analisis Integrasi QRIS dengan Backend Sistem	132
4.1.7	Analisis <i>Offline Fault Tolerance</i> pada <i>Hardware IoT</i>	132
4.2	Hasil Pengujian	132
4.2.1	Pengujian Blackbox	132

4.2.2	<i>User Acceptance Test</i>	148
BAB 5	Penutup	154
5.1	Kesimpulan	154
5.2	Saran	154
DAFTAR PUSTAKA		155
LAMPIRAN		L - 1
A	Lampiran Jawaban Kuesioner <i>User Acceptance Testing User</i>	L - 1
B	Lampiran Jawaban Kuesioner <i>User Acceptance Testing Admin</i>	L - 3
C	Lampiran Jawaban Kuesioner <i>User Acceptance Testing SuperAdmin</i>	L - 6

DAFTAR GAMBAR

2.1	Ilustrasi Protokol OAuth 2.0	14
2.2	Ilustrasi REST API	15
3.1	Diagram Tahapan Proyek Akhir	25
3.2	Use Case Diagram Integrasi	30
3.3	Use Case Diagram SuperAdmin (Admin Pusat)	31
3.4	Use Case Diagram Admin (Admin Cabang) 1	32
3.5	Use Case Diagram Admin (Admin Cabang) 2	33
3.6	Use Case Diagram User (Pengunjung)	34
3.7	Diagram Aktivitas Pengguna: Registrasi Email	35
3.8	Diagram Aktivitas Pengguna: Verifikasi Email	36
3.9	Diagram Aktivitas Pengguna: Registrasi dan Login dengan Google OAuth2	36
3.10	Diagram Aktivitas Pengguna: Reservasi	37
3.11	Diagram Aktivitas Pengguna: Checkin dan Stay	38
3.12	Diagram Aktivitas Pengguna: Checkout dan Review	39
3.13	Diagram Aktivitas Pengguna: Verifikasi Identitas (KYC)	40
3.14	Diagram Aktivitas Pengguna: Payout Referral	40
3.15	Diagram Aktivitas Pengguna: Sinkronisasi Data Reservasi OTA	41
3.16	Diagram Aktivitas Admin: Login Email dan Google OAuth2	42
3.17	Diagram Aktivitas Admin: Kelola Cabang (Hotel)	42
3.18	Diagram Aktivitas Admin: Kelola Jenis Pod	43
3.19	Diagram Aktivitas Admin: Kelola Pod	43
3.20	Diagram Aktivitas Admin: Moderasi Review	44
3.21	Diagram Aktivitas Admin: Kelola Voucher Cabang	44
3.22	Diagram Aktivitas Admin: Kelola Pindah Pod	45
3.23	Diagram Aktivitas Admin: Kelola Pembersihan Pod	45
3.24	Diagram Aktivitas SuperAdmin: Login Email dan Google OAuth2	46
3.25	Diagram Aktivitas SuperAdmin: Kelola Data Master	46
3.26	Diagram Aktivitas SuperAdmin: Kelola Data Cabang (Hotel)	47
3.27	Diagram Aktivitas SuperAdmin: Kelola Data Staff	47
3.28	Diagram Aktivitas SuperAdmin: Kelola Data Verifikasi Identitas (KYC)	48
3.29	Diagram Aktivitas SuperAdmin: Kelola Data Voucher	48
3.30	Diagram Aktivitas SuperAdmin: Kelola Payout Request	49
3.31	ERD User, Verifikasi Identitas, Admin, dan Token Verifikasi Global	50
3.32	ERD Kota, Cabang(Hotel), Fasilitas, Jenis Pod, Pod, Fasilitas, Staff (Admin), dan Gambar	51

3.33	ERD Transaksi, Item Transaksi, Booking, dan QR Code	52
3.34	ERD Voucher, Referral, dan Payout Request	53
3.35	ERD OTA Platform, OTA Transaction, dan Review	54
3.36	ERD Dyanmic Pricing, Pod Cleaning, dan Switch Pod	55
3.37	Diagram Sekuensial Implementasi <i>Offline Fault Tolerance</i> pada Rotasi Kunci Pod	56
3.38	Diagram Sekuensial Integrasi <i>WebHook Payment Gateway</i>	57
3.39	Diagram Sekuensial Integrasi <i>Passwordless</i> dengan OAuth2 Google	58
3.40	Diagram Arsitektur Peluncuran Aplikasi Tamu.id	59
4.1	Tampilan Halaman Pendaftaran User	81
4.2	Tampilan Halaman Login User.....	82
4.3	Tampilan Halaman Profil User	85
4.4	Tampilan Halaman Landing Page User	85
4.5	Komponen Pencarian User	87
4.6	Tampilan Hasil Pencarian User	89
4.7	Tampilan Hasil Pencarian User	89
4.8	Tampilan Proses Pembayaran detail Reservasi	90
4.9	Tampilan Proses Pembayaran Metode Pembayaran.....	90
4.10	Tampilan Proses Pembayaran kode QRIS	91
4.11	Tampilan Proses Pembayaran <i>Pending</i>	91
4.12	Tampilan Proses Pembayaran <i>Success</i>	92
4.13	Tampilan List Reservasi User	93
4.14	Tampilan Proses pilih Pod User	95
4.15	Tampilan Kunci Pod User	95
4.16	Tampilan Checkout dan Memberikan Ulasan User	96
4.17	Tampilan list Transaksi User	97
4.18	Tampilan detail Transaksi User	97
4.19	Tampilan Halaman Referral User.....	98
4.20	Tampilan Halaman Payout User.....	98
4.21	Tampilan Halaman Verifikasi Identitas User	99
4.22	Tampilan Halaman Login Admin.....	100
4.23	Tampilan Halaman Pilih Cabang Admin	100
4.24	Tampilan Halaman Dasbor Admin Cabang	101
4.25	Tampilan Halaman Dasbor Okupansi Admin Cabang	102
4.26	Tampilan Halaman Profil Admin Cabang	103
4.27	Tampilan Admin Kelola Data Cabang	103
4.28	Tampilan Admin Kelola Data Cabang	104
4.29	Tampilan Admin Kelola jenis pod cabang	105
4.30	Tampilan Admin Kelola jenis pod cabang	106
4.31	Tampilan Admin Kelola <i>OTA</i> Cabang	107

4.32	Tampilan Admin Kelola Staff Cabang	108
4.33	Tampilan Admin Kelola Voucher Cabang	109
4.34	Tampilan Admin Kelola Review Cabang	110
4.35	Tampilan Halaman Laporan Harian Admin Cabang	112
4.36	Tampilan Halaman Daftar Tamu Check-in Admin Cabang	113
4.37	Tampilan Halaman Proses Pindah Pod User oleh Admin Cabang	113
4.38	Tampilan Halaman Riwayat Pindah Pod pada Cabang	114
4.39	Tampilan Halaman Daftar Pod yang Harus Dibersihkan	115
4.40	Tampilan Halaman Proses Pembersihan Pod oleh Admin Cabang	115
4.41	Tampilan Halaman Pod Selesai Dibersihkan oleh Admin Cabang	116
4.42	Tampilan Admin Kelola Harga Dinamis Cabang	117
4.43	Tampilan Transaksi pada Cabang	118
4.44	Tampilan Input <i>OTA</i> pada Cabang	118
4.45	Tampilan List Input <i>OTA</i> pada Cabang	119
4.46	Tampilan Halaman Login Superadmin	119
4.47	Tampilan Halaman Dashboard Superadmin	120
4.48	Tampilan Halaman Pengaturan Akun Superadmin	120
4.49	Tampilan Halaman Pengaturan Kota Superadmin	121
4.50	Tampilan Halaman Pengaturan Fasilitas Superadmin	122
4.51	Tampilan Halaman Pengaturan <i>OTA</i> Superadmin	122
4.52	Tampilan Halaman Pengaturan Cabang Superadmin	123
4.53	Tampilan Halaman Pengaturan Staff Superadmin	123
4.54	Tampilan Halaman Pengaturan User Superadmin	124
4.55	Tampilan Halaman List Transaksi Superadmin	124
4.56	Tampilan Halaman Lihat Detail Transaksi Superadmin	125
4.57	Tampilan Halaman Pengaturan Voucher Superadmin	126
4.58	Tampilan Halaman list Permintaan KYC User di Superadmin	126
4.59	Tampilan Halaman Proses KYC User di Superadmin	128
4.60	Tampilan Halaman List Permintaan Payout User di Superadmin	128
4.61	Tampilan Halaman Proses Payout Referral User di Superadmin	129
4.62	Tampilan List Hardware	130
4.63	Tampilan QR Code Berhasil	131

DAFTAR TABEL

2.1	Tabel Perbandingan Studi Pustaka	12
2.2	Notasi <i>Use Case Diagram</i>	17
2.3	Notasi <i>Activity Diagram</i>	18
2.4	Notasi <i>Entity Relationship Diagram</i>	18
2.5	Bobot Jawaban <i>UAT</i> berdasarkan skala <i>Likert</i>	21
2.6	Kategori Kelayakan <i>UAT</i>	21
3.1	Skenario Pengujian Black Box (User)	60
3.2	Skenario Pengujian Black Box (Admin)	68
3.3	Skenario Pengujian Black Box (SuperAdmin)	73
3.4	Kuesioner <i>User Acceptance Test</i> - Perspektif <i>User</i>	77
3.5	Kuesioner <i>UAT</i> - Perspektif Admin Cabang	77
3.6	Kuesioner <i>UAT</i> - Perspektif SuperAdmin	79
4.1	Hasil Pengujian Black Box (User)	133
4.2	Hasil Pengujian Black Box (Admin)	140
4.3	Hasil Pengujian Black Box (SuperAdmin)	145
4.4	Daftar Responden <i>User Acceptance Test</i> (<i>UAT</i>)	148
4.5	Hasil Kuesioner <i>User Acceptance Test</i> - Perspektif <i>User</i>	149
4.6	Hasil Kuesioner <i>User Acceptance Test</i> - Perspektif Admin Cabang	150
4.7	Hasil Kuesioner <i>User Acceptance Test</i> - Perspektif SuperAdmin	152

DAFTAR KODE

4.1	Implementasi Frontend Login User	82
4.2	Implementasi Integrasi <i>Oauth2 Google</i> pada Sistem Autentikasi Pengguna . . .	83
4.3	Implementasi <i>Landing Page User</i>	86
4.4	Implementasi Komponen Pencarian untuk Reservasi	86
4.5	Implementasi Backend Pencarian Pod untuk Reservasi User	88
4.6	Implementasi Tampilan Hasil Pencarian User	88
4.7	Implementasi Integrasi Core API QRIS Midtrans Saat Checkout	91
4.8	Implementasi Integrasi <i>Webhook Midtrans</i> di <i>Backend</i>	92
4.9	Implementasi Otomasi <i>Lifecycle Stays</i> dan mendapatkan <i>List Stays</i>	94
4.10	Implementasi Rotasi dan Regenerasi Kunci Pod	96
4.11	Implementasi Login Admin dan Pilih Cabang	100
4.12	Implementasi Dasbor Admin Cabang	102
4.13	Implementasi Dasbor Okupansi Admin Cabang	102
4.14	Implementasi Halaman Kelola Data Cabang	104
4.15	Implementasi Kelola Jenis Pod Admin	105
4.16	Implementasi Kelola Pod Admin	106
4.17	Implementasi Kelola <i>OTA</i> Admin	107
4.18	Implementasi Kelola oleh Staff Admin	108
4.19	Implementasi Kelola Voucher oleh Admin	109
4.20	Implementasi Moderasi Review oleh Admin	111
4.21	Implementasi Laporan Harian Admin	112
4.22	Implementasi Pindah Pod <i>User</i> oleh Admin	114
4.23	Implementasi Generate Kunci Pod Admin	116
4.24	Implementasi Harga Dinamis berdasarkan Tanggal	117
4.25	Implementasi Validasi ketika <i>Redeem</i> kode Referral	127
4.26	Implementasi Perhitungan Komisi dan Diskon Referral	129
4.27	Implementasi <i>Customized Linkedlist</i> dengan <i>Unix Timestamp</i> pada <i>Hardware IoT</i> 131	
4.28	Implementasi Integrasi <i>Backend</i> dengan <i>Hardware IoT</i>	131

Rancang bangun Aplikasi Manajemen Hotel Kapsul multi-Cabang berbasis Website (Tamu.id)

Oleh :
Aliif Arief Maulana
21/479029/SV/19418

INTISARI

Digitalisasi mendorong transformasi signifikan dalam industri perhotelan, termasuk pada segmen hotel kapsul yang menonjolkan efisiensi ruang dan biaya. Sayangnya, keunggulan fisik tersebut belum selalu diiringi sistem digital yang efisien. Penelitian ini berfokus pada perancangan dan pengembangan *Tamu.id*, sebuah platform manajemen hotel kapsul berbasis web yang komprehensif dan terintegrasi. Sistem ini dibangun dengan arsitektur multi-pengguna (superadmin waralaba, admin hotel, dan tamu) guna mendukung pengelolaan operasional secara terpusat. Fitur utama mencakup sinkronisasi pemesanan dari *Online Travel Agent* (OTA), pengaturan harga dinamis berbasis waktu, dan *Content Management System* (CMS) untuk mengelola informasi properti. Untuk menunjang pertumbuhan bisnis, aplikasi menyediakan fitur promosi berupa *referral* dan voucher diskon yang dapat dikonfigurasi secara fleksibel. Pengembangan sistem dilakukan menggunakan metodologi *Agile* dengan pendekatan *Kanban* agar proses iterasi berjalan fleksibel dan adaptif terhadap perubahan kebutuhan selama siklus pengembangan. Dari sisi pengguna, sistem dirancang mengutamakan kemudahan dan keamanan, dimulai dari proses login yang cepat melalui integrasi Google OAuth2 tanpa password. Untuk mencegah terjadinya *race condition* dalam pemesanan, diterapkan mekanisme *optimistic locking*. Proses pembayaran berlangsung secara *real-time* melalui QRIS yang diintegrasikan dengan layanan Midtrans menggunakan *webhook*. Inovasi utama dari sistem ini adalah implementasi *chained QR Code Rotation* untuk regenerasi kunci kamar digital yang dapat bekerja bahkan dalam kondisi perangkat IoT offline, memastikan akses yang aman dan andal. Evaluasi sistem mencakup pengujian fungsional melalui metode *blackbox* yang menunjukkan seluruh fitur berfungsi sesuai perancangan awal. Selain itu, dilakukan uji penerimaan pengguna menggunakan kuesioner dengan skala Likert yang menunjukkan tingkat kelayakan sangat tinggi: 94,6% untuk *role user*, 96% untuk *superadmin*, dan 100% untuk *admin*. Hasil dari penelitian ini adalah sebuah sistem yang tidak hanya meningkatkan efisiensi operasional hotel kapsul, tetapi juga memberikan pengalaman digital yang modern dan *seamless* bagi pengguna, baik dari sisi pengelola maupun tamu.

Kata kunci: Sistem Informasi, Reservasi, Website, Hotel Kapsul, OAuth2, Webhook, E-Ticketing, Kode QR, QRIS, Harga Dinamis, Booking Online, IoT, CMS

Design and Development of a Web-Based Multi-Tenant Capsule Hotel Management Application (Tamu.id)

by:
Aliif Arief Maulana
21/479029/SV/19418

ABSTRACT

Digitalization has driven significant transformation within the hospitality industry, including in the capsule hotel segment which emphasizes spatial and cost efficiency. Unfortunately, these physical advantages are not always accompanied by efficient digital systems. This study focuses on the design and development of *Tamu.id*, a comprehensive and integrated web-based capsule hotel management platform. The system is built with a multi-user architecture (franchise superadmin, hotel admin, and guest) to support centralized operational management. Core features include booking synchronization from *Online Travel Agents* (OTA), time-based dynamic pricing configuration, and a *Content Management System* (CMS) for property information management. To support business growth, the application also provides promotional features such as referrals and discount vouchers, which can be configured flexibly. System development was carried out using the *Agile* methodology with a *Kanban* approach to ensure a flexible and adaptive iterative process in response to evolving requirements throughout the development cycle. From the user perspective, the system prioritizes simplicity and security, starting with a fast login process through Google OAuth2 integration without requiring passwords. To prevent race conditions during the booking process, an *optimistic locking* mechanism is implemented. Payments are processed in *real-time* via QRIS, integrated with the Midtrans service using a *webhook* mechanism. A key innovation in this system is the implementation of *chained QR Code Rotation* for regenerating digital room keys, which can function even when IoT devices are offline—ensuring secure and reliable access. System evaluation includes functional testing using the *blackbox* method, which demonstrated that all features operate as initially designed. In addition, user acceptance testing was conducted using a Likert scale questionnaire, which showed a very high level of feasibility: 94.6% for user roles, 96% for superadmin, and 100% for admin. The outcome of this study is a system that not only enhances the operational efficiency of capsule hotels but also delivers a modern and seamless digital experience for both hotel operators and guests.

Keywords: Information System, Website, Reservation, Capsule Hotel, OAuth2, Webhook, E-Ticketing, QR Code, QRIS, Dynamic Pricing, Online Booking, IoT, CMS

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam era digital, industri perhotelan global telah mengalami transformasi fundamental yang didorong oleh adopsi teknologi informasi. Sistem manajemen properti atau Property Management System (PMS) telah menjadi pusat operasional bagi hotel modern, mengintegrasikan fungsi-fungsi krusial mulai dari reservasi, manajemen front office, penagihan, hingga pelaporan. Implementasi PMS yang efektif tidak lagi menjadi pilihan, melainkan sebuah keharusan strategis bagi hotel untuk dapat bersaing, meningkatkan efisiensi operasional, dan memberikan pengalaman yang memuaskan bagi tamu [1]. Seiring dengan evolusi teknologi, sistem ini terus berkembang menjadi platform terpadu yang mampu mengelola berbagai kanal distribusi dan strategi bisnis yang kompleks.

Hotel kapsul merupakan bentuk akomodasi yang berasal dari Jepang dan kini telah menyebar ke berbagai negara, termasuk Indonesia. Konsepnya yang menawarkan ruang tidur minimalis namun fungsional menjadikannya pilihan populer di kalangan wisatawan, terutama mereka yang mencari penginapan hemat biaya dan efisien ruang. Di Indonesia, hotel kapsul telah berkembang pesat di kota-kota besar seperti Jakarta, Bandung, dan Yogyakarta. Beberapa contoh hotel kapsul yang populer di Indonesia antara lain Bobobox, Digital Airport Hotel, dan Tab Capsule Hotel [2].

Pertumbuhan hotel kapsul di Indonesia didorong oleh meningkatnya mobilitas urban dan pergeseran perilaku konsumen ke arah digital, yang menuntut akomodasi praktis serta terjangkau. Meskipun model bisnis ini efisien, banyak pengelola masih menghadapi tantangan operasional akibat proses pemesanan yang manual dan sistem pembayaran yang tidak terintegrasi. Solusi yang dibutuhkan adalah sistem manajemen terpadu berbasis website atau aplikasi yang mampu memfasilitasi proses bisnis secara real-time untuk berbagai tingkat pengguna, mulai dari pemilik waralaba hingga pengguna akhir. Penerapan teknologi informasi yang komprehensif seperti ini terbukti secara signifikan dapat meningkatkan kinerja organisasi dan efisiensi operasional dalam industri perhotelan [3].

Seiring dengan perkembangan teknologi finansial di Indonesia, Bank Indonesia telah meluncurkan standar QR Code untuk pembayaran yang dikenal dengan QRIS (Quick Response Code Indonesian Standard) pada tahun 2019. QRIS memungkinkan berbagai aplikasi pembayaran digital seperti GoPay, OVO, dan Dana untuk menggunakan satu kode QR yang sama, sehingga memudahkan proses transaksi bagi konsumen dan pelaku usaha [4]. Adopsi QRIS telah meningkat secara signifikan di Indonesia, dengan jutaan transaksi yang dilakukan setiap bulannya. Integrasi QRIS dalam sistem manajemen hotel kapsul akan

memungkinkan tamu untuk melakukan pembayaran secara cepat dan aman, serta membantu pengelola hotel dalam memantau transaksi secara real-time [5]

Dalam industri perhotelan, strategi harga dinamis atau dynamic pricing telah menjadi praktik umum untuk memaksimalkan pendapatan. Dengan strategi ini, harga kamar dapat disesuaikan berdasarkan permintaan pasar, musim, atau waktu tertentu. Misalnya, harga kamar dapat dinaikkan saat permintaan tinggi seperti musim liburan, dan diturunkan saat permintaan rendah untuk menarik lebih banyak tamu [6]. Penerapan harga dinamis dalam sistem manajemen hotel kapsul akan memberikan fleksibilitas bagi pengelola hotel dalam menentukan harga yang kompetitif dan sesuai dengan kondisi pasar. Hal ini juga akan meningkatkan efisiensi operasional dan pendapatan hotel secara keseluruhan.

Keamanan dan kenyamanan dalam proses autentikasi pengguna menjadi aspek penting dalam pengembangan aplikasi berbasis web. OAuth2 adalah protokol otorisasi yang memungkinkan aplikasi pihak ketiga untuk mengakses sumber daya pengguna tanpa harus mengetahui kredensial pengguna tersebut [7]. Dengan mengintegrasikan OAuth2, pengguna dapat melakukan login ke sistem manajemen hotel kapsul menggunakan akun Google mereka tanpa perlu membuat akun baru atau mengingat kata sandi tambahan. Hal ini tidak hanya meningkatkan keamanan, tetapi juga memberikan pengalaman pengguna yang lebih baik.

Dalam arsitektur pembayaran digital, webhook digunakan sebagai mekanisme notifikasi real-time yang esensial untuk mengirimkan data secara otomatis dari penyedia layanan pembayaran ke sistem merchant, seperti pada sistem hotel kapsul. Pendekatan yang digerakkan oleh peristiwa (event-driven) ini, misalnya saat mengonfirmasi pembayaran QRIS dari Midtrans, secara drastis mengurangi kebutuhan verifikasi manual dan menekan biaya server dengan menghindari metode polling yang tidak efisien [8]. Dengan demikian, sistem manajemen hotel dapat secara instan memperbarui status pemesanan menjadi lunas sesaat setelah transaksi berhasil, tanpa intervensi manusia. Alhasil, integrasi webhook tidak hanya mengoptimalkan efisiensi operasional, tetapi juga secara signifikan meningkatkan kualitas pengalaman bagi tamu dan pengelola hotel.

Penerapan teknologi QR Code merevolusi sistem kontrol akses di perhotelan modern, menggantikan kunci fisik dengan solusi digital yang lebih aman dan efisien bagi tamu hotel kapsul. Untuk mengatasi potensi kendala koneksi internet pada perangkat IoT (kunci pintu), implementasi struktur data LinkedList digunakan untuk menyinkronkan daftar kredensial yang valid ke dalam memori perangkat; setiap node pada list ini menyimpan data QR Code unik beserta Unix timestamp yang secara presisi mendefinisikan waktu mulai dan berakhirnya masa berlaku kunci tersebut. Saat beroperasi dalam mode offline, perangkat kunci pintu akan memvalidasi QR Code yang dipindai oleh tamu dengan mencocokkannya terhadap data dan rentang waktu yang tersimpan secara lokal di memorinya, sebuah prinsip yang sejalan dengan arsitektur keamanan untuk sistem akses IoT [9]. Dengan demikian, sistem cerdas ini tidak

hanya secara drastis meningkatkan kenyamanan dan keamanan bagi tamu, tetapi juga menjamin reliabilitas akses kamar secara penuh bahkan ketika terjadi gangguan konektivitas jaringan.

Agar dapat bersaing secara efektif, sistem manajemen hotel kapsul modern harus mampu melakukan sinkronisasi dua arah dengan berbagai Online Travel Agency (OTA) melalui channel manager terpusat, serta dilengkapi Content Management System (CMS) yang fleksibel untuk menjaga akurasi informasi properti. Meskipun OTA krusial untuk akuisisi pelanggan, strategi jangka panjang yang vital adalah mendorong pergeseran pelanggan untuk melakukan pemesanan melalui platform mandiri seperti Tamu.id. Keberhasilan migrasi ini bergantung pada kemampuan platform direct-booking untuk menawarkan proposisi nilai yang unik, misalnya melalui fitur eksklusif, pengalaman pengguna yang lebih baik, atau insentif khusus yang tidak tersedia di OTA [10]. Dengan demikian, hotel tidak hanya mengurangi risiko overbooking dan inefisiensi, tetapi juga membangun loyalitas pelanggan secara langsung dan meningkatkan margin keuntungan dengan memotong biaya komisi yang tinggi.

Dalam pengembangan sistem manajemen hotel kapsul Tamu.id, filosofi Agile dengan metode Kanban memungkinkan proses kerja yang iteratif dan sangat responsif terhadap perubahan kebutuhan. Dengan memvisualisasikan alur kerja pada papan Kanban, tim pengembang dapat secara proaktif mengidentifikasi hambatan untuk meningkatkan efisiensi secara keseluruhan [11]. Transparansi proses ini juga mendorong kolaborasi yang lebih erat antara tim teknis dan para pemangku kepentingan, sehingga memastikan adanya keselarasan visi. Pada akhirnya, pendekatan ini menjamin bahwa produk akhir yang dikembangkan benar-benar dapat memenuhi kebutuhan spesifik pengguna secara lebih efektif dan tepat sasaran.

1.2 Rumusan Masalah

Rumusan masalah yang dapat diambil dari latar belakang yang sudah dijabarkan sebelumnya antara lain:

1. Bagaimana mendesain dan mengimplementasikan sistem manajemen hotel kapsul berbasis web yang terintegrasi dan *Seamless*?
2. Bagaimana mengintegrasikan sistem pembayaran digital yang mudah dan aman?
3. Bagaimana memastikan sistem yang dikembangkan dapat diterima oleh pengguna?

1.3 Tujuan dan Manfaat Proyek Akhir

1.3.1 Tujuan Proyek Akhir

Berdasarkan rumusan masalah yang telah ditetapkan, tujuan dari proyek akhir ini adalah sebagai berikut:

1. Merancang dan membangun sebuah sistem manajemen hotel kapsul berbasis web yang

terintegrasi, mulai dari pengelolaan operasional terpusat (ketersediaan kamar, kebersihan, dan harga dinamis) hingga pengalaman pengguna yang *seamless* melalui autentikasi *passwordless* (OAuth2) dan sistem kunci digital berbasis QR Code yang andal bahkan saat *offline* (dengan implementasi *LinkedList*).

2. Mengintegrasikan sistem pembayaran digital yang aman dan mudah melalui *payment gateway* Midtrans, dengan memanfaatkan *webhook* untuk notifikasi transaksi *real-time* dan menyediakan QRIS sebagai metode pembayaran utama yang universal bagi pengguna.
3. Mengembangkan fitur-fitur yang dirancang untuk meningkatkan penerimaan dan kepuasan pengguna, yang mencakup antarmuka yang ramah pengguna (*user-friendly*), sistem ulasan dan peringkat (*rating and review*), serta program loyalitas seperti *voucher* dan *affiliate marketing* yang memberikan keuntungan finansial bagi tamu.

1.3.2 Manfaat Proyek Akhir

Penyelesaian proyek akhir ini diharapkan dapat memberikan manfaat yang signifikan bagi berbagai pihak terkait, yang dapat dirincikan sebagai berikut:

A. Manfaat bagi Pengelola Hotel (Tamu.id)

- **Peningkatan Efisiensi Operasional:** Sistem terpusat memungkinkan pengelolaan data reservasi dari berbagai sumber (termasuk OTA), status kebersihan kamar, proses pindah kamar, dan penerapan harga dinamis secara otomatis. Hal ini mengurangi pekerjaan manual, meminimalkan *human error*, dan mempercepat alur kerja harian.
- **Peningkatan Pendapatan dan Profitabilitas:** Dengan platform reservasi mandiri yang kaya fitur (seperti *voucher* dan promosi), ketergantungan pada Online Travel Agency (OTA) yang membebankan komisi tinggi dapat dikurangi. Fitur *affiliate marketing* dengan kode *referral* juga membuka kanal pemasaran baru yang berpotensi meningkatkan okupansi.
- **Pengambilan Keputusan Berbasis Data:** Adanya fitur *rating* dan ulasan dari tamu memberikan masukan langsung yang berharga untuk perbaikan kualitas layanan. Data operasional yang terkumpul juga dapat dianalisis untuk membuat keputusan bisnis yang lebih strategis.

B. Manfaat bagi Tamu (Pengguna Akhir)

- **Pengalaman Pengguna yang Lebih Baik:** Proses dari pemesanan hingga *check-in* menjadi lebih mudah dan cepat. Pengguna dapat melakukan reservasi melalui antarmuka yang responsif, *login* tanpa kata sandi menggunakan akun Google, melakukan pembayaran dengan QRIS, dan masuk ke kamar hanya dengan memindai QR Code dari ponsel mereka.

- **Keamanan dan Kepercayaan:** Integrasi dengan *payment gateway* terpercaya dan mekanisme autentikasi modern (OAuth2, JWT) memberikan jaminan keamanan data dan transaksi. Sistem ulasan yang transparan juga membantu calon tamu dalam membuat keputusan yang lebih baik.
- **Keuntungan Tambahan:** Tamu dapat memperoleh keuntungan finansial melalui diskon yang didapat dari kode *referral* atau bahkan mendapatkan komisi dengan berpartisipasi dalam program afiliasi, memberikan nilai lebih dari sekadar menginap.

C. Manfaat dari Sisi Pengembangan Teknologi

- **Implementasi Arsitektur Modern:** Proyek ini menjadi studi kasus penerapan arsitektur sistem berbasis web modern yang menggabungkan *backend (Rest API)*, *payment gateway* (Midtrans), protokol otentikasi (OAuth2, JWT), dan integrasi dengan perangkat IoT (*smart lock*) secara efisien dan aman.
- **Solusi Inovatif untuk Masalah Offline:** Penggunaan struktur data *LinkedList* untuk sinkronisasi kredensial QR Code ke perangkat IoT merupakan sebuah solusi konkret dan inovatif untuk mengatasi potensi masalah konektivitas internet pada sistem kontrol akses.

1.4 Batasan Masalah

Karena keterbatasan waktu dan sumber daya, Tamu.id memiliki beberapa batasan masalah yang perlu diperhatikan, yaitu:

1. Aplikasi hanya dapat diakses pada web browser dan membutuhkan jaringan internet.
2. User tidak dapat melakukan pemesanan tanpa login ke dalam sistem.
3. User tidak dapat melakukan pemesanan lebih dari satu jenis Pod dalam satu waktu dengan pemesanan via Platform Tamu.id.
4. Integrasi Payment gateway yang digunakan adalah Midtrans dengan metode pembayaran QRIS saja menimbang QRIS adalah standar QR Code yang dikeluarkan oleh Bank Indonesia dan hampir semua bank dan e-wallet di Indonesia sudah mendukung QRIS.
5. Karena keterbatasan dokumen perizinan dan waktu belum ada integrasi API otomatis via webhook dengan online travel agent (OTA) seperti Traveloka, Agoda, dan Booking.com.
6. Integrasi API Backend dengan IoT diilustrasikan dengan menggunakan aplikasi web simulasi.

1.5 Sistematika Penulisan

Sistematika penulisan laporan Proyek Akhir ini disusun secara sistematis agar pembaca dapat memahami alur pembahasan dengan jelas dan runtut. Penulisan dibagi ke dalam beberapa

bab, mulai dari pendahuluan hingga lampiran, dengan rincian sebagai berikut:

BAB I : PENDAHULUAN

Bab ini memberikan gambaran umum mengenai latar belakang permasalahan yang melatarbelakangi dilakukannya proyek akhir, perumusan masalah yang akan dikaji, tujuan yang hendak dicapai, serta manfaat yang diharapkan baik secara teoritis maupun praktis. Selain itu, bab ini juga memuat batasan masalah yang menjadi ruang lingkup kajian agar fokus pembahasan lebih terarah. Di akhir bab, dijelaskan pula sistematika penulisan yang memberikan panduan mengenai struktur dan isi masing-masing bab dalam laporan ini.

BAB II : STUDI PUSTAKA

Bab ini membahas berbagai teori, konsep, dan referensi dari penelitian atau sumber ilmiah terdahulu yang relevan dengan topik proyek akhir. Kajian pustaka ini mencakup pembahasan mengenai sistem e-ticketing, mekanisme reservasi daring, pemanfaatan teknologi QRCode, serta ulasan terhadap perangkat teknologi yang digunakan dalam pengembangan sistem, baik dari sisi perangkat lunak maupun perangkat keras. Studi pustaka ini menjadi dasar pijakan dalam merumuskan solusi dan pendekatan teknis yang digunakan dalam proyek akhir ini.

BAB III : METODE PENELITIAN

Bab ini menjelaskan pendekatan dan metode yang digunakan selama pelaksanaan proyek akhir. Penjelasan mencakup tahapan-tahapan dalam pengembangan sistem, mulai dari perencanaan, perancangan, implementasi, hingga pengujian. Penggunaan filosofi Agile dengan metode Kanban dijelaskan sebagai strategi manajemen proyek yang fleksibel dan adaptif. Selain itu, dibahas pula integrasi sistem secara teknis, seperti penerapan one-time token untuk penggabungan otentikasi OAuth2 milik Google dengan sistem login konvensional menggunakan email dan sandi, integrasi API dan webhook untuk sistem pembayaran otomatis, serta rancangan struktur data linked list berbasis timestamp sebagai mekanisme rotasi kunci pada sistem QRCode yang terintegrasi dengan perangkat IoT secara offline dan fault-tolerant.

BAB IV : HASIL DAN PEMBAHASAN

Bab ini menyajikan hasil dari proses implementasi sistem yang telah dirancang pada bab sebelumnya. Uraian dilakukan secara sistematis mulai dari hasil pengembangan antarmuka pengguna, proses backend, serta integrasi sistem secara keseluruhan. Hasil pengujian terhadap sistem turut disertakan dalam bab ini, meliputi User Acceptance Testing (UAT) untuk menilai kesesuaian sistem terhadap kebutuhan pengguna, Black Box Testing untuk menguji fungsionalitas sistem untuk mengevaluasi performa sistem dalam kondisi beban tinggi. Pembahasan disusun secara analitis untuk mengkaji sejauh mana sistem memenuhi tujuan dan spesifikasi yang telah dirumuskan sebelumnya.

BAB V : PENUTUP

Bab ini merupakan bagian akhir dari laporan proyek akhir yang berisi kesimpulan dari seluruh proses pelaksanaan proyek, mulai dari identifikasi masalah hingga hasil pengujian sistem. Kesimpulan disusun berdasarkan temuan yang diperoleh selama pelaksanaan proyek. Selain itu, bab ini juga memuat saran-saran yang dapat digunakan sebagai acuan bagi pengembangan sistem lanjutan atau penelitian selanjutnya yang relevan dengan topik ini, dengan harapan dapat menyempurnakan dan memperluas cakupan sistem yang telah dikembangkan.

DAFTAR PUSTAKA : DAFTAR PUSTAKA

Bagian ini memuat seluruh referensi dan sumber literatur yang digunakan sebagai landasan teori dan acuan teknis dalam penyusunan laporan proyek akhir. Daftar pustaka dapat berupa buku, jurnal ilmiah, artikel daring, dokumentasi perangkat lunak, maupun sumber lainnya yang kredibel dan relevan.

LAMPIRAN : LAMPIRAN

Bagian lampiran menyajikan dokumen-dokumen pendukung yang tidak dicantumkan dalam bab utama, namun memiliki relevansi penting terhadap keseluruhan isi laporan. Lampiran dapat mencakup tangkapan layar sistem, potongan kode program penting, dokumentasi pengujian, konfigurasi sistem, diagram tambahan, serta dokumen lainnya yang mendukung validitas dan keterlacakan hasil proyek akhir.

BAB II

TINJAUAN PUSTAKA

2.1 Studi Pustaka

Laporan proyek akhir ini didasarkan pada berbagai penelitian terdahulu yang relevan dengan pengembangan aplikasi berbasis web untuk reservasi hotel. Fokus penelitian ini mencakup integrasi QR Code untuk sistem kunci digital, penggunaan OAuth2 sebagai metode otentikasi tanpa kata sandi, serta implementasi *webhook* untuk integrasi pembayaran. Studi pustaka ini bertujuan memberikan wawasan terkini terkait teknologi yang digunakan untuk membangun sistem penjualan tiket yang efisien, aman, dan mudah digunakan.

Penelitian sebelumnya [12] berjudul *Rancang Bangun Sistem Informasi Reservasi Hotel "Hotel Hebat" Berbasis Website* mengembangkan sistem informasi reservasi hotel yang memungkinkan pengguna untuk melakukan pemesanan kamar secara online. Sistem ini dibangun dengan menggunakan bahasa pemrograman PHP dan database MySQL. Hasil penelitian menunjukkan bahwa sistem informasi reservasi hotel ini dapat meningkatkan efisiensi dalam proses pemesanan kamar, mengurangi kesalahan manual, dan memberikan kemudahan bagi pengguna dalam mengakses informasi terkait ketersediaan kamar.

Selain itu, penelitian lain [13] berjudul *Rancang Bangun Sistem Reservasi Hotel pada Hotel Larona Berbasis Website* juga mengembangkan sistem informasi reservasi hotel berbasis web. Penelitian ini menggunakan metode pengembangan perangkat lunak Waterfall dan bahasa pemrograman PHP dengan database MySQL. Hasil penelitian menunjukkan bahwa semua komponen dalam sistem informasi reservasi hotel ini dapat berfungsi dengan baik.

Pada penelitian ini [14] yang melakukan perancangan *Sistem Informasi Pemesanan Tiket Bus Damri di Bandara menggunakan QR Code dan Web Base* bahwa digitalisasi dalam konteks pemesanan tiket dapat mempermudah baik dari sisi pengguna maupun pihak penyedia layanan. penelitian ini [14] dilakukan dengan tujuan untuk meningkatkan visibilitas dan efisiensi dalam proses pemesanan tiket bus Damri di Bandara XYZ. Hasil penelitian menunjukkan bahwa penggunaan QR Code dalam sistem pemesanan tiket bus Damri di Bandara Soekarno-Hatta dapat meningkatkan visibilitas dengan sistem informasi yang berbasis website dan efisiensi dengan QR Code dalam proses pengelolaan tiket bus Damri di Bandara XYZ.

Penelitian juga dilakukan oleh [15] dengan topik *Perancangan Aplikasi Pemesanan Tiket Masuk Pada Central Park Zoo Medan*, sistem ini dibangun dengan memadukan antara aplikasi berbasis web dan mobile yaitu di sisi admin menggunakan aplikasi berbasis web dan di sisi pengguna menggunakan aplikasi berbasis mobile, aplikasi web dibangun dengan

menggunakan bahasa pemrograman PHP dan database MySQL, sedangkan aplikasi mobile dibangun dengan menggunakan bahasa pemrograman Java berbasis Android. Hasil penelitian menunjukkan bahwa aplikasi pemesanan tiket masuk pada Central Park Zoo Medan dapat mempermudah pengunjung dalam melakukan pemesanan tiket secara online dengan berbagai metode pembayaran dan mempermudah pihak pengelola dalam melakukan pengelolaan data pengunjung serta mempercepat proses validasi tiket masuk.

Dengan sudut pandang yang lebih menarik pada penelitian [16] dengan judul *Development of IoT-Based E-ticket Selling and Management System with QR Code Scanner (Tickets.now)* yang mengintegrasikan teknologi Internet of Things (IoT) dalam pengembangan sistemnya yaitu dengan menggunakan ESP32-Cam dengan modul WiFi sebagai QR Code Scanner berbeda dengan penelitian lain yang menggunakan perangkat lunak kamera seperti smartphone ataupun webcam. Hasil penelitian menunjukkan bahwa penggunaan teknologi IoT dalam sistem e-ticketing juga dapat dilakukan dan tentunya memiliki kelebihan dan kekurangan masing-masing menyesuaikan dengan sumber daya yang dimiliki.

Mirip seperti penelitian dengan IoT sebelumnya, pada penelitian ini [17] dengan judul *Implementasi Pembayaran Dan Palang Otomatis Pada Sistem Smart Parking Di Lahan Parkir Menggunakan Metode QR Code* memiliki pembahasan mengenai IoT yang lebih komprehensif ditunjukkan dengan integrasi perangkat *Hardware* yang lebih lengkap, selain itu juga tentunya menggunakan QR Code sebagai basis Tiket dengan website yang dikembangkan dengan PHP dan MySQL serta TriPay sebagai payment gateway dalam sistem smart parking. Hasil penelitian menunjukkan bahwa implementasi pembayaran dan palang otomatis pada sistem smart parking di lahan parkir menggunakan metode QR Code dapat mempermudah pengguna dalam melakukan pembayaran dan mempercepat proses keluar masuk lahan parkir ditunjukkan dengan hasil uji coba fungsionalitas manual baik secara software yaitu website Admin dan website User untuk kemudian melakukan pembayaran, serta hardware yaitu palang otomatis berbasis QR Code dengan pengujian QoS (*Quality of Service*), Pengujian *Throughput*, dan Pengujian *Packet Loss*.

Penelitian berkaitan dengan payment gateway tripay juga dilakukan [18] dengan judul *Pembangunan Sistem Transaksi Penjualan Barang dan Jasa (Studi Kasus: Startup Botanik Kota)* yang membuat website untuk mendigitalisasi UMKM dalam proses operasinya dengan konsep MVC (model, view, controller) menggunakan Yii Framework, MySQL, dan Payment Gateway Tripay, metode pengembangan dilakukan secara *Waterfall*. Hasil penelitian menunjukkan bahwa pembangunan sistem transaksi penjualan barang dan jasa pada startup Botanik Kota dapat meningkatkan efektifitas dan efisiensi dalam operasionalnya ditunjukkan dengan pengujian unit menggunakan cyclomatic complexity paling tinggi bernilai 10 dan uji fungsi terhadap 3 sampel komponen dan integrasi perangkat lunak bernilai 100% valid.

Penelitian [19] juga dilakukan dengan judul *Implementasi Backend System Untuk*

Integrasi Payment Gateway Pada Sistem Pembayaran Kost Menggunakan Express.js yang bertujuan untuk mempermudah proses pembayaran kost dengan menggunakan Express.js sebagai backend system, MySQL sebagai database, dan Payment Gateway Midtrans, yang menarik dari penelitian ini adalah fokusnya yang ada pada membandingkan keamanan dari sisi integrasi payment gateway khususnya midtrans yaitu via *snap* tanpa backend dan *core API* menggunakan backend. Hasil penelitian menunjukkan bahwa implementasi backend system untuk integrasi payment gateway pada sistem pembayaran kost menggunakan Express.js dapat meningkatkan keamanan sistem pembayaran kost dibandingkan dengan sistem pembayaran menggunakan midtrans snap tanpa backend, disini secara tidak langsung menyebutkan bahwa sistem dengan backend yang mengimplemenasikan webhook lebih baik dalam hal keamanan maupun efektifitas dan efisiensi.

Ada juga penelitian [20] dengan judul *Sistem Pengesahan Dokumen dengan QR Code dan Memanfaatkan JSON Web Token (JWT) untuk Mengurangi Penyimpanan: Studi Kasus Universitas Esa Unggul Kampus Bekasi* yang bertujuan untuk mempermudah proses pengesahan dokumen dengan menggunakan QR Code dan JSON Web Token (JWT) untuk mengurangi penyimpanan, penelitian ini menggunakan metode Waterfall dengan menggunakan PHP, MySQL, Laragon, Codeigniter, dan JWT. Hasil penelitian menunjukkan bahwa sistem pengesahan dokumen dengan QR Code dan memanfaatkan JSON Web Token (JWT) untuk mengurangi penyimpanan dapat mempermudah proses pengesahan dokumen dan mengurangi penyimpanan data yang tidak perlu dengan kemampuan JWT yang dapat menyimpan data dalam bentuk token stateless yang aman dan dapat diverifikasi keabsahannya tanpa perlu selalu terhubung dengan database.

Terakhir penelitian [21] dengan judul *Student Mobile Attendance Application Using QRCode and Integrated with SSO at Universitas Sebelas Maret* yang bertujuan untuk mempermudah proses absensi mahasiswa oleh dosen via Aplikasi Android yang terintegrasi dengan SSO (Single Sign On) dalam proses autentikasinya, pengembangan aplikasi dilakukan dengan menggunakan metode SDLC dengan bahasa pemrograman Java sedangkan untuk autentikasinya itulah yang mengimplementasikan JWT lalu untuk absensi menggunakan QR Code. Hasil penelitian menunjukkan bahwa aplikasi absensi mahasiswa menggunakan QR Code dan terintegrasi dengan SSO di Universitas Sebelas Maret dapat mempermudah proses absensi mahasiswa oleh dosen dan mempercepat proses autentikasi mahasiswa.

Berikut adalah pada tabel 2.1 merupakan ringkasan dari berbagai penelitian yang telah dilakukan sebelumnya, yang menunjukkan fokus, metode, teknologi yang digunakan, dan hasil yang diperoleh dari masing-masing penelitian tersebut.

Penelitian	Fokus	Metode	Teknologi	Hasil
[12]	Sistem reservasi hotel berbasis web	Rancang Bangun	PHP, MySQL	Meningkatkan efisiensi pemesanan kamar & mengurangi kesalahan manual.
[13]	Sistem reservasi hotel berbasis web	Waterfall	PHP, MySQL	Semua komponen berfungsi dengan baik dalam sistem reservasi hotel.
[14]	Digitalisasi tiket bus Damri	Studi Kasus	Web, QR Code	Digitalisasi tiket meningkatkan visibilitas & efisiensi layanan.
[15]	Penggunaan tiket online untuk pengelolaan pengunjung	Studi Kasus	Web (PHP, MySQL), Mobile (Java, Android)	Aplikasi tiket online mempermudah pengelolaan pengunjung & validasi tiket.
[16]	Integrasi IoT dalam e-ticketing	Pengembangan Sistem (IoT)	ESP32-Cam, QR Code	Integrasi IoT dalam e-ticketing memungkinkan otomatisasi lebih lanjut.
[17]	Penerapan smart parking berbasis QR Code	Pengujian Kinerja (QoS, Throughput, Packet Loss)	PHP, MySQL, TriPay, QR Code, IoT	QR Code & otomatisasi smart parking mempercepat pembayaran & akses.
[18]	Digitalisasi transaksi UMKM	Waterfall, Uji Unit (Cyclomatic Complexity)	Yii Framework, MySQL, TriPay	Digitalisasi transaksi UMKM meningkatkan efisiensi operasional.
[19]	Perbandingan integrasi Midtrans (Snap vs Core API)	Perbandingan Teknis	Express.js, MySQL, Midtrans (Core API)	Backend webhook lebih aman dan efisien dibandingkan Snap tanpa backend.
[20]	Verifikasi QR Code menggunakan JWT	Waterfall	PHP, MySQL, Codeigniter, JWT, QR Code	QR Code + JWT mengurangi penyimpanan data & mempermudah verifikasi.

Lanjutan di halaman berikutnya...

Penelitian	Fokus	Metode	Teknologi	Hasil
[21]	Implementasi SSO dengan QR Code	SDLC, Implementasi SSO	Java (Android), SSO, JWT, QR Code	Integrasi QR Code & SSO mempercepat autentikasi & absensi mahasiswa.

Tabel 2.1 Tabel Perbandingan Studi Pustaka

Dari berbagai penelitian tersebut banyak menggunakan PHP sebagai bahasa pemrograman utama, MySQL sebagai database, dan QR Code sebagai metode autentikasi. Namun bukan itu sebenarnya yang menjadi titik beratnya namun yang menjadi persamaan dengan penelitian kali ini lebih ke implementasi dan integrasi yang dilakukan, seperti pada penelitian [16] yang mengintegrasikan IoT dalam e-ticketing, [17] yang mengintegrasikan hardware pada smart parking, [18] yang mengintegrasikan payment gateway Tripay pada UMKM, dan [19] yang membandingkan keamanan integrasi payment gateway Midtrans. Hal ini menunjukkan bahwa implementasi dan integrasi teknologi yang tepat dapat memberikan nilai tambah yang signifikan bagi sistem yang dikembangkan. yang menjadi pembeda dan keunggulan dari penelitian ini adalah integrasi berbagai teknologi seperti OAuth2 untuk autentikasi, webhook untuk integrasi pembayaran, dan QR Code untuk sistem kunci digital dengan regenerasi kunci yang mengimplementasikan *Customized LinkedList* sebagai struktur data untuk menyimpan data QR Code yang valid, serta penerapan harga dinamis untuk meningkatkan efisiensi operasional hotel kapsul.

2.2 Dasar Teori

Pembuatan proyek akhir ini tentunya merujuk pada beberapa teori dan konsep yang relevan. Studi pustaka ini bertujuan untuk memberikan landasan teoritis yang mendukung pengembangan proyek akhir ini. Beberapa konsep yang menjadi dasar dalam proyek akhir ini antara lain:

2.2.1 Hotel Kapsul

Hotel kapsul adalah jenis akomodasi yang menawarkan ruang tidur kecil dan efisien, biasanya dalam bentuk kapsul atau pod. Konsep ini berasal dari Jepang dan telah menyebar ke berbagai negara di seluruh dunia. Hotel kapsul dirancang untuk memberikan pengalaman menginap yang nyaman dengan harga terjangkau, terutama bagi pelancong yang mencari akomodasi sementara [2], istilah kamar/room dalam hotel konvensional bila dalam hotel kapsul berubah sesuai dengan bentuknya sebut saja pada platform Tamu.id kamar disebut sebagai Pod, dan dalam satu kapsul hotel terdiri dari beberapa jenis Pod dan tiap jenis pod memiliki spesifikasi dan jumlah Pod yang berbeda-beda.

2.2.2 Sistem Informasi

Sistem informasi adalah kumpulan komponen yang saling berinteraksi untuk mengumpulkan, mengelola, memproses, dan menyebarluaskan informasi guna mendukung pengambilan keputusan yang efektif dan efisien [22]. Sistem informasi terdiri dari beberapa elemen utama, yaitu perangkat keras, perangkat lunak, manusia, data, dan prosedur yang saling berhubungan untuk mencapai tujuan tertentu.

2.2.3 Website

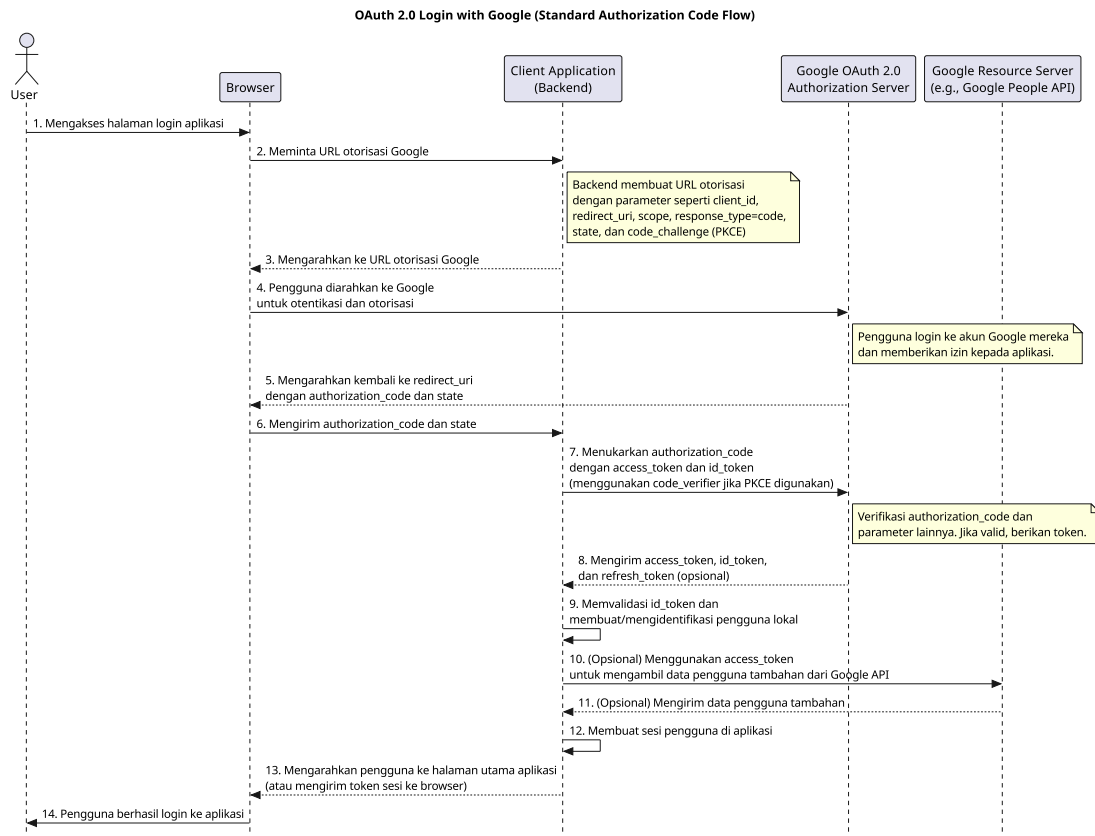
Website adalah sekumpulan halaman web yang saling terhubung dan dapat diakses melalui internet menggunakan protokol HTTP atau HTTPS [23]. Website dirancang untuk menyediakan informasi, layanan, atau hiburan kepada pengguna. Dalam konteks modern, website menjadi elemen penting dalam kehidupan sehari-hari, mulai dari e-commerce, pendidikan, hingga media sosial.

Website terdiri dari dua komponen utama: front-end dan back-end. Front-end mengacu pada tampilan antarmuka pengguna yang dapat diakses melalui browser, sedangkan back-end mencakup server, database, dan logika aplikasi yang mendukung fungsi website tersebut.

Perkembangan teknologi web, seperti HTML5, CSS3, dan JavaScript, telah memungkinkan website menjadi lebih interaktif dan responsif. Selain itu, keberadaan sistem manajemen konten (CMS) seperti WordPress dan Joomla memudahkan pengguna tanpa latar belakang teknis untuk mengelola konten website mereka.

2.2.4 OAuth 2.0

OAuth 2.0 adalah protokol otorisasi yang dirancang untuk memberikan akses terbatas ke sumber daya pengguna di server tanpa perlu membagikan kredensial mereka secara langsung [24]. Protokol ini biasanya digunakan dalam aplikasi web, perangkat mobile, dan layanan pihak ketiga lainnya.



Gambar 2.1 Ilustrasi Protokol OAuth 2.0

Seperti pada gambar 2.1, prinsip kerja OAuth 2.0 melibatkan beberapa entitas, yaitu resource owner (pemilik sumber daya), client (aplikasi pihak ketiga), authorization server (server otorisasi), dan resource server (server sumber daya). Dalam skenario OAuth 2.0, pengguna memberikan izin kepada aplikasi pihak ketiga untuk mengakses data mereka melalui token akses yang diterbitkan oleh server otorisasi.

OAuth 2.0 sering digunakan oleh platform besar seperti Google, Facebook, dan GitHub untuk memungkinkan integrasi dengan layanan mereka. Keamanan dalam OAuth 2.0 sangat bergantung pada implementasi yang benar, termasuk penggunaan HTTPS, pengelolaan token dengan benar, dan mitigasi serangan seperti CSRF.

2.2.5 Webhook

Webhook adalah mekanisme komunikasi antara server yang memungkinkan pemberitahuan secara real-time melalui HTTP POST ketika suatu peristiwa tertentu terjadi [25]. Webhook sering digunakan dalam pengintegrasian aplikasi untuk memastikan data tetap sinkron tanpa perlu polling terus-menerus.

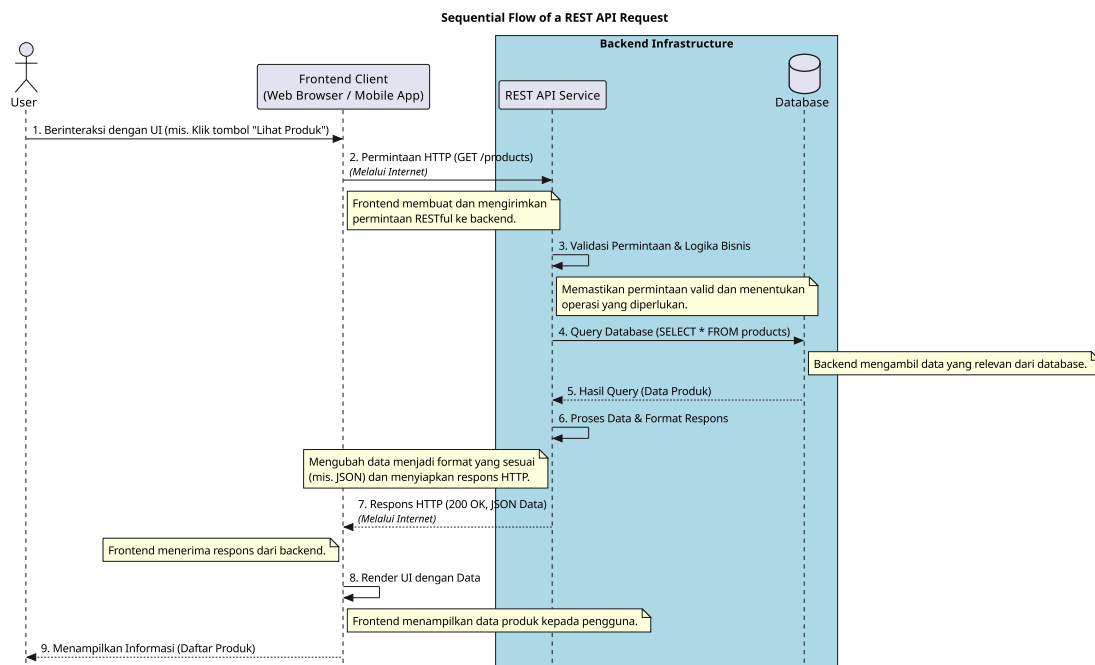
Dalam praktiknya, webhook bekerja dengan cara mengirimkan payload berupa data JSON atau XML ke URL tertentu yang telah ditentukan oleh pengguna. Misalnya, layanan pembayaran online dapat menggunakan webhook untuk memberi tahu merchant ketika

pembayaran berhasil dilakukan.

Keuntungan utama webhook adalah efisiensi karena server hanya mengirimkan data saat diperlukan, dibandingkan dengan polling yang memeriksa perubahan secara berkala. Namun, implementasi webhook memerlukan pengamanan ekstra untuk mencegah akses tidak sah, seperti menggunakan token validasi atau enkripsi data.

2.2.6 Rest API (*Representational State Transfer Application Programming Interface*)

REST API (*Representational State Transfer Application Programming Interface*) adalah arsitektur layanan web yang dirancang untuk mendukung komunikasi antara klien dan server melalui protokol HTTP. REST diperkenalkan oleh Roy Fielding dalam disertasinya pada tahun 2000, dengan prinsip utama bahwa setiap sumber daya pada server diidentifikasi melalui URI (*Uniform Resource Identifier*) [26].



Gambar 2.2 Ilustrasi REST API

Seperti pada gambar 2.2, REST API memanfaatkan metode HTTP seperti GET, POST, PUT, dan DELETE untuk melakukan operasi pada sumber daya. Pendekatan ini memungkinkan pengembangan layanan web yang skalabel, sederhana, dan mudah diintegrasikan dengan berbagai platform. REST API banyak digunakan dalam pengembangan aplikasi modern karena sifatnya yang ringan dan kompatibel dengan format data seperti JSON dan XML [27], Beberapa karakteristik utama REST API meliputi:

1. Client-Server: Memisahkan antara klien dan server, memungkinkan evolusi independen dari kedua komponen.
2. Stateless: Setiap permintaan dari klien ke server harus berisi semua informasi yang

dibutuhkan untuk memahami dan memproses permintaan tersebut.

3. Cacheability: Respons dapat ditandai sebagai dapat di-cache atau tidak untuk mengoptimalkan efisiensi.
4. Layered System : Klien tidak perlu mengetahui detail implementasi server, memungkinkan fleksibilitas arsitektur.

REST API telah menjadi standar de facto untuk menghubungkan berbagai aplikasi, terutama dalam ekosistem aplikasi berbasis cloud dan microservices [28].

2.2.7 Software Development Life Cycle dengan Metode Kanban Board

Software Development Life Cycle (SDLC) adalah kerangka kerja sistematis untuk merencanakan, membangun, menguji, dan memelihara perangkat lunak. Dalam penelitian ini, alur kerja pengembangan dikelola menggunakan metode *Kanban Board*, yang merupakan bagian dari pendekatan *Agile*. Dengan demikian, proses pengembangan mengikuti prinsip *Agile*, terutama dalam hal fleksibilitas, visibilitas progres, dan perbaikan berkelanjutan.

Kanban Board menyediakan pendekatan visual dalam mengelola tugas melalui tiga kolom utama: *To Do*, *In Progress*, dan *Complete*. Setiap tugas direpresentasikan sebagai kartu yang berpindah antar kolom sesuai dengan statusnya. Pendekatan ini membantu membatasi pekerjaan yang berjalan sekaligus (*Work In Progress/WIP*), menjaga aliran kerja yang berkelanjutan, serta memberikan transparansi terhadap progres proyek [11].

Penerapan *Kanban Board* dalam SDLC dilakukan secara bertahap, dimulai dari penempatan aktivitas seperti analisis kebutuhan dan perancangan sistem pada kolom *To Do*, pengembangan fitur pada *In Progress*, hingga penyelesaian tugas yang dipindahkan ke kolom *Complete*. Skema ini mendukung iterasi yang adaptif serta memudahkan pemantauan dan pengendalian proyek *Agile* secara sederhana namun efektif [29].

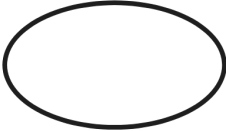
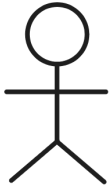
2.2.8 Unified Modeling Language (UML)

Unified Modeling Language (UML) merupakan bahasa pemodelan standar yang digunakan untuk menentukan, memvisualisasikan, membangun, dan mendokumentasikan artefak perangkat lunak serta sistem lainnya. UML memfasilitasi pemahaman antar tim dalam proses pengembangan dengan menyediakan serangkaian notasi yang konsisten dan formal. Komponen utama dalam UML mencakup diagram struktural seperti diagram kelas dan diagram komponen, serta diagram perilaku seperti diagram aktivitas dan diagram use case. UML dirancang untuk mendukung berbagai metodologi pengembangan perangkat lunak dan menjadi alat penting dalam rekayasa perangkat lunak modern [30].

2.2.9 Use Case Diagram

Use case diagram digunakan untuk memahami interaksi antara sistem dan aktor, yang merupakan entitas eksternal yang berinteraksi dengan sistem tanpa menjadi bagian dari sistem.

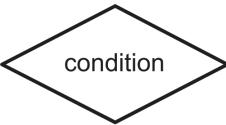
Diagram ini menggambarkan fungsi-fungsi utama sistem, yang disebut sebagai use case, serta hubungan antara aktor dan use case. Elemen-elemen dalam use case diagram meliputi aktor, use case, dan hubungan yang menghubungkan keduanya, membantu menyederhanakan representasi kebutuhan fungsional sistem [30].

Notasi	Nama	Keterangan
	<i>Use Case</i>	Deskripsi serangkaian tindakan yang dilakukan oleh suatu sistem yang menghasilkan efek terukur bagi seorang aktor.
	<i>Actor</i>	Mendefinisikan serangkaian peran yang dimainkan pengguna saat berinteraksi dengan <i>use case</i>
	<i>Association</i>	Garis menghubungkan satu objek dengan objek lainnya <i>use case</i>
	<i>Include</i>	Menjelaskan secara eksplisit bahwa <i>use case</i> adalah sumbernya <i>use case</i>
	<i>Extend</i>	Pada titik tertentu, <i>use case</i> target memperluas perilaku <i>use case</i> sumber. <i>use case</i>
	<i>System</i>	Mendefinisikan paket yang membatasi tampilan sistem dalam konteks tertentu.
	<i>Generalization</i>	Hubungan antara <i>use case</i> yang umum dan <i>use case</i> yang khusus, apabila salah satu mempunyai fungsi yang lebih umum dibandingkan dengan yang lain.

Tabel 2.2 Notasi *Use Case Diagram*

2.2.10 Activity Diagram

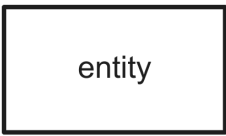
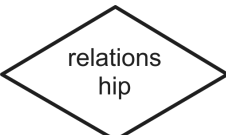
Activity diagram digunakan untuk menggambarkan alur kerja atau proses bisnis dalam suatu sistem. Diagram ini menampilkan aktivitas-aktivitas yang dilakukan dan urutan alurnya. Setiap aktivitas merepresentasikan tindakan tertentu yang dilakukan oleh aktor atau sistem. Elemen penting dalam activity diagram meliputi nodes (kegiatan atau tindakan), edges (aliran), decision points, dan forks. Diagram ini sangat bermanfaat untuk menganalisis alur proses dan mengidentifikasi area untuk optimasi [31].

Notasi	Nama	Keterangan
	<i>Initial Node</i>	Menandakan titik awal dari proses atau alur aktivitas.
	<i>Final Node</i>	Menandakan akhir dari proses atau alur aktivitas.
	<i>Activity</i>	Menggambarkan aktivitas tertentu dalam proses atau alur kerja.
	<i>Condition</i>	Menunjukkan keputusan atau kondisi yang menentukan percabangan alur aktivitas.

Tabel 2.3 Notasi *Activity Diagram*

2.2.11 *Entities Relationship Diagram (ERD)*

Entity Relationship Diagram (ERD) digunakan untuk memodelkan struktur data dalam suatu sistem. Diagram ini merepresentasikan entitas, atribut, dan hubungan antar entitas. Entitas biasanya digambarkan sebagai persegi panjang, atribut sebagai elips, dan hubungan sebagai belah ketupat. ERD membantu memvisualisasikan bagaimana data diatur dan dihubungkan, mempermudah perancangan database secara terstruktur [32].

Notasi	Nama	Keterangan
	<i>Association</i>	Menggambarkan hubungan antara dua entitas dalam model data.
	<i>Entity</i>	Merepresentasikan objek atau konsep yang memiliki data yang disimpan dalam sistem.
	<i>Attribute</i>	Menunjukkan informasi atau properti yang dimiliki oleh sebuah entitas atau hubungan.
	<i>Relation</i>	Menjelaskan hubungan antara dua atau lebih entitas, termasuk kardinalitasnya.

Tabel 2.4 Notasi *Entity Relationship Diagram*

2.2.12 Teknologi dan Kakas Pengembangan

Dalam pengembangan proyek akhir ini, penulis menggunakan beberapa teknologi dan kakas pengembangan yang mendukung proses pengembangan. Beberapa teknologi dan kakas pengembangan yang digunakan antara lain:

- **VueJS:** VueJS adalah pustaka JavaScript yang digunakan untuk membangun antarmuka pengguna yang interaktif dan responsif. VueJS menggunakan konsep komponen yang dapat digunakan kembali untuk memisahkan tampilan dari logika aplikasi, memungkinkan pengembangan yang lebih efisien dan mudah diatur [33].
- **Python:** Python adalah bahasa pemrograman tingkat tinggi yang digunakan untuk pengembangan aplikasi web, analisis data, dan kecerdasan buatan. Python memiliki sintaks yang sederhana dan mudah dipahami, serta mendukung berbagai pustaka dan kerangka kerja yang mempermudah pengembangan perangkat lunak [34].
- **Flask:** Flask adalah kerangka kerja web berbasis Python yang digunakan untuk membangun aplikasi web dan API. Flask dirancang untuk menjadi ringan dan fleksibel, memungkinkan pengembang untuk memilih komponen yang sesuai dengan kebutuhan proyek mereka. Flask juga mendukung berbagai ekstensi yang mempermudah pengembangan aplikasi web [35].
- **PostgreSQL:** PostgreSQL adalah sistem manajemen basis data relasional (RDBMS) open-source yang digunakan untuk menyimpan dan mengelola data. PostgreSQL mendukung berbagai fitur seperti transaksi, indeks, dan fungsi yang memungkinkan pengembang untuk membangun aplikasi yang aman dan efisien [36].
- **Git:** Git adalah sistem kontrol versi yang digunakan untuk mengelola perubahan kode sumber dalam pengembangan perangkat lunak. Git memungkinkan pengembang untuk bekerja secara kolaboratif, melacak perubahan kode, dan mengelola versi kode dengan mudah [37].
- **GitLab:** GitLab adalah platform pengembangan perangkat lunak berbasis Git yang menyediakan repositori kode, manajemen proyek, dan alat kolaborasi untuk tim pengembang. GitLab memungkinkan pengembang untuk menyimpan kode sumber, melacak masalah, dan berkolaborasi dalam proyek secara efisien [38].
- **Visual Studio Code:** Visual Studio Code adalah editor kode sumber ringan dan kuat yang dikembangkan oleh Microsoft. Visual Studio Code menyediakan berbagai fitur seperti penyorotan sintaks, debugging, dan ekstensi yang mempermudah pengembangan perangkat lunak [39].
- **Google Cloud Run:** Google Cloud Run adalah layanan komputasi tanpa server yang memungkinkan pengembang untuk menjalankan aplikasi kontainer di infrastruktur

Google Cloud. Cloud Run secara otomatis mengelola skala dan sumber daya yang dibutuhkan untuk menjalankan aplikasi, memungkinkan pengembang untuk fokus pada pengembangan tanpa perlu khawatir tentang manajemen infrastruktur [40].

- **Hoppscotch:** hoppscotch adalah aplikasi web open-source yang digunakan untuk menguji dan mengelola API. hoppscotch menyediakan antarmuka yang intuitif dan mudah digunakan untuk membuat dan mengirim permintaan HTTP ke API, serta melihat respons yang dihasilkan [41].
- **JSON Web Token (JWT):** JSON Web Token (JWT) adalah standar terbuka yang digunakan untuk membuat token akses yang aman dan dapat diverifikasi. JWT digunakan dalam aplikasi web modern untuk otentikasi dan otorisasi pengguna, serta pertukaran informasi yang aman antara klien dan server [42].
- **SQLAlchemy:** SQLAlchemy adalah ORM (Object-Relational Mapping) yang digunakan untuk menghubungkan aplikasi Backend python yang menggunakan Flask dengan basis data. SQLAlchemy menyediakan API yang intuitif dan deklaratif untuk berinteraksi dengan basis data, memungkinkan pengembang untuk fokus pada logika aplikasi tanpa perlu menulis query SQL secara manual [43].
- **Google Cloud Storage:** Google Cloud Storage adalah server penyimpanan objek (object storage) yang merupakan salah satu lini produk dari google cloud [44].
- **Docker:** Docker adalah platform open-source yang digunakan untuk mengemas, mengirim, dan menjalankan aplikasi dalam kontainer. Docker memungkinkan pengembang untuk membuat lingkungan yang konsisten dan portabel untuk aplikasi, serta mempercepat siklus pengembangan dan penyebaran aplikasi [45].
- **Latex:** Latex adalah sistem penyiapan dokumen yang digunakan untuk membuat dokumen yang berkualitas tinggi, seperti artikel ilmiah, laporan, dan buku. Latex menggunakan sintaks markup untuk mengatur struktur dokumen, memungkinkan pengembang untuk fokus pada konten tanpa perlu khawatir tentang tata letak [46].

2.2.13 Pengujian Sistem

Setelah pengembangan sistem selesai, langkah selanjutnya adalah melakukan pengujian untuk memastikan bahwa sistem berfungsi dengan baik dan sesuai dengan kebutuhan pengguna. Pengujian sistem adalah proses verifikasi dan validasi yang dilakukan untuk mengevaluasi kualitas sistem dan memastikan bahwa sistem memenuhi spesifikasi yang telah ditentukan. Beberapa jenis pengujian sistem yang umum dilakukan antara lain:

1. **Pengujian Black Box:** Pengujian ini berfokus pada validasi fungsionalitas sistem tanpa memperhatikan struktur internal atau kode sumbernya. Tujuan utama dari pengujian ini adalah untuk memastikan bahwa masukan (input) menghasilkan keluaran (output) yang

sesuai dengan spesifikasi [47].

2. **User Acceptance Testing (UAT)** : UAT adalah tahap akhir pengujian yang melibatkan pengguna akhir untuk memastikan bahwa sistem memenuhi kebutuhan bisnis dan dapat digunakan sebagaimana mestinya. Pada UAT, setiap fungsi sistem diuji berdasarkan skenario yang dirancang sesuai dengan kebutuhan pengguna [48]. Tingkat keberhasilan UAT dapat dihitung dengan rumus berikut:

Jawaban	Bobot
Sangat Setuju(SS)	5
Setuju(S)	4
Cukup Setuju(CS)	3
Tidak Setuju(TS)	2
Sangat Tidak Setuju(STS)	1

Tabel 2.5 Bobot Jawaban UAT berdasarkan skala *Likert*

Penentuan skor UAT dihitung menggunakan rumus pada Persamaan 2.1 dan hasil dari Skor tersebut akan dibandingkan dengan pengelompokkan pada tabel 2.5 berdasarkan skala *Likert*.

$$Hasil(\%) = \left(\frac{A}{B \times N} \right) \times 100\% \quad (2.1)$$

Keterangan:

A = total skor yang diperoleh dari semua responden

B = maksimum skor yang dapat diperoleh dari setiap responden

N = total responden

Persentase akhir UAT kemudian dapat dijadikan sebagai indikator kelayakan sesuai kategori yang ditunjukkan pada Tabel 2.6.

No.	Skor	Kategori Kelayakan
1.	< 21%	Sangat Tidak Layak
2.	21-40%	Tidak Layak
3.	41-60%	Cukup Layak
4.	61-80%	Layak
5.	81-100%	Sangat Layak

Tabel 2.6 Kategori Kelayakan UAT

BAB III

METODE PROYEK AKHIR

Metodologi yang diterapkan pada penelitian menjadi pokok bahasan pada bab ini. Alur perencanaan penelitian, pengumpulan dan analisis data, serta hasil penelitian akan dijelaskan secara terperinci. Sebagai tambahan, penjelasan dalam bab ini juga meliputi komponen-komponen lain seperti pendekatan penelitian, teknik pengumpulan data, jenis data yang terkumpul, metode analisis, dan instrumen pendukung yang digunakan.

3.1 Bahan

Bahan berupa data yang digunakan penulis untuk mengerjakan penelitian ini diambil langsung dari sistem hotel kapsul Tamu.id cabang jalan Magelang Yogyakarta yang sudah ada dengan observasi langsung ke lokasi. Data yang diambil berupa data detail cabang, gambar, fasilitas, jenis pod, pod, staff, harga, dan lain-lain yang diambil pada periode bulan Maret 2025 hingga April 2025.

3.2 Peralatan dan Kakas

Peralatan dan kakas yang digunakan dalam pengembangan aplikasi Tamu.id dan proyek akhir ini adalah sebagai berikut:

3.2.1 Perangkat Lunak

1. Visual Studio Code: Editor kode sumber yang digunakan untuk menulis kode program.
2. Flask: Framework Python yang digunakan untuk membangun backend aplikasi.
3. VueJS: Library JavaScript yang digunakan untuk membangun frontend aplikasi.
4. Tailwind CSS: Framework CSS yang digunakan untuk mendesain antarmuka pengguna.
5. PrimeVue: Komponen UI yang digunakan untuk mempercepat pengembangan antarmuka pengguna terutama bagian Super Admin dan Admin.
6. Axios: Library JavaScript yang digunakan untuk melakukan permintaan HTTP dari frontend ke backend.
7. JSON Web Token (JWT): Standar terbuka yang digunakan untuk mengamankan komunikasi antara frontend dan backend.
8. Docker: Platform yang digunakan untuk mengemas aplikasi dan dependensinya ke dalam kontainer, sehingga memudahkan pengembangan, pengujian, dan penyebaran.
9. GitLab: Platform pengembangan perangkat lunak yang digunakan untuk mengelola kode sumber.

10. PostgreSQL: Sistem manajemen basis data relasional yang digunakan untuk menyimpan data aplikasi.
11. Google Cloud Run: serverless platform tempat deploy aplikasi backend.
12. Google Cloud Storage: Layanan penyimpanan objek yang digunakan untuk menyimpan file statis.
13. Google Cloud SQL: Layanan basis data terkelola yang digunakan untuk menyimpan data aplikasi di lingkungan Produksi.
14. Cloudflare Pages: Layanan hosting statis yang digunakan untuk menyajikan aplikasi frontend.
15. Midtrans Core API: Layanan pembayaran yang digunakan untuk memproses transaksi pembayaran.
16. Ngrok: Alat yang digunakan untuk membuat terowongan ke localhost, memungkinkan akses ke aplikasi yang sedang dikembangkan di mesin lokal.
17. Hoppscotch: Alat pengujian API yang digunakan untuk menguji endpoint API.
18. PostgreSQL: Sistem manajemen basis data relasional yang digunakan untuk menyimpan data aplikasi.
19. Latex: Bahasa markup yang digunakan untuk menulis dokumen proyek akhir.
20. Draw.io (VSCode Extension): Ekstensi Visual Studio Code yang digunakan untuk membuat diagram alur.
21. PlantUML: Alat yang digunakan untuk membuat diagram UML, seperti diagram kelas, diagram urutan, dan diagram aktivitas.
22. ClickUp: Alat manajemen proyek yang digunakan untuk mengelola tugas-tugas dalam proyek akhir ini.

3.2.2 Perangkat Keras

Berikut spesifikasi perangkat keras yang digunakan dalam pengembangan aplikasi AtleTiket dan proyek akhir ini:

1. Laptop Thinkpad T480s
 - Prosesor: Intel Core i5-8350U
 - RAM: 16 GB DDR4 2400 MHz
 - Penyimpanan: 256 GB SSD NVMe M.2

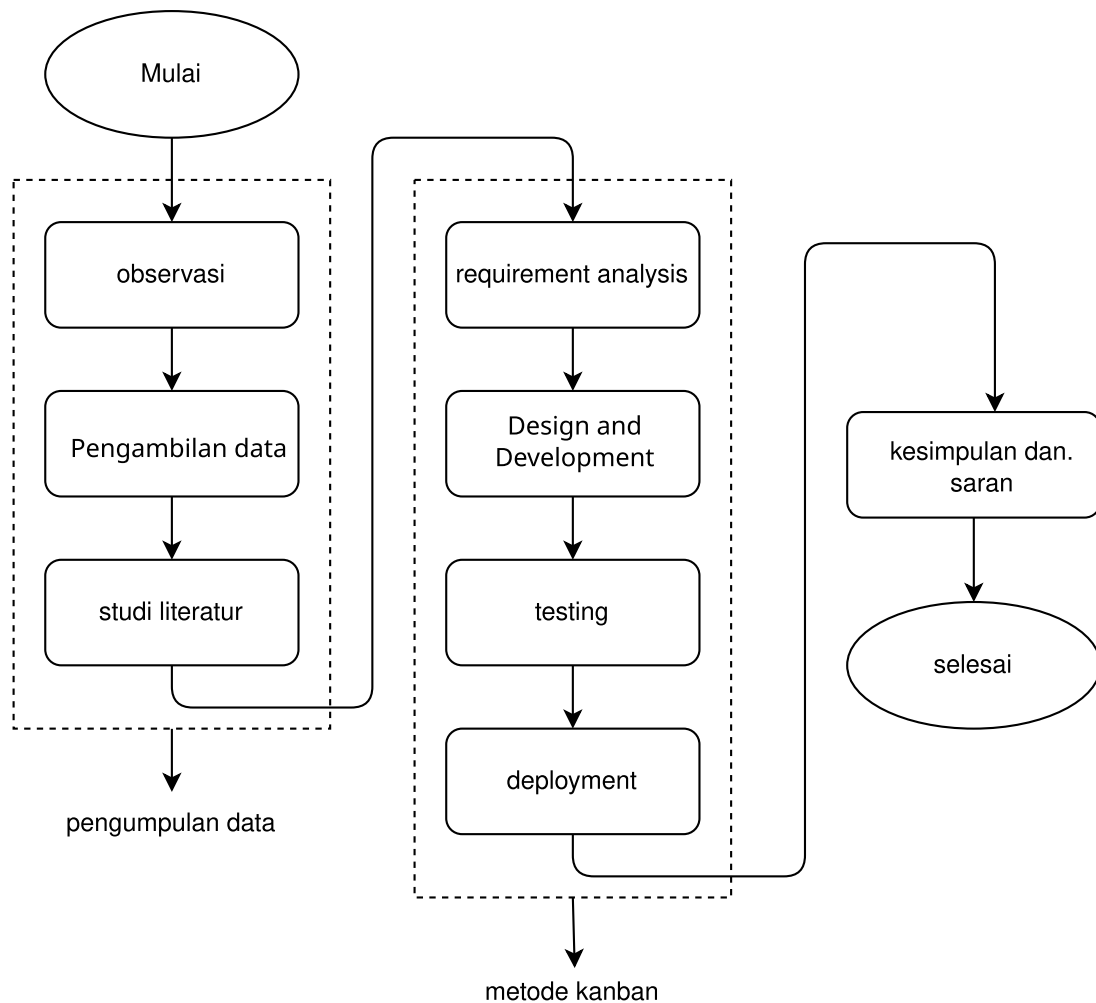
- Kartu Grafis: Intel UHD Graphics
- Sistem Operasi: Ubuntu 24.02 LTS

2. Smartphone Infinix Hot 9

- Prosesor: MediaTek Helio A25
- RAM: 4 GB
- Penyimpanan: 128 GB
- Sistem Operasi: Android 10
- Browser: Google Chrome
- Resolusi Layar: 720 x 1600 piksel
- Ukuran Layar: 6.6 inci

3.3 Tahapan Proyek Akhir

Tahapan dalam proyek akhir ini dibagi menjadi beberapa tahapan yang harus dilalui oleh penulis yang terdiri dari 3 tahapan utama, yaitu tahap pengumpulan data, tahap pengembangan aplikasi dengan metode kanban, dan di akhir yaitu kesimpulan dan saran. Berikut adalah tahapan-tahapan tersebut pada gambar 3.1:



Gambar 3.1 Diagram Tahapan Proyek Akhir

Berikut uraian lebih lengkap mengenai tahapan pada gambar 3.1:

1. Tahap Pengumpulan Data

Tahap ini merupakan tahap awal yang dilakukan oleh penulis untuk mengumpulkan data-data yang dibutuhkan dalam proyek akhir ini, yaitu dengan survei langsung ke hotel kapsul Tamu.id yang sudah ada. Penulis melakukan survei untuk mendapatkan data-data yang dibutuhkan dalam proyek akhir ini, seperti data hotel, fasilitas, jenis pod, dan pod. Penulis juga melakukan wawancara dengan pihak hotel kapsul Tamu.id untuk mendapatkan informasi lebih lanjut mengenai sistem yang sudah ada.

2. Tahap Pengembangan Aplikasi dengan Metode Kanban

Tahap ini merupakan tahap pengembangan aplikasi Tamu.id yang dilakukan oleh penulis dengan menggunakan metode Kanban. Penulis melakukan pengembangan aplikasi dengan menggunakan metode Kanban untuk mempermudah dalam mengelola proyek akhir ini. Penulis juga menggunakan alat bantu seperti ClickUp untuk mengelola proyek akhir ini. Dalam tahap ini, penulis melakukan pengembangan aplikasi dengan

menggunakan bahasa pemrograman Python dan JavaScript. Penulis juga menggunakan framework Flask untuk membangun backend aplikasi dan VueJS untuk membangun frontend aplikasi. Penulis juga menggunakan Tailwind CSS untuk mendesain antarmuka pengguna. Penulis juga menggunakan PostgreSQL sebagai sistem manajemen basis data relasional yang digunakan untuk menyimpan data aplikasi.

3. Tahap Kesimpulan dan Saran

Tahap ini merupakan tahap akhir yang dilakukan oleh penulis untuk menyimpulkan hasil dari proyek akhir ini. Penulis juga akan memberikan saran-saran untuk pengembangan aplikasi Tamu.id ke depannya.

3.4 Analisis Kebutuhan Sistem

Pada tahap ini penulis melakukan proses analisis kebutuhan sesuai dengan hasil wawancara dengan *stakeholder*. Tujuan utama dari analisis kebutuhan ini adalah untuk menentukan kebutuhan sistem yang akan dibangun, baik kebutuhan fungsional maupun non-fungsional. Kebutuhan fungsional adalah kebutuhan yang berkaitan dengan fungsi-fungsi yang harus ada dalam sistem, sedangkan kebutuhan non-fungsional adalah kebutuhan yang berkaitan dengan kualitas sistem seperti keamanan, performa, dan lain-lain.

3.4.1 Analisis Pengguna Sistem

Pada tahap ini penulis melakukan analisis terkait pengguna sistem dan *role* yang ada dalam sistem. Pengguna sistem Tamu.id terdiri dari beberapa *role* sebagai berikut:

1. Admin Pusat (SuperAdmin) Superadmin adalah pengguna yang memiliki hak akses penuh terhadap sistem, termasuk mengelola data cabang, admin untuk cabang, fasilitas, kota, voucher multi cabang, verifikasi identitas pengguna, verifikasi pencairan referral pengguna, dan lain-lain.
2. Admin Cabang (Admin) Admin cabang adalah pengguna yang memiliki hak akses untuk mengelola data pada cabang tertentu, seperti mengelola data fasilitas, voucher cabang, jenis pod cabang, pod cabang, dan melakukan aktifitas operasional pada cabang tersebut.
3. Pengunjung (User) Pengunjung adalah pengguna yang dapat mendaftar, membuat akun, dan melakukan pemesanan tiket untuk mengunjungi cabang tertentu. Pengunjung juga dapat melakukan identitas untuk mengakses lebih banyak fitur seperti mendapatkan penghasilan dari referral, mengelola data diri, dan melakukan pencairan referral.

3.4.2 Analisis Kebutuhan Fungsional

Analisis kebutuhan fungsional dilakukan untuk menentukan fungsi-fungsi yang harus ada dalam sistem. Kebutuhan fungsional yang diperlukan dalam pengembangan aplikasi Tamu.id adalah sebagai berikut:

1. Admin Pusat (SuperAdmin)

- (a) SuperAdmin dapat melihat grafik dan statistik global dari seluruh cabang sebagai garis besar informasi keseluruhan sistem Tamu.id untuk membantu pengambilan keputusan.
- (b) Superadmin dapat melakukan *CRUD (Create, Read, Update, Delete)* pada data master Cabang, Kota, *Online Travel Agent (OTA)*, Staff, fasilitas, dan voucher multi cabang.
- (c) Superadmin dapat memberikan ataupun mengedit hak akses pada admin cabang untuk mengelola satu atau beberapa cabang.
- (d) Superadmin dapat melakukan verifikasi identitas pengguna baik itu menerima ataupun menolak verifikasi identitas pengguna dengan alasan yang jelas serta melihat catatan verifikasi identitas yang telah dilakukan.
- (e) SuperAdmin dapat menolak ataupun menerima permintaan pencairan referral pengguna dengan serta melihat catatan pencairan referral yang telah dilakukan.
- (f) SuperAdmin dapat mengubah data dirinya sendiri seperti nama, email, dan password.

2. Admin Cabang (Admin)

- (a) Admin dapat melihat grafik dan statistik cabang untuk membantu pengambilan keputusan.
- (b) Admin dapat mengelola data cabangnya sendiri seperti mengubah gambar, nama, alamat, dan informasi lainnya.
- (c) Admin dapat mengelola data admin untuk cabang tersebut, termasuk menambah, mengubah, dan menghapus admin cabang.
- (d) Admin dapat mengaktifkan atau menonaktifkan *Online Travel Agent (OTA)* yang bekerja sama dengan cabang tersebut.
- (e) Admin dapat melakukan *CRUD (Create, Read, Update, Delete)* pada data master jenis pod cabang, pod cabang, voucher cabang.
- (f) Admin dapat melihat dan mengelola review dari pengguna yang telah mengunjungi cabang tersebut.
- (g) Admin dapat melakukan *CRUD (Create, Read, Update, Delete)* pada pengaturan harga dinamis untuk jenis pod pada cabang tersebut dengan rentang waktu tertentu.
- (h) Admin dapat menginputkan data Transaksi Reservasi via *Online Travel Agent (OTA)* yang bekerja sama dengan cabang tersebut untuk kemudian nantinya diclaim oleh

pengguna yang bersangkutan.

- (i) Admin dapat memindahkan pengunjung(*User*) yang sedang checkin pada pod ke pod lain yang tersedia jika terjadi masalah pada pod yang sedang digunakan serta melihat catatan pemindahan pod yang telah dilakukan.
- (j) Admin dapat melakukan pembersihan pada pod yang telah digunakan oleh pengunjung untuk memastikan kebersihan dan kenyamanan pod serta melihat catatan pembersihan pod yang telah dilakukan.
- (k) Admin dapat melakukan *monitoring dan controlling* pada pod cabangnya, termasuk melihat status pod dan mengubah status pod jika diperlukan.
- (l) Admin dapat mengubah data dirinya sendiri seperti nama, email, dan password.

3. Pengunjung (*User*)

- (a) Pengunjung dapat mendaftar dan membuat akun untuk mengakses fitur-fitur yang ada di aplikasi Tamu.id via Email atau Google OAuth2.
- (b) Pengunjung dapat melakukan verifikasi identitas untuk mendapatkan akses lebih banyak fitur seperti referral, pencairan referral, dan lain-lain.
- (c) Pengunjung dapat melakukan pencairan referral yang telah didapatkan dari pengguna lain dengan minimal saldo referral yang telah ditentukan.
- (d) Pengunjung dapat melakukan pencarian dengan memilih cabang dan menetapkan tanggal checkin dan checkout untuk melihat ketersediaan pod pada cabang tersebut.
- (e) Pengunjung dapat melihat detail cabang, jenis pod, dan fasilitas yang tersedia pada cabang tersebut.
- (f) Pengunjung dapat melakukan pemesanan tiket dengan memilih jenis pod, tanggal checkin, dan checkout, serta melakukan pembayaran melalui QRIS.
- (g) Pengunjung dapat memasukkan kode voucher yang valid dan juga kode referral yang valid untuk mendapatkan potongan harga pada pemesanan tiket.
- (h) Pengunjung yang telah melakukan reservasi via *Online Travel Agent (OTA)* dapat mengsinkronisasi data reservasi tersebut dengan memasukkan kode reservasi yang diberikan oleh OTA.
- (i) Pengunjung dapat melakukan checkin pada tanggal dan waktu yang telah ditentukan dan memilih pod yang tersedia untuk digunakan sesuai dengan jenis pod yang telah dipesan.

- (j) Pengunjung yang telah melakukan checkin dapat mengakses pod dengan menggunakan QR Code yang dihasilkan oleh sistem.
- (k) Pengunjung dapat melakukan checkout pada pod yang telah digunakan dan sistem akan otomatis mengubah status pod menjadi dalam pembersihan.
- (l) Pengunjung dapat melihat riwayat transaksi dan detail transaksi yang telah dilakukan,
- (m) Pengunjung dapat memberikan ulasan dan rating terhadap cabang yang telah dikunjungi.
- (n) Pengunjung dapat mengelola data diri seperti nama, username, email, nomor telepon, dan password.
- (o) Pengunjung dapat melihat dan mengelola saldo referral yang dimiliki, termasuk melakukan permintaan pencairan saldo referral.
- (p) Pengunjung dapat melihat promo yang sedang berlaku.

3.4.3 Analisis Kebutuhan Non-Fungsional

Pada bagian ini analisis kebutuhan Non Fungsional dilakukan untuk menentukan kebutuhan sistem yang berkaitan dengan kualitas sistem seperti keamanan, performa, dan lain-lain. Kebutuhan non-fungsional yang diperlukan dalam pengembangan aplikasi Tamu.id adalah sebagai berikut:

1. Walaupun pod dalam keadaan offline sementara ataupun dalam interval belum mengambil data kunci valid terbaru, pengguna yang baru checkin harus dapat langsung mengakses pod dengan QR Code yang telah dihasilkan sebelumnya.
2. Sistem dapat mengamankan pesanan pengguna yang masih dalam proses pembayaran sehingga tidak terjadinya *double booking* pada pod yang sama serta dapat otomatis menghapus pesanan yang tidak dibayar dalam waktu satu jam untuk menghindari pemborosan kapasitas pod serta sistem harus dapat otomatis mencheckout pengguna yang tidak melakukan checkout sesuai waktu yang telah ditentukan.
3. Sistem dapat berjalan pada peramban (*browser*) modern seperti Google Chrome, Mozilla Firefox, Microsoft Edge, dan Safari serta memiliki tampilan yang responsif pada perangkat seluler dan desktop.

3.4.4 Pengembangan Sistem

Pengembangan sistem Tamu.id dilakukan dengan menggunakan filosofi *Agile* dengan metode *Kanban*. Pendekatan ini dipilih karena memungkinkan penulis untuk melakukan pengembangan sistem secara iteratif dan inkremental, sehingga dapat dengan cepat

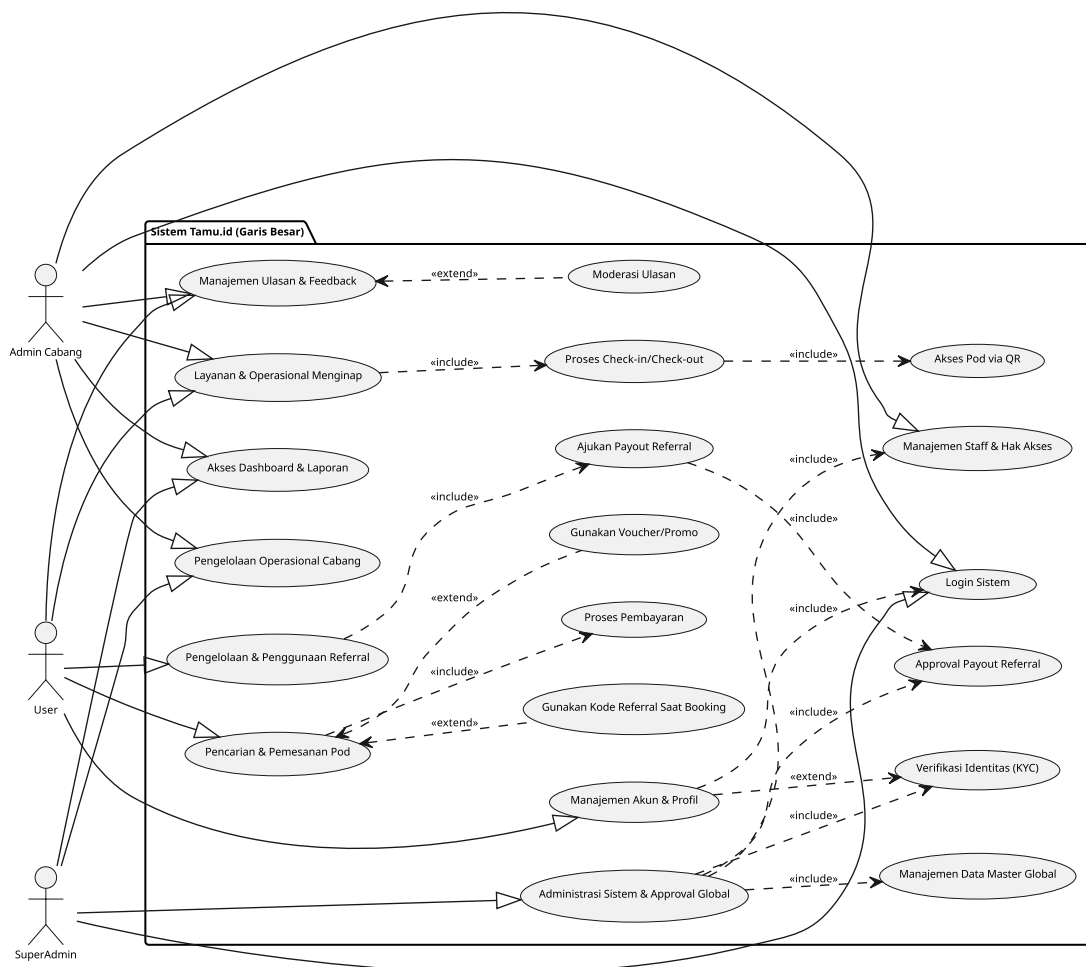
menyesuaikan diri dengan perubahan kebutuhan yang mungkin terjadi selama proses pengembangan. Pengembangan sistem dilakukan dalam beberapa tahap, yaitu perencanaan, pengembangan, pengujian, dan peluncuran.

3.5 Rancangan Sistem

Pada tahap ini penulis melakukan proses perancangan sistem berdasarkan kebutuhan fungsional dan non-fungsional yang telah dianalisis sebelumnya. Rancangan sistem ini mencakup arsitektur sistem, desain basis data, *Unified Modeling Language (UML)* diagram yang mencakup beberapa diagram, desain antarmuka pengguna, rancangan alur kerja, dan lain-lain yang diperlukan untuk membangun sistem Tamu.id.

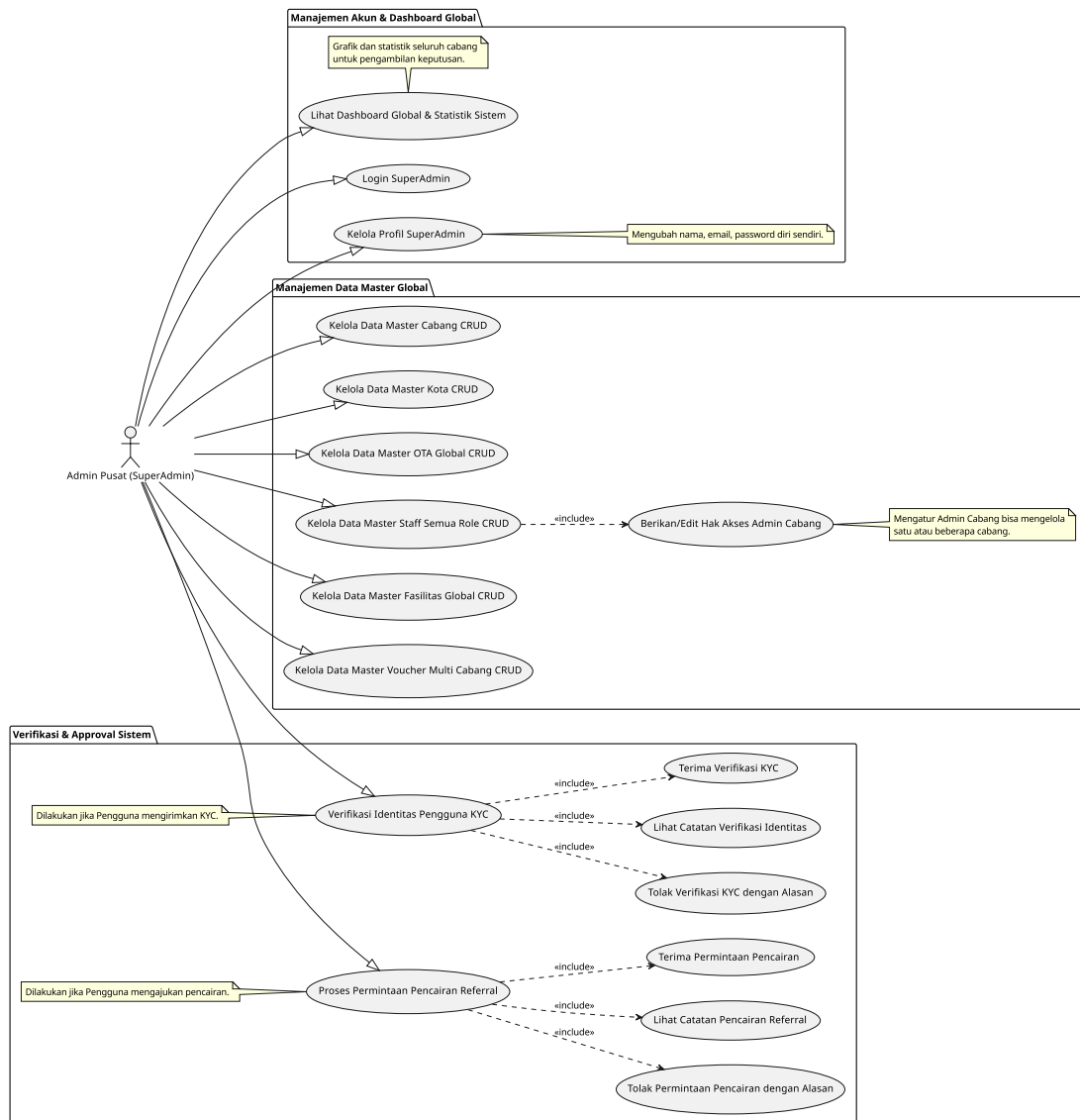
3.5.1 Use Case Diagram

Pada tahap ini *Use Case Diagram* dibuat untuk menggambarkan interaksi antara pengguna sistem dan sistem itu sendiri. Use Case Diagram ini mencakup aktor-aktor yang terlibat, seperti SuperAdmin, Admin Cabang, dan Pengunjung, serta use case yang menggambarkan fungsi-fungsi yang dapat dilakukan oleh masing-masing aktor. *Use Case Diagram* ini akan membantu dalam memahami kebutuhan sistem dan bagaimana pengguna akan berinteraksi dengan sistem sesuai pada gambar 3.2 hingga 3.6.



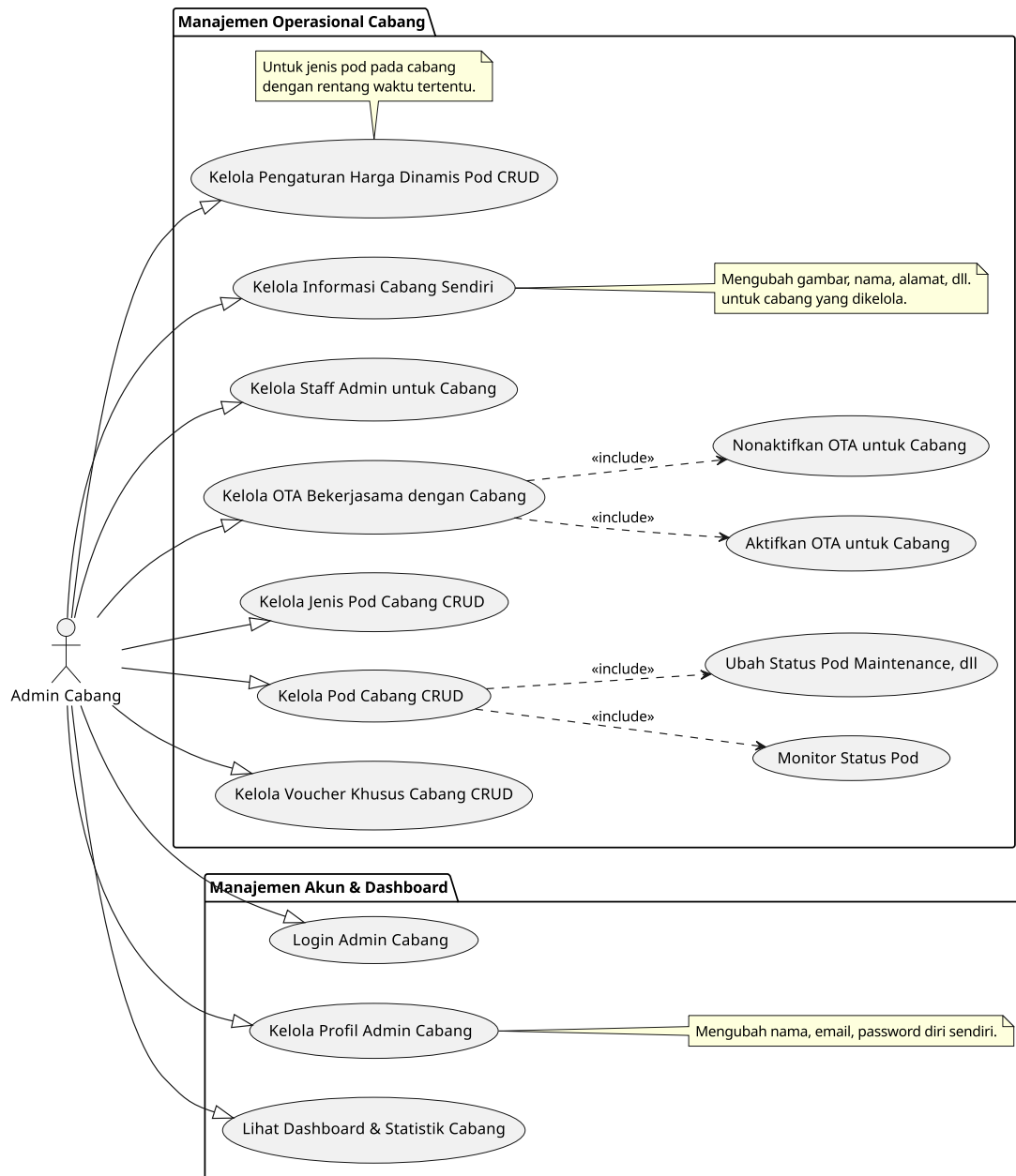
Gambar 3.2 Use Case Diagram Integrasi

Pada gambar 3.2 di atas merupakan gambaran kasar namun keseluruhan dari sistem yang akan dibangun. Terdapat beberapa aktor yang berinteraksi dengan sistem, yaitu SuperAdmin, Admin Cabang, dan User. SuperAdmin bertanggung jawab untuk mengelola seluruh sistem, termasuk pengelolaan Admin Cabang, sedangkan Admin Cabang memiliki tanggung jawab terbatas pada cabang mereka masing-masing. Pengguna atau User dapat melakukan reservasi kamar, memberikan ulasan, dan berpartisipasi dalam program loyalitas.



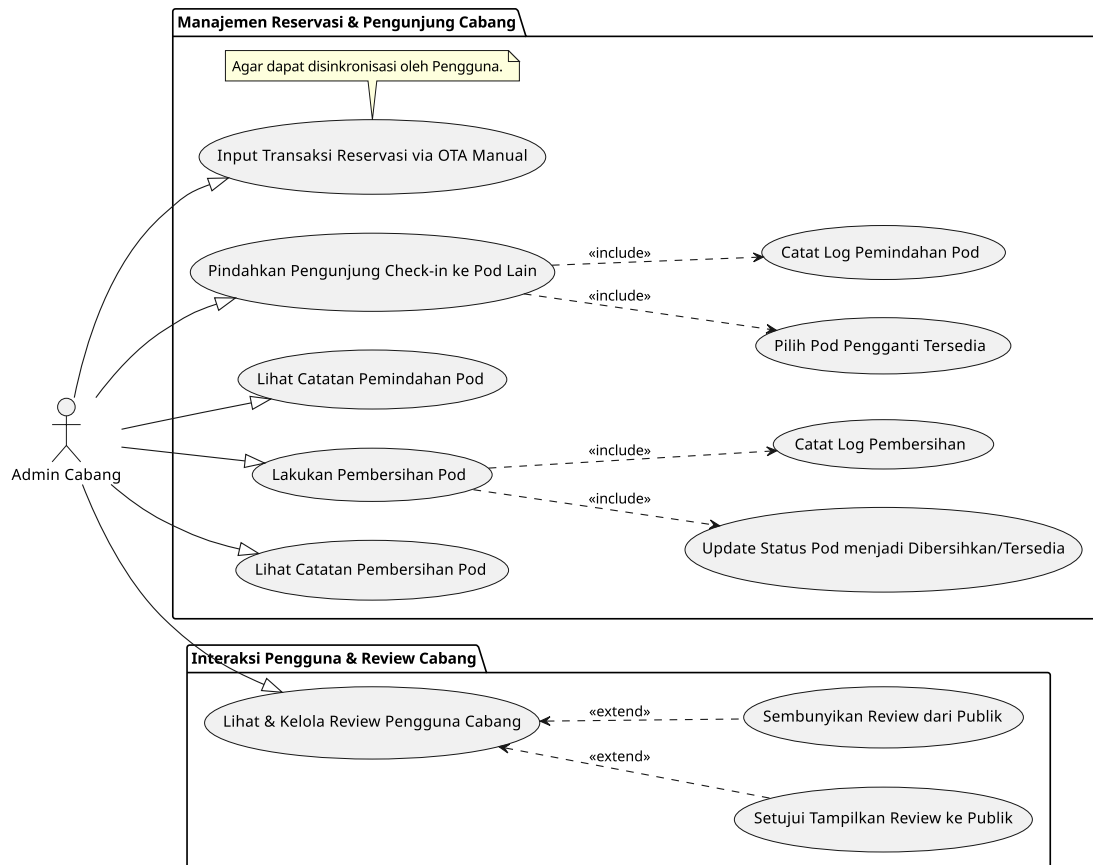
Gambar 3.3 Use Case Diagram SuperAdmin (Admin Pusat)

Pada gambar 3.3, ditampilkan use case diagram untuk SuperAdmin (Admin Pusat). SuperAdmin memiliki akses penuh untuk mengelola seluruh sistem, termasuk pengelolaan Admin Cabang, pengaturan data - dat master, dasbor global, serta verisikasi dan sistem persetujuan.



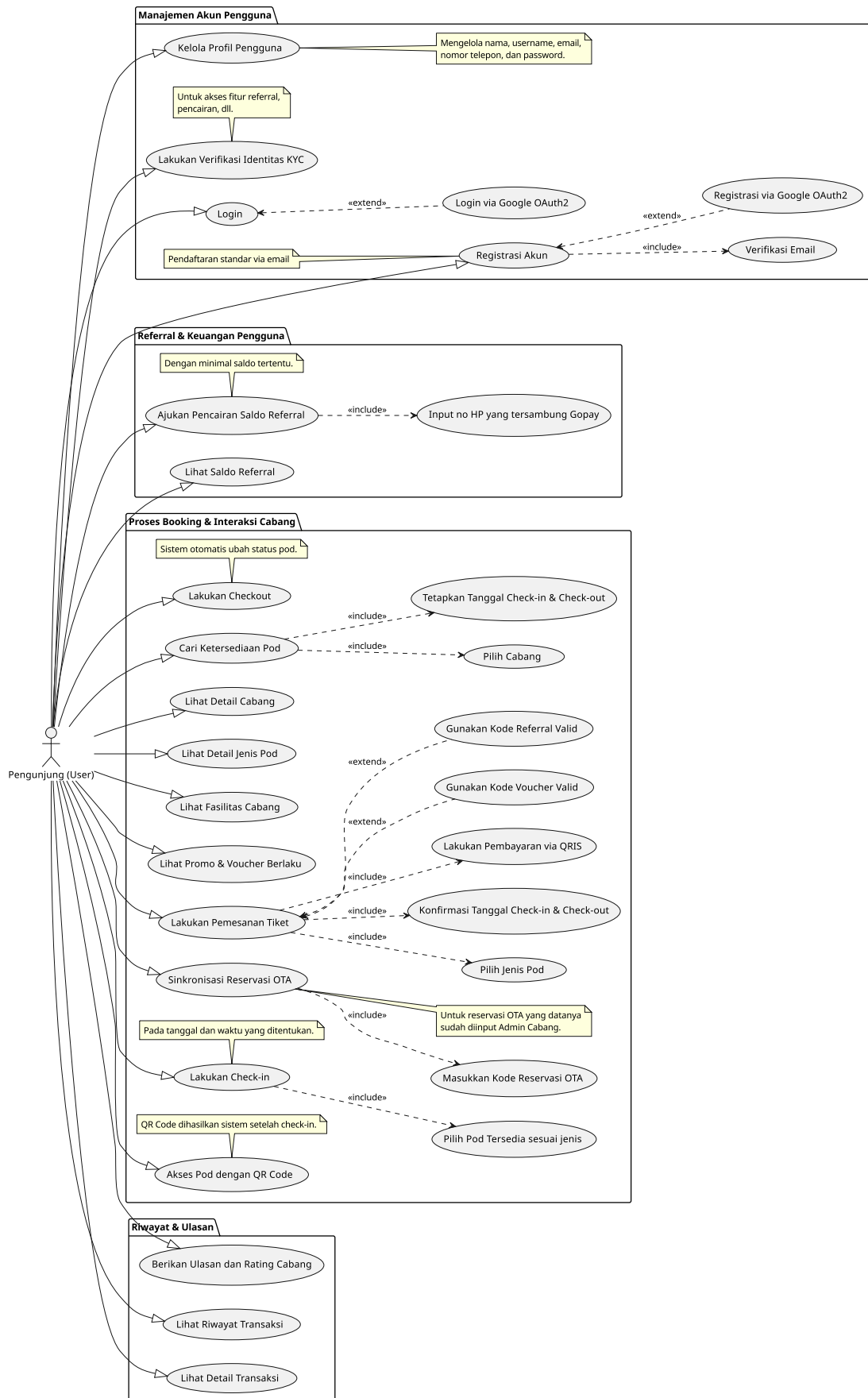
Gambar 3.4 Use Case Diagram Admin (Admin Cabang) 1

Pada gambar 3.4, ditampilkan use case diagram untuk Admin Cabang. Admin Cabang memiliki tanggung jawab untuk mengelola data cabang mereka, termasuk pengelolaan kamar, fasilitas, dan layanan yang tersedia di cabang tersebut. Admin Cabang juga dapat mengelola reservasi yang dilakukan oleh pengguna di cabang mereka.



Gambar 3.5 Use Case Diagram Admin (Admin Cabang) 2

Pada gambar 3.5, ditampilkan use case diagram tambahan untuk Admin Cabang. Diagram ini mencakup fungsi-fungsi tambahan yang mungkin tidak tercakup dalam diagram sebelumnya, seperti manajemen operasional harian, input data via *Online Travel Agent (OTA)*, dan sistem manajemen *Review* dan *Rating*.



Gambar 3.6 Use Case Diagram User (Pengunjung)

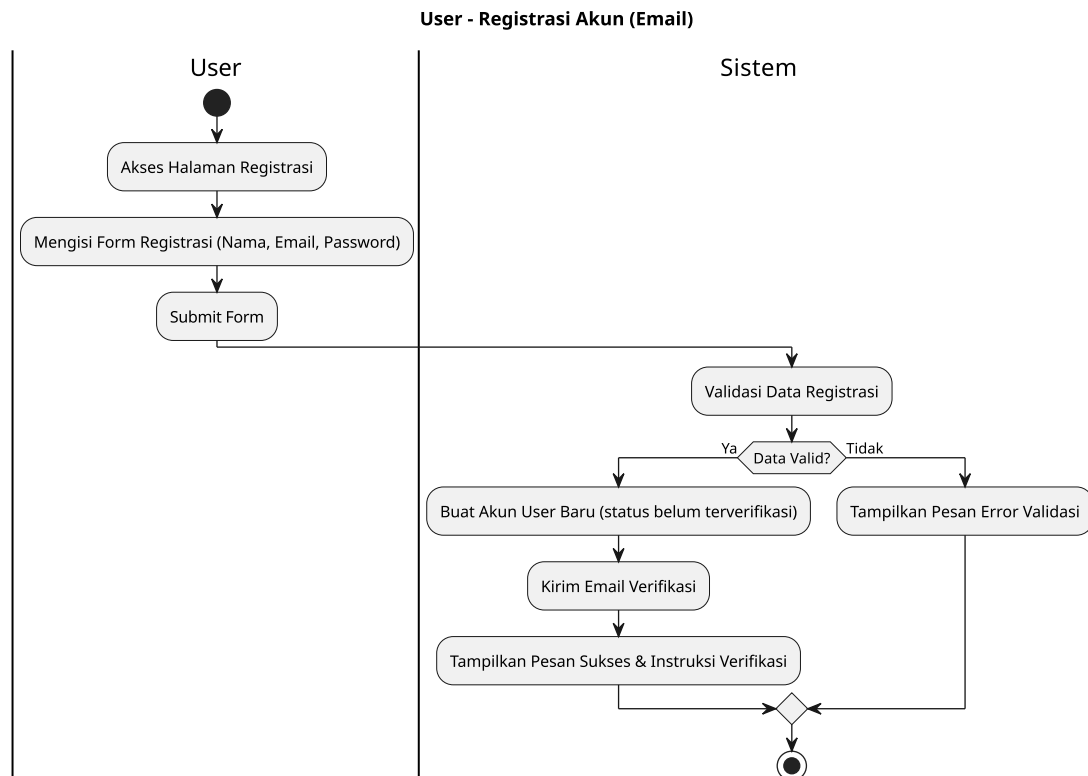
Pada gambar 3.6, ditampilkan use case diagram untuk User (Pengunjung). Pengguna dapat melakukan reservasi kamar, memberikan ulasan, dan berpartisipasi dalam program loyalitas. Diagram ini mencakup fungsi-fungsi yang dapat diakses oleh pengguna, seperti pencarian kamar, pemesanan, dan manajemen akun pengguna.

3.5.2 Activity Diagram

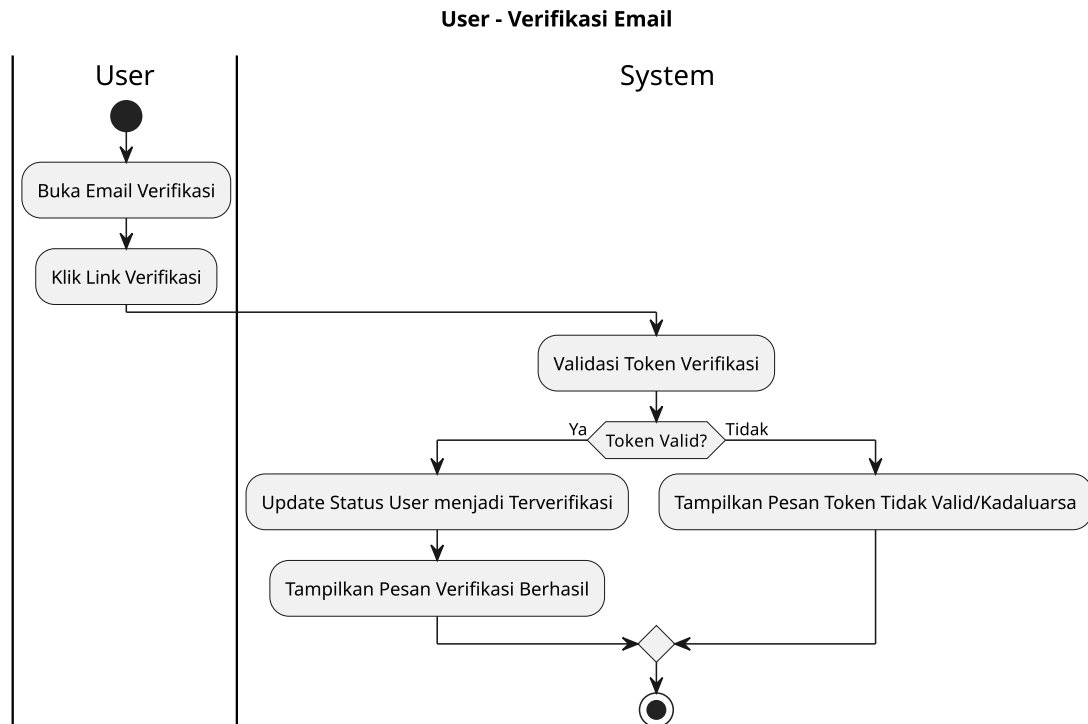
Pada tahap ini, *Activity Diagram* dibuat untuk menggambarkan alur kerja dari berbagai aktivitas yang dilakukan oleh pengguna dalam sistem. Berikut adalah beberapa diagram aktivitas yang telah dibuat beserta gambar dan penjelasannya untuk tiap proses yang ada dalam sistem.

A. Diagram Aktivitas Pengguna

Pada gambar 3.7 hingga 3.14, ditampilkan diagram aktivitas yang menggambarkan berbagai proses yang dilakukan oleh pengguna dalam sistem. Diagram ini mencakup proses registrasi, verifikasi email, reservasi kamar, check-in, stay, checkout, verifikasi identitas (KYC), dan payout referral. Setiap diagram memberikan gambaran yang jelas tentang langkah-langkah yang harus diikuti oleh pengguna dalam setiap proses.

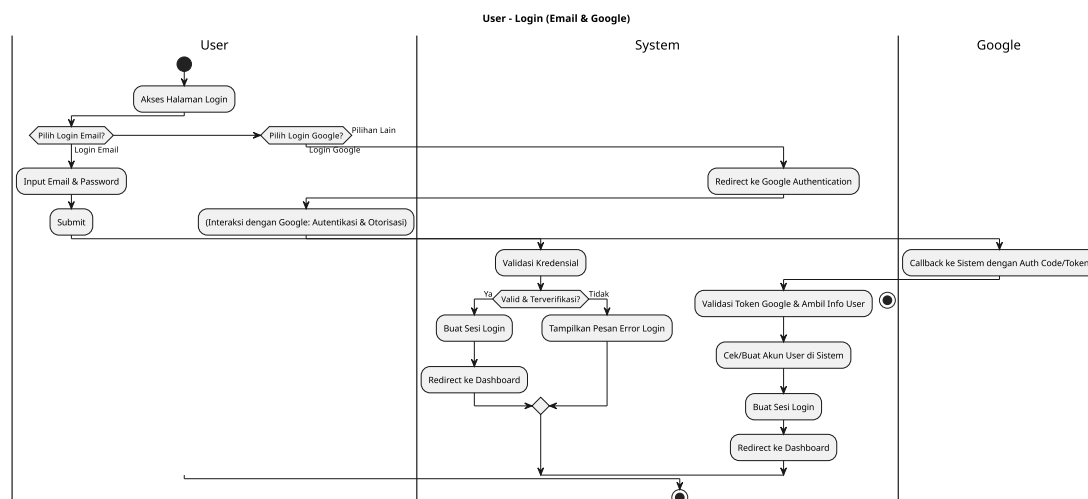


Gambar 3.7 Diagram Aktivitas Pengguna: Registrasi Email



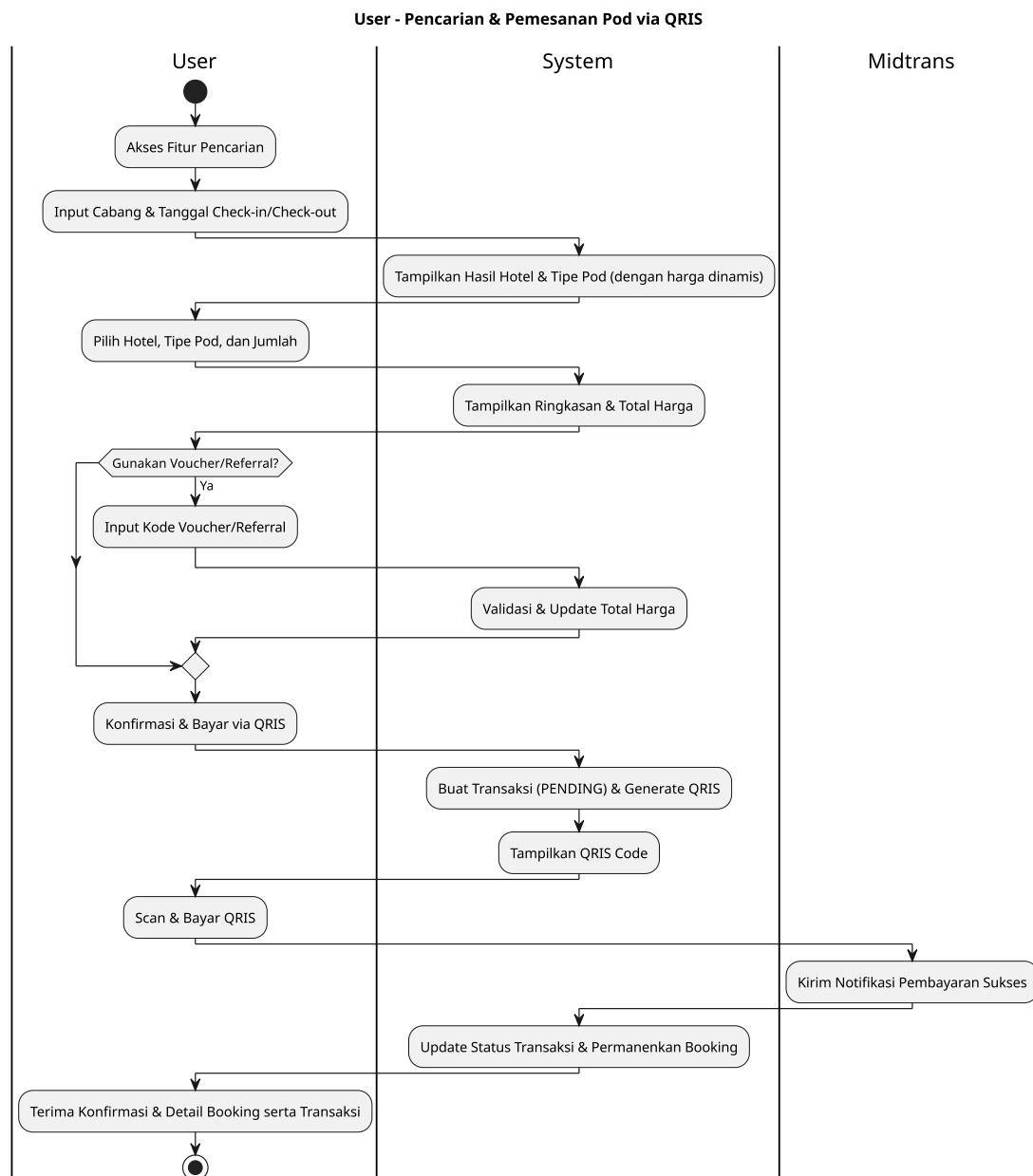
Gambar 3.8 Diagram Aktivitas Pengguna: Verifikasi Email

Pada gambar 3.7 dan 3.8, ditampilkan proses registrasi pengguna dan verifikasi email. Proses ini dimulai dengan pengguna mengisi formulir registrasi, kemudian sistem mengirimkan email verifikasi. Pengguna harus mengklik tautan verifikasi dalam email untuk menyelesaikan proses registrasi.



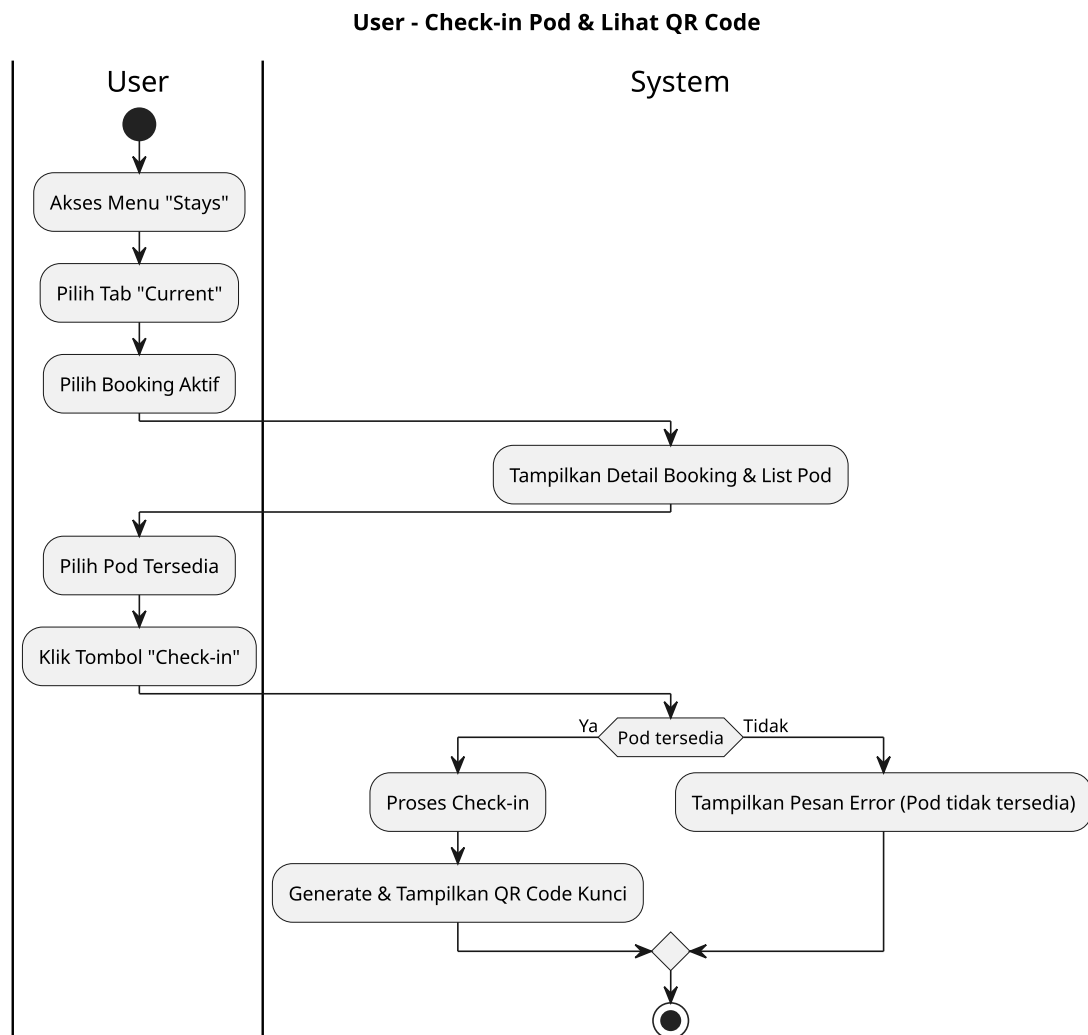
Gambar 3.9 Diagram Aktivitas Pengguna: Registrasi dan Login dengan Google OAuth2

Pada gambar 3.9, ditampilkan proses registrasi dan login menggunakan Google OAuth2. Pengguna dapat memilih untuk mendaftar atau masuk ke sistem menggunakan akun Google mereka, yang memudahkan proses autentikasi.



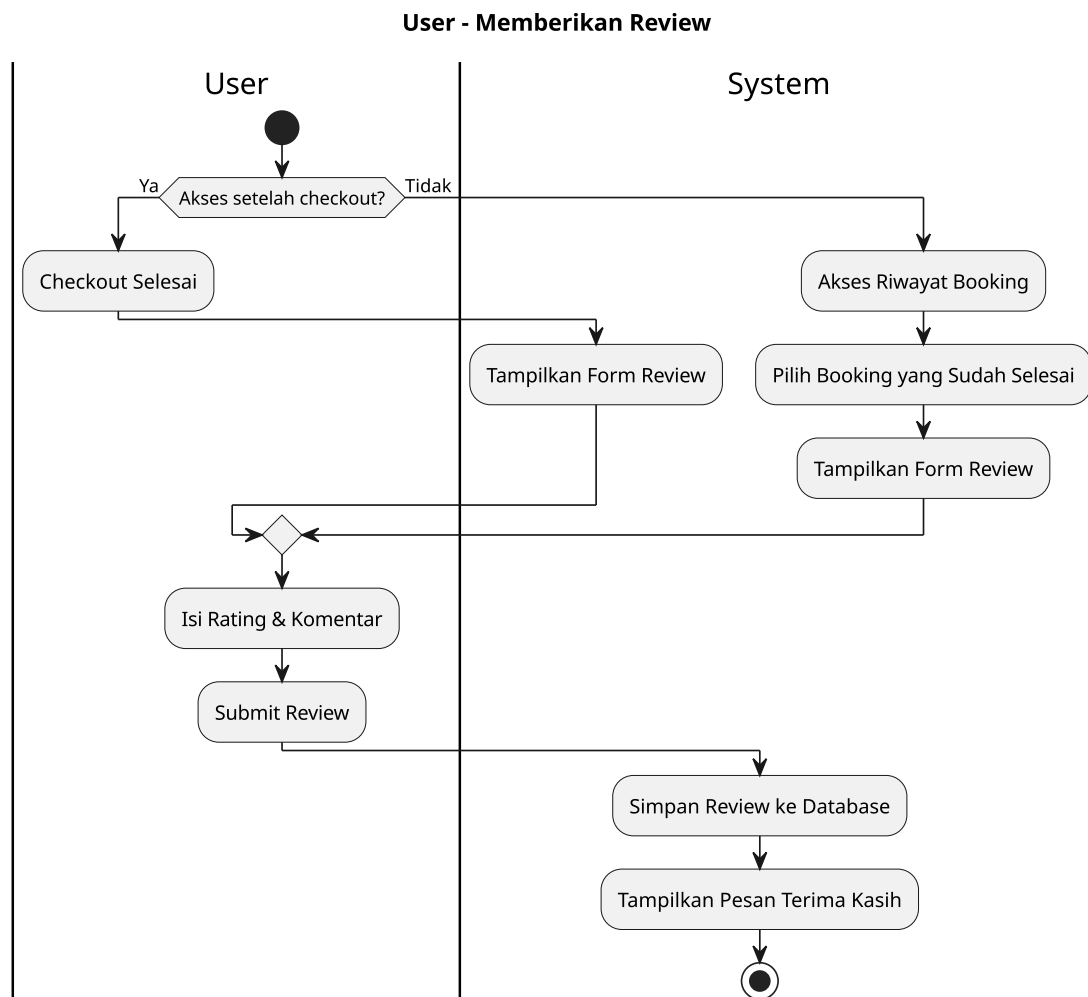
Gambar 3.10 Diagram Aktivitas Pengguna: Reservasi

Pada gambar 3.10, ditampilkan proses reservasi kamar oleh pengguna. Pengguna dapat memilih kamar atau pod yang tersedia, menentukan tanggal check-in dan check-out, serta melakukan pembayaran untuk reservasi tersebut.



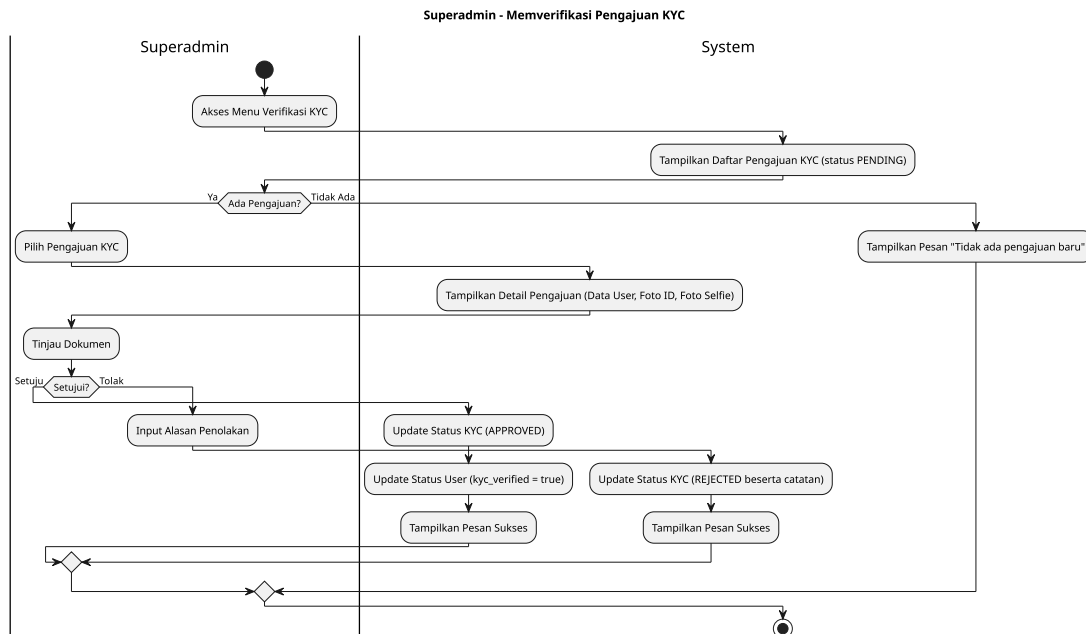
Gambar 3.11 Diagram Aktivitas Pengguna: Checkin dan Stay

Pada gambar 3.11, ditampilkan proses check-in dan stay. Setelah reservasi berhasil, pengguna dapat melakukan check-in ke kamar atau pod yang telah dipesan untuk mendapatkan kunci akses yang berupa kode QR untuk discan pada pintu masuk kamar atau pod agar dapat masuk ke dalamnya.



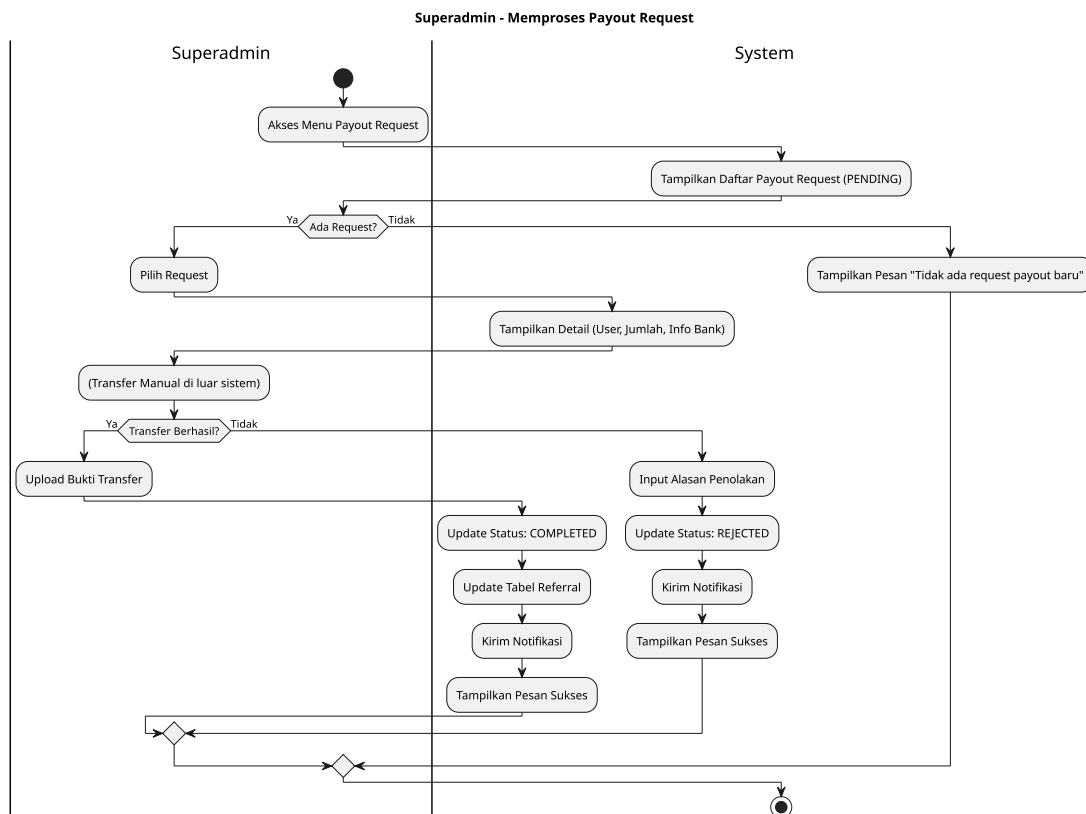
Gambar 3.12 Diagram Aktivitas Pengguna: Checkout dan Review

Pada gambar 3.12, ditampilkan proses checkout dan review. Setelah masa tinggal selesai, pengguna dapat melakukan checkout dari kamar atau pod yang telah dipesan. Pengguna juga dapat memberikan ulasan tentang pengalaman mereka selama menginap.



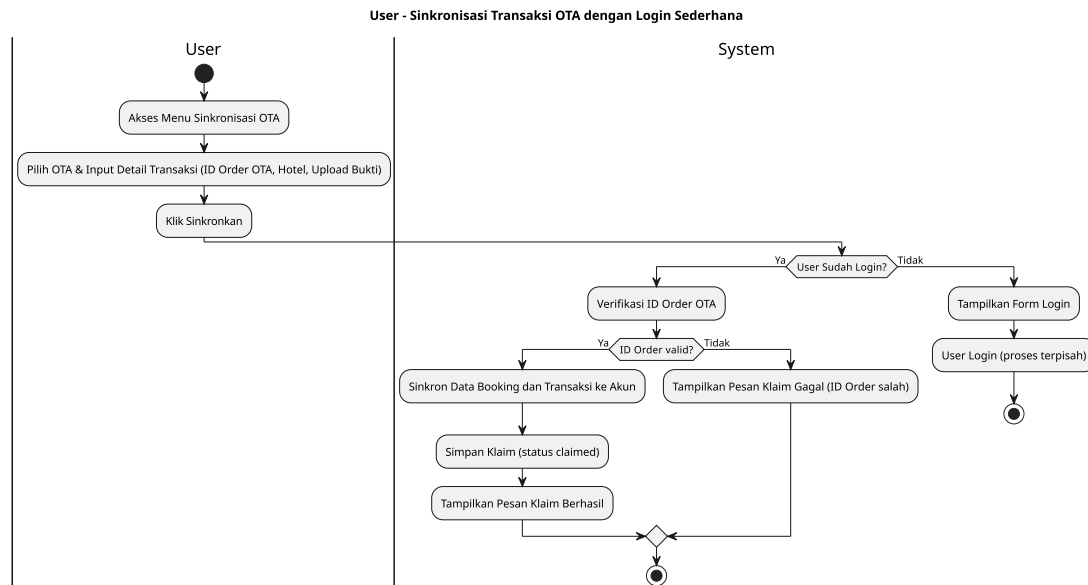
Gambar 3.13 Diagram Aktivitas Pengguna: Verifikasi Identitas (KYC)

Pada gambar 3.13, ditampilkan proses verifikasi identitas (KYC) yang dilakukan oleh pengguna. Proses ini penting untuk memastikan keamanan dan keaslian identitas pengguna dalam sistem. Pengguna diminta untuk mengunggah dokumen identitas yang diperlukan, yang kemudian akan diverifikasi oleh SuperAdmin.



Gambar 3.14 Diagram Aktivitas Pengguna: Payout Referral

Pada gambar 3.14, ditampilkan proses payout referral. Pengguna yang berhasil mereferensikan teman atau orang lain untuk menggunakan layanan dapat menerima payout sebagai imbalan. Proses ini melibatkan verifikasi referral dan pemrosesan pembayaran oleh SuperAdmin.

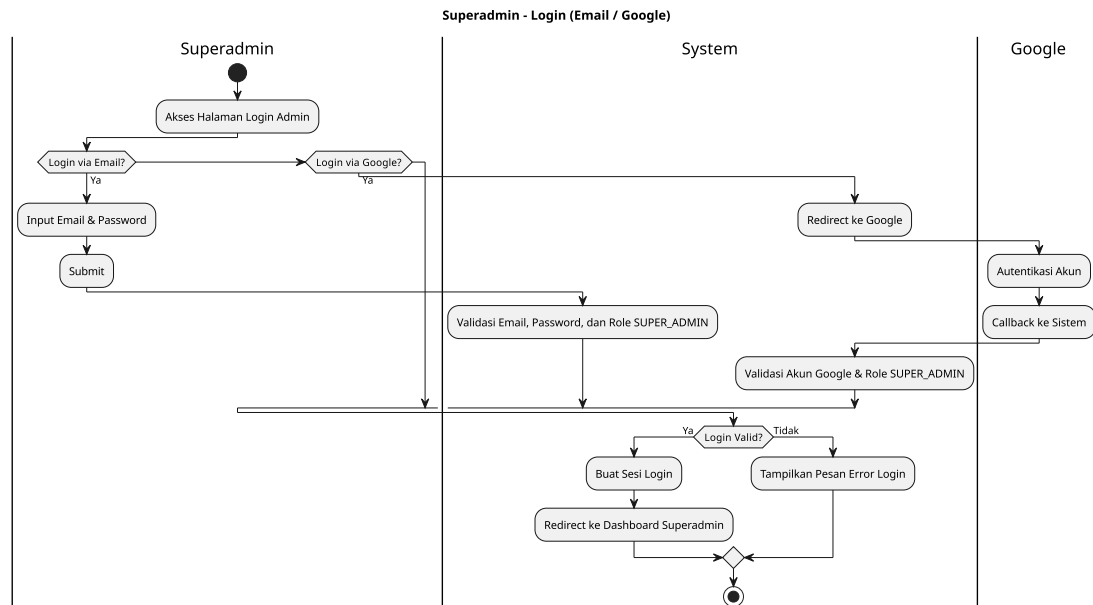


Gambar 3.15 Diagram Aktivitas Pengguna: Sinkronisasi Data Reservasi OTA

Pada gambar 3.15, ditampilkan proses sinkronisasi data reservasi dengan *Online Travel Agent (OTA)*. Proses ini memungkinkan pengguna untuk mengklaim reservasi yang dibuat melalui OTA, sehingga data reservasi dapat terintegrasi dengan sistem utama.

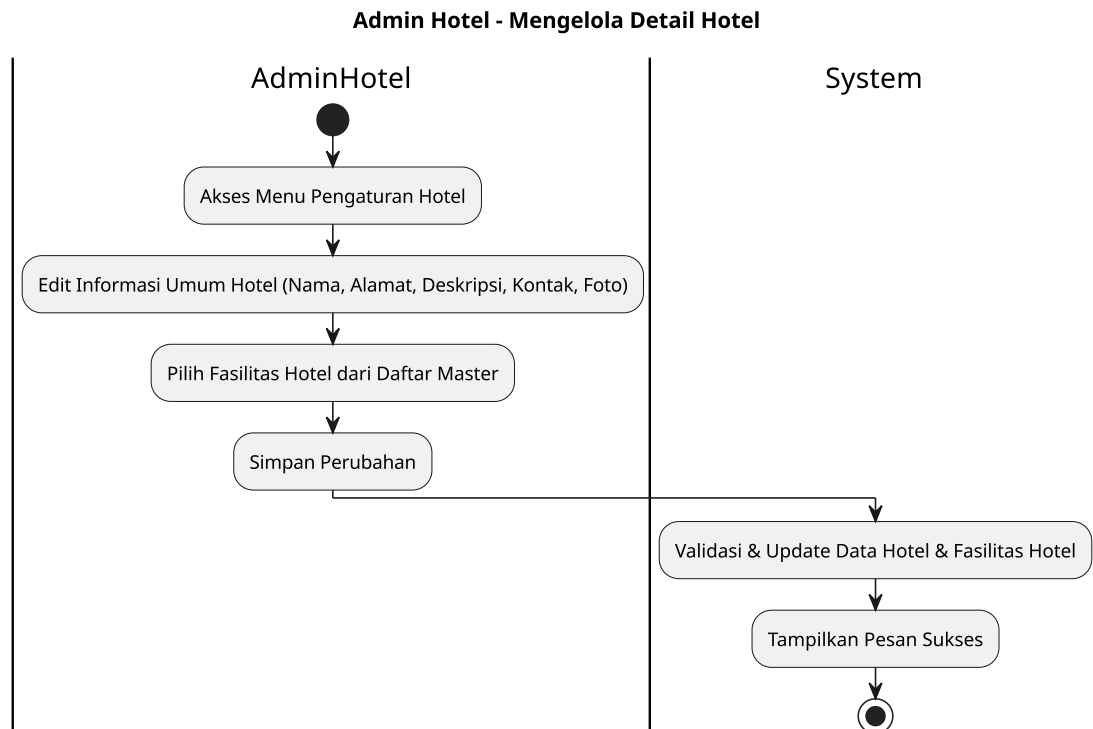
B. Diagram Aktivitas Admin Cabang

Pada gambar 3.16 hingga 3.23, ditampilkan diagram aktivitas yang menggambarkan berbagai proses yang dilakukan oleh Admin Cabang dalam sistem. Diagram ini mencakup proses login, pengelolaan cabang (hotel), jenis pod, pod, moderasi review, pengelolaan voucher cabang, pengelolaan pindah pod, dan pengelolaan pembersihan pod.



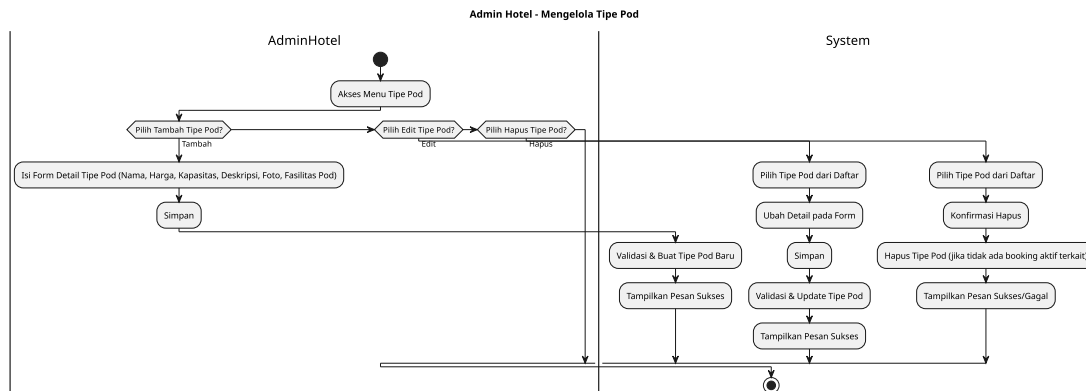
Gambar 3.16 Diagram Aktivitas Admin: Login Email dan Google OAuth2

Pada gambar 3.16, ditampilkan proses login Admin Cabang menggunakan email dan Google OAuth2. Admin Cabang dapat masuk ke sistem untuk mengakses fitur-fitur yang tersedia bagi mereka.



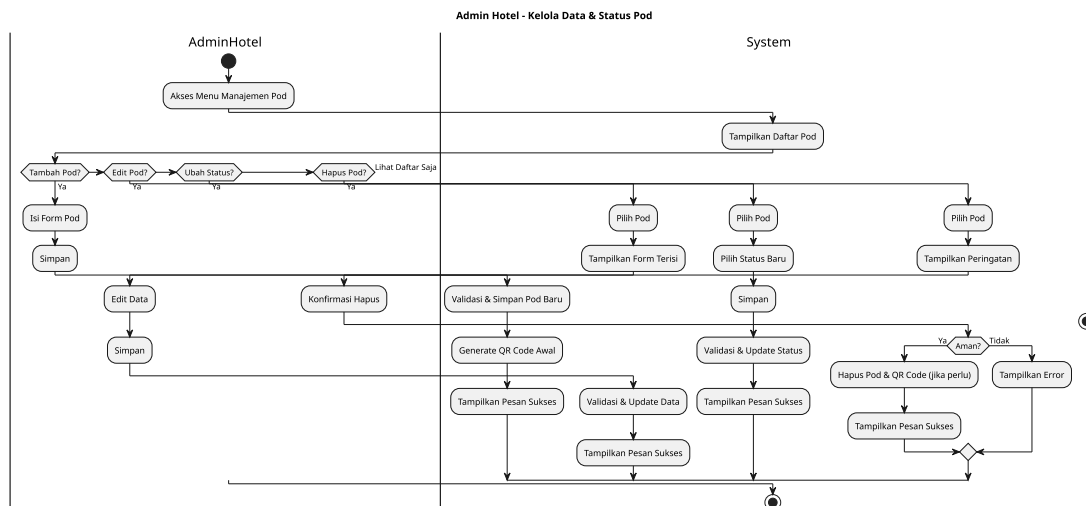
Gambar 3.17 Diagram Aktivitas Admin: Kelola Cabang (Hotel)

Pada gambar 3.17, ditampilkan proses pengelolaan cabang (hotel) oleh Admin Cabang. Admin Cabang dapat menambahkan, mengedit, atau menghapus informasi tentang cabang mereka, termasuk fasilitas yang tersedia.



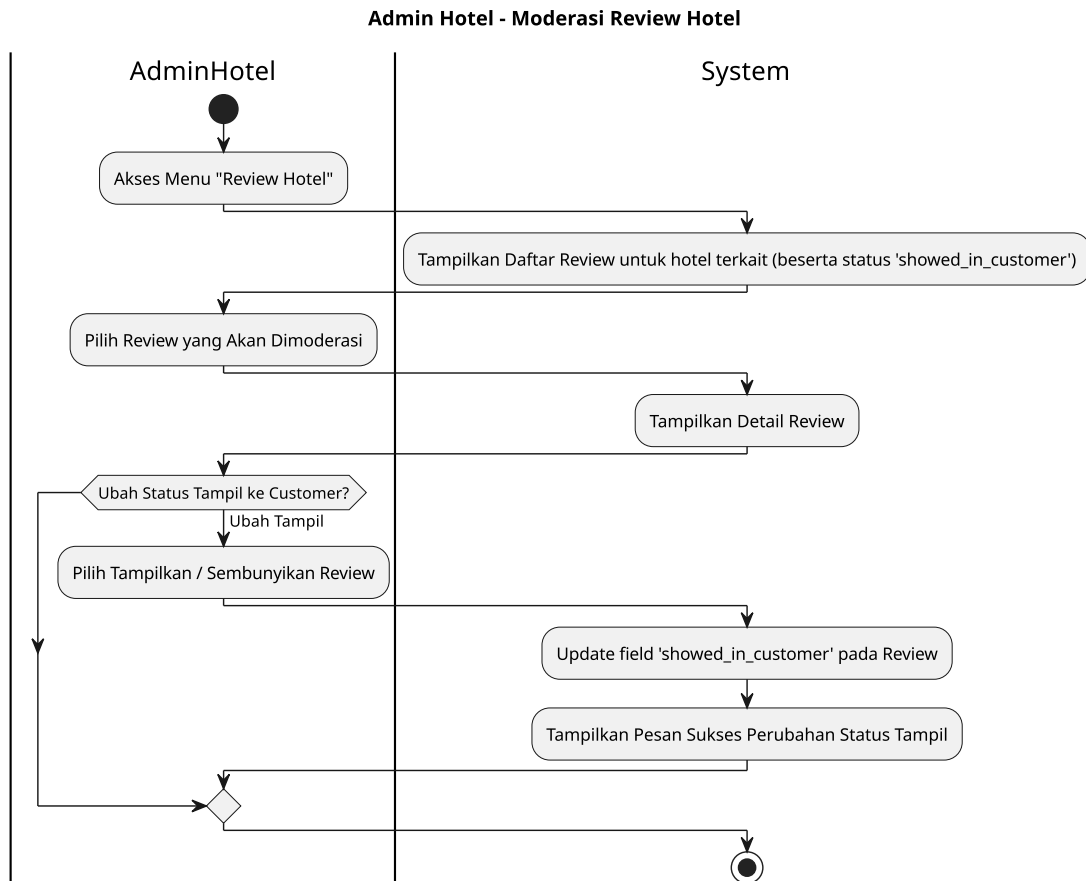
Gambar 3.18 Diagram Aktivitas Admin: Kelola Jenis Pod

Pada gambar 3.18, ditampilkan proses pengelolaan jenis pod oleh Admin Cabang. Admin Cabang dapat menambahkan, mengedit, atau menghapus jenis pod yang tersedia di cabang mereka, termasuk informasi tentang fasilitas dan harga.



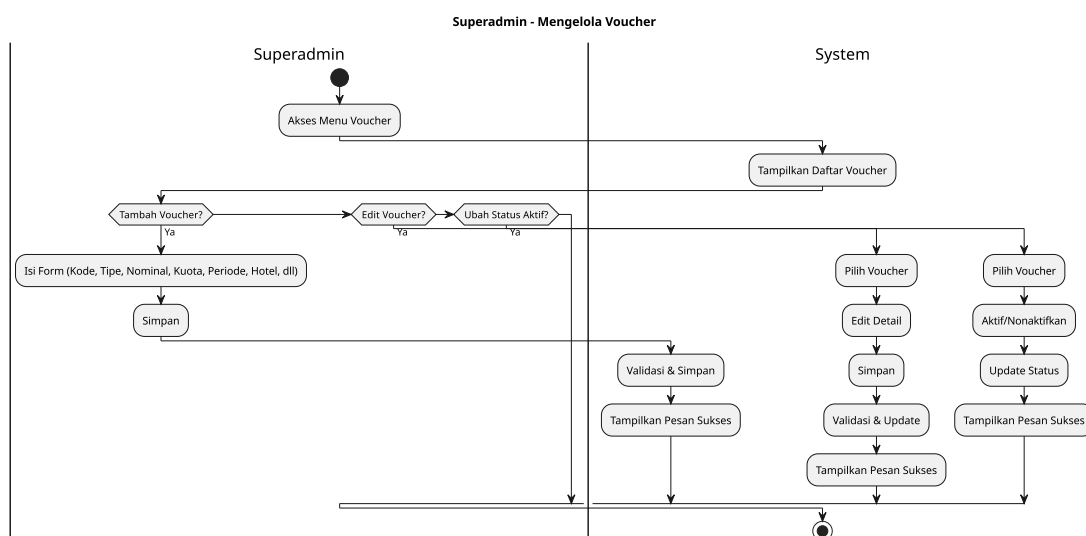
Gambar 3.19 Diagram Aktivitas Admin: Kelola Pod

Setelah menambahkan jenis pod, Admin Cabang dapat mengelola pod yang tersedia di cabang mereka. Pada gambar 3.19, ditampilkan proses pengelolaan pod, termasuk penambahan, pengeditan, dan penghapusan pod yang tersedia untuk reservasi.



Gambar 3.20 Diagram Aktivitas Admin: Moderasi Review

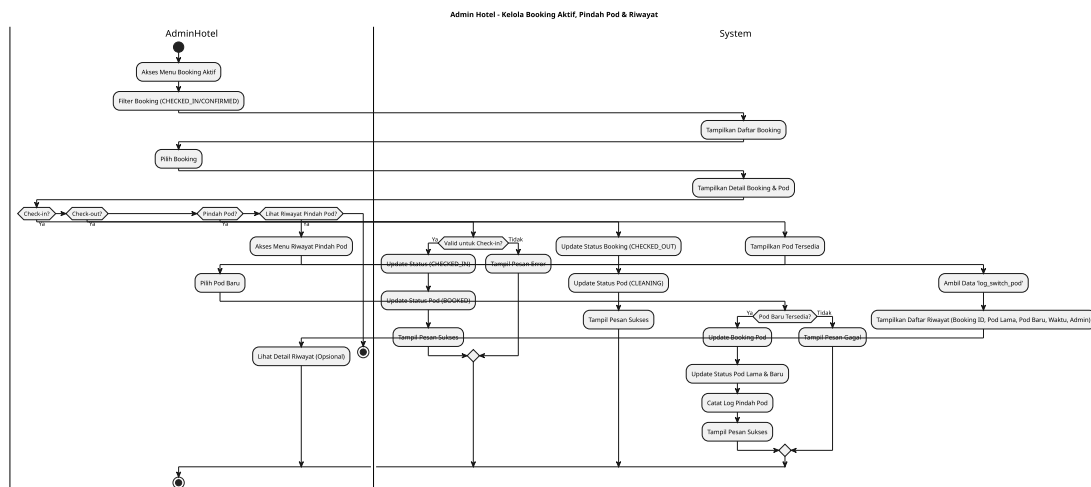
Pada gambar 3.20, ditampilkan proses moderasi review oleh Admin Cabang. Admin Cabang dapat meninjau, menyetujui, atau menolak ulasan yang diberikan oleh pengguna tentang pod atau cabang mereka.



Gambar 3.21 Diagram Aktivitas Admin: Kelola Voucher Cabang

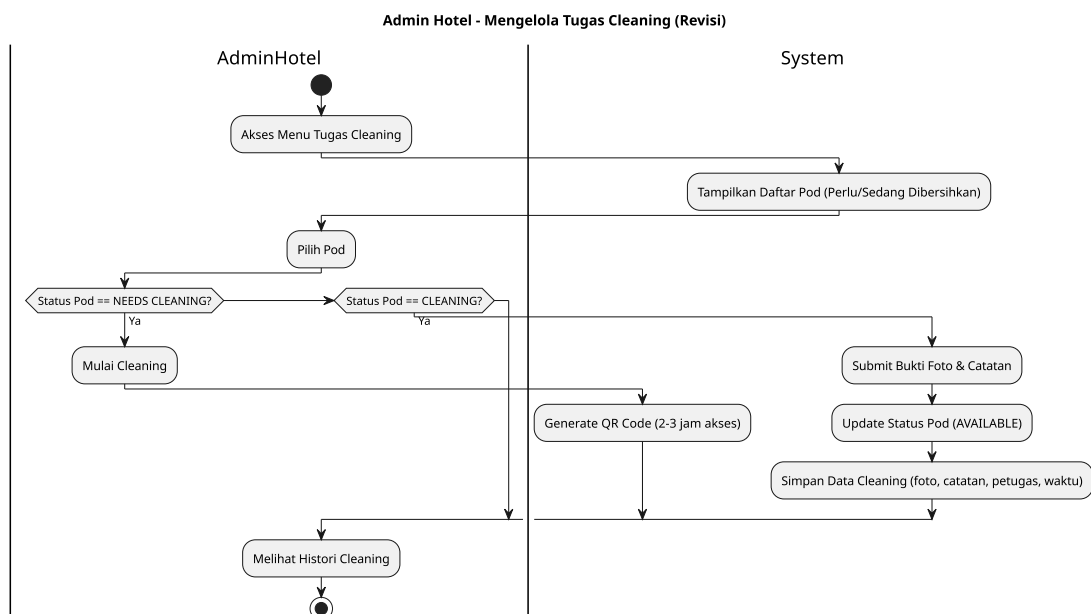
Pada gambar 3.21, ditampilkan proses pengelolaan voucher cabang oleh Admin Cabang.

Admin Cabang dapat membuat, mengedit, atau menghapus voucher yang dapat digunakan oleh pengguna untuk mendapatkan diskon atau penawaran khusus di cabang mereka.



Gambar 3.22 Diagram Aktivitas Admin: Kelola Pindah Pod

Pada gambar 3.22, ditampilkan proses pengelolaan pindah pod oleh Admin Cabang. Admin Cabang dapat mengelola permintaan pindah pod dari pengguna, termasuk memproses permintaan dan memastikan ketersediaan pod yang diminta.

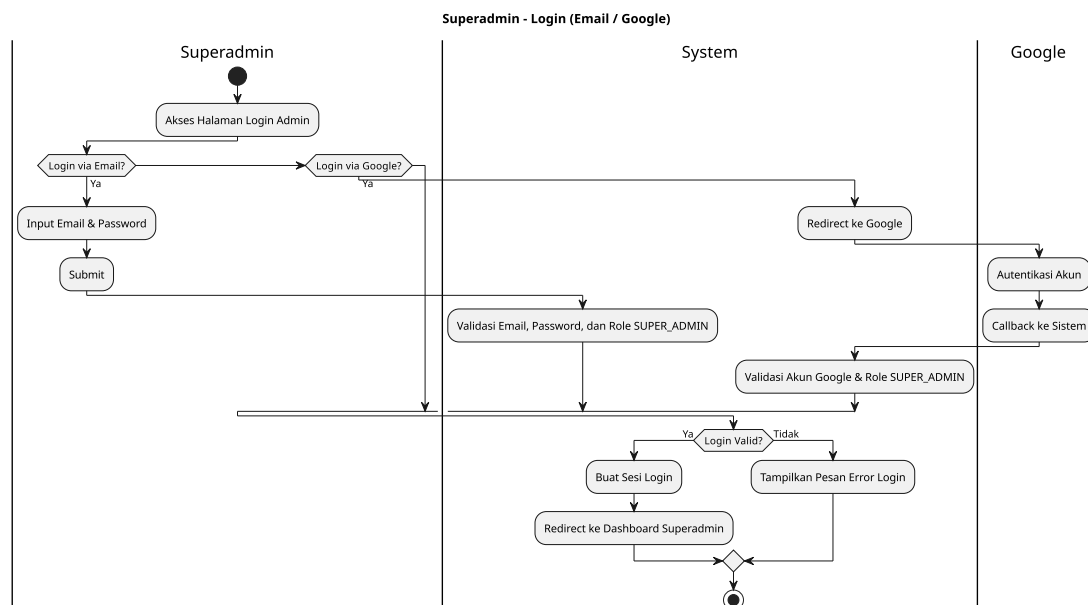


Gambar 3.23 Diagram Aktivitas Admin: Kelola Pembersihan Pod

Pada gambar 3.23, ditampilkan proses pengelolaan pembersihan pod oleh Admin Cabang. Admin Cabang dapat melakukan pembersihan pod setelah pengguna check-out, memastikan bahwa pod siap digunakan oleh pengguna berikutnya.

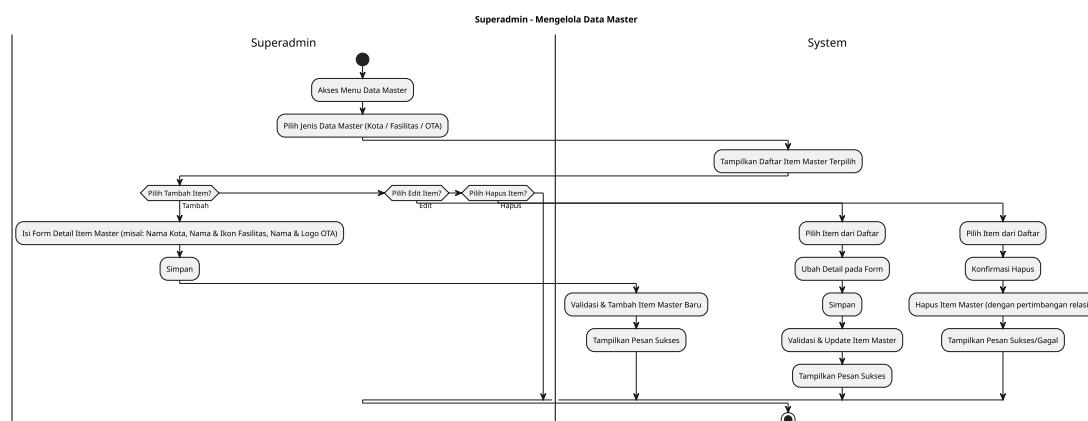
C. Diagram Aktivitas SuperAdmin

Pada gambar 3.24 hingga 3.30, ditampilkan diagram aktivitas yang menggambarkan berbagai proses yang dilakukan oleh SuperAdmin dalam sistem. Diagram ini mencakup proses login, pengelolaan data master, cabang (hotel), staff, verifikasi identitas (KYC), voucher, dan payout request.



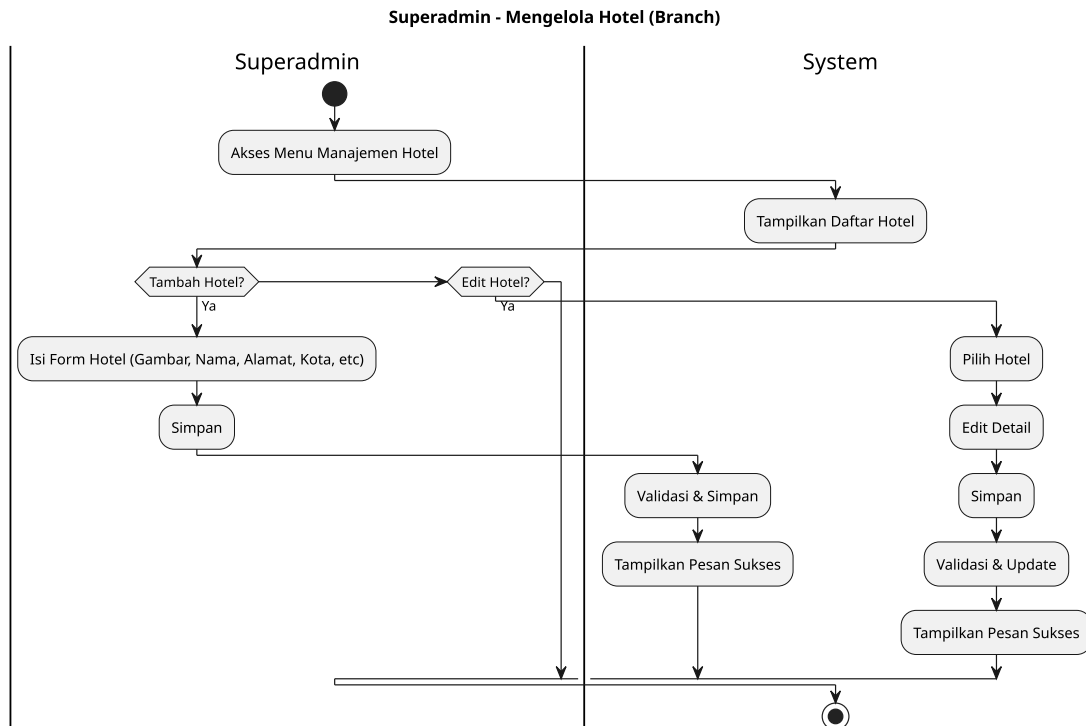
Gambar 3.24 Diagram Aktivitas SuperAdmin: Login Email dan Google OAuth2

Pada gambar 3.24, ditampilkan proses login SuperAdmin menggunakan email dan Google OAuth2. SuperAdmin dapat masuk ke sistem untuk mengakses fitur-fitur yang tersedia bagi mereka.



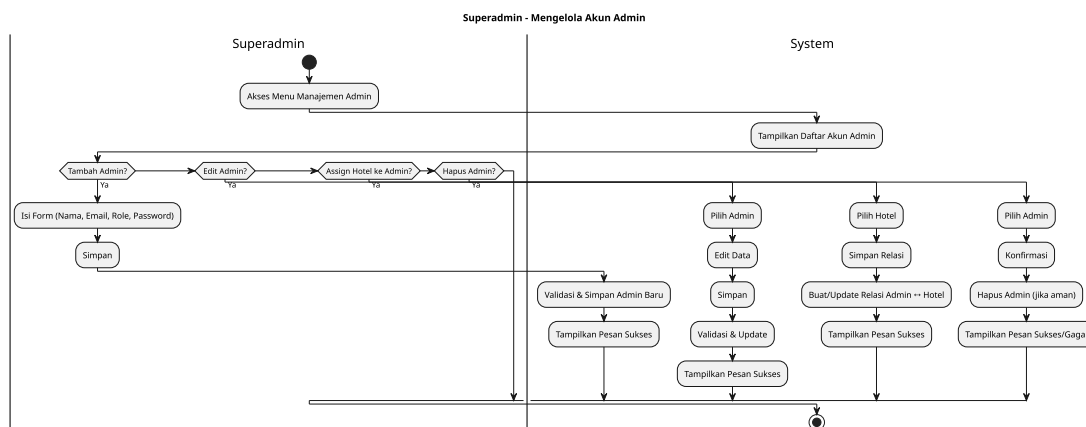
Gambar 3.25 Diagram Aktivitas SuperAdmin: Kelola Data Master

Pada gambar 3.25, ditampilkan proses pengelolaan data master oleh SuperAdmin. SuperAdmin dapat mengelola data master yang mencakup informasi dasar sistem seperti fasilitas, Kota, Cabang, dan Karyawan.



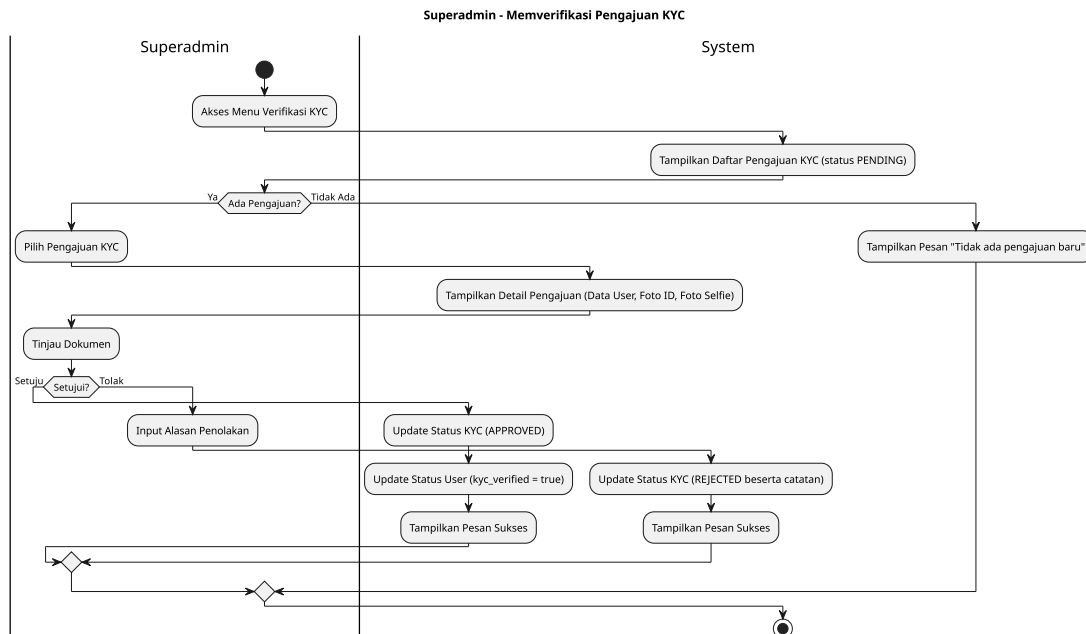
Gambar 3.26 Diagram Aktivitas SuperAdmin: Kelola Data Cabang (Hotel)

Pada gambar 3.26, ditampilkan proses pengelolaan data cabang (hotel) oleh SuperAdmin. SuperAdmin dapat menambahkan, mengedit, atau menghapus informasi tentang cabang yang ada dalam sistem.



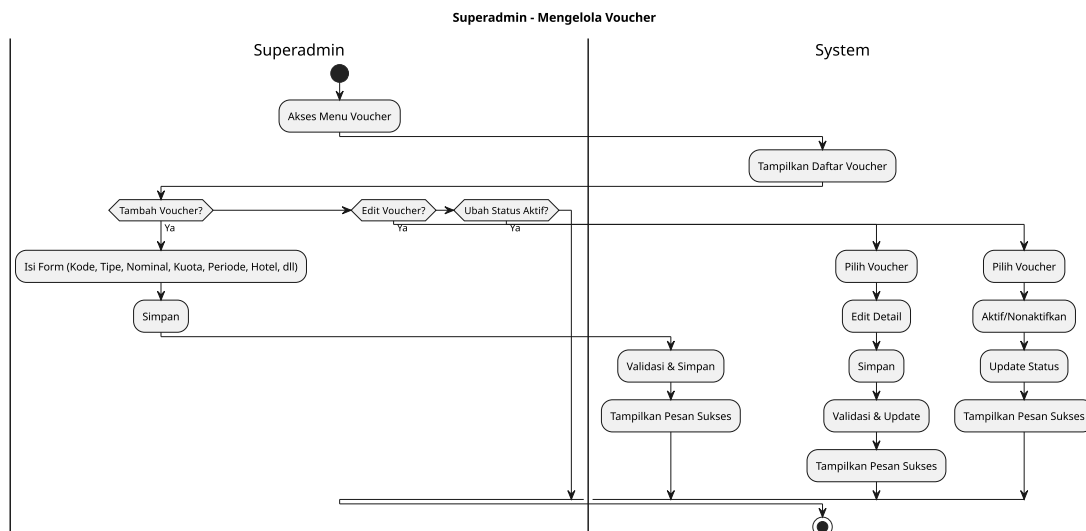
Gambar 3.27 Diagram Aktivitas SuperAdmin: Kelola Data Staff

Pada gambar 3.27, ditampilkan proses pengelolaan data staff oleh SuperAdmin. SuperAdmin dapat menambahkan, mengedit, atau menghapus informasi tentang staff yang bekerja di cabang-cabang yang ada dalam sistem.



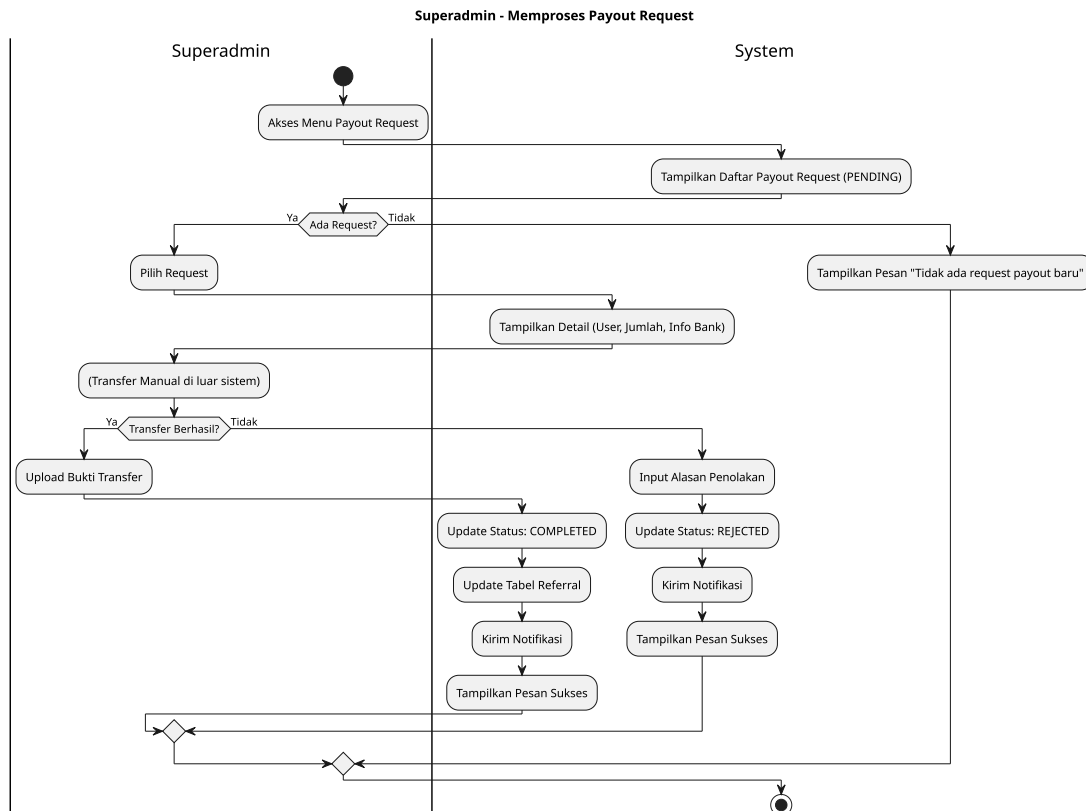
Gambar 3.28 Diagram Aktivitas SuperAdmin: Kelola Data Verifikasi Identitas (KYC)

Pada gambar 3.28, ditampilkan proses pengelolaan data verifikasi identitas (KYC) oleh SuperAdmin. SuperAdmin dapat mengelola proses verifikasi identitas pengguna, termasuk persetujuan atau penolakan dokumen yang diunggah oleh pengguna.



Gambar 3.29 Diagram Aktivitas SuperAdmin: Kelola Data Voucher

Pada gambar 3.29, ditampilkan proses pengelolaan data voucher oleh SuperAdmin. SuperAdmin dapat membuat, mengedit, atau menghapus voucher yang dapat digunakan oleh pengguna untuk mendapatkan diskon atau penawaran khusus baik itu secara global maupun cabang tertentu.

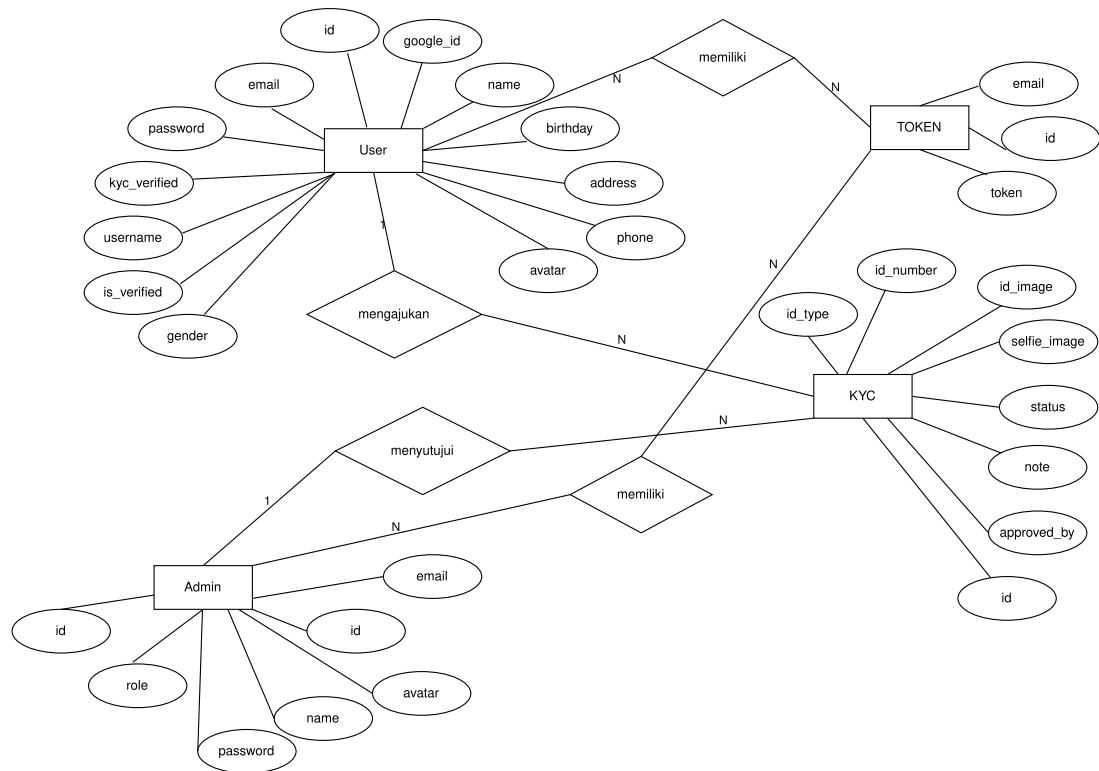


Gambar 3.30 Diagram Aktivitas SuperAdmin: Kelola Payout Request

Pada gambar 3.30, ditampilkan proses pengelolaan payout request oleh SuperAdmin. SuperAdmin dapat memproses permintaan payout dari pengguna yang telah mengajukan payout referral. Proses ini melibatkan verifikasi dan persetujuan payout sebelum dana dikirim ke pengguna.

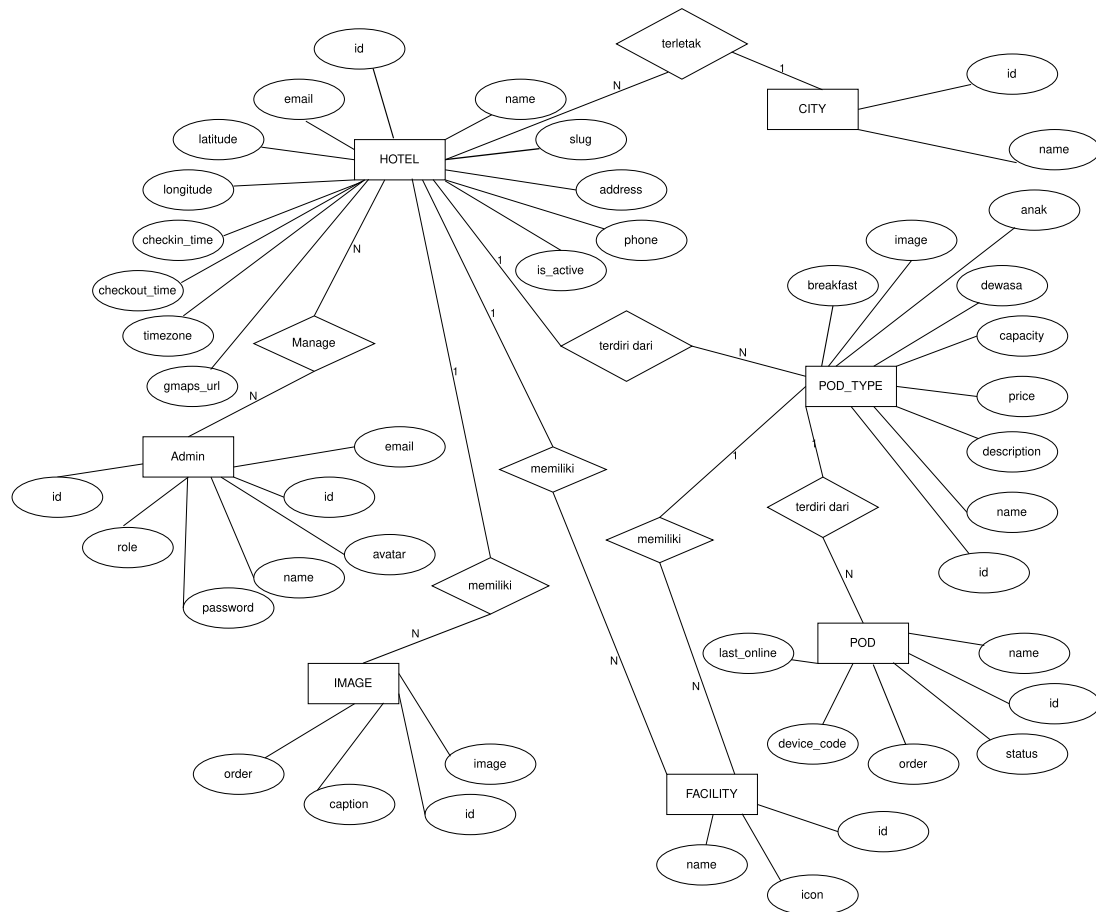
3.5.3 Entity Relationship Diagram

Pada tahap ini, *Entity Relationship Diagram* (ERD) dibuat untuk menggambarkan struktur basis data yang akan digunakan dalam sistem. ERD ini mencakup entitas-entitas utama, atribut-atributnya, serta hubungan antar entitas. ERD ini akan membantu dalam merancang basis data yang efisien dan sesuai dengan kebutuhan sistem.



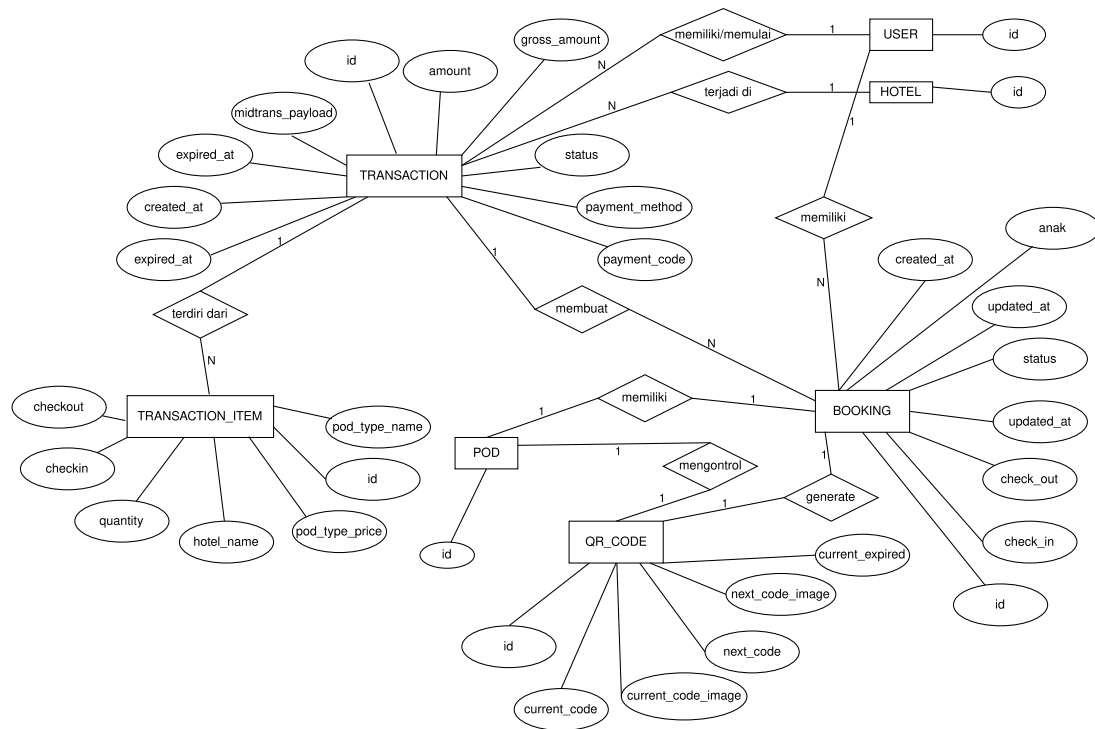
Gambar 3.31 ERD User, Verikasi Identitas, Admin, dan Token Verifikasi Global

Pada gambar 3.31, ditampilkan ERD yang mencakup entitas-entitas utama seperti User, Verifikasi Identitas, Admin, dan Token Verifikasi Global. Entitas User menyimpan informasi pengguna, sedangkan Verifikasi Identitas menyimpan data terkait verifikasi identitas pengguna. Entitas Admin menyimpan informasi tentang admin yang mengelola sistem, dan Token Verifikasi Global digunakan untuk mengelola proses verifikasi dan otentikasi pengguna dengan strategi *One Time Token*.



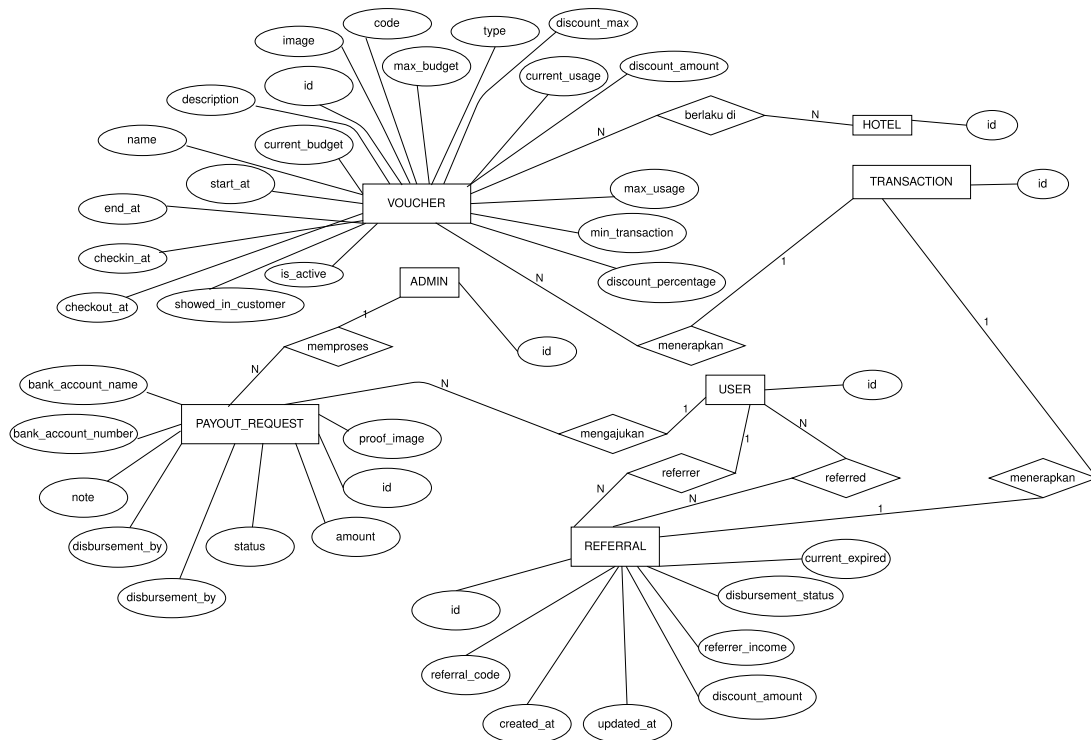
Gambar 3.32 ERD Kota, Cabang(Hotel), Fasilitas, Jenis Pod, Pod, Fasilitas, Staff (Admin), dan Gambar

Pada gambar 3.32, ditampilkan ERD yang mencakup entitas Kota, Cabang (Hotel), Fasilitas, Jenis Pod, Pod, Fasilitas, Staff (Admin), dan Gambar. Entitas Kota menyimpan informasi tentang lokasi cabang, sedangkan Cabang menyimpan data terkait hotel yang dikelola. Fasilitas dan Jenis Pod menyimpan informasi tentang fasilitas yang tersedia di setiap cabang dan jenis pod yang ditawarkan. Entitas Pod menyimpan data terkait pod yang tersedia, dan Staff (Admin) menyimpan informasi tentang admin yang mengelola cabang tersebut. Gambar digunakan untuk menyimpan representasi visual dari entitas lainnya.



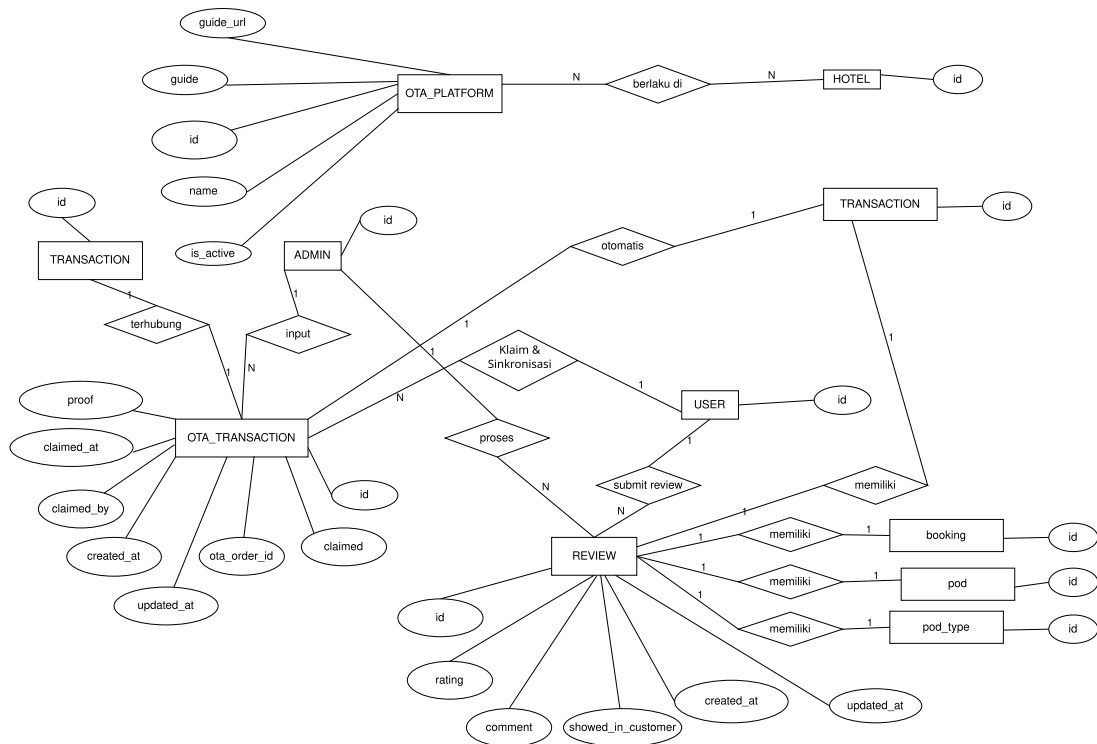
Gambar 3.33 ERD Transaksi, Item Transaksi, Booking, dan QR Code

Pada gambar 3.33, ditampilkan ERD yang mencakup entitas Transaksi, Item Transaksi, Booking, dan QR Code. Entitas Transaksi menyimpan informasi tentang transaksi yang dilakukan oleh pengguna, sedangkan Item Transaksi menyimpan data terkait item yang dibeli dalam transaksi tersebut. Entitas Booking menyimpan informasi tentang reservasi yang dilakukan oleh pengguna, dan QR Code digunakan untuk mengelola kode unik yang terkait dengan setiap transaksi atau reservasi.



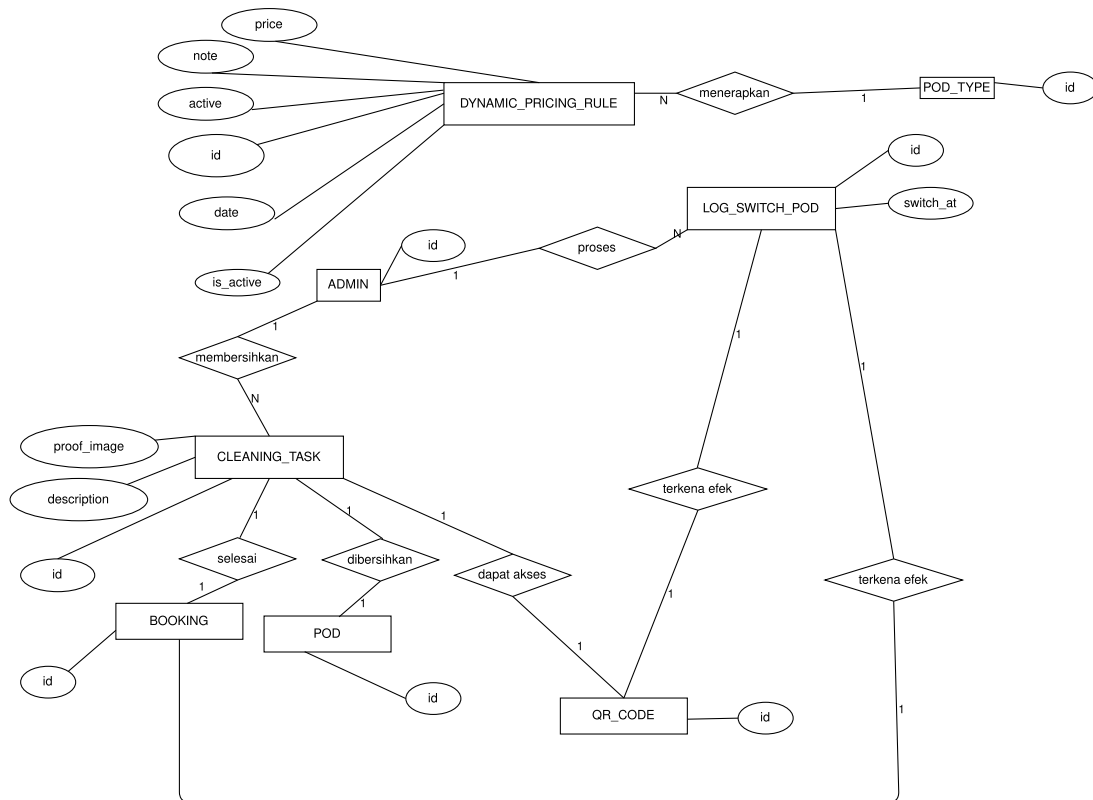
Gambar 3.34 ERD Voucher, Referral, dan Payout Request

Pada gambar 3.34, ditampilkan ERD yang mencakup entitas Voucher, Referral, dan Payout Request. Entitas Voucher menyimpan informasi tentang voucher yang digunakan oleh pengguna, sedangkan Referral menyimpan data terkait program referral yang memungkinkan pengguna untuk mengajak orang lain bergabung. Entitas Payout Request digunakan untuk mengelola permintaan pembayaran yang diajukan oleh pengguna yang berpartisipasi dalam program referral.



Gambar 3.35 ERD OTA Platform, OTA Transaction, dan Review

Pada gambar 3.35, ditampilkan ERD yang mencakup entitas OTA Platform, OTA Transaction, dan Review. Entitas OTA Platform menyimpan informasi tentang platform *Online Travel Agent* yang digunakan untuk integrasi, sedangkan OTA Transaction menyimpan data terkait transaksi yang dilakukan melalui platform tersebut. Entitas Review digunakan untuk mengelola ulasan dan penilaian yang diberikan oleh pengguna terhadap layanan yang mereka terima.

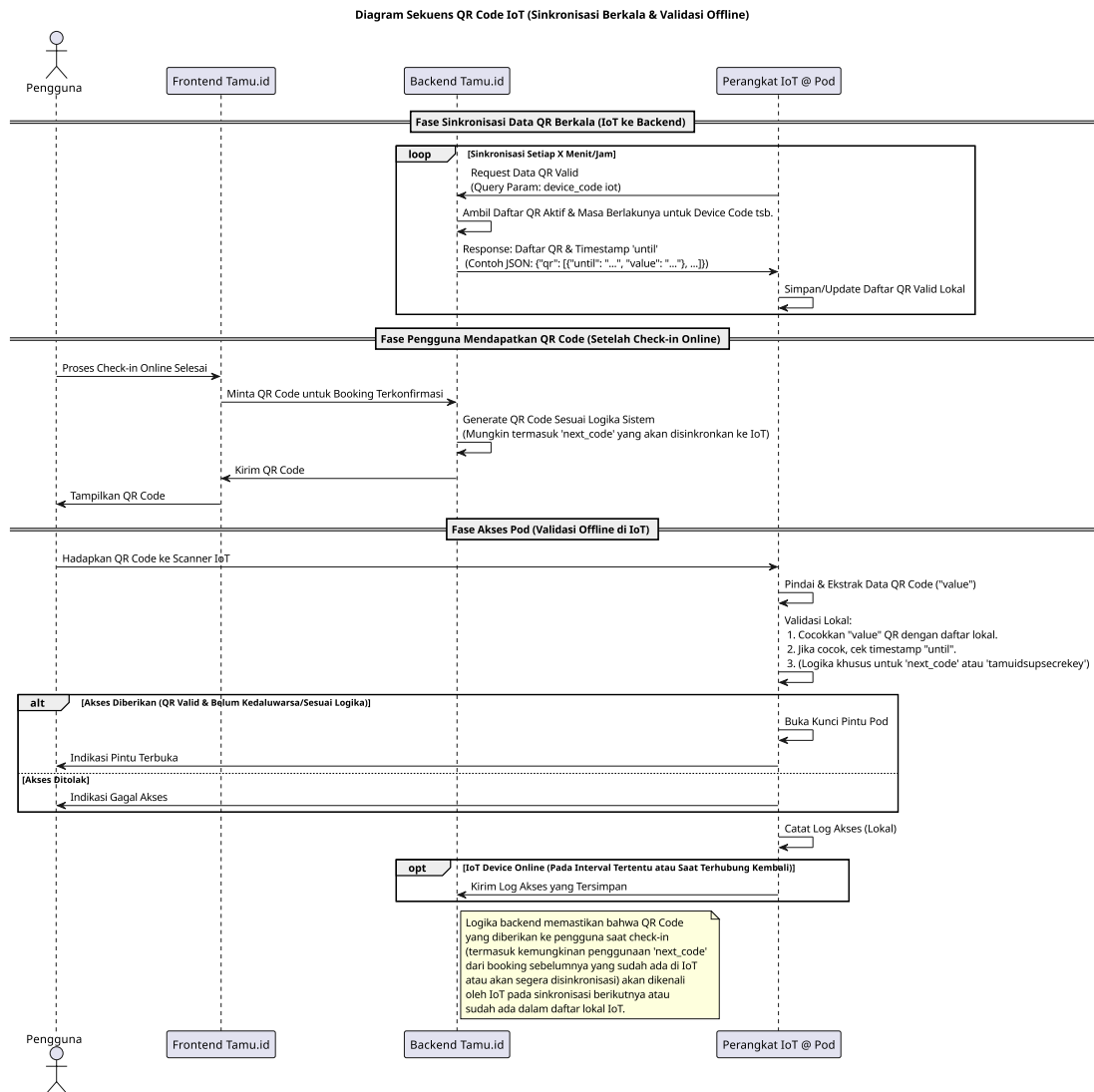


Gambar 3.36 ERD Dyanmic Pricing, Pod Cleaning, dan Switch Pod

Pada gambar 3.36, ditampilkan ERD yang mencakup entitas Dynamic Pricing, Pod Cleaning, dan Switch Pod. Entitas Dynamic Pricing menyimpan informasi tentang penetapan harga dinamis berdasarkan permintaan dan ketersediaan, sedangkan Pod Cleaning menyimpan data terkait proses pembersihan pod. Entitas Switch Pod digunakan untuk mengelola proses pindah pod yang dilakukan oleh admin cabang.

3.5.4 Ilustrasi Implementasi *Offline Fault Tolerance* pada rotasi Kunci Pod

Salah satu fitur penting dalam sistem Tamu.id adalah kemampuan untuk menangani situasi di mana koneksi internet tidak tersedia atau terputus, terutama saat pengguna melakukan checkin pada pod. Untuk mengatasi masalah ini, sistem dirancang dengan fitur *offline fault tolerance* yang memungkinkan pengguna tetap dapat mengakses pod meskipun koneksi internet tidak tersedia pada tenggang waktu tertentu di suatu cabang, dengan struktur data dan mekanisme seperti ini juga yang membuat ketika user melakukan checkin dan mendapatkan QR Code, QR Code tersebut langsung dapat digunakan untuk membuka pintu pod tanpa delay menunggu fetching data dari pod ke *backend*.

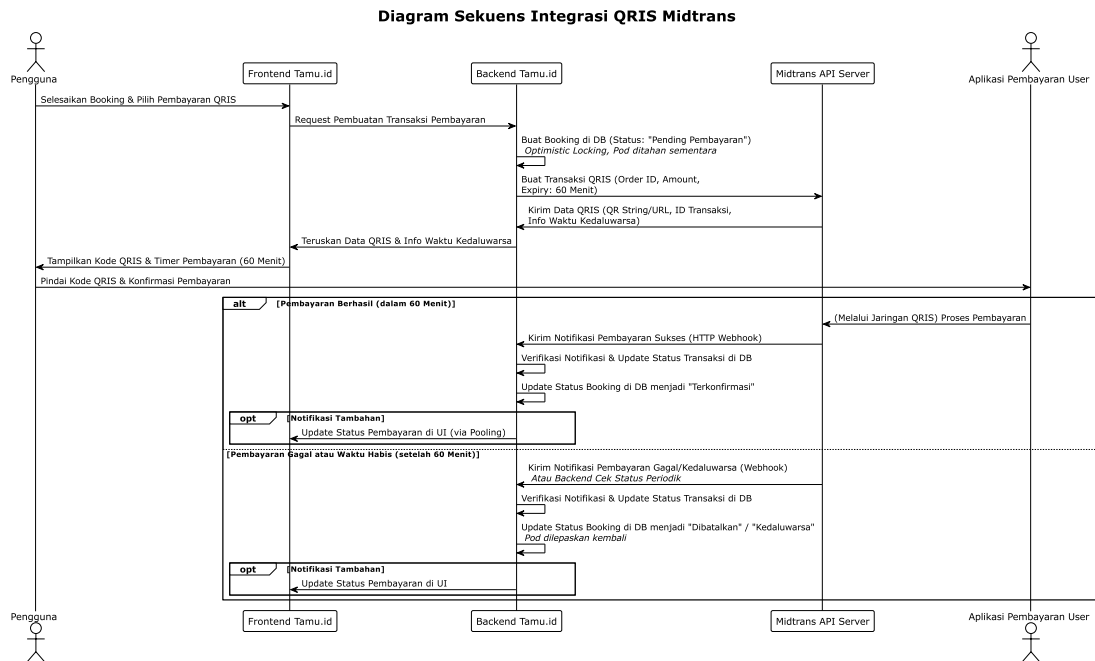


Gambar 3.37 Diagram Sekuensial Implementasi *Offline Fault Tolerance* pada Rotasi Kunci Pod

Ilustrasi implementasi *offline fault tolerance* pada rotasi kunci pod dilakukan untuk memastikan bahwa sistem tetap dapat berfungsi dengan baik meskipun terjadi gangguan pada koneksi internet atau perangkat IoT. Hal ini penting untuk menjaga pengalaman pengguna yang baik dan memastikan bahwa pengguna dapat mengakses pod yang telah mereka pesan tanpa masalah.

3.5.5 Ilustrasi Integrasi *WebHook Payment Gateway*

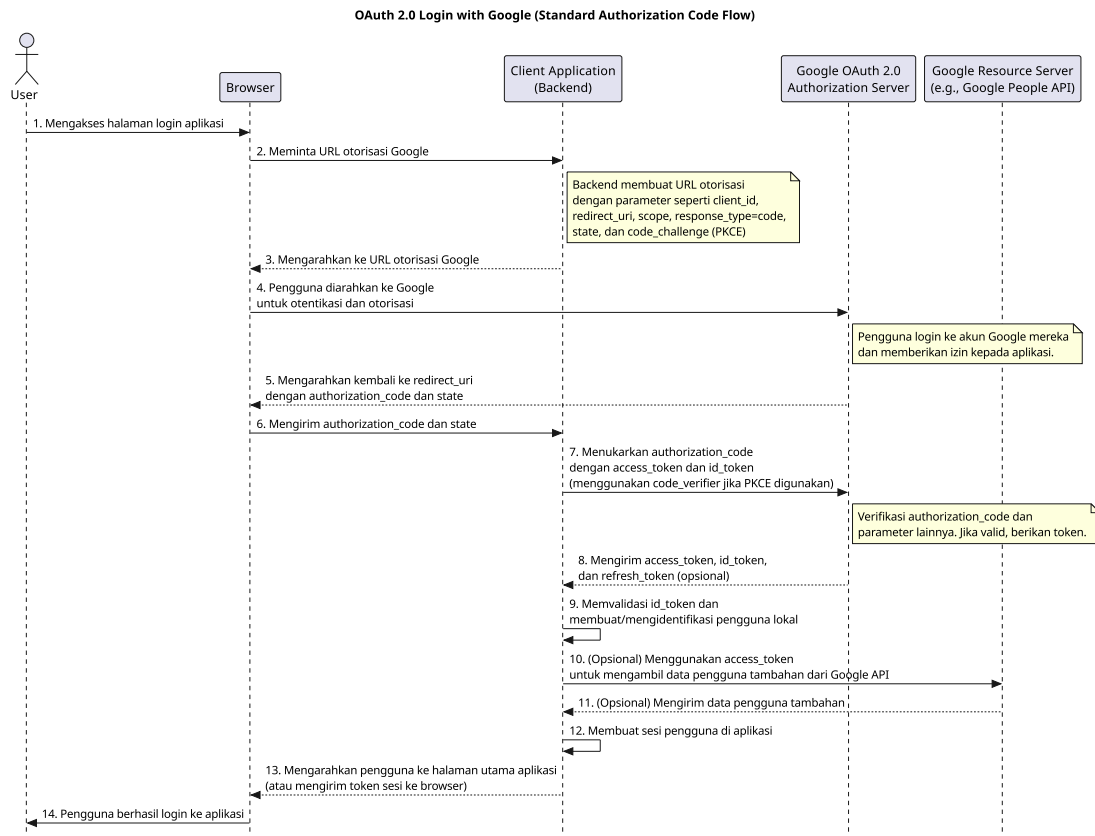
Ilustrasi integrasi *WebHook Payment Gateway* dilakukan untuk memastikan bahwa sistem dapat menerima notifikasi pembayaran secara real-time dari penyedia layanan pembayaran. Hal ini penting untuk memastikan bahwa status pembayaran pengguna dapat diperbarui secara otomatis dan pengguna dapat melanjutkan proses checkin setelah pembayaran berhasil dilakukan.



Gambar 3.38 Diagram Sekuensial Integrasi *WebHook Payment Gateway*

3.5.6 Ilustrasi Integrasi *Passwordless* dengan OAuth2 Google

Ilustrasi integrasi *passwordless* dengan OAuth2 Google dilakukan untuk memberikan kemudahan bagi pengguna dalam melakukan pendaftaran dan masuk ke dalam sistem. Dengan menggunakan OAuth2 Google, pengguna dapat masuk ke dalam sistem Tamu.id tanpa perlu mengingat password, sehingga meningkatkan pengalaman pengguna dan keamanan sistem.

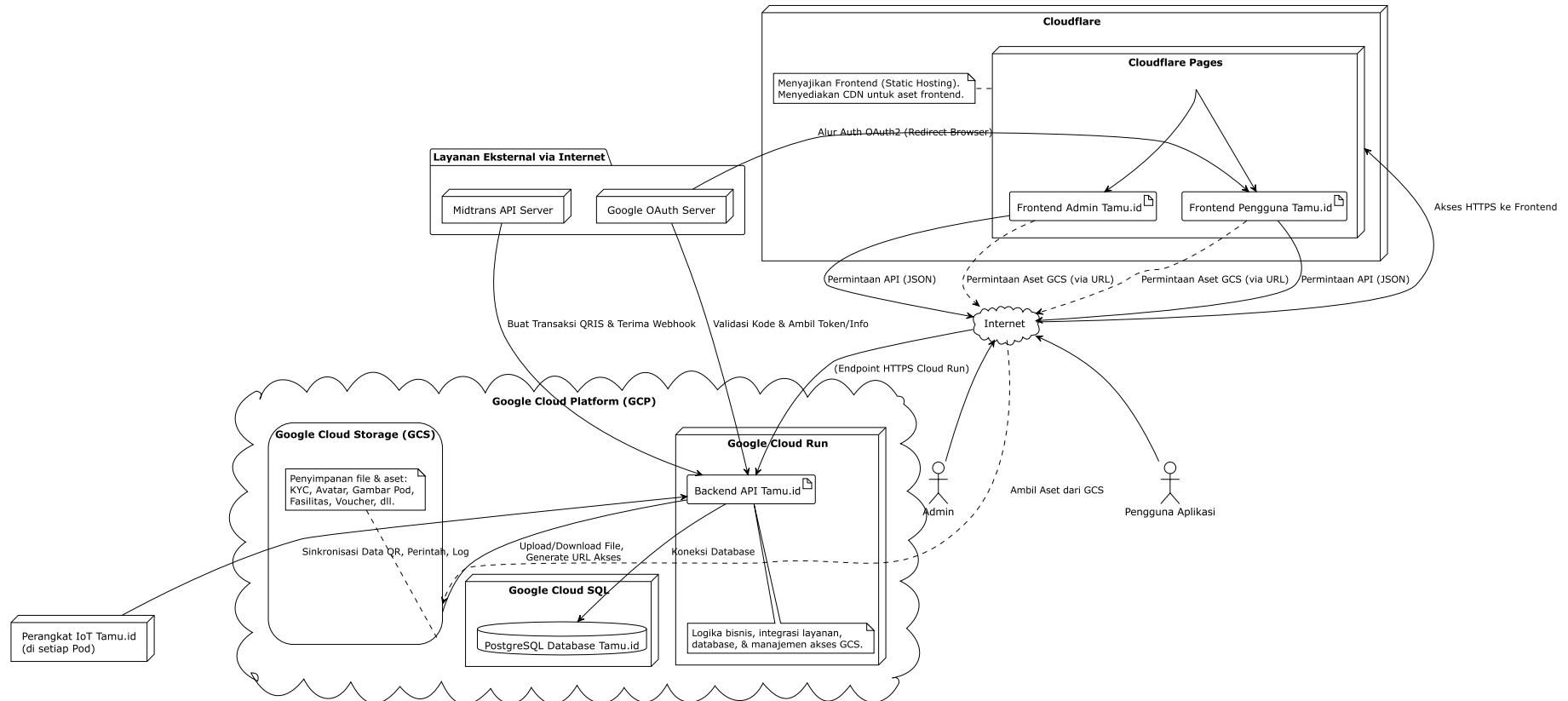


Gambar 3.39 Diagram Sekuensial Integrasi *Passwordless* dengan OAuth2 Google

3.5.7 Peluncuran Aplikasi (*Deployment*)

Peluncuran aplikasi Tamu.id dilakukan setelah semua tahap pengembangan, pengujian, dan perbaikan bug selesai. Aplikasi ini diluncurkan ke lingkungan produksi dengan menggunakan layanan cloud seperti Google Cloud Platform (GCP) untuk memastikan ketersediaan, skalabilitas, dan keamanan sistem. Proses peluncuran meliputi konfigurasi server, pengaturan basis data, dan pengaturan keamanan untuk memastikan bahwa aplikasi dapat berjalan dengan baik di lingkungan produksi. Pada gambar 3.40 dapat dilihat arsitektur peluncuran aplikasi Tamu.id yang mencakup berbagai komponen seperti server, basis data, dan layanan cloud yang digunakan untuk mendukung operasional aplikasi.

Diagram Arsitektur Deploy Aplikasi Tamu.id



Gambar 3.40 Diagram Arsitektur Peluncuran Aplikasi Tamu.id

3.5.8 Pengujian Sistem

Pengujian sistem dilakukan untuk memastikan bahwa sistem Tamu.id berfungsi sesuai dengan kebutuhan yang telah ditentukan. Pengujian ini mencakup beberapa jenis pengujian, yaitu pengujian fungsional, pengujian keamanan, pengujian performa, dan pengujian penerimaan pengguna.

A. Pengujian Black Box

Pengujian black box dilakukan untuk menguji fungsionalitas sistem tanpa memperhatikan struktur internalnya. Pengujian ini dilakukan dengan menggunakan skenario pengujian yang telah disusun berdasarkan kebutuhan fungsional yang telah dianalisis sebelumnya. Pengujian ini mencakup pengujian pada setiap fitur yang ada dalam sistem, seperti pendaftaran pengguna, pemesanan tiket, checkin, checkout, dan lain-lain, pengujian dilakukan untuk tiga peran pengguna yaitu pada tabel 3.1 untuk pengguna, tabel 3.2 untuk admin cabang, dan tabel 3.3 untuk superadmin. Pengujian ini dilakukan untuk memastikan bahwa setiap fitur berfungsi sesuai dengan yang diharapkan dan tidak ada bug yang ditemukan.

Tabel 3.1 Skenario Pengujian Black Box (User)

Skenario Pengujian	Langkah Pengujian	Hasil yang diharapkan	Kesesuaian Hasil
Fitur Registrasi Akun			
Registrasi berhasil dengan email baru	1. Buka halaman registrasi. 2. Isi nama lengkap. 3. Masukkan email yang belum terdaftar (format valid). 4. Masukkan kata sandi kuat. 5. Konfirmasi kata sandi dengan benar. 6. (Jika ada) Centang persetujuan Syarat & Ketentuan. 7. Klik tombol "Daftar".	- Pesan sukses registrasi ditampilkan. - Pengguna diarahkan ke halaman login atau dashboard. - Email konfirmasi terkirim.	

Registrasi gagal karena email sudah terdaftar	1. Buka halaman registrasi. 2. Isi nama lengkap. 3. Masukkan email yang sudah terdaftar. 4. Masukkan kata sandi. 5. Konfirmasi kata sandi. 6. Klik tombol "Daftar".	- Pesan error "Email sudah terdaftar" ditampilkan. - Pengguna tetap di halaman registrasi.	
Registrasi gagal karena format email salah	1. Buka halaman registrasi. 2. Masukkan email dengan format salah (misal: user@.com, user.com). 3. Isi field lainnya. 4. Klik "Daftar".	- Pesan error "Format email tidak valid" ditampilkan.	
Registrasi gagal karena konfirmasi kata sandi tidak cocok	1. Buka halaman registrasi. 2. Isi nama lengkap, email baru, kata sandi. 3. Masukkan konfirmasi kata sandi yang berbeda. 4. Klik tombol "Daftar".	- Pesan error "Konfirmasi kata sandi tidak cocok" ditampilkan.	
Registrasi menggunakan akun Google (sukses, pengguna baru)	1. Buka halaman registrasi/login. 2. Klik tombol "Daftar/Masuk dengan Google". 3. Pilih akun Google yang valid dan belum terdaftar di sistem. 4. Izinkan akses jika diminta.	- Akun berhasil terbuat/terhubung. - Pengguna otomatis berhasil masuk.	
Fitur Login Pengguna			

Login berhasil dengan email dan kata sandi valid	1. Buka halaman login. 2. Masukkan email terdaftar. 3. Masukkan kata sandi yang benar. 4. Klik tombol "Masuk".	- Login berhasil. - Pengguna berhasil masuk.	
Login gagal (email tidak terdaftar)	1. Buka halaman login. 2. Masukkan email yang tidak terdaftar. 3. Masukkan kata sandi. 4. Klik tombol "Masuk".	- Pesan error "Email atau kata sandi salah" ditampilkan.	
Login gagal (kata sandi salah)	1. Buka halaman login. 2. Masukkan email terdaftar. 3. Masukkan kata sandi yang salah. 4. Klik tombol "Masuk".	- Pesan error "Email atau kata sandi salah" ditampilkan.	
Login gagal (akun belum diverifikasi email, jika ada)	1. Registrasi dengan email (misal: REG_01), jangan verifikasi email. 2. Coba login dengan email dan kata sandi tersebut.	- Pesan error "Akun belum diverifikasi. Silakan cek email Anda." atau sejenisnya.	
Login menggunakan akun Google (sukses, akun sudah ada)	1. Buka halaman login. 2. Klik "Masuk dengan Google". 3. Pilih akun Google yang sudah terdaftar/terhubung sebelumnya.	- Login berhasil. - Pengguna diarahkan ke dashboard.	
Fitur Lupa Kata Sandi			

Proses lupa kata sandi berhasil	1. Di halaman login, klik "Lupa Kata Sandi?". 2. Masukkan email terdaftar. 3. Klik "Kirim Instruksi". 4. Buka email, klik link reset. 5. Masukkan kata sandi baru dan konfirmasinya. 6. Klik "Reset Kata Sandi".	- Pesan sukses "Kata sandi berhasil direset". - Dapat login dengan kata sandi baru.	
Lupa kata sandi dengan email tidak terdaftar	1. Di halaman login, klik "Lupa Kata Sandi?". 2. Masukkan email yang tidak terdaftar. 3. Klik "Kirim Instruksi".	- Pesan error "Email tidak ditemukan" atau instruksi tidak terkirim.	
Fitur Kelola Profil Pengguna			
Mengubah semua data profil yang diizinkan (sukses)	1. Login. 2. Ke halaman "Profil Saya" lalu "Edit Profil". 3. Ubah nama, nomor telepon, alamat, gender, tanggal lahir. 4. Klik "Simpan".	- Pesan sukses "Profil berhasil diperbarui". - Semua perubahan tercermin dengan benar.	
Mengubah data profil dengan format salah (misal: telepon)	1. Login. 2. Ke "Edit Profil". 3. Masukkan nomor telepon dengan format salah (misal: huruf). 4. Klik "Simpan".	- Pesan error "Format nomor telepon tidak valid". - Data tidak tersimpan.	
Fitur Verifikasi Identitas (KYC)			

Mengajukan verifikasi KYC (KTP) dengan data valid	1. Login. 2. Ke halaman KYC. 3. Pilih jenis ID "KTP". 4. Masukkan nomor KTP valid. 5. Unggah foto KTP jelas. 6. Unggah foto selfie jelas. 7. Klik "Ajukan".	- Pengajuan KYC berhasil dikirim. - Status KYC menjadi "Menunggu Verifikasi".	
Mengajukan KYC (Passport) dengan data valid	1. Login. 2. Ke halaman KYC. 3. Pilih jenis ID "Passport". 4. Masukkan nomor Passport valid. 5. Unggah foto Passport jelas. 6. Unggah foto selfie jelas. 7. Klik "Ajukan".	- Pengajuan KYC berhasil dikirim. - Status KYC menjadi "Menunggu Verifikasi".	
Mengajukan KYC dengan file gambar terlalu besar/format salah	1. Login. 2. Ke halaman KYC. 3. Isi data. 4. Unggah file gambar dengan ukuran < batas maksimal atau format bukan JPG/PNG. 5. Klik "Ajukan".	- Pesan error "Ukuran file terlalu besar" atau "Format file tidak didukung".	
Melihat status KYC (Pending, Approved, Rejected)	1. Login. 2. Navigasi ke halaman KYC atau Profil.	- Status KYC pengguna ditampilkan dengan benar sesuai kondisi aktual.	
Fitur Pencarian dan Reservasi			
Mencari pod (tanggal check-out sebelum check-in)	1. Login. 2. Pilih Cabang. 3. Pilih tanggal check-in. 4. Pilih tanggal check-out lebih awal dari check-in. 5. Klik "Cari Pod".	- Pesan error "Tanggal check-out tidak valid".	

Melakukan pemesanan pod dengan voucher kadaluarsa	1. Lakukan pencarian pod. 2. Pilih tipe pod. 3. Di halaman pembayaran, masukkan kode voucher yang sudah kadaluarsa. 4. Klik "Terapkan Voucher".	- Pesan error "Voucher tidak valid atau sudah kadaluarsa".	
Pemesanan pod dengan jumlah melebihi kapasitas tipe pod	1. Lakukan pencarian pod. 2. Pilih tipe pod. 3. Coba pesan jumlah tamu yang melebihi kapasitas maksimal dewasa/anak tipe pod tersebut.	- Pesan error "Jumlah tamu melebihi kapasitas" atau input dibatasi.	
Pemesanan pod, pembayaran berhasil (QRIS)	1. Lakukan pencarian pod. 2. Pilih tipe pod. 3. Lanjutkan ke pembayaran. 4. Pilih metode QRIS, pindai QR. 5. Konfirmasi pembayaran di aplikasi e-wallet.	- Pembayaran berhasil. - Tampilan menunggu dibayar berubah menjadi terbayar - Transaksi tercatat.	
Pemesanan pod, menggunakan kode referral teman (valid, aturan disesuaikan)	1. Lakukan pencarian pod. 2. Pilih tipe pod. 3. Di halaman pembayaran, masukkan kode referral valid milik teman. 4. Klik "Terapkan". 5. Selesaikan pembayaran.	- Diskon referral berhasil diterapkan. - Harga total berkurang. - Pemesanan berhasil.	
Fitur Sinkronisasi OTA			

Sinkronisasi reservasi OTA dengan kode valid	1. Login. 2. Ke halaman "Sinkronisasi OTA" atau "Klaim Reservasi". 3. Pilih OTA. 4. Masukkan kode booking OTA / nomor order OTA yang valid (sudah diinput Admin Cabang). 5. Klik "Sinkronisasi".	- Reservasi OTA berhasil disinkronkan/diklaim. - Muncul di daftar booking pengguna.	
Sinkronisasi reservasi OTA dengan kode tidak ditemukan/salah	1. Login. 2. Ke halaman "Sinkronisasi OTA". 3. Masukkan kode booking OTA yang salah. 4. Klik "Sinkronisasi".	- Pesan error "Reservasi tidak ditemukan" atau "Kode booking salah".	
Fitur Operasional Menginap			
Check-in berhasil	1. Login. 2. Buka halaman stays lalu klik tab current 3. Coba klik tombol "Check-in" 4. Pilih pod yang diinginkan 5. klik checkin	- berhasil check-in dan mendapatkan kunci pod yang diinginkan berupa QRCode	
Check-in gagal (di luar jam operasional atau sebelum waktunya check-in)	1. Login. 2. Buka halaman stays. 3. Coba klik tombol "Check-in" sebelum jam check-in yang diizinkan atau setelah batas waktu check-in.	- Tombol "Check-in" tidak ada untuk booking yang statusnya masih upcoming	

Check-out	1. Login. 2. Buka detail booking aktif yang sudah check-in. 3. Klik tombol "Check-out" 4. muncul modal rating (user wajib mengisi rating untuk isi review opsional) 5. klik kirim	- Check-out berhasil. - Status booking "Completed". - rating dan review juga berhasil dikirim	
Fitur Referral			
Melihat riwayat pendapatan referral	1. Login. 2. Ke halaman Referral. 3. Pilih tab "Riwayat Pendapatan".	- Menampilkan daftar transaksi yang menghasilkan pendapatan referral dengan statusnya.	
Mengajukan pencairan referral (KYC belum disetujui)	1. Login, KYC masih "Pending" atau "Rejected". 2. Ke halaman Referral. 3. Coba klik "Cairkan Dana".	- Tombol "Cairkan Dana" tidak aktif atau pesan error "Verifikasi KYC diperlukan untuk pencairan".	
Fitur Ulasan			
Memberikan ulasan tanpa rating bintang	1. Login, setelah checkout. 2. Ke Riwayat Booking, pilih tab oldest booking. 3. Klik "Beri Ulasan". 4. Hanya tulis komentar, tidak memilih bintang. 5. Coba Kirim.	- tombol kirim review tidak dapat di klik	
Mencoba memberi ulasan untuk booking yang belum selesai/dibatalkan	1. Login. 2. Cari booking yang statusnya "Pending Payment" atau "Cancelled". 3. Coba cari opsi "Beri Ulasan".	- Opsi "Beri Ulasan" tidak tersedia untuk booking tersebut.	

Tabel 3.2 Skenario Pengujian Black Box (Admin)

Skenario Pengujian	Langkah Pengujian	Hasil yang diharapkan	Kesesuaian Hasil
Fitur Login dan Profil Admin Cabang			
Login Admin Cabang berhasil	1. Buka halaman login admin. 2. Masukkan email Admin Cabang valid. 3. Masukkan kata sandi benar. 4. Klik "Masuk".	- Login berhasil. - Diarahkan ke halaman pilih cabang yang ingin dikelola Cabang.	
Login Admin Cabang gagal (kata sandi salah)	1. Buka halaman login admin. 2. Masukkan email Admin Cabang valid. 3. Masukkan kata sandi salah. 4. Klik "Masuk".	- Pesan error "Email atau kata sandi salah". - Tetap di halaman login.	
Mengubah profil Admin Cabang (misal: nama)	1. Login sebagai Admin Cabang. 2. Navigasi ke "Profil Saya". 3. Klik "Edit Profil". 4. Ubah nama. 5. Klik "Simpan".	- Nama Admin Cabang berhasil diperbarui.	
Fitur Pengelolaan Operasional Cabang			
Mengedit semua informasi detail cabang yang diizinkan	1. Login. 2. Ke "Pengaturan data Cabang". 3. Ubah nama, alamat, telepon, email, deskripsi, jam check-in/out, gambar utama. 4. Klik "Simpan".	- Semua perubahan berhasil disimpan dan ditampilkan dengan benar.	

Menambah tipe pod baru dengan semua field valid	1. Login. 2. Ke "Menu jenis pod" 3. Klik "Tambah Baru". 4. Isi nama, deskripsi, harga, kapasitas (dewasa, anak), gambar, fasilitas terkait, status breakfast. 5. Klik "Simpan".	- Tipe pod baru berhasil ditambahkan dengan semua detailnya.	
Menghapus tipe pod yang masih memiliki pod aktif	1. Login. 2. Ke "Manajemen Pod" - > "Tipe Pod". 3. Pilih tipe pod yang memiliki pod terdaftar di bawahnya. 4. Klik "Hapus". 5. Konfirmasi.	- Pesan error "Tidak dapat menghapus tipe pod karena masih ada pod yang terhubung" atau sejenisnya. - Tipe pod tidak terhapus.	
Menambah pod baru dengan device code IoT unik	1. Login. 2. Ke "Manajemen Pod" - > "pilih jenis pod yang pod nya ingin ditambahkan". 3. Klik "Tambah Pod". 4. Isi nama/nomor pod, pilih tipe pod, masukkan device code IoT yang unik. 5. Klik "Simpan".	- Pod baru berhasil ditambahkan. - Status awal "Tersedia".	
Menambah pod baru dengan device code IoT yang sudah ada	1. Login. 2. Ke "Manajemen Pod" - > "Daftar Pod". 3. Klik "Tambah Pod". 4. Masukkan device code IoT yang sudah digunakan pod lain. 5. Klik "Simpan".	- Pesan error "Device code sudah digunakan". - Penambahan pod gagal.	

Mengubah status pod menjadi "Perlu Dibersihkan" secara manual	1. Login. 2. Ke "Manajemen Pod" -> "Daftar Pod". 3. Pilih pod yang "Tersedia", klik "Edit". 4. Ubah status ke "Perlu Dibersihkan". 5. Klik "Simpan".	- Status pod berhasil diubah.	
Membuat voucher diskon nominal untuk cabang	1. Login. 2. Ke "Manajemen Voucher Cabang". 3. Klik "Tambah Voucher". 4. Isi kode, nama, deskripsi, tipe "NOMINAL", jumlah diskon, min transaksi, periode berlaku, maks penggunaan. 5. Klik "Simpan".	- Voucher cabang berhasil dibuat.	
Membuat voucher diskon persentase dengan batas maks diskon	1. Login. 2. Ke "Manajemen Voucher Cabang". 3. Klik "Tambah Voucher". 4. Isi kode, nama, tipe "PERSENTASE", nilai persentase, diskon maksimal, min transaksi, periode, kuota. 5. Klik "Simpan".	- Voucher cabang berhasil dibuat.	

Menetapkan harga dinamis untuk akhir pekan	1. Login. 2. Ke "Harga Dinamis". 3. Pilih tipe pod. 4. Pilih rentang tanggal mencakup beberapa akhir pekan. 5. Masukkan harga lebih tinggi. 6. Klik "Simpan".	- Harga dinamis untuk akhir pekan berhasil disimpan.	
Menetapkan harga dinamis yang tumpang tindih dengan aturan lain	1. Login. 2. Ke "Harga Dinamis". 3. Buat aturan harga dinamis baru yang tanggalnya tumpang tindih dengan aturan yang sudah ada untuk tipe pod yang sama.	- Sistem memberikan peringatan bahwa aturan harga ada yang tumpang tindih dan aturan harga baru tidak berhasil disimman	
Fitur Manajemen Reservasi dan Tamu Cabang			
Input manual transaksi OTA dengan semua field lengkap	1. Login. 2. Ke "Transaksi OTA". 3. Klik "Tambah". 4. Isi Jenis OTA, ID Order OTA, nama tamu jika ada, etanggal check-in/out, tipe pod, jumlah tamu, dan harga di OTA 5. Klik "Simpan".	- Transaksi OTA berhasil dicatat.	
Input manual transaksi OTA dengan field wajib kosong	1. Login. 2. Ke "Transaksi OTA". 3. Klik "Tambah". 4. Kosongkan salah satu field wajib (misal: ID Order OTA). 5. Coba klik "Simpan".	- Tombol simpan tidak bisa diklik karena masih ada field wajib yang kosong	

Memindahkan tamu ke pod lain karena ada masalah di pod asal	1. Login. 2. Cari booking tamu aktif. 3. Pilih "Pindah Pod", berikan alasan (misal: AC rusak). 4. Pilih pod lain yang kosong, tipe sama. 5. Konfirmasi.	- Tamu berhasil dipindahkan. - Pod asal mungkin statusnya berubah jadi "Maintenance".	
Mencatat pembersihan pod dengan unggah foto bukti	1. Login. 2. Ke "Log Kebersihan" atau daftar pod. 3. Pilih pod "Perlu Dibersihkan". 4. Klik "Sudah Dibersihkan". 5. Unggah foto bukti kebersihan. 6. (Jika ada) Tambahkan catatan. 7. Simpan.	- Status pod "Tersedia". - Foto bukti dan catatan tersimpan di log.	
Fitur Manajemen Ulasan Cabang			
Menyetujui ulasan pengguna yang konstruktif	1. Login. 2. Ke "Manajemen Ulasan". 3. Pilih ulasan "Pending" yang isinya baik. 4. Klik "Setujui/Tampilkan".	- Ulasan tampil di halaman publik.	
Menolak ulasan pengguna yang mengandung spam/kata kasar	1. Login. 2. Ke "Manajemen Ulasan". 3. Pilih ulasan yang berisi spam. 4. Klik "Tolak/Sembunyikan".	- Ulasan tidak tampil dan statusnya diperbarui.	

Tabel 3.3 Skenario Pengujian Black Box (SuperAdmin)

Skenario Pengujian	Langkah Pengujian	Hasil yang diharapkan	Kesesuaian Hasil
Fitur Login & Dashboard Global SuperAdmin			
Login SuperAdmin berhasil	1. Buka halaman login admin. 2. Masukkan email SuperAdmin. 3. Masukkan kata sandi yang benar. 4. Klik "Masuk".	- Login berhasil. - Diarahkan ke dashboard SuperAdmin.	
Melihat statistik global di dashboard SuperAdmin	1. Login sebagai SuperAdmin. 2. Amati informasi di dashboard (total cabang, total pengguna, total pendapatan global, dll.).	- Menampilkan data statistik agregat dari semua cabang yang akurat.	
Fitur Manajemen Data Master Global			
Menambah data cabang (hotel) baru	1. Login sebagai SuperAdmin. 2. Navigasi ke "Manajemen Cabang/Hotel". 3. Klik "Tambah Cabang Baru". 4. Isi semua field yang diperlukan (nama, alamat, kota, kontak, slug unik, dll.). 5. Klik "Simpan".	- Cabang baru berhasil ditambahkan ke sistem. - Cabang muncul di daftar cabang.	

Mengedit data cabang (hotel) yang sudah ada	1. Login sebagai SuperAdmin. 2. Navigasi ke "Manajemen Cabang/Hotel". 3. Pilih cabang, klik "Edit". 4. Ubah salah satu field (misal: email kontak). 5. Klik "Simpan".	- Data cabang berhasil diperbarui.	
Menambah data kota baru	1. Login sebagai SuperAdmin. 2. Navigasi ke "Manajemen Kota". 3. Klik "Tambah Kota". 4. Masukkan nama kota. 5. Klik "Simpan".	- Kota baru berhasil ditambahkan. - Kota dapat dipilih saat menambah/mengedit cabang.	
Menambah akun Admin Cabang baru dan mengassign ke cabang	1. Login sebagai SuperAdmin. 2. Navigasi ke "Manajemen Staff". 3. Klik "Tambah Staff". 4. Isi detail staff (nama, email, password), pilih peran "Admin Cabang". 5. Assign ke satu atau beberapa cabang. 6. Klik "Simpan".	- Akun Admin Cabang baru berhasil dibuat. - Admin Cabang tersebut dapat login dan mengelola cabang yang di-assign.	
Menghapus akun staff (Admin Cabang)	1. Login sebagai SuperAdmin. 2. Navigasi ke "Manajemen Staff". 3. Cari akun Admin Cabang. 4. Klik opsi "Hapus".	- Akun Admin Cabang berhasil dihapus. - Admin Cabang tersebut tidak dapat login.	

Menambah fasilitas global baru	1. Login sebagai SuperAdmin. 2. Navigasi ke "Manajemen Fasilitas". 3. Klik "Tambah Fasilitas". 4. Masukkan nama fasilitas dan unggah ikon. 5. Klik "Simpan".	- Fasilitas baru berhasil ditambahkan. - Fasilitas dapat di-assign ke hotel atau tipe pod.	
Membuat voucher global untuk semua cabang	1. Login. 2. Ke "Manajemen Voucher". 3. Klik "Tambah Voucher". 4. Isi detail (kode, nama, tipe, nilai, periode, kuota). 5. Pilih opsi "Berlaku di semua cabang". 6. Klik "Simpan".	- Voucher global berhasil dibuat. - Dapat digunakan pengguna di semua cabang.	
Fitur Verifikasi & Approval Sistem			
Menyetujui permintaan verifikasi KYC pengguna yang valid	1. Login sebagai SuperAdmin. 2. Navigasi ke "Verifikasi KYC". 3. Pilih permintaan KYC yang berstatus "Pending" dengan data dan foto valid. 4. Klik "Setujui".	- Status KYC pengguna berubah menjadi "Approved". - Pengguna mendapatkan pembaharuan. - Pengguna dapat mengakses fitur yang memerlukan KYC.	
Menolak permintaan verifikasi KYC pengguna (misal: foto buram)	1. Login sebagai SuperAdmin. 2. Navigasi ke "Verifikasi KYC". 3. Pilih permintaan KYC yang fotonya buram. 4. Klik "Tolak". 5. Masukkan alasan penolakan (misal: "Foto tidak jelas").	- Status KYC pengguna berubah menjadi "Rejected". - Pengguna mendapatkan pembaharuan beserta alasan penolakan.	

Menyetujui permintaan pencairan saldo referral yang valid	1. Login sebagai SuperAdmin. 2. Navigasi ke "Permintaan Payout Referral". 3. Pilih permintaan yang valid (saldo pengguna cukup, data bank benar). 4. Klik "Setujui". 5. Proses transfer dana dan unggah buktinya.	- Status permintaan payout berubah menjadi "Approved". - Pengguna mendapatkan pembaharuan.	
Menolak permintaan pencairan saldo referral (saldo kurang)	1. Login sebagai SuperAdmin. 2. Navigasi ke "Permintaan Payout Referral". 3. Pilih permintaan dimana saldo aktual pengguna kurang dari yang diajukan. 4. Klik "Tolak". 5. Masukkan alasan.	- Status permintaan payout berubah menjadi "Rejected". - Pengguna mendapatkan pembaharuan.	

B. User Acceptance Testing

Pengujian penerimaan pengguna dilakukan untuk memastikan bahwa sistem memenuhi kebutuhan dan harapan pengguna. Pengujian ini melibatkan pengguna akhir yang akan menggunakan sistem Tamu.id untuk melakukan berbagai aktivitas seperti pemesanan tiket, checkin, checkout, dan lain-lain. Tujuan dari pengujian ini adalah untuk memastikan bahwa sistem mudah digunakan, intuitif, dan memenuhi kebutuhan pengguna. Penilaian dilakukan berdasarkan lima parameter dalam skala *Likert* yaitu Sangat Tidak Setuju (STS), Tidak Setuju (TS), Netral (N), Setuju (S), dan Sangat Setuju (SS), pengujian ini dilakukan untuk tiga peran pengguna yaitu pada tabel 3.4 untuk pengguna, tabel 3.5 untuk admin cabang, dan tabel 4.7 untuk superadmin. Pengujian ini dilakukan untuk memastikan bahwa sistem mudah digunakan, intuitif, dan memenuhi kebutuhan pengguna.

Tabel 3.4 Kuesioner *User Acceptance Test* - Perspektif *User*

No.	Pertanyaan	Jawaban				
		STS	TS	N	S	SS
1.	Apakah proses pendaftaran akun dan login ke aplikasi Tamu.id mudah dipahami dan dilakukan?					
2.	Apakah tampilan antarmuka aplikasi Tamu.id menarik dan mudah digunakan secara umum?					
3.	Apakah informasi mengenai cabang dan tipe pod (harga, fasilitas, ketersediaan) disajikan dengan jelas?					
4.	Apakah alur proses pencarian dan pemesanan pod mudah diikuti dan efisien?					
5.	Apakah proses pembayaran saat melakukan pemesanan mudah dan terasa aman?					
6.	Apakah proses check-in dan check-out melalui aplikasi berjalan dengan lancar?					
7.	Apakah penggunaan QR Code untuk akses pod (jika ada fitur ini) mudah dan praktis?					
8.	Apakah saya mudah menemukan riwayat pemesanan dan detail transaksi saya?					
9.	Apakah fitur untuk memberikan ulasan dan rating setelah menginap mudah digunakan?					
10.	Secara keseluruhan, apakah Anda puas dengan fungsionalitas dan kemudahan penggunaan aplikasi Tamu.id?					
Total Bobot						
Skor						
Total Skor						

Tabel 3.5 Kuesioner UAT - Perspektif Admin Cabang

No.	Pertanyaan	Jawaban				
		STS	TS	N	S	SS
1.	Apakah proses login ke dashboard Admin Cabang mudah dan aman?					

2.	Apakah tampilan dashboard Admin Cabang informatif untuk memantau operasional harian cabang?					
3.	Apakah pengelolaan informasi umum cabang (seperti detail kontak, foto, atau deskripsi) mudah dilakukan?					
4.	Apakah fitur untuk menambah dan mengelola tipe pod serta unit pod individual di cabang saya mudah digunakan?					
5.	Apakah proses untuk mengatur harga pod (termasuk harga dinamis jika digunakan) cukup jelas dan mudah diterapkan?					
6.	Apakah pengelolaan data reservasi tamu (melihat daftar booking, detail, dan status) efisien?					
7.	Apakah fitur untuk menginput atau mengelola data reservasi yang berasal dari Online Travel Agent (OTA) mudah dioperasikan?					
8.	Apakah proses untuk mengelola status kebersihan dan ketersediaan pod (misalnya, dari "Perlu Dibersihkan" menjadi "Tersedia") sederhana?					
9.	Apakah fitur untuk melihat dan menanggapi (memoderasi) ulasan dari pengguna untuk cabang saya mudah digunakan?					
10.	Secara keseluruhan, apakah dashboard Admin Cabang ini efektif membantu saya dalam mengelola operasional cabang sehari-hari?					
Total Bobot						
Skor						
Total Skor						

Tabel 3.6 Kuesioner UAT - Perspektif SuperAdmin

No.	Pertanyaan	Jawaban				
		STS	TS	N	S	SS
1.	Apakah proses login ke dashboard SuperAdmin mudah dan aman?					
2.	Apakah tampilan dashboard global SuperAdmin menyajikan informasi ringkasan sistem secara efektif dan mudah dipahami?					
3.	Apakah pengelolaan data master sistem (seperti daftar Cabang/Hotel, Kota, OTA global) mudah dilakukan?					
4.	Apakah fitur untuk menambah, mengedit, dan mengelola akun staf (Admin Cabang, SuperAdmin lain) serta hak aksesnya intuitif?					
5.	Apakah proses verifikasi identitas pengguna (KYC) dari sisi SuperAdmin berjalan dengan alur yang jelas dan efisien?					
6.	Apakah proses approval atau penolakan permintaan pencairan saldo referral pengguna mudah dikelola dan dilacak?					
7.	Apakah fitur untuk membuat dan mengelola voucher yang berlaku secara global atau untuk banyak cabang mudah digunakan?					
8.	Apakah sistem SuperAdmin menyediakan alat yang memadai untuk memantau aktivitas penting dan kesehatan sistem secara keseluruhan?					
9.	Apakah laporan global atau data analitik yang dihasilkan sistem (jika ada) membantu dalam analisis dan pengambilan keputusan strategis?					
10.	Secara keseluruhan, apakah antarmuka SuperAdmin efektif dalam membantu saya menjalankan tugas administrasi sistem dan pengawasan global?					
Total Bobot						

Skor					
Total Skor					

BAB IV

HASIL DAN PEMBAHASAN

4.1 Hasil dan Pembahasan

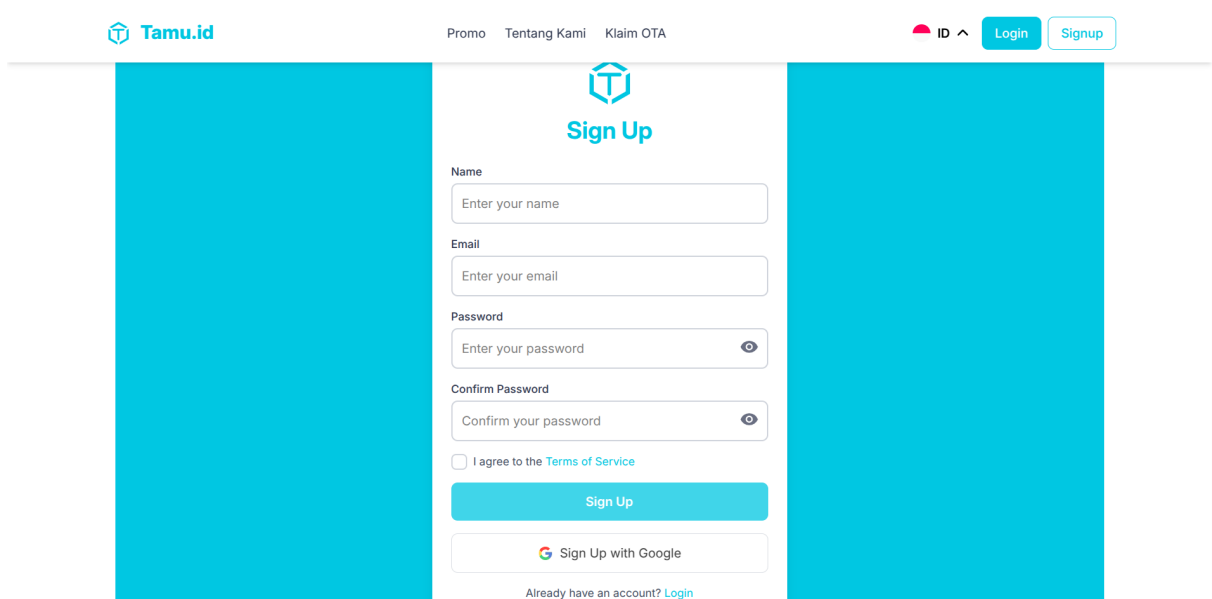
Dari hasil analisis yang telah dilakukan maka selanjutnya adalah mengimplementasikan sistem yang telah dirancang, berikut adalah hasil implementasi sistem yang telah dibuat.

4.1.1 Halaman User

Halaman user adalah halaman yang ditujukan untuk pengguna yang ingin melakukan reservasi dan menikmati layanan yang tersedia di Tamu.id.

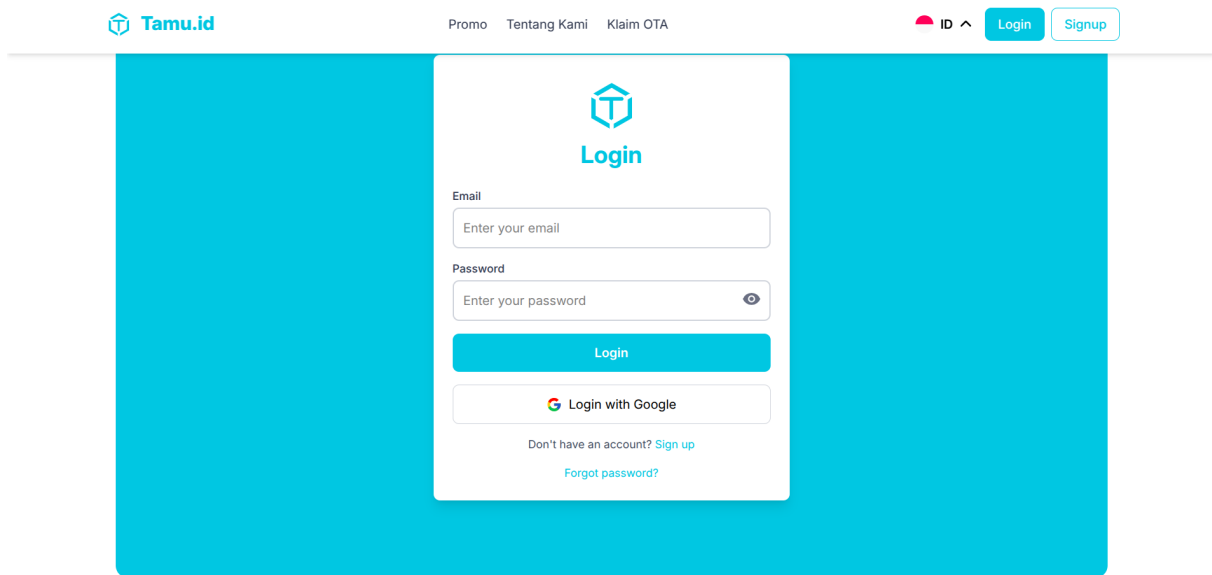
A. Registrasi, Login dan kelola Data User

Pada halaman pendaftaran user, pengguna dapat melakukan pendaftaran dengan dua metode yaitu dengan menggunakan email atau menggunakan akun google langsung yang membuatnya lebih instan dan mudah. Tampilan halaman pendaftaran user dapat dilihat pada Gambar 4.1.



Gambar 4.1 Tampilan Halaman Pendaftaran User

Setelah pengguna melakukan pendaftaran, pengguna dapat melakukan login ke dalam aplikasi Tamu.id. Tampilan halaman login user dapat dilihat pada Gambar 4.2.



Gambar 4.2 Tampilan Halaman Login User

Pada kode 4.1 berikut adalah implementasi frontend untuk halaman login dan pendaftaran user. Kode ini mengatur tampilan dan interaksi pengguna untuk proses login dan pendaftaran.

```

1 <div class="space-y-4">
2   <div>
3     <label for="email" class="block text-sm font-medium text-gray-700">
4       Email</label>
5     >
6     <input
7       id="email"
8       v-model="email"
9       type="email"
10      placeholder="Enter your email"
11      class="w-full p-3 border-2 border-gray-300 rounded-lg focus:outline-none mt-1"
12      required
13    />
14  </div>
15  <div>
16    <label for="password" class="block text-sm font-medium text-gray-700">
17      Password</label>
18    >
19    <div class="relative">
20      <input
21        id="password"
22        :type="showPassword ? 'text' : 'password'"
23        v-model="password"
24        placeholder="Enter your password"
25        class="w-full p-3 border-2 border-gray-300 rounded-lg focus:outline-none pr-10 mt-1"
26        required
27      />
28      <button
29        @click="showPassword = !showPassword"
30        class="absolute right-3 top-1/2 -translate-y-1/2 text-gray-500 hover:text-gray-700 cursor-pointer"
31      >

```

```

32         <material-symbols-visibility
33         v-if="!showPassword"
34         width="24"
35         height="24"
36         />
37         <material-symbols-visibility-off v-else width="24" height="24" />
38     </button>
39 </div>
40 </div>
41 <button
42     @click="login"
43     class="w-full cursor-pointer bg-primary text-white p-3 rounded-lg hover:bg-primary-
44         dark transition duration-200 flex items-center justify-center font-semibold"
45     :disabled="loading"
46 >
47     <span
48         v-if="loading"
49         class="animate-spin border-2 border-white border-t-transparent rounded-full w-5 h-5 mr
50         -2"
51     ></span>
52     {{ loading ? "Logging in..." : "Login" }}
53 </button>
54 <button
55     @click="gotoGoogleLogin"
56     class="w-full flex items-center justify-center space-x-2 border border-gray-300 p-3
57         rounded-lg hover:bg-gray-100 transition duration-200 cursor-pointer"
58 >
59     <flat-color-icons-google class="w-5 h-5" />
60     <span>Login with Google</span>
61 </button>
62 </div>

```

Kode 4.1: Implementasi Frontend Login User

Salah satu fitur utama yaitu integrasi dengan OAuth Google yang memungkinkan pengguna untuk masuk ke aplikasi menggunakan akun Google mereka. Hal ini memberikan kemudahan bagi pengguna yang tidak ingin mengingat password tambahan. Implementasi fitur ini dapat dilihat pada kode 4.2.

```

1 def google_login():
2     try:
3         GOOGLE_CLIENT_ID = os.getenv("GOOGLE_CLIENT_ID")
4         GOOGLE_REDIRECT_URI = os.getenv("GOOGLE_USER_REDIRECT_URI")
5         google_auth_url = (
6             "https://accounts.google.com/o/oauth2/auth"
7             "?response_type=code"
8             f"&client_id={GOOGLE_CLIENT_ID}"
9             f"&redirect_uri={GOOGLE_REDIRECT_URI}"
10            "&scope=openid%20email%20profile"
11        )
12        return redirect(google_auth_url)
13    except Exception as e:
14        logging.error(f"Error in google_login: {e}")
15        return jsonify({"message": "Internal server error"}), 500
16
17 def google_callback():
18     try:

```

```

19     code = request.args.get("code")
20     if not code:
21         return jsonify({"message": "Code not provided"}), 400
22     # change code with access token
23     token_url = "https://oauth2.googleapis.com/token"
24     GOOGLE_CLIENT_ID = os.getenv("GOOGLE_CLIENT_ID")
25     GOOGLE_CLIENT_SECRET = os.getenv("GOOGLE_CLIENT_SECRET")
26     GOOGLE_REDIRECT_URI = os.getenv("GOOGLE_USER_REDIRECT_URI")
27     token_data = {
28         "code": code,
29         "client_id": GOOGLE_CLIENT_ID,
30         "client_secret": GOOGLE_CLIENT_SECRET,
31         "redirect_uri": GOOGLE_REDIRECT_URI,
32         "grant_type": "authorization_code"
33     }
34     token_response = requests.post(token_url, data=token_data)
35     token_data = token_response.json()
36     access_token = token_data.get("access_token")
37     if not access_token:
38         return jsonify({"message": "Invalid code"}), 400
39
40     # get user info
41     user_info_res = requests.get(
42         "https://www.googleapis.com/oauth2/v1/userinfo",
43         headers={"Authorization": f"Bearer {access_token}"},
44     )
45     user_info = user_info_res.json()
46     email = user_info.get("email")
47
48     user = User.query.filter_by(email=email).first()
49     token = str(uuid.uuid4())
50     new_token = Token(token=token, email=email)
51     db.session.add(new_token)
52     db.session.commit()
53     return redirect(f"{os.getenv('FRONTEND')}/sso?token={token}")
54 except Exception as e:
55     logging.error(f"Error in google_callback: {e}")
56     return jsonify({"message": "Internal server error"}), 500

```

Kode 4.2: Implementasi Integrasi *Oauth2 Google* pada Sistem Autentikasi Pengguna

Gambar 4.3 Tampilan Halaman Profil User

Setelah pengguna berhasil melakukan login, mereka dapat mengelola data pribadi mereka seperti pada gambar 4.3.

B. Landing Page dan Proses Reservasi User

Landing page adalah halaman utama yang ditampilkan kepada pengguna dalam sistem. Halaman ini memberikan informasi umum tentang layanan yang tersedia, termasuk fitur-fitur utama, manfaat, dan cara menggunakan aplikasi.

Gambar 4.4 Tampilan Halaman Landing Page User

Pada gambar 4.4, terlihat bahwa halaman ini dirancang untuk memberikan pengalaman pengguna yang intuitif dan menarik. Halaman ini dapat diakses oleh pengguna yang telah

melakukan login maupun yang belum, sehingga memberikan fleksibilitas dalam penggunaan aplikasi.

```
1
2 <div>
3   <Hero />
4   <Feature />
5   <Flow />
6   <Gallery />
7   <Faq />
8   <Map />
9   <Testimonial />
10 </div>
11
```

Kode 4.3: Implementasi *Landing Page User*

Pada kode 4.3, implementasi frontend untuk landing page user ditampilkan. Dapat terlihat dengan menggunakan Vue.js kita dapat memecah halaman menjadi komponen-komponen yang lebih kecil, sehingga memudahkan pengelolaan dan pengembangan aplikasi.

```
1 <div class="flex flex-col lg:flex-row items-end gap-4">
2   <div class="flex-1 w-full">
3     <label class="block font-semibold text-white">Lokasi</label>
4     <Combobox v-model="selectedBranch">
5       <div class="relative w-full">
6         <ComboboxButton class="w-full">
7           <div
8             class="flex items-center p-3 border rounded-lg bg-white/20 backdrop-blur-md"
9             >
10            <mdi-location class="w-5 h-5 mr-2 text-gray-200" />
11            <ComboboxInput
12              class="w-full outline-none bg-transparent text-white placeholder-gray-300"
13              @change="query = $event.target.value"
14              :display-value="(branch) => branch?.name"
15              placeholder="Pilih Lokasi"
16            />
17          </div>
18        </ComboboxButton>
19        <ComboboxOptions
20          class="absolute w-full bg-white/90 rounded-lg mt-1 shadow-lg z-50 text-black"
21        >
22          <ComboboxOption
23            v-for="branch in filteredBranches"
24            :key="branch.id"
25            :value="branch"
26            class="p-2 hover:bg-gray-200 cursor-pointer flex items-center hover:rounded-lg"
27          >
28            <lucide-hotel class="w-5 h-5 mr-2" />
29            {{ branch.name }}
30          </ComboboxOption>
31        </ComboboxOptions>
32      </div>
33    </Combobox>
34  </div>
35
36  <div class="flex-1 w-full">
37    <label class="block font-semibold text-white">Durasi & Tanggal</label>
```

```

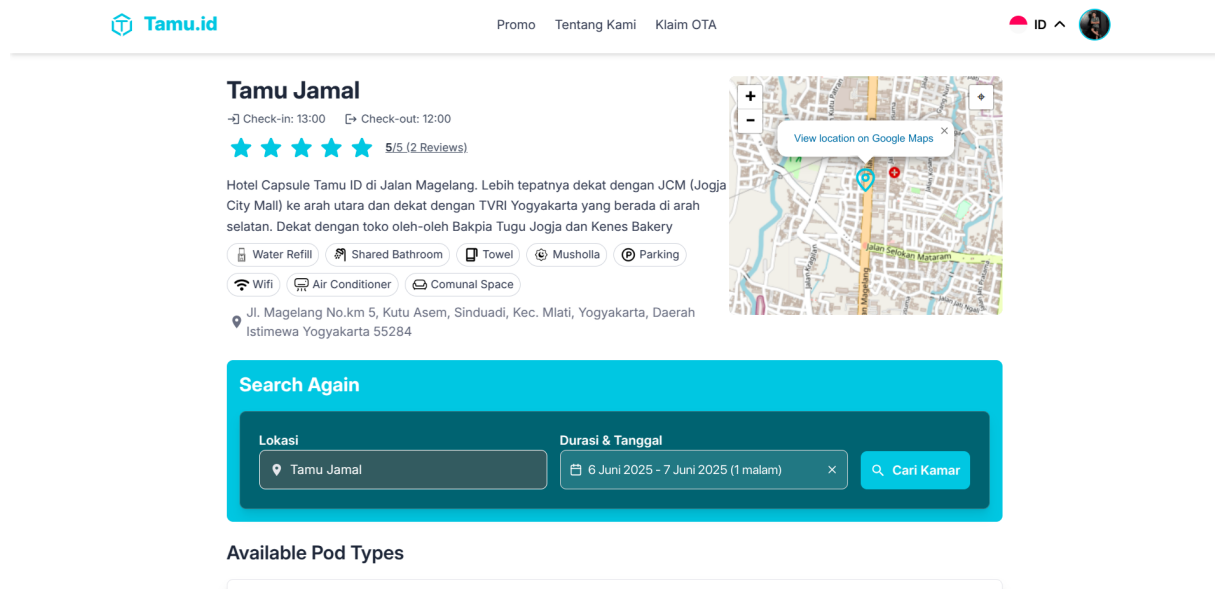
38 <VueDatePicker
39   v-model="dateRange"
40 />
41 </div>
42 <div class="w-full md:w-auto">
43   <button
44     @click="searchRooms"
45     class="w-full md:w-fit bg-primary text-white p-3 rounded-lg cursor-pointer font-
46       semibold flex items-center justify-center"
47   >
48     <mdi-magnify class="w-5 h-5 mr-2" />
49     Cari Kamar
50   </button>
51 </div>

```

Kode 4.4: Implementasi Komponen Pencarian untuk Reservasi

Kode 4.4 menunjukkan implementasi komponen pencarian untuk reservasi. Komponen ini memungkinkan pengguna untuk mencari ketersediaan Pod pada tanggal dan cabang tertentu. Pengguna dapat memasukkan tanggal dan cabang yang diinginkan, dan sistem akan menampilkan daftar Pod yang tersedia untuk reservasi pada tanggal tersebut.

Halaman reservasi adalah salah satu fitur utama dalam aplikasi yang memungkinkan pengguna untuk melakukan reservasi Pod yang tersedia secara online.



Gambar 4.5 Komponen Pencarian User

Pada gambar 4.5, terlihat bahwa komponen pencarian ini dirancang untuk memberikan kemudahan bagi pengguna dalam mencari Pod yang sesuai dengan kebutuhan mereka. Pengguna dapat memilih tanggal dan cabang yang diinginkan

Kode implementasi backend untuk proses pencarian Pod yang tersedia pada tanggal tertentu dapat dilihat pada kode 4.5.

```
1 def count_available_pods(pod_type_id, checkin_date, checkout_date):
2     total_pods = db.session.query(Pod).filter(Pod.pod_type_id == pod_type_id).count()
3     overlapping_bookings = db.session.query(Booking).filter(
4         Booking.pod_type_id == pod_type_id,
5         Booking.status.in_(
6             BookingStatusEnum.PENDING,
7             BookingStatusEnum.BOOKED,
8             BookingStatusEnum.CHECKED_IN,
9         ),
10        db.or_(
11            db.and_(Booking.check_in <= checkin_date, Booking.check_out > checkin_date),
12            db.and_(Booking.check_in >= checkin_date, Booking.check_in < checkout_date)
13        )
14    ).count()
15    return total_pods - overlapping_bookings
```

Kode 4.5: Implementasi Backend Pencarian Pod untuk Reservasi User

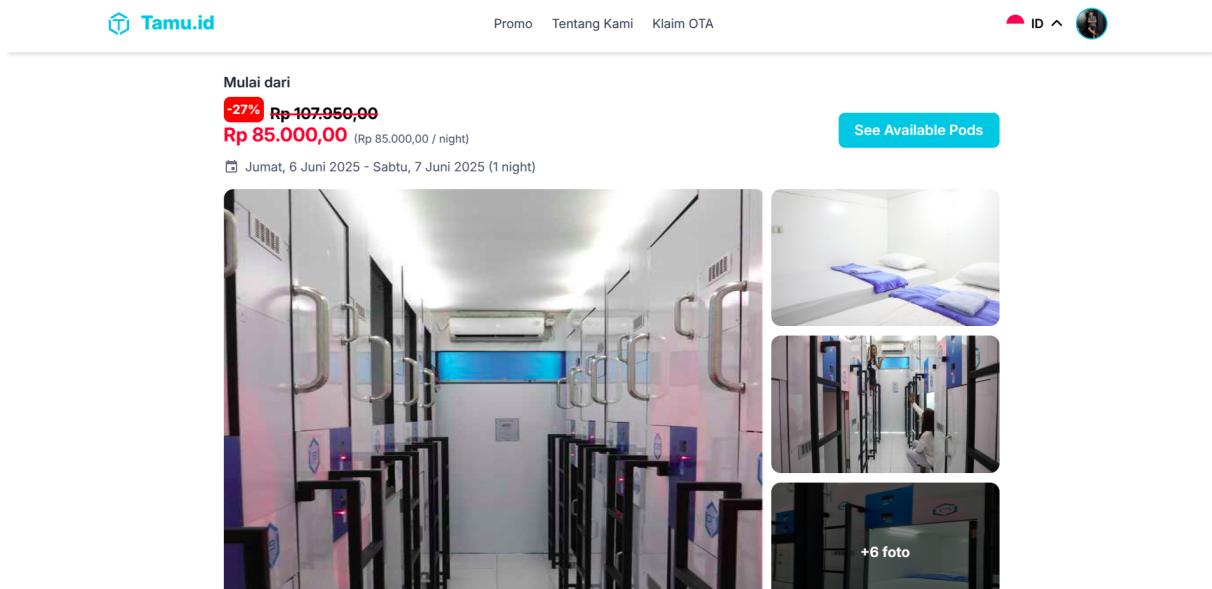
Selanjutnya kode backend akan mengolah permintaan pencarian Pod yang telah dilakukan oleh pengguna. Kode ini akan memeriksa ketersediaan Pod pada tanggal dan cabang yang diminta, dan mengembalikan daftar Pod yang tersedia untuk reservasi, setelah itu data dikirimkan ke frontend untuk ditampilkan kepada pengguna.

```
1 const fetchHotel = async (slug, checkin, checkout) => {
2     try {
3         isLoading.value = true;
4         const response = await axios.get(
5             `${
6                 import.meta.env.VITE_API_URL
7             }/hotel/${slug}?checkin=${checkin}&checkout=${checkout}`
8         );
9
10        hotel.value = response.data.hotel;
11        roomCounts.value = hotel.value.pod_types.reduce((acc, room) => {
12            //based on room.id
13            acc[room.id] = 1; // Set default count to 1 for each room type
14            return acc;
15        }, {});
16        // console.log("hotel", hotel.value.images);
17    } catch (error) {
18        console.error("Error fetching hotel data:", error);
19        router.push({ name: "notfound" });
20        // toast.error("Gagal mengambil data hotel");
21    } finally {
22        isLoading.value = false;
23    }
24 };
```

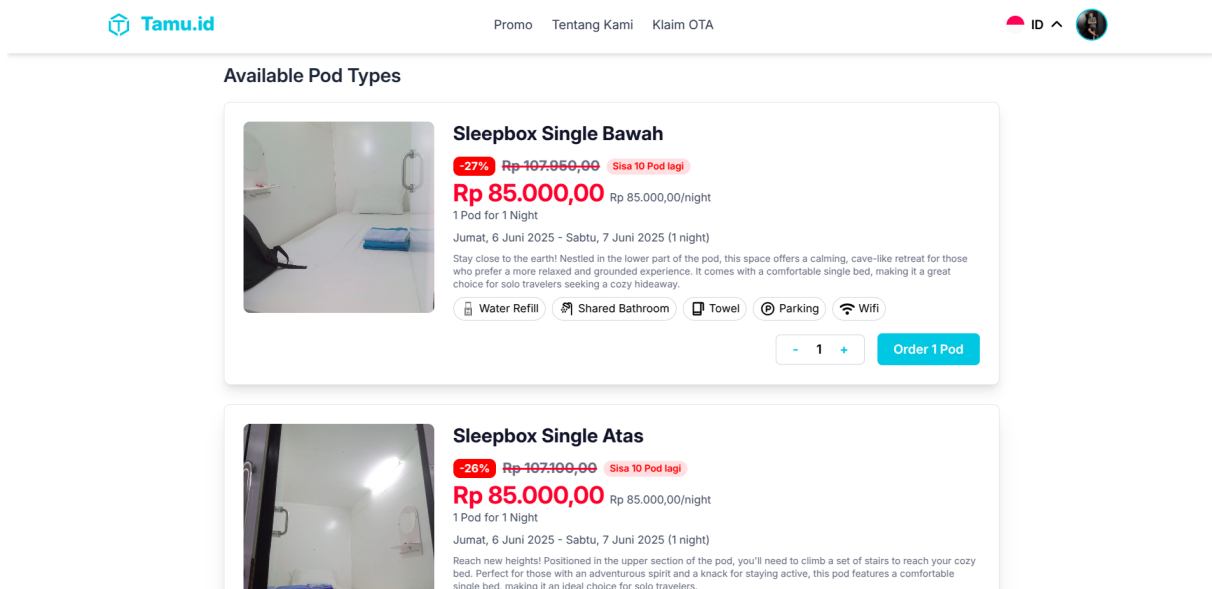
Kode 4.6: Implementasi Tampilan Hasil Pencarian User

Setelah pengguna memasukkan tanggal dan cabang yang diinginkan, sistem akan menampilkan daftar Pod yang tersedia untuk reservasi pada tanggal tersebut. Pada halaman

ini, frontend akan menampilkan detail cabang, tanggal, serta daftar Pod yang dapat dipesan, dengan berinteraksi langsung melalui API yang telah diimplementasikan pada bagian backend sebelumnya. Pengguna dapat memilih Pod yang diinginkan dan melanjutkan ke proses reservasi. Tampilan hasil pencarian ditunjukkan pada gambar 4.6 dan gambar 4.7, sementara implementasi antarmuka frontend untuk halaman ini ditampilkan pada kode 4.4.



Gambar 4.6 Tampilan Hasil Pencarian User



Gambar 4.7 Tampilan Hasil Pencarian User

Setelah pengguna memilih Pod yang diinginkan, maka akan lanjut ke halaman pembayaran yang terdiri dari tiga proses yaitu pada gambar 4.8 merupakan proses pertama pengguna dapat melihat detail reservasi yang dia lakukan.

Tamu.id Promo Tentang Kami Klaim OTA ID

Segera selesaikan pesanan dalam waktu 59:25 [Batalan Pesanan](#)

Reservation Detail

Alamat Pod
Jl. Magelang No.km 5, Kutu Asem, Sinduadi, Kec. Mlati, Yogyakarta, Daerah Istimewa Yogyakarta 55284

Phone
089533185022

Check-in
Rabu, 18 Juni 2025, 12:00

Stay Duration
1 Night

Total Pod
1 Pod

Email
jamal@tamu.id

Check-out
Kamis, 19 Juni 2025, 11:00

Pod Type
Sleepbox Single Bawah

Room Capacity
1 Adult, 0 Children

1 Sleepbox Single Bawah

Harga	Rp 85.000,00
Additional Tax	Rp 0,00
Service Fee	Free
Total	Rp 85.000,00

[Continue to Payment](#)

Customer Details

Name: Alif Arief

Email: alif.arief.m@gmail.com

Gambar 4.8 Tampilan Proses Pembayaran detail Reservasi

Pada gambar 4.9, pengguna dapat memilih metode pembayaran yang diinginkan yaitu QRIS lalu dalam tahap ini pengguna dapat memasukkan kode voucher dan kode referral untuk mendapatkan potongan harga.

Tamu.id Promo Tentang Kami Klaim OTA ID

Segera selesaikan pesanan dalam waktu 58:59 [Batalan Pesanan](#)

Pilih Metode Pembayaran

Metode Pembayaran

QRIS
Scan pakai e-wallet favorit kamu

Terapkan Diskon

Kode Voucher

Masukkan kode promo

[Apply Voucher](#)

✓ alifam ✕

Tamu Jamal

Check-in Rabu, 18 Juni 2025, 12:00

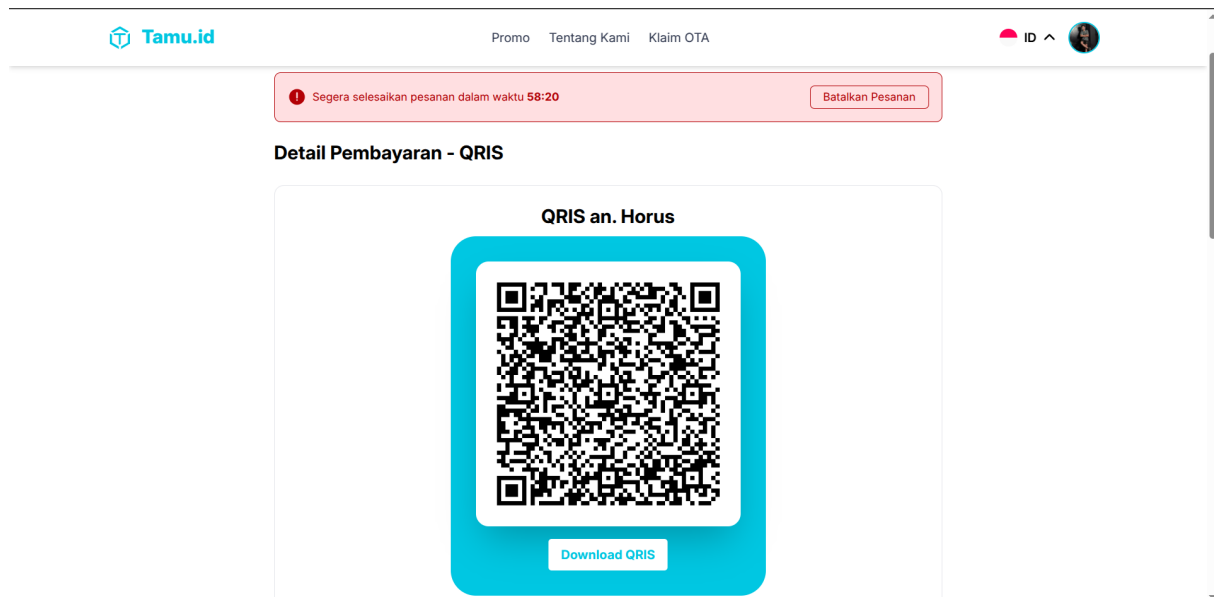
Check-out Kamis, 19 Juni 2025, 11:00

Jumlah Malam 1 Night

Sleepbox Single Bawah (x1)	Rp 85.000,00
Diskon Referral	Rp 5.950,00
Additional Tax	Rp 0,00
Service Fee	Free
Total	Rp 79.050,00

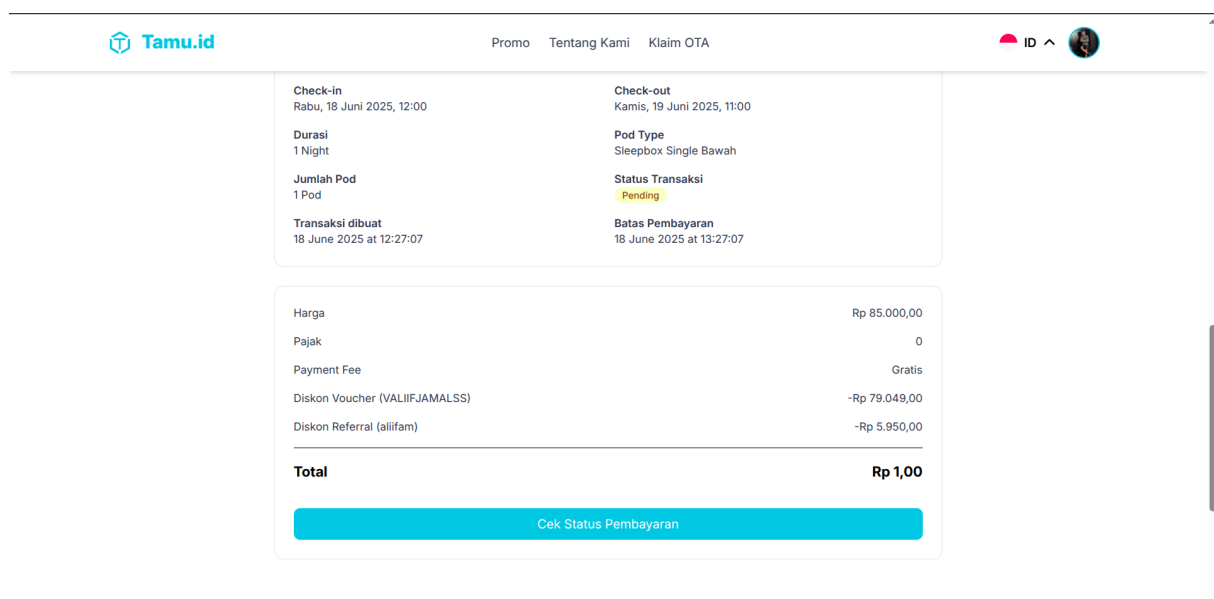
[Bayar Sekarang](#)

Gambar 4.9 Tampilan Proses Pembayaran Metode Pembayaran



Gambar 4.10 Tampilan Proses Pembayaran kode QRIS

Pada gambar 4.10 dan 4.11 setelah itu sistem akan terhubung dengan *payment gateway* dan menampilkan kode QRIS yang dapat di scan oleh pengguna untuk melakukan pembayaran, untuk implementasi integrasinya ada pada kode 4.7.



Gambar 4.11 Tampilan Proses Pembayaran *Pending*

```

1 payload = {
2   "payment_type": payment_type,
3   "transaction_details": {
4     "order_id": transaction.id,
5     "gross_amount": transaction.gross_amount - referral_discount - voucher_discount
6   },
7   "item_details": item_details,
8   "customer_details": {

```

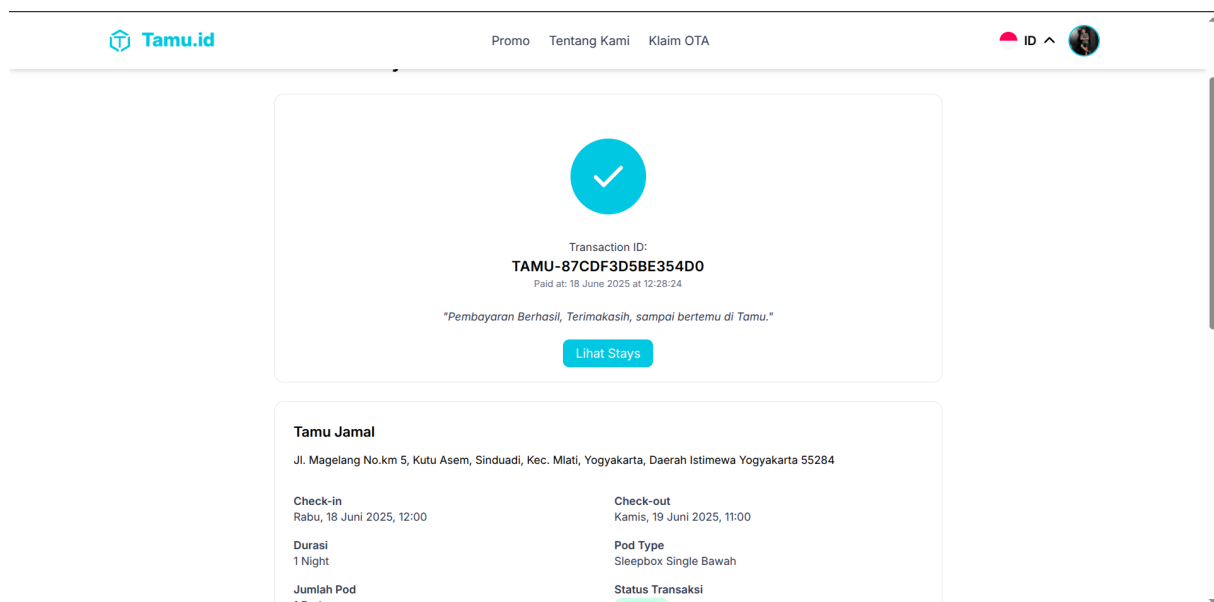
```

9     "first_name": user.name,
10    "email": user.email,
11    "phone": user.phone
12  },
13  "qris": { "acquirer": "gopay" },
14  "custom_expiry": {
15    "order_time": datetime.fromtimestamp(transaction.created_at, tz=timezone(timedelta(
16      hours=7))).strftime('%Y-%m-%d %H:%M:%S %Z'),
17    "expiry_duration": transaction_expired_time,
18    "unit": "minute"
19  }
20 }
21 headers = {
22   "Content-Type": "application/json",
23   "Accept": "application/json",
24   "Authorization": f"Basic {auth_key}",
25   "X-Override-Notification": webhook_url
26 }
27 response = requests.post(f"{midtrans_url}/charge", json=payload, headers=headers)
28 response_data = response.json()

```

Kode 4.7: Implementasi Integrasi Core API QRIS Midtrans Saat Checkout

Setelah pengguna melakukan pembayaran, maka akan muncul halaman konfirmasi bahwa reservasi berhasil dilakukan secara *Realtime*. Pengguna dapat melihat detail reservasi yang telah dilakukan pada gambar 4.12. proses *Realtime* dapat berjalan dengan peran integrasi antara sistem *Backend* dan Midtrans dengan metode *Webhook* yang ada pada kode 4.8



Gambar 4.12 Tampilan Proses Pembayaran *Success*

```

1 payload = {
2   "payment_type": payment_type,
3   "transaction_details": {
4     "order_id": transaction.id,
5     "gross_amount": transaction.gross_amount - referral_discount - voucher_discount
6   },

```

```

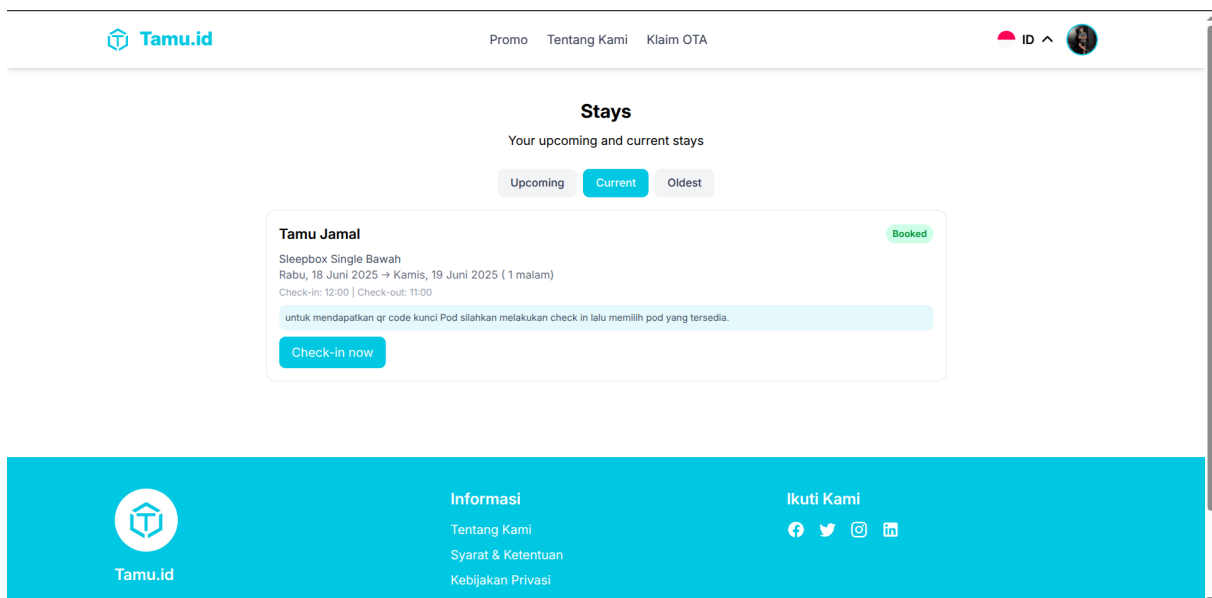
7  "item_details": item_details,
8  "customer_details" : {
9      "first_name": user.name,
10     "email": user.email,
11     "phone": user.phone
12 },
13 "qris": { "acquirer": "gopay" },
14 "custom_expiry": {
15     "order_time": datetime.fromtimestamp(transaction.created_at, tz=timezone(timedelta(
16         hours=7))).strftime('%Y-%m-%d %H:%M:%S %Z'),
17     "expiry_duration": transaction_expired_time,
18     "unit": "minute"
19 }
20 headers = {
21     "Content-Type": "application/json",
22     "Accept": "application/json",
23     "Authorization": f"Basic {auth_key}",
24     "X-Override-Notification": webhook_url
25 }
26 response = requests.post(f"{midtrans_url}/charge", json=payload, headers=headers)
27 response_data = response.json()

```

Kode 4.8: Implementasi Integrasi *Webhook Midtrans* di *Backend*

C. Proses Menginap User

Setelah melakukan reservasi tentunya pengguna akan menginap di jenis Pod yang telah dipesan, berikut proses yang dilakukan oleh pengguna ketika menginap di Pod yang telah dipesan.



Gambar 4.13 Tampilan List Reservasi User

Pada gambar 4.13, pengguna dapat melihat daftar reservasi yang telah dilakukan, terdiri dari tiga status yaitu *upcoming*, *current*, dan *oldest*. Pengguna dapat melakukan check-in pada

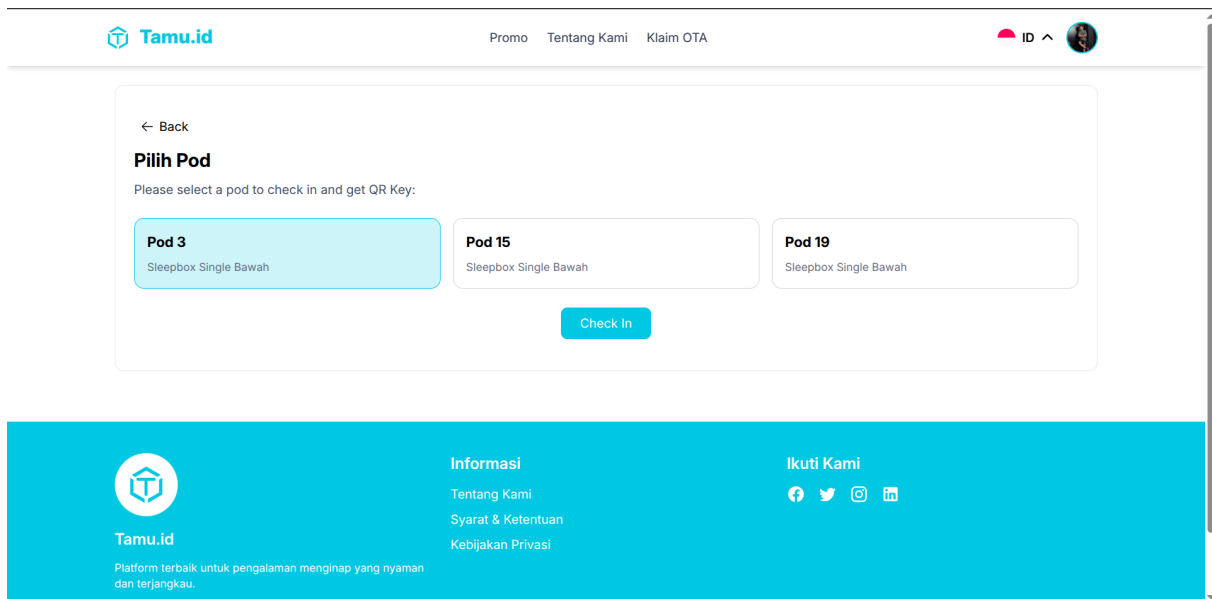
reservasi yang berstatus *current* yaitu reservasi yang waktu check-in sudah tiba. Pengguna dapat melakukan check-in dengan menekan tombol *Check In* pada reservasi yang berstatus *current*. Dapat dilihat pada kode 4.9 yang mendapatkan list stays untuk pengguna dengan cara mengekstrak kode JWT dan melakukan otomasi misal ada yang sudah waktunya checkout maka otomatis akan tercheckout sehingga meminimalkan kesalahan dan proses manual.

```

1 def get_stays():
2     try:
3         user_id = get_jwt_identity()
4         user = User.query.filter_by(id=user_id).first()
5         if not user:
6             return jsonify({"error": "User not found"}), 404
7         automate_booking_lifecycle()
8         stays = Booking.query.options(
9             joinedload(Booking.pod_type).joinedload(PodType.hotel),
10            joinedload(Booking.pod),
11        ).filter(
12            Booking.user_id == user.id,
13            Booking.status.in_([BookingStatusEnum.BOOKED, BookingStatusEnum.CHECKED_IN,
14                               BookingStatusEnum.CHECKED_OUT, BookingStatusEnum.CANCELLED]),
15        ).order_by(Booking.check_in.desc()).all()
16        results = []
17        for stay in stays:
18            results.append({
19                "id": stay.id,
20                "hotel_name": stay.pod_type.hotel.name if stay.pod_type and stay.pod_type.hotel else None,
21                "hotel_check_in": stay.pod_type.hotel.checkin_time.strftime("%H:%M") if stay.pod_type and stay.pod_type.hotel else None,
22                "hotel_check_out": stay.pod_type.hotel.checkout_time.strftime("%H:%M") if stay.pod_type and stay.pod_type.hotel else None,
23                "room_type": stay.pod_type.name if stay.pod_type else None,
24                "room_type_id": stay.pod_type.id if stay.pod_type else None,
25                "check_in": stay.check_in.isoformat() if stay.check_in else None,
26                "check_out": stay.check_out.isoformat() if stay.check_out else None,
27                "night" : (stay.check_out - stay.check_in).days if stay.check_in and stay.check_out else None,
28                "status": stay.status.value,
29                "pod": {
30                    "id": stay.pod.id,
31                    "name": stay.pod.name,
32                } if stay.pod else None,
33            })
34        return jsonify(results), 200
35    except Exception as e:
36        logging.error(str(e), exc_info=True)
37        return jsonify({"error": "Internal server error"}), 500

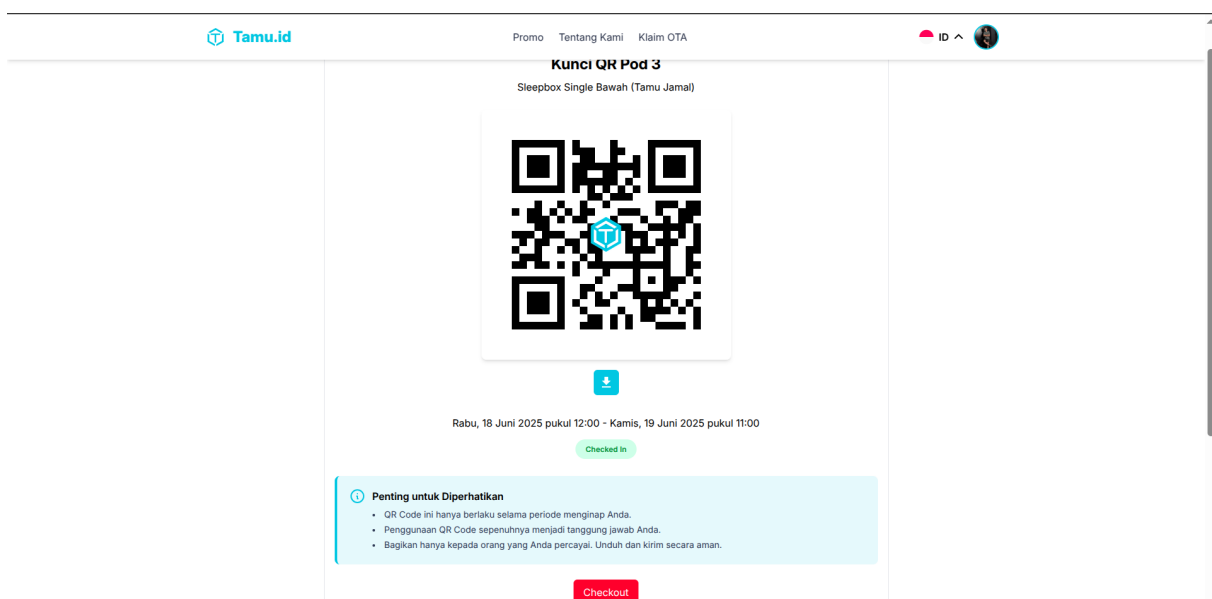
```

Kode 4.9: Implementasi Otomasi *Lifecycle Stays* dan mendapatkan *List Stays*



Gambar 4.14 Tampilan Proses pilih Pod User

Setelah klik tombol *Check In*, pengguna akan diarahkan ke halaman 4.14 untuk memilih Pod yang akan ditempati. Pengguna dapat memilih Pod yang diinginkan dan melanjutkan ke proses check-in.



Gambar 4.15 Tampilan Kunci Pod User

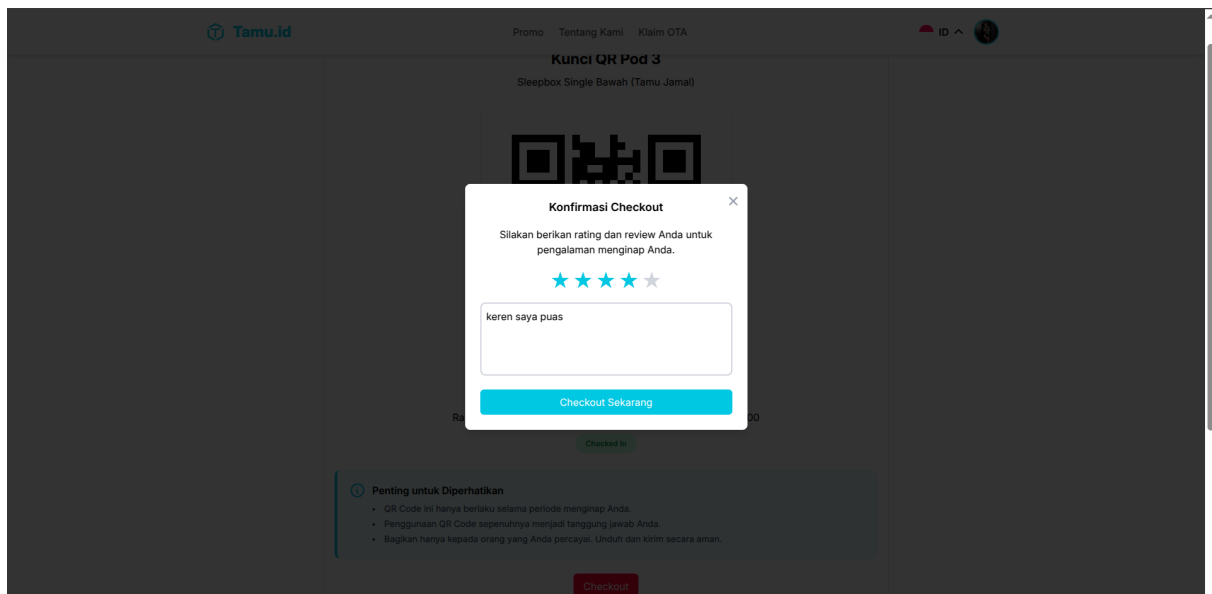
Setelah memilih Pod yang diinginkan, pengguna akan diarahkan ke halaman 4.15 dan mendapatkan kunci Pod yang telah dipilih yang kuncinya merupakan QR Code yang dapat di scan oleh pengguna untuk membuka Pod yang telah dipilih, dapat dilihat pada kode 4.10 merupakan implementasi regenerasi kunci Pod yang mengimplementasikan *Customized Linkedlist* dengan *Unix Timestamp* untuk keamanannya.

```

1 hotel = Hotel.query.filter_by(id=pod.pod_type.hotel_id).first()
2 hotel_timezone = hotel.timezone if hotel and hotel.timezone else "Asia/Jakarta"
3 checkout_datetime = datetime.combine(booking.check_out, hotel.checkout_time)
4 hotel_tz = pytz.timezone(hotel_timezone)
5 localized_checkout = hotel_tz.localize(checkout_datetime)
6 current_expired = int(localized_checkout.timestamp())
7 qrcode = QRCode.query.filter_by(pod_id=pod.id).first()
8 next_code = qr_key_generator()
9 next_code_image = generate_qr(next_code)
10 qrcode.current_code=qrcode.next_code
11 qrcode.current_code_image=qrcode.next_code_image
12 qrcode.next_code=next_code
13 qrcode.next_code_image=next_code_image
14 qrcode.current_expired = current_expired
15 qrcode.booking_id = booking.id
16 booking.status = BookingStatusEnum.CHECKED_IN
17 pod.status = PodStatusEnum.OCCUPIED

```

Kode 4.10: Implementasi Rotasi dan Regenerasi Kunci Pod

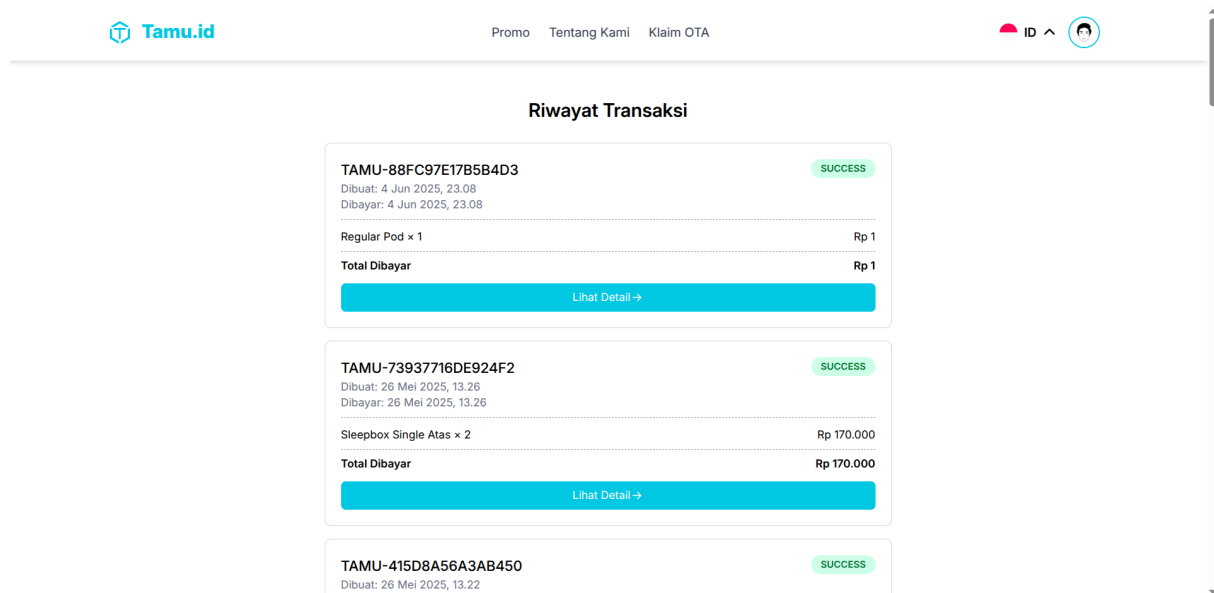


Gambar 4.16 Tampilan Checkout dan Memberikan Ulasan User

Setelah pengguna selesai menginap, maka pengguna dapat melakukan checkout dengan menekan tombol *Checkout* pada gambar 4.16. Setelah itu pengguna dapat memberikan ulasan terhadap Pod yang telah ditempati. Ulasan ini akan membantu pengguna lain dalam memilih Pod yang sesuai dengan kebutuhan mereka.

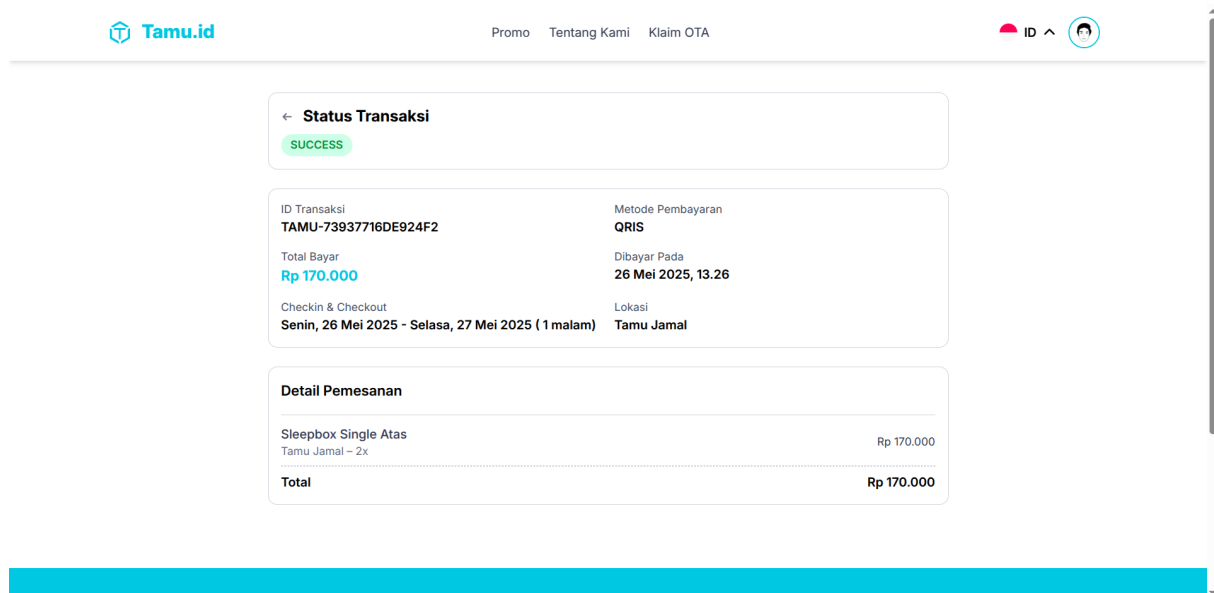
D. Transaksi User

Pada gambar 4.17, pengguna dapat melihat daftar transaksi yang telah dilakukan. Daftar transaksi ini mencakup semua reservasi yang telah dilakukan oleh pengguna, baik yang sedang berlangsung maupun yang telah selesai.



Gambar 4.17 Tampilan list Transaksi User

Lebih detail lagi pada gambar 4.18, pengguna dapat melihat detail dari transaksi yang telah dilakukan. Detail ini mencakup informasi seperti tanggal reservasi, jenis Pod yang dipesan, voucher serta kode referral yang digunakan, dan status pembayaran. Hal ini memberikan transparansi kepada pengguna mengenai transaksi yang telah mereka lakukan.

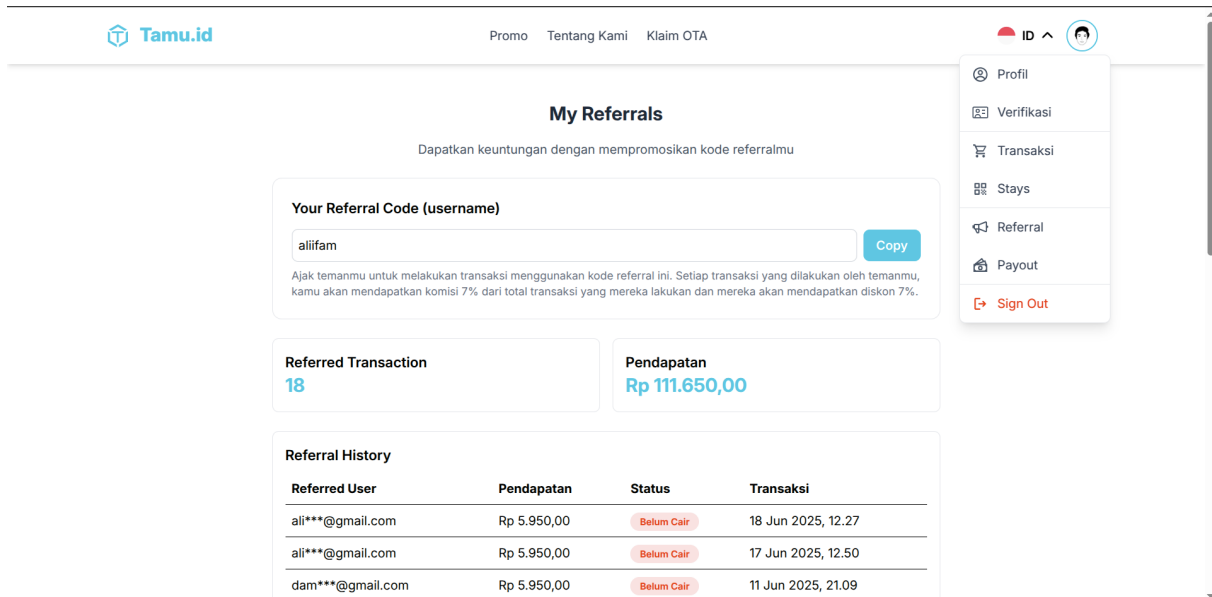


Gambar 4.18 Tampilan detail Transaksi User

E. Sistem Loayalitas User dan Verifikasi Identitas

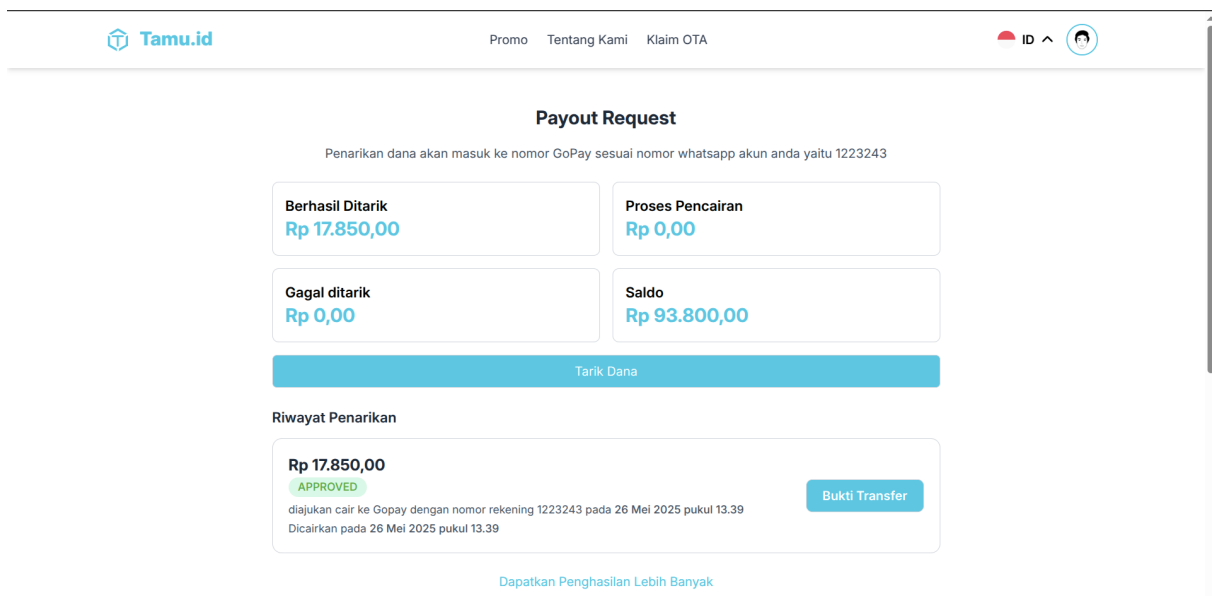
Untuk meningkatkan pengalaman pengguna, Tamu.id juga menyediakan sistem loyalitas yang memungkinkan pengguna untuk mendapatkan uang tambahan dengan mereferralkan kode mereka kepada pengguna lain. Pengguna dapat memasukkan kode referral pada saat melakukan

reservasi, dan jika kode tersebut valid, pengguna yang mereferralkan akan mendapatkan uang tambahan dan yang direferralkan akan mendapatkan potongan harga.



Gambar 4.19 Tampilan Halaman Referral User

Pada gambar 4.19, pengguna dapat melihat kode referral mereka yang dapat dibagikan kepada pengguna lain. Pengguna juga dapat melihat jumlah uang tambahan yang telah didapatkan dari sistem loyalitas ini.



Gambar 4.20 Tampilan Halaman Payout User

Pada gambar 4.21, pengguna dapat melakukan penarikan uang tambahan yang telah didapatkan dari sistem loyalitas. Pengguna dapat memasukkan jumlah uang yang ingin ditarik dan sistem akan memproses permintaan tersebut.

Verifikasi Identitas APPROVED

Verikasi identitasmu lebih awal untuk checkin lebih cepat

Yeay Selamat! Pengajuan verifikasi identitas anda telah disetujui pada 15 Mei 2025 pukul 00.52

Foto Kartu Identitas



Foto Selfie



ID Type

KTP
v

ID Number

3216010409030003

Gambar 4.21 Tampilan Halaman Verifikasi Identitas User

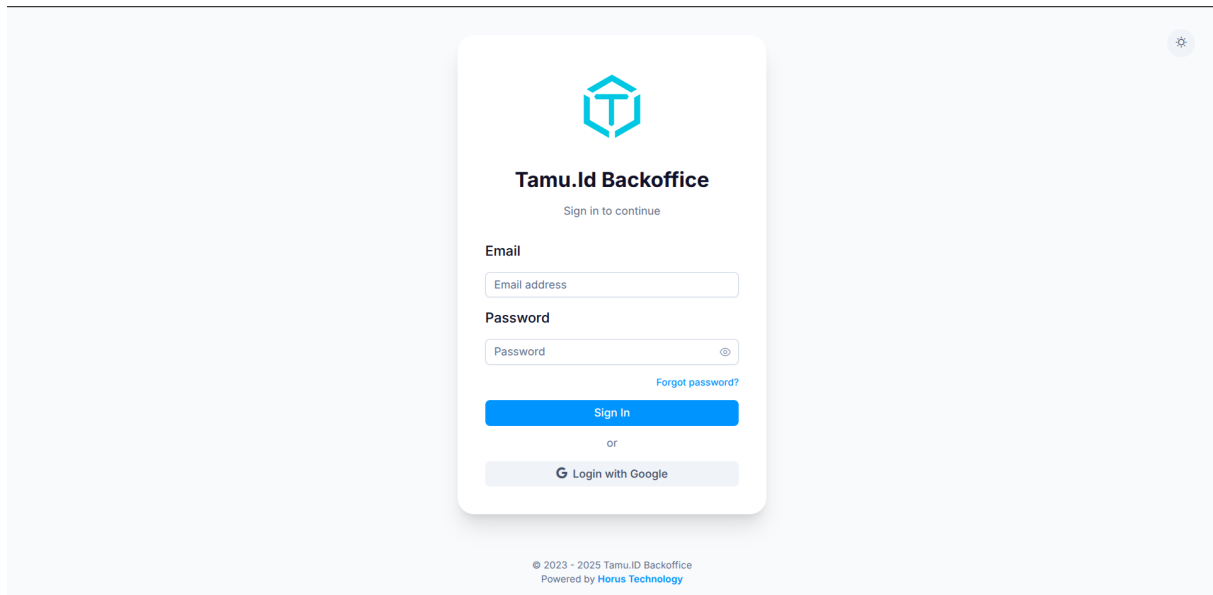
Sebelum mengikuti sistem loyalitas, pengguna harus melakukan verifikasi identitas terlebih dahulu. Hal ini dilakukan untuk memastikan bahwa pengguna yang mengikuti sistem loyalitas adalah pengguna yang sah dan tidak melakukan kecurangan. Pada gambar 4.21, pengguna dapat melihat halaman verifikasi identitas yang harus diisi sebelum mengikuti sistem loyalitas.

4.1.2 Halaman Admin

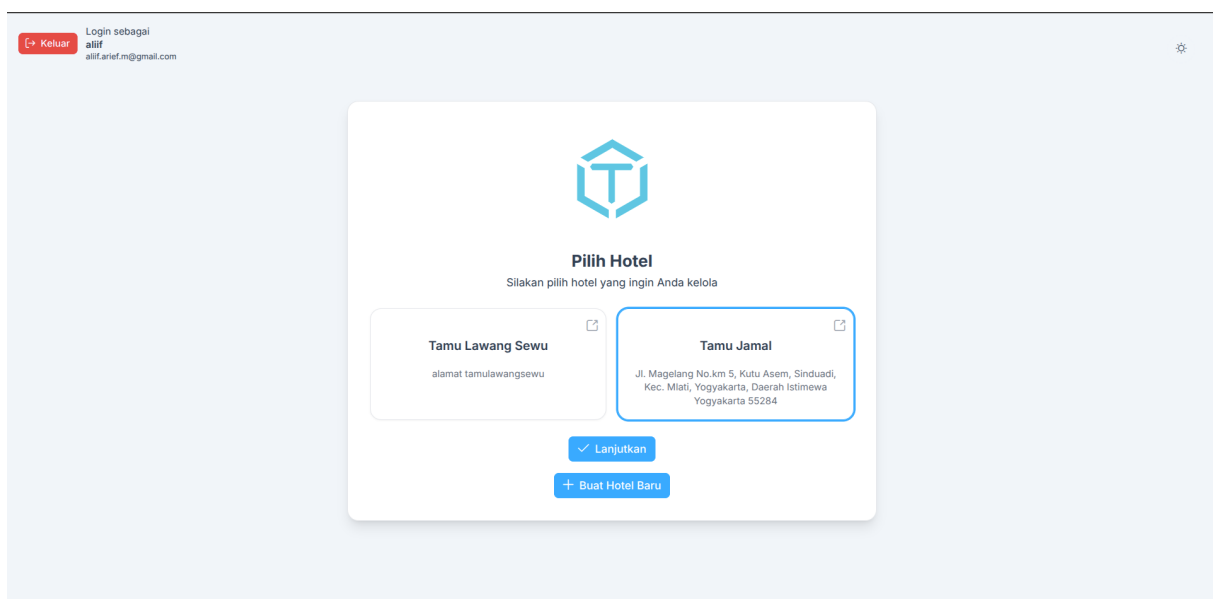
Halaman Admin atau Admin cabang merupakan halaman yang digunakan oleh admin untuk mengelola operasional dan data cabang mereka, berikut implementasi halaman admin yang telah dibuat.

A. Login, Pilih Cabang, Dashboard, dan Pengaturan Akun

Pada halaman ini, admin dapat melakukan login ke dalam aplikasi Tamu.id. Tampilan halaman login admin dapat dilihat pada Gambar 4.22. Admin dapat login dengan menggunakan dua metode yaitu email dan password ataupun langsung menggunakan akun Google.



Gambar 4.22 Tampilan Halaman Login Admin



Gambar 4.23 Tampilan Halaman Pilih Cabang Admin

Setelah Admin berhasil login, admin akan diarahkan ke halaman pilih cabang. Pada halaman ini, admin dapat memilih cabang yang akan dikelola. Admin dapat mengelola lebih dari satu cabang, Tampilan halaman pilih cabang admin dapat dilihat pada Gambar 4.23, Potongan implementasi kode untuk mendapatkan data cabang yang dikelola oleh admin saat setelah login dapat dilihat pada kode 4.11.

```

1 admin_hotels = []
2 if admin.role != RoleEnum.SUPERADMIN:
3     logging.info(f"Admin ID: {admin.id}, {admin.role.value}")
4     hotels = (
5         db.session.query(Hotel)

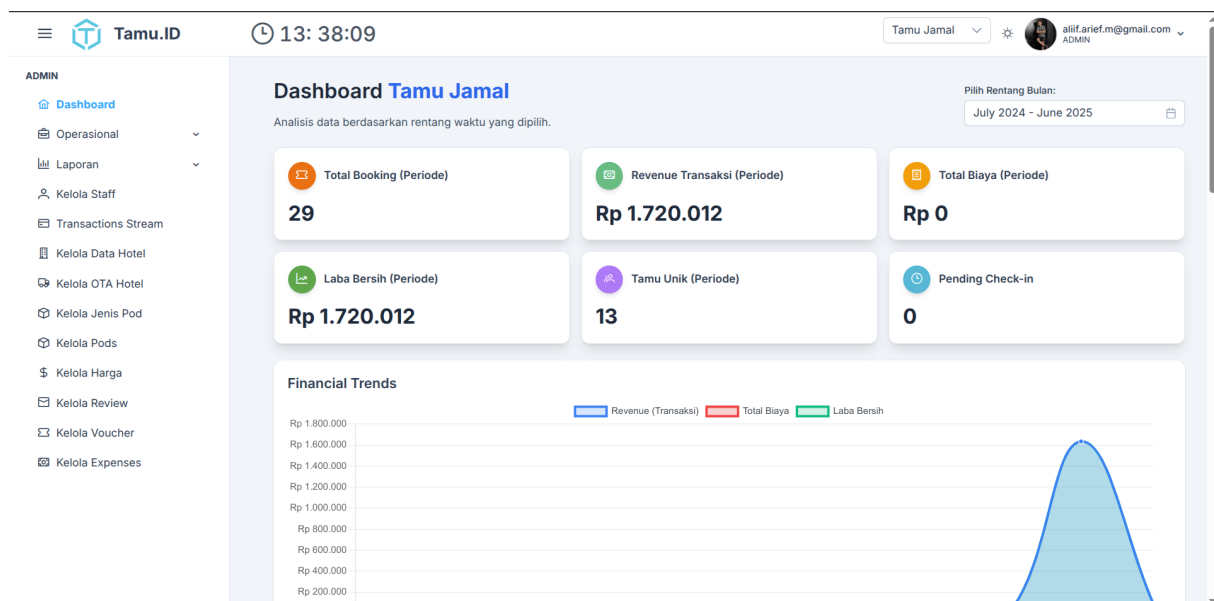
```

```

6      .join(AdminHotel, AdminHotel.hotel_id == Hotel.id)
7      .filter(AdminHotel.admin_id == admin.id)
8      .all()
9  )
10  admin_hotels = [
11      {field: getattr(hotel, field) for field in [
12          "id", "name", "address", "slug"
13      ]}
14      for hotel in hotels
15  ]
16  access_token = create_access_token(identity=admin.id, additional_claims={"role": admin.role.
17      value})
18  refresh_token = create_refresh_token(identity=admin.id, additional_claims={"role": admin.role.
19      value})
20  # returning all admin data
21  return jsonify({
22      "message": "Login success",
23      "admin" : {
24          "id": admin.id,
25          "email": admin.email,
26          "avatar": admin.avatar,
27          "role": admin.role.value,
28          "name": admin.name,
29          "hotels": admin_hotels
30      },
31      "access_token": access_token,
32      "refresh_token": refresh_token
33  })

```

Kode 4.11: Implementasi Login Admin dan Pilih Cabang



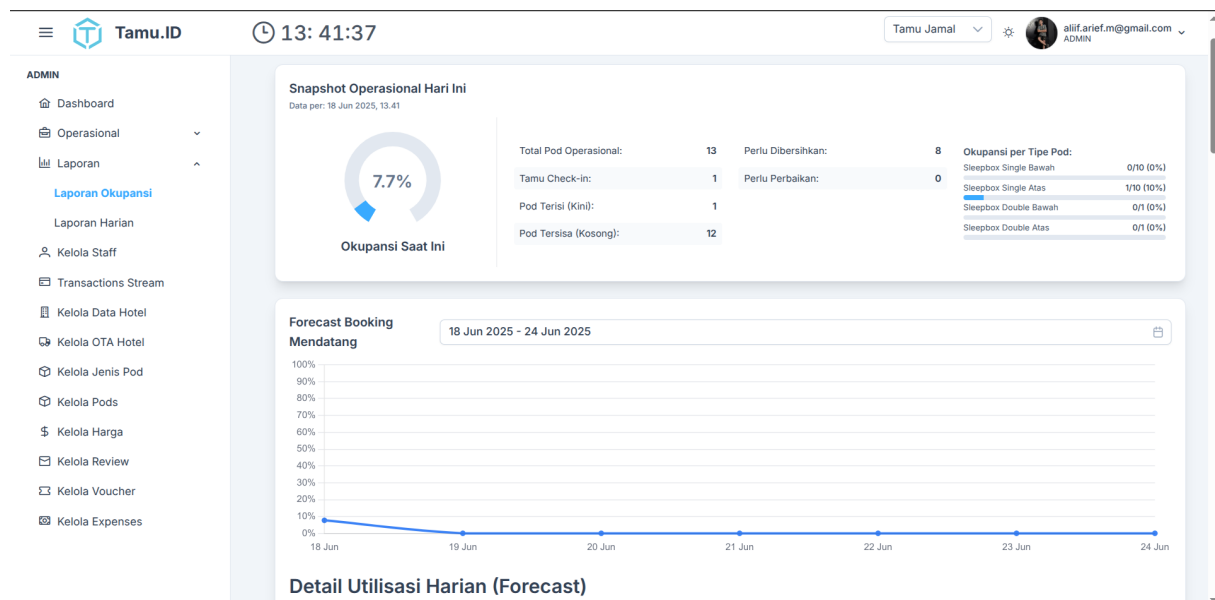
Gambar 4.24 Tampilan Halaman Dasbor Admin Cabang

Setelah memilih cabang, admin akan diarahkan ke halaman dasbor admin cabang. Pada halaman ini, admin dapat melihat informasi garis besar mengenai cabang yang dikelola seperti

pada Gambar 4.24, Potongan implementasi kode untuk mendapatkan data yang ditampilkan pada dasbor cabang yang dikelola oleh admin saat setelah login dapat dilihat pada kode 4.12.

```
1 return jsonify({
2     "hotel_name": hotel_info.name,
3     "stats_kpi": stats_kpi_data,
4     "financial_trend_chart": financial_trend_chart_data,
5     "best_selling_pods_by_revenue": best_pods_by_revenue_data,
6     "top_pods_by_sold": top_pods_by_sold_data,
7     "monthly_booking_growth": monthly_booking_growth_data,
8     "payment_method_trends": payment_method_line_chart_data,
9     "busiest_booking_days": busiest_booking_days_data,
10    "recent_activity": recent_activity_data,
11    "top_booking_durations": top_booking_durations_data,
12    "top_expenses_category": top_expenses_category_data,
13 })
```

Kode 4.12: Implementasi Dasbor Admin Cabang



Gambar 4.25 Tampilan Halaman Dasbor Okupansi Admin Cabang

Pada gambar 4.25 adalah tampilan halaman dasbor okupansi admin cabang. Pada halaman ini, admin dapat melihat informasi okupansi cabang yang dikelola secara real-time, selain itu juga pada halaman ini admin dapat melihat perkiraan tingkat okupansi beberapa hari ke depan dan juga melihat performa okupansi cabang dalam beberapa hari terakhir. Hal ini dapat membantu admin dalam mengelola okupansi cabang dan membuat keputusan yang lebih baik, Potongan implementasi kode untuk mendapatkan data yang ditampilkan pada dasbor okupansi cabang yang dikelola oleh admin saat setelah login dapat dilihat pada kode 4.13.

```
1 return jsonify({
2     "current_snapshot": current_snapshot_data,
3     "future_forecast": {"actual_start_date": query_forecast_start_date.isoformat(), "
4         actual_end_date": query_forecast_end_date.isoformat(), "daily_utilization":
5         future_forecast_data_list},
```

```

4      "historical_utilization_trend": {"actual_start_date": query_historical_start_date.
      isoformat(), "actual_end_date": query_historical_end_date.isoformat(), "
      daily_utilization": historical_utilization_list}
5  }, 200

```

Kode 4.13: Implementasi Dasbor Okupansi Admin Cabang

Gambar 4.26 Tampilan Halaman Profil Admin Cabang

Pada gambar 4.26 adalah tampilan halaman profil admin cabang. Pada halaman ini, admin dapat melihat informasi profil mereka seperti nama, email, dan nomor telepon. Admin juga dapat mengubah informasi profil mereka jika diperlukan.

B. Kelola Data Cabang

Gambar 4.27 Tampilan Admin Kelola Data Cabang

The screenshot shows the 'Kelola Data Hotel' page in the Tamu.ID admin panel. The left sidebar lists various management options. The main form is for updating a hotel branch, with fields for check-in/out times, coordinates, a Google Maps link, and city selection. A map of Yogyakarta is displayed, and a list of facilities is shown below it. A 'Simpan' button is used to save the changes.

Gambar 4.28 Tampilan Admin Kelola Data Cabang

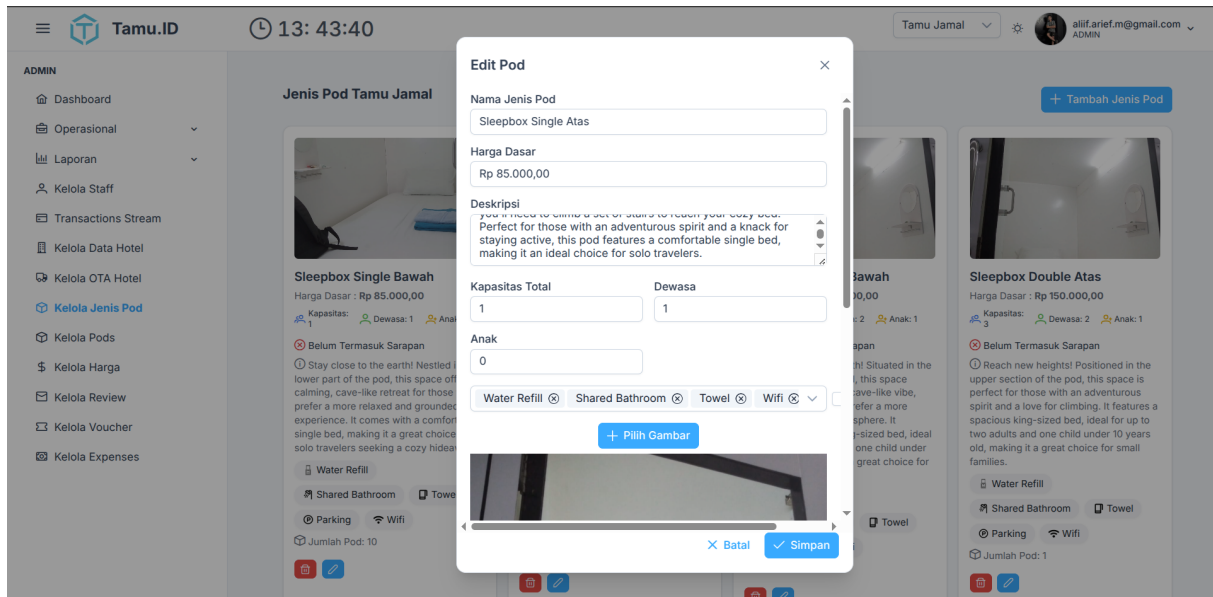
Untuk melakukan update data cabang, admin dapat mengakses halaman kelola data cabang seperti pada gambar 4.27 dan 4.28, pada halaman ini, admin dapat mengubah informasi cabang seperti nama, alamat, nomor telepon, dan deskripsi cabang. Hal ini dapat membantu admin dalam mengelola data cabang dan memastikan bahwa informasi yang ditampilkan di aplikasi Tamu.id selalu akurat dan terbaru. Potongan implementasi kode untuk mendapatkan data cabang yang dikelola oleh admin dapat dilihat pada kode 4.14.

```

1 const fetchHotels = async () => {
2   const response = await axios.get(`${import.meta.env.VITE_API_URL}/hotel/${id}`, {
3     headers: { Authorization: `Bearer ${authStore.accessToken}` }
4   });
5   if (response.status !== 200) {
6     console.error('Failed to fetch hotels:', response);
7     return;
8   }
9   data.value = response.data;
10  data.value.facilities_id = response.data.facilities.map((facility) => facility);
11  data.value.city_id = response.data.city_id;
12  images.value = response.data.images.map((image) => ({
13    url: image.url,
14    caption: image.caption,
15    order: image.order
16  }));
17  console.log('Hotel data:', data.value);
18  initMap();
19  data.value.latitude = response.data.latitude;
20  data.value.longitude = response.data.longitude;
21 };

```

Kode 4.14: Implementasi Halaman Kelola Data Cabang



Gambar 4.29 Tampilan Admin Kelola jenis pod cabang

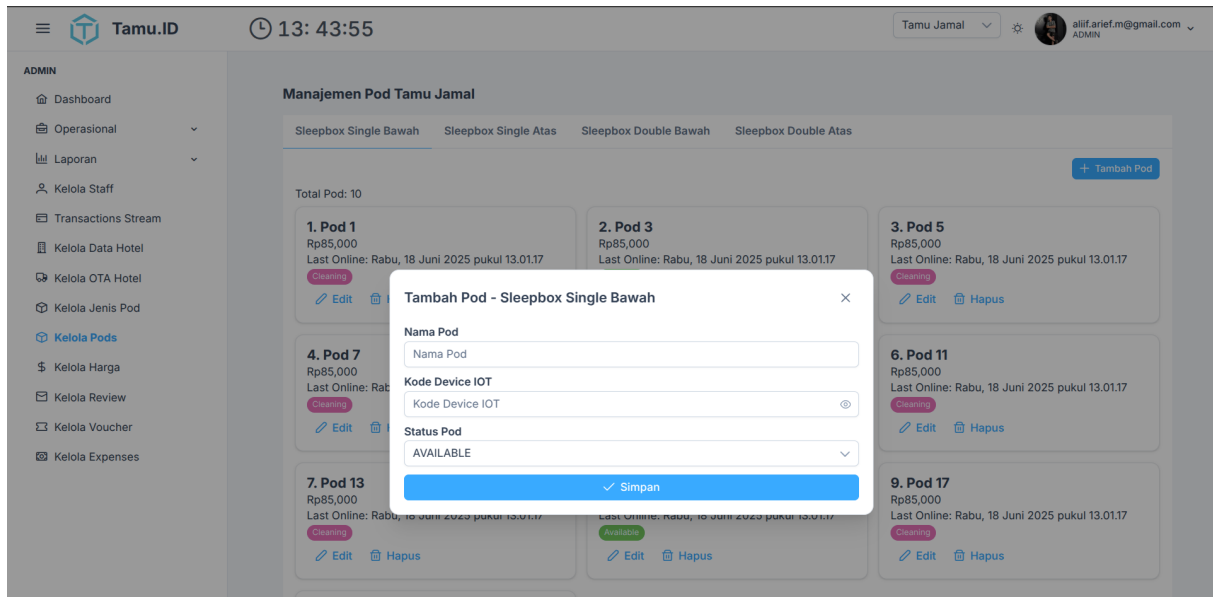
Yang paling utama dari sebuah cabang adalah tiap cabang memiliki jenis pod yang dapat dipesan oleh *User*, oleh karena itu pada gambar 4.29 *Admin* dapat mengelola jenis pod yang ada pada cabang yang ia kelola, dari mulai membuat, mengedit, dan menghapusnya. Potongan implementasi kode untuk mendapatkan data jenis pod yang dikelola oleh admin dapat dilihat pada kode 4.15.

```

1 @admin_bp.route("/hotel/<hotel_id>/podtype", methods=["POST"])
2 @jwt_required_role([RoleEnum.ADMIN])
3 def create_pod_type_route(hotel_id):
4     return create_pod_type(hotel_id)
5 @admin_bp.route("/hotel/<hotel_id>/podtype/<pod_type_id>", methods=["PUT"])
6 @jwt_required_role([RoleEnum.ADMIN])
7 def update_pod_type_route(hotel_id, pod_type_id):
8     return update_pod_type(hotel_id, pod_type_id)
9 @admin_bp.route("/hotel/<hotel_id>/podtype/<pod_type_id>", methods=["DELETE"])
10 @jwt_required_role([RoleEnum.ADMIN])
11 def delete_pod_type_route(hotel_id, pod_type_id):
12     return delete_pod_type(hotel_id, pod_type_id)
13 @admin_bp.route("/hotel/<hotel_id>/podtype", methods=["GET"])
14 @jwt_required_role([RoleEnum.ADMIN])
15 def get_pod_types_route(hotel_id):
16     return get_pod_types(hotel_id)

```

Kode 4.15: Implementasi Kelola Jenis Pod Admin



Gambar 4.30 Tampilan Admin Kelola jenis pod cabang

Setelah memiliki jenis pod maka selanjutnya untuk masing - masing jenis pod tentunya memiliki anggota yaitu entitas Pod, oleh karena itu *Admin* dapat mengelola Pod yang ada pada cabangnya dari mulai membuat, mengedit, dan menghapusnya sesuai pada gambar 4.30, potongan implementasi kode untuk tampilan halaman kelola pod admin dapat dilihat pada kode 4.16.

```

1 <TabView v-else v-model:activeIndex="activeTabIndex" scrollable>
2   <TabPanel v-for="(podType, index) in podTypes" :key="podType.id" :header="podType.name"
      :headerStyle="{ fontWeight: 'bold' }">
3     <div class="flex justify-end mb-3">
4       <Button label="Tambah Pod" icon="pi pi-plus" class="p-button-sm" @click="
            openAddPodModal(podType) " />
5     </div>
6     <!-- display total pod if any -->
7     <div v-if="filteredPods(podType.id).length" class="mb-2">Total Pod: {{ filteredPods(
            podType.id).length }}</div>
8     <div v-if="filteredPods(podType.id).length" class="grid md:grid-cols-2 lg:grid-cols-3
            gap-4">
9       <div v-for="pod in filteredPods(podType.id)" :key="pod.id" class="border rounded-
            xl p-4 shadow hover:shadow-md bg-white dark:bg-zinc-800 dark:border-zinc-600">
10        
11        <div class="font-bold text-xl">{{ pod.order }}. {{ pod.name }}</div>
12        <div class="text">Rp{{ pod.pod_type.price.toLocaleString() }}</div>
13        <!-- last online unix timestamp to wib -->
14        <div class="mt-1">
15          Last Online:
16          {{
17            new Date(pod.last_online * 1000).toLocaleString('id-ID', {
18              timeZone: 'Asia/Jakarta',
19              weekday: 'long', // Senin, Selasa, ...
20              year: 'numeric',
21              month: 'long', // Januari, Februari, ...
22              day: 'numeric',
23              hour: '2-digit',

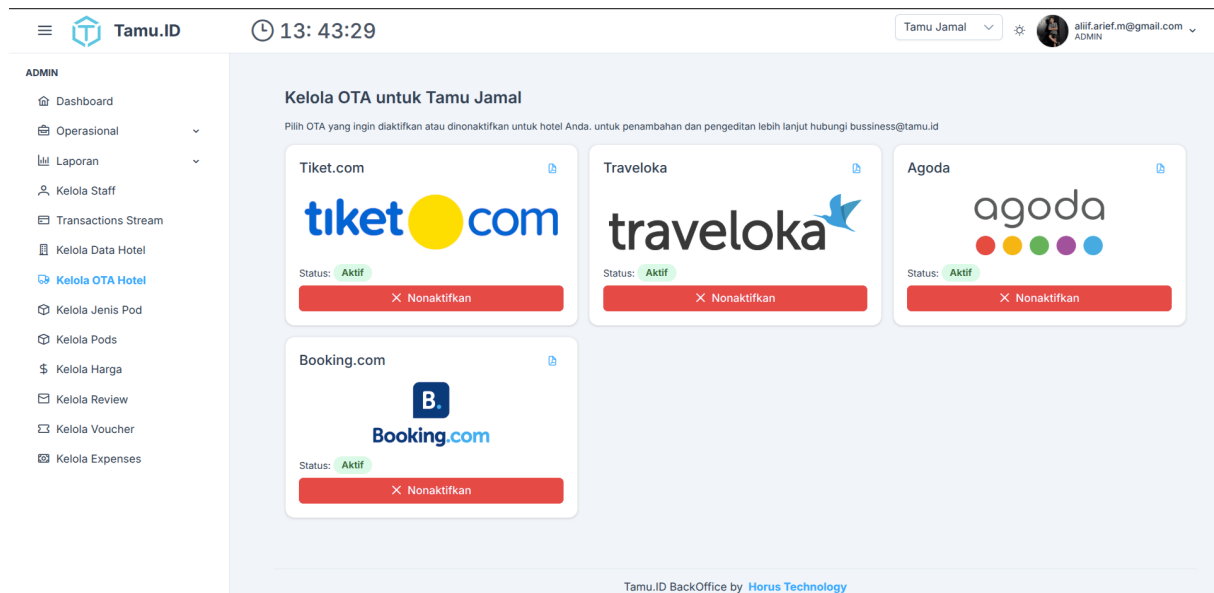
```

```

24         minute: '2-digit',
25         second: '2-digit'
26     })
27     }}
28 </div>
29 <div class="mt-2">
30     <Button label="Edit" icon="pi pi-pencil" class="p-button-text p-0 text-xs"
31         @click="openEditPodModal(pod)" />
32     <Button label="Hapus" icon="pi pi-trash" class="p-button-text p-0 text-xs
33         text-red-500" @click="deletePod(pod.id, pod.name)" />
34 </div>
35 </div>
36 <div v-else class="text-gray-500 dark:text-gray-300 h-[300px] text-center">Belum ada
37     pod untuk tipe ini.</div>
</TabPanel>
</TabView>

```

Kode 4.16: Implementasi Kelola Pod Admin



Gambar 4.31 Tampilan Admin Kelola OTA Cabang

Untuk tiap cabang memiliki kebebasan untuk melakukan integrasi dengan *Online Travel Agency* lain misal Traveloka, Tiket.com, Booking.com, dll. ataupun tidak sama sekali oleh karena itu pada gambar 4.31 Admin dapat mengelola *Online Travel Agency* mana yang aktif pada cabang yang ia kelola, potongan implementasi kode untuk halaman kelola OTA admin dapat dilihat pada kode 4.17.

```

1 <div class="grid md:grid-cols-2 lg:grid-cols-3 gap-4">
2   <Card v-for="ota in otas" :key="ota.id" class="shadow-md border">
3     <template #title>
4       <div class="flex items-center justify-between">
5         <span>{{ ota.name }}</span>
6         <Button v-if="ota.guide" icon="pi pi-file-pdf" class="p-button-text p-button-
          sm" @click="downloadGuide(ota.guide)" title="Lihat Panduan" />

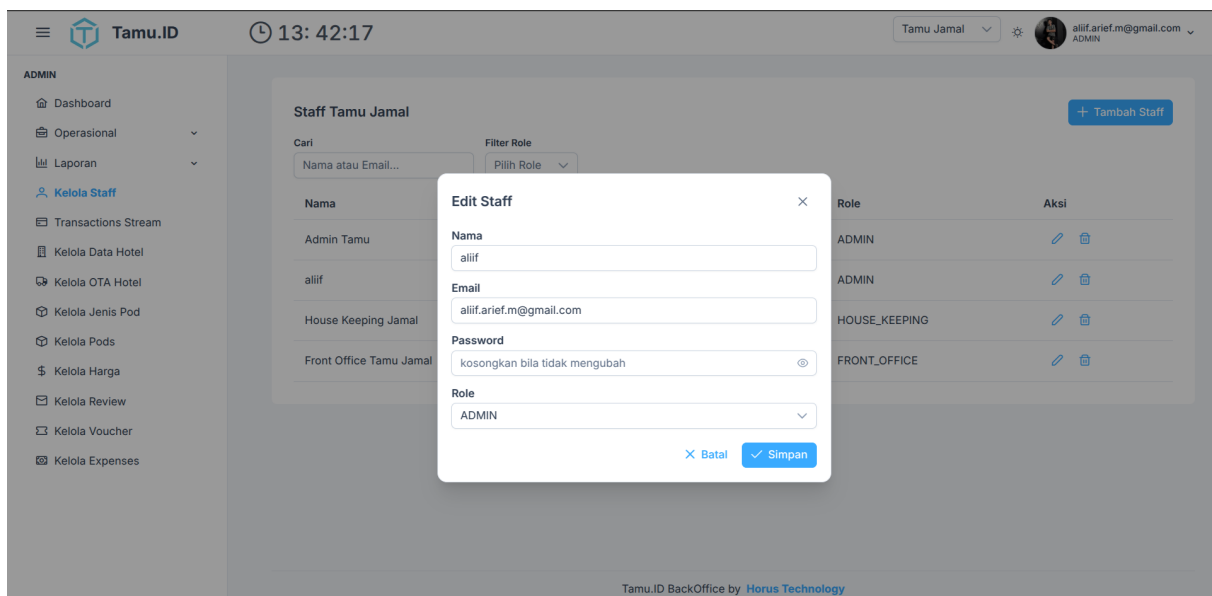
```

```

7         </div>
8     </template>
9
10    <template #content>
11        
13        <div class="text-sm mb-2">
14            Status:
15            <span :class="['px-3 py-1 rounded-full text-sm font-semibold', isActive(ota.id
16                ) ? 'bg-green-100 text-green-800' : 'bg-red-100 text-red-800']">
17                {{ isActive(ota.id) ? 'Aktif' : 'Nonaktif' }}
18            </span>
19        </div>
20        <Button
21            :label="isActive(ota.id) ? 'Nonaktifkan' : 'Aktifkan'"
22            :icon="isActive(ota.id) ? 'pi pi-times' : 'pi pi-check'"
23            :class="isActive(ota.id) ? 'p-button-danger' : 'p-button-primary'"
24            @click="isActive(ota.id) ? deactivateOta(ota.id) : activateOta(ota.id)"
25            class="w-full"
26        />
27    </template>
28 </Card>
29 </div>

```

Kode 4.17: Implementasi Kelola *OTA* Admin



Gambar 4.32 Tampilan Admin Kelola Staff Cabang

Setiap cabang dapat dikelola oleh lebih dari satu Admin atau Staff oleh karena itu pada gambar 4.32 Staff dengan peran Admin dapat melakukan pengaturan untuk staff pada cabang yang ia kelola, dari mulai membuat, mengedit, dan menghapus staff yang ada pada cabang tersebut. Potongan implementasi kode untuk kelola staff yang dilakukan oleh admin dapat dilihat pada kode 4.18.

```

1 def get_staffs_by_hotel(hotel_id):

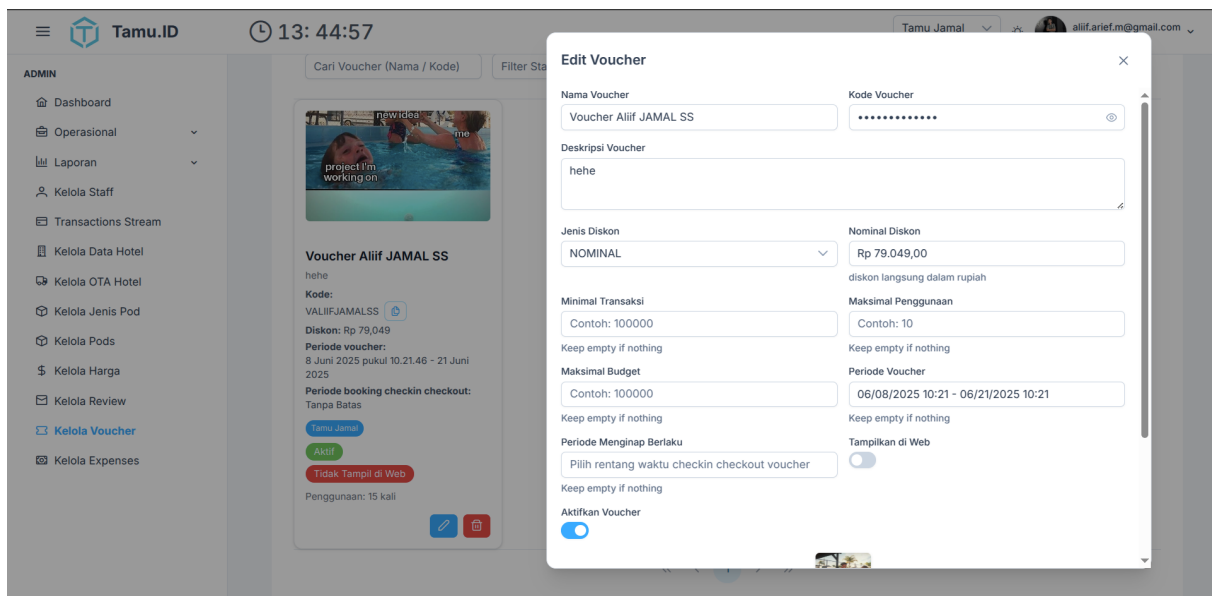
```

```

2  try:
3      hotel = Hotel.query.get(int(hotel_id))
4      if not hotel:
5          return jsonify({"error": "Hotel not found"}), 404
6      staffs = (
7          db.session.query(Admin)
8          .join(AdminHotel, AdminHotel.admin_id == Admin.id)
9          .filter(AdminHotel.hotel_id == hotel.id)
10         .all()
11     )
12     result = [{
13         "id": s.id,
14         "name": s.name,
15         "email": s.email,
16         "role": s.role.value if isinstance(s.role, RoleEnum) else s.role,
17         "hotel_ids": get_staff_hotels(s.id),
18     } for s in staffs]
19     return jsonify(result), 200
20 except:
21     # log error stack
22     logging.error("Error getting staffs by hotel", exc_info=True)
23     return jsonify({"error": "Internal Server Error"}), 500

```

Kode 4.18: Implementasi Kelola oleh Staff Admin



Gambar 4.33 Tampilan Admin Kelola Voucher Cabang

Untuk setiap cabang dapat mengadakan promo mandiri yaitu seperti pada gambar 4.33, Admin dapat membuat Voucher promo khusus untuk cabang yang ia kelolal, dari mulai membuat, mengedit, dan menghapus voucher yang ada pada cabang tersebut. Potongan implementasi kode untuk kelola voucher yang dilakukan oleh admin dapat dilihat pada kode 4.19.

```

1 def get_vouchers_for_hotel(hotel_id):
2     try:

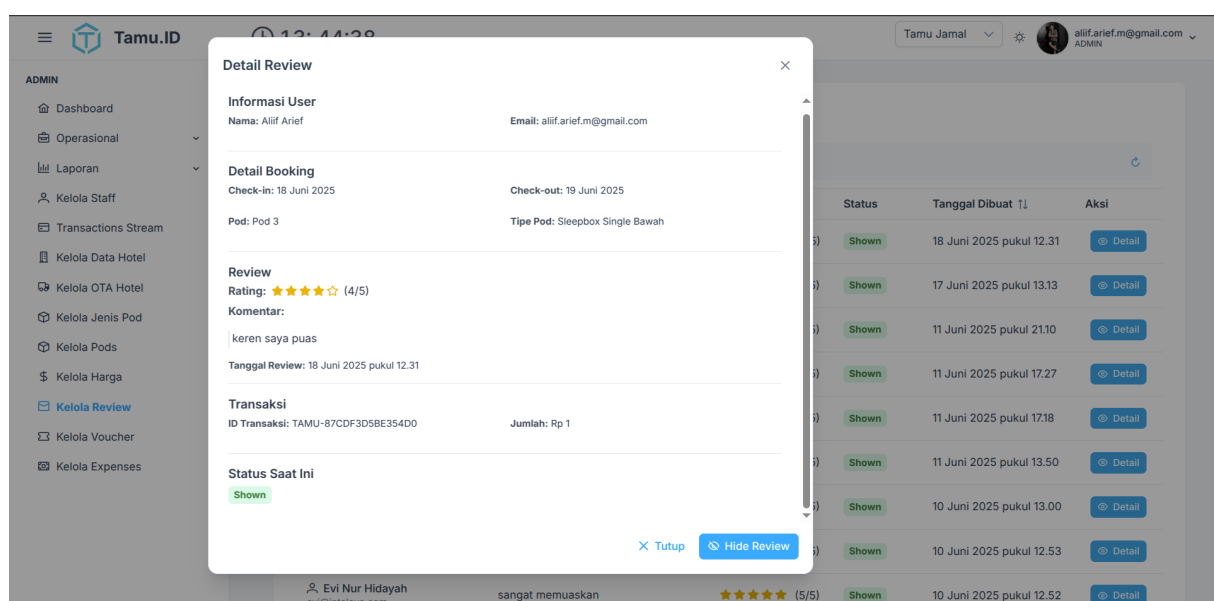
```

```

3     vouchers = Voucher.query.join(VoucherHotel).filter(VoucherHotel.hotel_id == int(
4         hotel_id)).order_by(Voucher.created_at.desc()).all()
5     voucher_list = []
6     for voucher in vouchers:
7         voucher_data = {
8             "id": voucher.id,
9             "name": voucher.name,
10            "description": voucher.description,
11            "code": voucher.code,
12            "discount_type": voucher.type.name,
13            "discount_max": voucher.discount_max,
14            "discount_amount": voucher.discount_amount,
15            "current_usage": voucher.current_usage or 0,
16            "current_budget": voucher.current_budget or 0,
17            "discount_percentage": voucher.discount_percentage,
18            "min_transaction": voucher.min_transaction,
19            "image": voucher.image,
20            "max_usage": voucher.max_usage,
21            "max_budget": voucher.max_budget,
22            "start_at": voucher.start_at,
23            "end_at": voucher.end_at,
24            "checkin_at": voucher.checkin_at,
25            "checkout_at": voucher.checkout_at,
26            "showed_in_customer": voucher.showed_in_customer,
27            "is_active": voucher.is_active,
28            "hotel_ids": [vh.hotel.id for vh in voucher.hotels],
29            "hotels": [{"id": vh.hotel.id, "name": vh.hotel.name} for vh in voucher.hotels
30                ],
31        }
32        voucher_list.append(voucher_data)
33    return jsonify(voucher_list), 200
34 except Exception:
35     logging.error("Failed to get vouchers", exc_info=True)
36     return jsonify({"message": "Failed to get vouchers"}), 500

```

Kode 4.19: Implementasi Kelola Voucher oleh Admin



Gambar 4.34 Tampilan Admin Kelola Review Cabang

Setelah *User* menginap pada suatu Pod maka selanjutnya *User* akan melakukan *Checkout* dan meninggalkan *Review* dan *Review* tentang pengalamannya selama menggunakan layanan, oleh karena itu untuk menghindari *Spam* pada gambar 4.34, Admin dapat mengelola dan mengontrolnya, dari mulai melihat, menghapus, dan moderasi review yang ada pada cabang tersebut. Potongan implementasi kode untuk kelola review yang dilakukan oleh admin dapat dilihat pada kode 4.20.

```

1 const approveReview = async (reviewId) => {
2   if (!reviewId) return;
3   try {
4     await axios.put(
5       `${import.meta.env.VITE_API_URL}/review/${reviewId}/approve`,
6       {},
7       {
8         headers: { Authorization: `Bearer ${authStore.accessToken}` }
9       }
10    );
11    toast.add({ severity: 'success', summary: 'Success', detail: 'Review approved successfully' });
12    fetchReviews();
13    if (selectedReview.value && selectedReview.value.id === reviewId) {
14      selectedReview.value.showed = true;
15    }
16    dialog.value = false;
17  } catch (error) {
18    console.error('Failed to approve review:', error);
19    toast.add({ severity: 'error', summary: 'Error', detail: 'Failed to approve review' });
20  }
21 };
22
23 const rejectReview = async (reviewId) => {
24   if (!reviewId) return;
25   try {
26     await axios.put(
27       `${import.meta.env.VITE_API_URL}/review/${reviewId}/hide`, // Assuming 'hide' is
28       {}, // the correct endpoint for reject/hidden
29       {
30         headers: { Authorization: `Bearer ${authStore.accessToken}` }
31       }
32     );
33     toast.add({ severity: 'success', summary: 'Success', detail: 'Review hidden successfully' }); // Updated message
34     fetchReviews();
35     if (selectedReview.value && selectedReview.value.id === reviewId) {
36       selectedReview.value.showed = false;
37     }
38     dialog.value = false;
39   } catch (error) {
40     console.error('Failed to hide review:', error);
41     toast.add({ severity: 'error', summary: 'Error', detail: 'Failed to hide review' });
42   }
43 };

```

Kode 4.20: Implementasi Moderasi Review oleh Admin

C. Operasional Harian Cabang

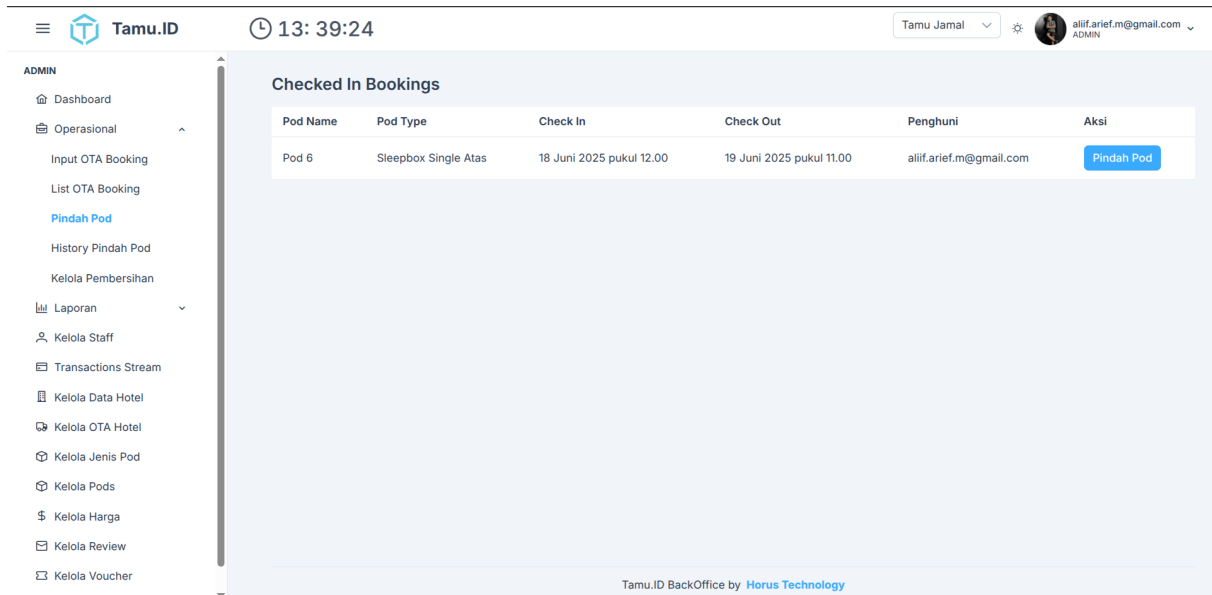
The screenshot shows the 'Laporan Harian: Tamu Jamal' page in the Tamu.ID admin system. The sidebar on the left lists various administrative functions under the 'ADMIN' category. The main content area features a table of daily reports for a specific guest. The table has six columns: 'Nama Tamu', 'No. POD', 'Tipe POD', 'Jadwal Check-out', 'Waktu Check-in', and 'Aksi'. A single entry is shown for 'A Alif Arief' in 'Pod 6' with a 'Sleepbox Single Atas' type, scheduled for '19 Jun 2025' at '12.42'. Above the table, there are three status indicators: 'Akan Check-in' (0), 'Sedang Check-in' (1), and 'Akan Check-out' (0). The top of the page displays the time '13:41:56' and the user 'Tamu Jamal'.

Gambar 4.35 Tampilan Halaman Laporan Harian Admin Cabang

Untuk mempermudah operasional harian cabang pada gambar 4.35 admin dapat melihat laporan harian cabang yang dikelola seperti siapa saja yang akan check-in, siapa saja yang sudah check-in, dan siapa saja yang sudah check-out. Hal ini dapat membantu admin dalam mengelola operasional harian cabang dan memastikan bahwa semua diketahui lebih awal oleh admin, potongan implementasi kode untuk mendapatkan data laporan harian cabang yang dikelola oleh admin dapat dilihat pada kode 4.21.

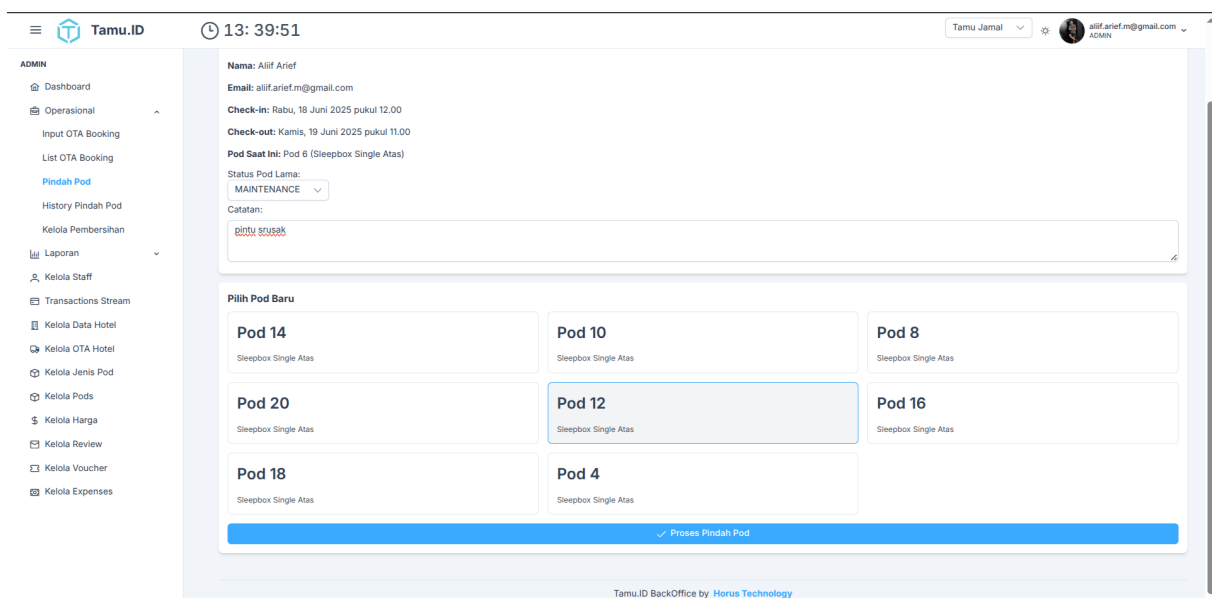
```
1 return jsonify({  
2     "date_in_hotel_tz": today_in_hotel_tz.isoformat(),  
3     "hotel_name": hotel.name,  
4     "expected_checkins": expected_checkins_list,  
5     "current_checkins": current_checkins_list,  
6     "expected_checkouts": expected_checkouts_list  
7 }, 200
```

Kode 4.21: Implementasi Laporan Harian Admin



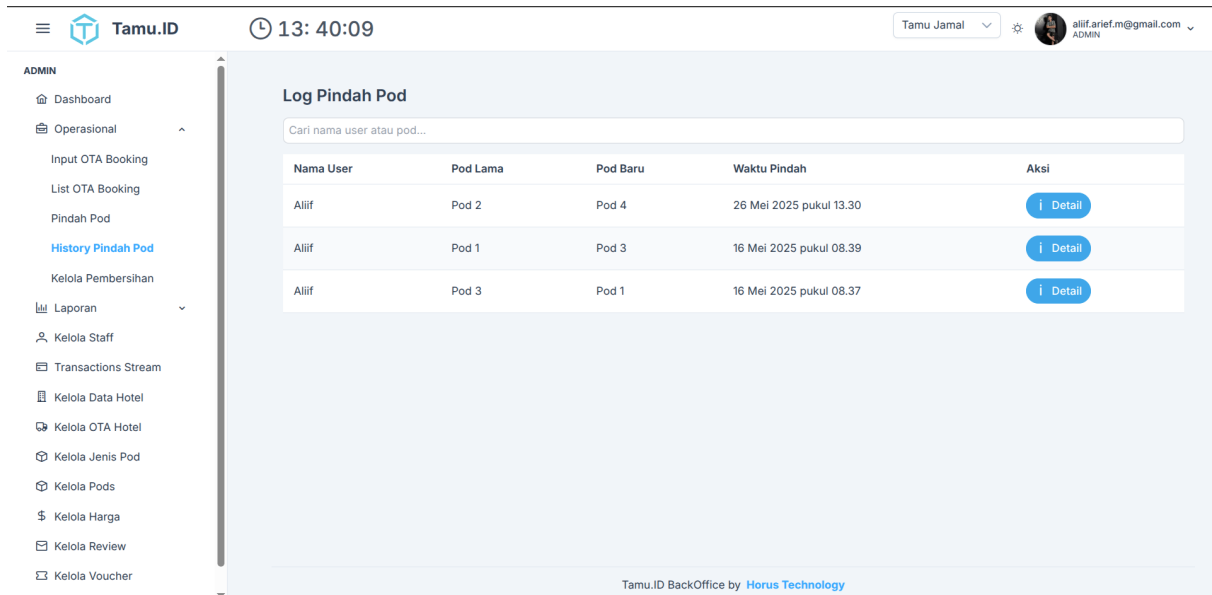
Gambar 4.36 Tampilan Halaman Daftar Tamu Check-in Admin Cabang

Pada gambar 4.36 adalah tampilan halaman daftar tamu yang sudah check-in. Pada halaman ini, admin dapat melihat daftar tamu yang sudah check-in pada cabang yang dikelola. Admin juga dapat melakukan tindakan seperti memindahkan tamu ke pod lain jika diperlukan. Hal ini dapat membantu admin dalam mengelola tamu yang sudah check-in dan memastikan bahwa semua tamu terlayani dengan baik.



Gambar 4.37 Tampilan Halaman Proses Pindah Pod User oleh Admin Cabang

Pada gambar 4.37 adalah tampilan halaman proses pindah pod user oleh admin cabang. Pada halaman ini, admin dapat memindahkan tamu ke pod lain jika diperlukan. status dari pod lama yang ditempati user juga dapat disesuaikan dengan alasan pemindahan tersebut.



Gambar 4.38 Tampilan Halaman Riwayat Pindah Pod pada Cabang

Untuk melihat riwayat pindah pod pada cabang, admin dapat mengakses halaman riwayat pindah pod seperti pada gambar 4.38. Pada halaman ini, admin dapat melihat riwayat pindah pod yang dilakukan oleh admin pada cabang yang dikelola.

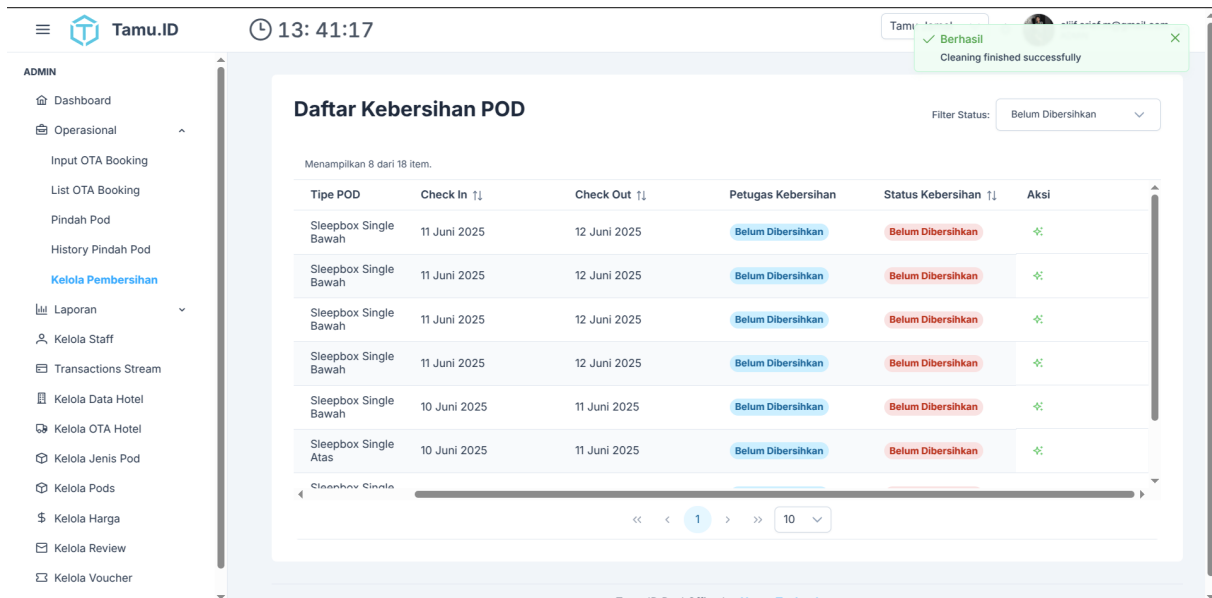
```

1 old_pod_status = old_pod_status.upper() if old_pod_status else PodStatusEnum.AVAILABLE.value
2 # update pod status
3 old_pod.status = PodStatusEnum[old_pod_status]
4 new_pod.status = PodStatusEnum.OCCUPIED
5 # generate new qr next and get qr old from new pod
6 old_qr_code = QRCode.query.filter_by(pod_id=old_pod.id).first()
7 new_qr_code = QRCode.query.filter_by(pod_id=new_pod.id).first()
8
9 hotel = Hotel.query.filter_by(id=new_pod.pod_type.hotel_id).first()
10 hotel_timezone = hotel.timezone if hotel and hotel.timezone else "Asia/Jakarta"
11 # get the timezone from hotel
12 checkout_datetime = datetime.combine(booking.check_out, hotel.checkout_time)
13 hotel_tz = pytz.timezone(hotel_timezone)
14 localized_checkout = hotel_tz.localize(checkout_datetime)
15 current_expired = int(localized_checkout.timestamp())
16
17 next_code = qr_key_generator()
18 next_code_image = generate_qr(next_code)
19
20 old_qr_code.booking_id = None
21
22 new_qr_code.booking_id = booking.id
23 new_qr_code.cleaning_id = None
24 new_qr_code.current_code = new_qr_code.next_code
25 new_qr_code.current_code_image = new_qr_code.next_code_image
26 new_qr_code.next_code = next_code
27 new_qr_code.next_code_image = next_code_image
28 new_qr_code.current_expired = current_expired
29
30 db.session.commit()

```

Kode 4.22: Implementasi Pindah Pod User oleh Admin

Kode 4.22 adalah potongan implementasi kode untuk memproses pindah pod yang dilakukan oleh admin pada cabang yang dikelola. Pada kode ini, admin dapat memindahkan tamu ke pod lain jika diperlukan. Status dari pod lama yang ditempati user juga dapat disesuaikan dengan alasan pemindahan tersebut.



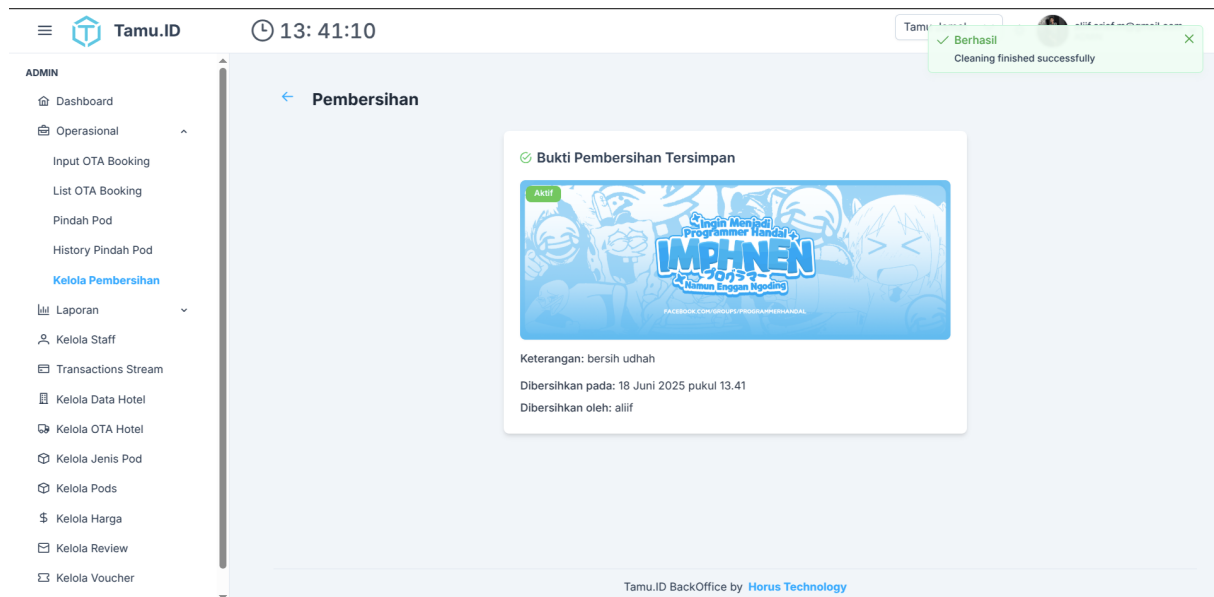
Gambar 4.39 Tampilan Halaman Daftar Pod yang Harus Dibersihkan

Setelah tamu check-out, admin dapat melihat daftar pod yang harus dibersihkan pada halaman daftar pod yang harus dibersihkan seperti pada gambar 4.39 dan melakukan tindakan seperti membersihkan pod tersebut.



Gambar 4.40 Tampilan Halaman Proses Pembersihan Pod oleh Admin Cabang

Pada gambar 4.40 adalah tampilan halaman proses pembersihan pod oleh admin cabang.



Gambar 4.41 Tampilan Halaman Pod Selesai Dibersihkan oleh Admin Cabang

Setelah proses pembersihan selesai maka tampilan akan berubah menjadi seperti pada gambar 4.41.

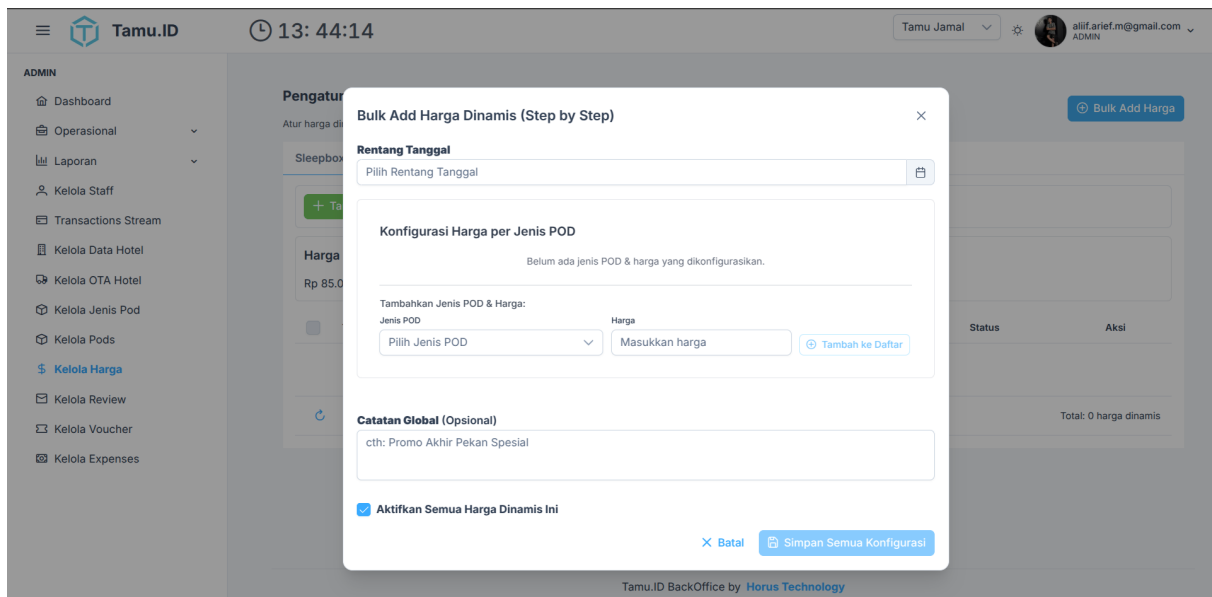
Saat proses pembersihan dimulai maka Admin akan mendapatkan kunci pod yang sedang dibersihkan, sehingga admin dapat mengakses pod tersebut untuk melakukan pembersihan, kunci tersebut hanya berlaku selama tiga jam dan bila lebih dari itu maka kunci harus diperbarui kembali, berikut potongan implementasi kode untuk mendapatkan kunci pod yang sedang dibersihkan oleh admin pada kode 4.23.

```

1 hotel = Hotel.query.filter_by(id=pod.pod_type.hotel_id).first()
2 hotel_timezone = hotel.timezone if hotel and hotel.timezone else "Asia/Jakarta"
3 expired_cleaning_token = datetime.now(pytz.timezone(hotel_timezone)) + timedelta(hours=3)
4 expired_cleaning_token = int(expired_cleaning_token.timestamp())
5 qr_code = QRCode.query.filter_by(pod_id=pod.id).first()
6 next_code = qr_key_generator()
7 next_code_image = generate_qr(next_code)
8 qr_code.current_code = qr_code.next_code
9 qr_code.current_code_image = qr_code.next_code_image
10 qr_code.next_code = next_code
11 qr_code.next_code_image = next_code_image
12 qr_code.current_expired = expired_cleaning_token
13 qr_code.cleaning_id = cleaning.id

```

Kode 4.23: Implementasi Generate Kunci Pod Admin



Gambar 4.42 Tampilan Admin Kelola Harga Dinamis Cabang

Pada gambar 4.42 adalah tampilan halaman admin cabang untuk mengelola harga dinamis. Pada halaman ini, admin dapat mengatur harga dinamis untuk cabang yang dikelola. Harga dinamis ini akan digunakan untuk menentukan harga reservasi tergantung pada tanggal reservasi dan jenis pod yang dipilih oleh tamu. Hal ini dapat membantu admin dalam mengelola harga reservasi dan memastikan bahwa harga yang ditawarkan sesuai dengan kondisi pasar.

```

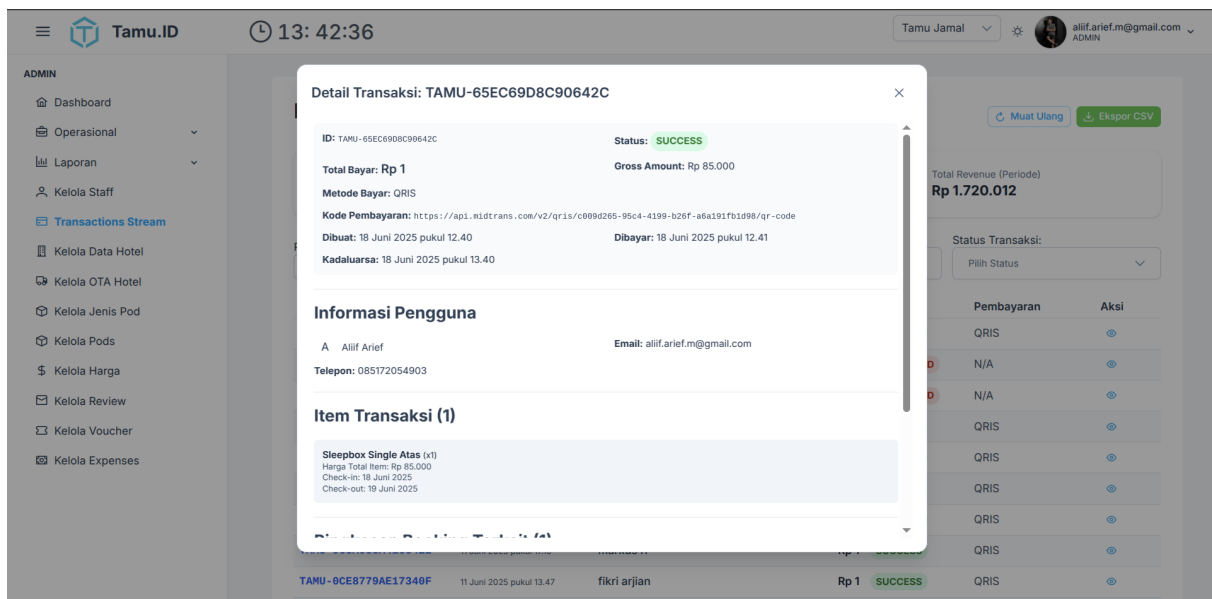
1 def calculate_dynamic_price(pod_type, checkin_date, checkout_date):
2     delta_days = (checkout_date - checkin_date).days
3     total_price = 0
4     pricing_map = {
5         dp.date: dp.price for dp in pod_type.dynamic_pricing if dp.active
6     }
7     for i in range(delta_days):
8         date = checkin_date + datetime.timedelta(days=i)
9         price = pricing_map.get(date, pod_type.price)
10        total_price += price
11    total_price = int(total_price)
12    per_night_price = total_price // delta_days if delta_days > 0 else 0
13    markup_percent = deterministic_markup_percentage(pod_type.id, checkin_date, checkout_date)
14    original_price = int(total_price * (1 + markup_percent / 100))
15    return total_price, original_price, markup_percent, per_night_price
16
17 def deterministic_markup_percentage(pod_type_id, checkin_date, checkout_date):
18     key = f"{pod_type_id}-{checkin_date}-{checkout_date}"
19     hash_value = hashlib.md5(key.encode()).hexdigest()
20     seed = int(hash_value, 16)
21     random.seed(seed)
22     return random.randint(10, 30) # percent 10-30%

```

Kode 4.24: Implementasi Harga Dinamis berdasarkan Tanggal

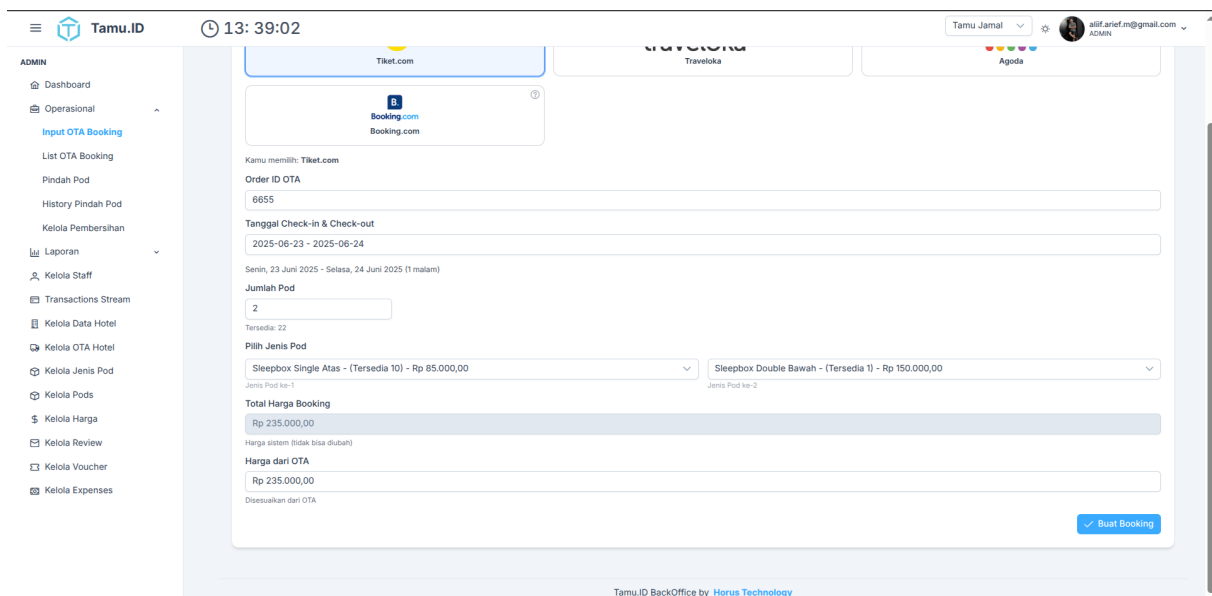
Setelah Admin memasukkan harga dinamis maka harga tersebut akan digunakan untuk

menentukan harga reservasi tergantung pada tanggal reservasi dan jenis pod yang dipilih oleh tamu, berikut potongan implementasi kode untuk mendapatkan harga dinamis berdasarkan tanggal pada kode 4.24.



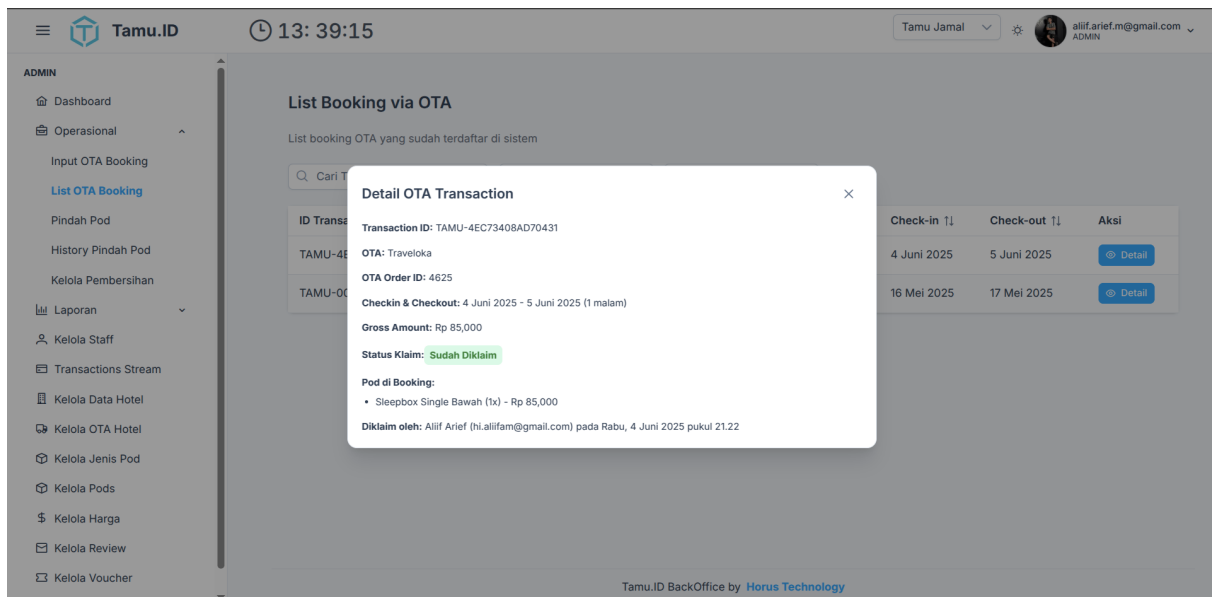
Gambar 4.43 Tampilan Transaksi pada Cabang

Pada gambar ?? adalah tampilan dimana Admin dapat melihat list dan detail transaksi yang terjadi untuk pod pada cabang yang ia kelola untuk visibilitas operasional yang lebih baik.



Gambar 4.44 Tampilan Input OTA pada Cabang

Integrasi antara *Online Travel Agency* dan sistem Tamu.id belum berjalan secara otomatis sehingga ketika ada yang melakukan reservasi via *Online Travel Agency* maka Admin harus menginputkan data reservasinya secara manual seperti pada gambar 4.44.



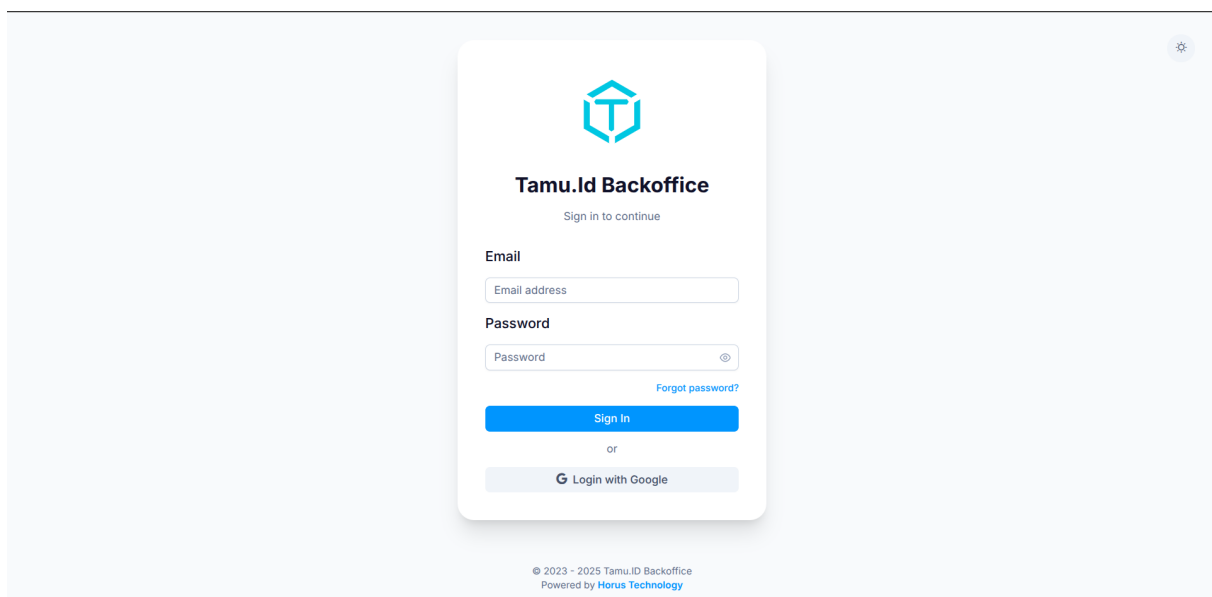
Gambar 4.45 Tampilan List Input OTA pada Cabang

Pada gambar 4.45 merupakan halaman list data untuk reservasi *User* yang dilakukan diluar platform yaitu dari *Online Travel Agency*, untuk visibilitas dan kontrol yang lebih baik tentunya data - data yang sudah dimasukkan harus dapat terlihat dengan detail.

4.1.3 Halaman Superadmin

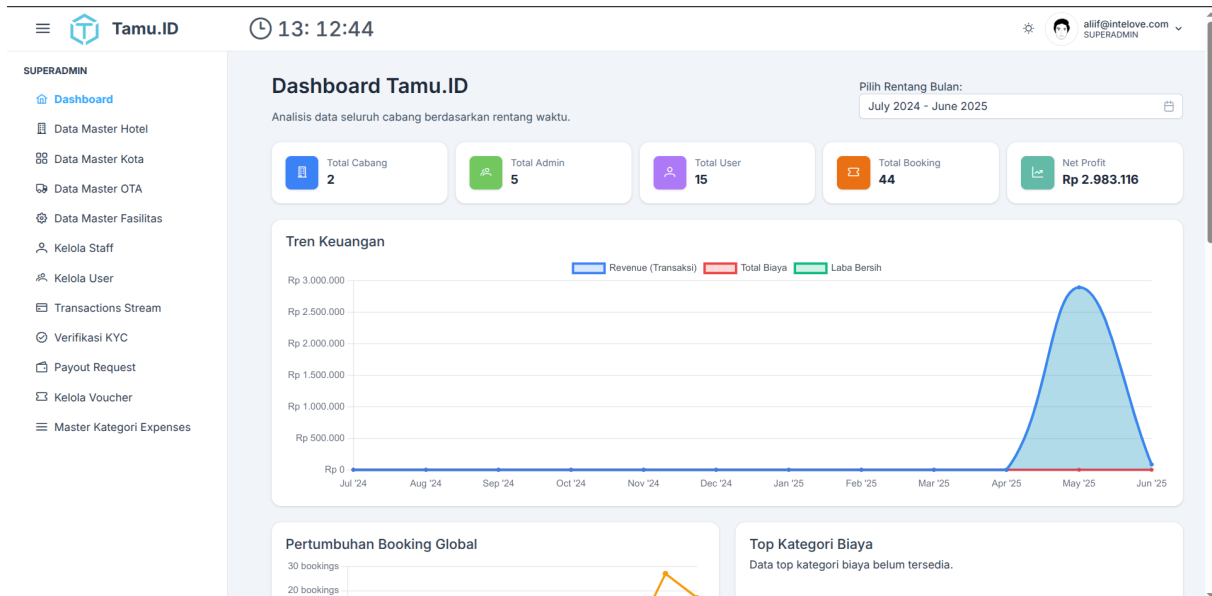
Halaman *SuperAdmin* merupakan halaman yang digunakan oleh pemilik waralaba untuk mengelola operasional Tamu.id secara terpusat dan global, berikut implementasi halaman *SuperAdmin* yang telah dibuat.

A. Login, Dashboard, dan Pengaturan Akun



Gambar 4.46 Tampilan Halaman Login Superadmin

Pada Gambar 4.46 diperlihatkan antarmuka halaman login untuk Superadmin. Halaman ini berfungsi sebagai gerbang utama bagi Superadmin untuk mengakses sistem. Di sini, Superadmin akan diminta untuk memasukkan kredensialnya, seperti nama pengguna dan kata sandi, sebelum dapat melanjutkan ke dasbor atau fitur lainnya, SuperAdmin juga dapat melakukan autentikasi dengan Google.



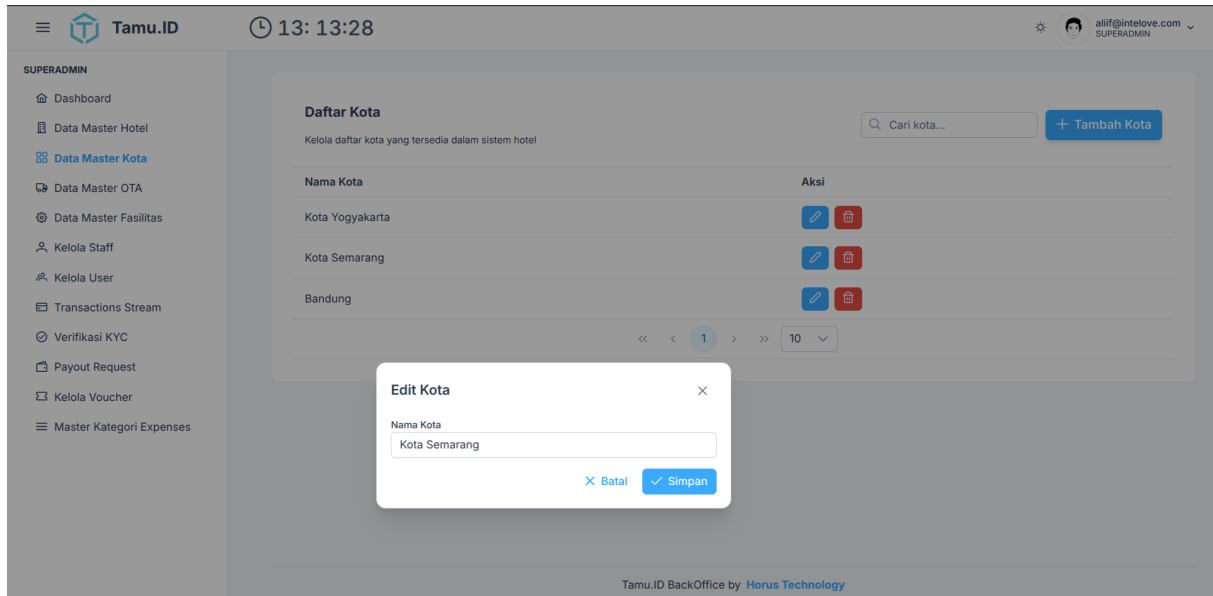
Gambar 4.47 Tampilan Halaman Dashboard Superadmin

Gambar 4.47 menampilkan halaman dasbor Superadmin, yang merupakan pusat kendali utama setelah berhasil login. Dasbor ini menyajikan gambaran global dari tiap cabang yang beroperasi sehingga setiap keputusan pusat dapat diambil berdasarkan data yang tersaji dengan benar.

Gambar 4.48 Tampilan Halaman Pengaturan Akun Superadmin

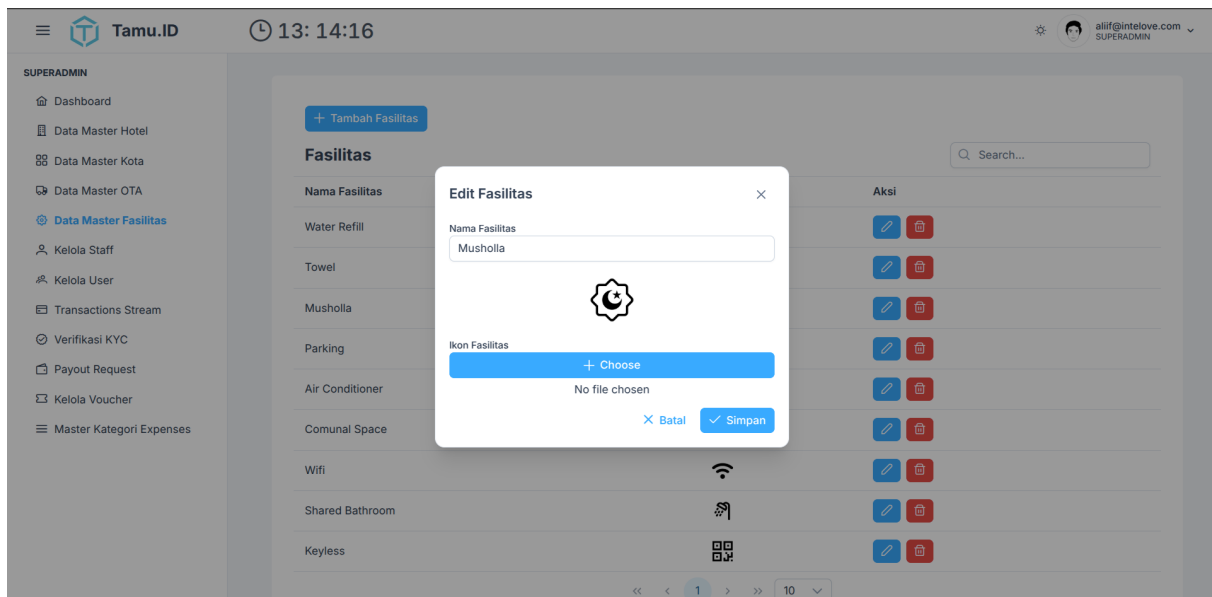
Pada Gambar 4.48 terlihat halaman pengaturan akun Superadmin. Halaman ini memungkinkan Superadmin untuk mengelola dan memperbarui informasi profilnya, seperti mengubah kata sandi atau detail pribadi lainnya. Fitur ini penting untuk menjaga keamanan akun dan memastikan bahwa informasi Superadmin selalu terkini, memberikan fleksibilitas dan kendali penuh kepada Superadmin atas akunnya.

B. Kelola Data Master



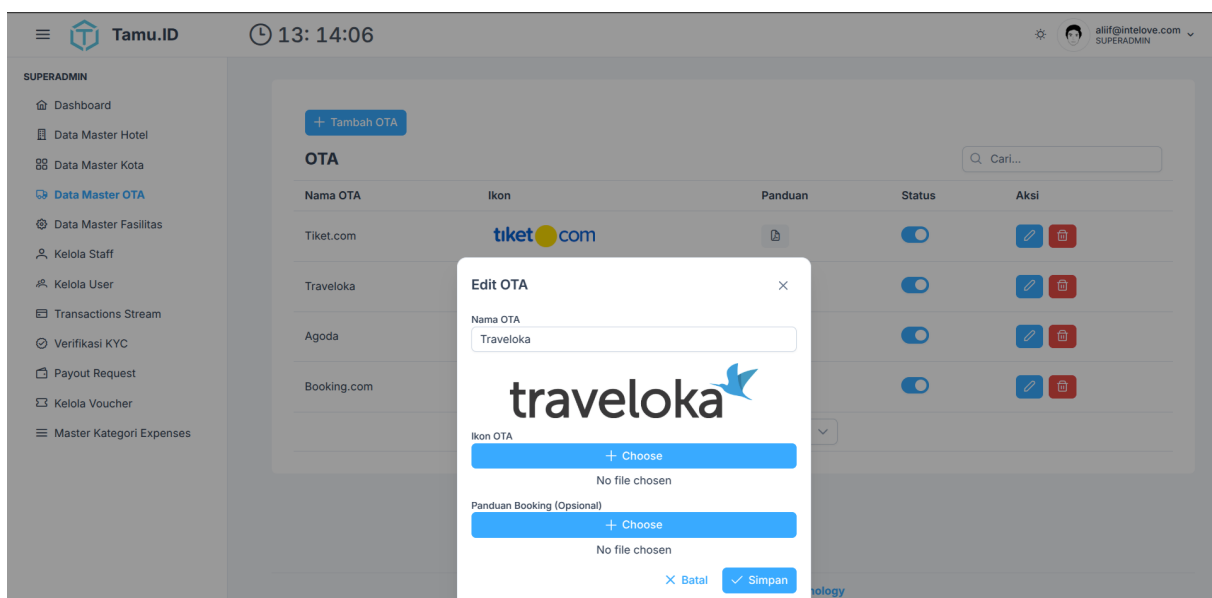
Gambar 4.49 Tampilan Halaman Pengaturan Kota Superadmin

Gambar 4.49 menampilkan antarmuka halaman pengaturan kota untuk Superadmin. Halaman ini berfungsi sebagai pusat kontrol di mana Superadmin dapat mengelola data kota, termasuk menambahkan kota baru, mengedit informasi kota yang sudah ada, atau menghapus data kota yang tidak relevan. Fitur ini penting untuk memastikan bahwa cakupan operasional sistem selalu akurat dan terbaru sesuai dengan wilayah geografis yang dilayani.



Gambar 4.50 Tampilan Halaman Pengaturan Fasilitas Superadmin

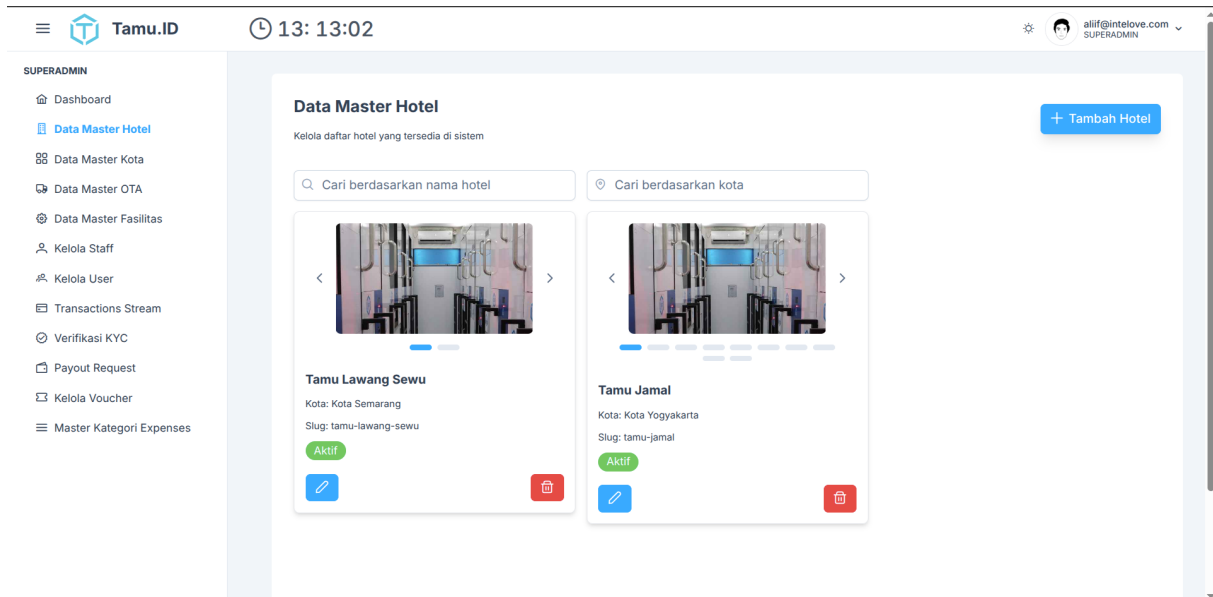
Pada Gambar 4.50 terlihat tampilan halaman pengaturan fasilitas Superadmin. Di sini, Superadmin memiliki kemampuan untuk menambah, mengubah, dan menghapus daftar fasilitas yang tersedia dalam sistem. Pengelolaan fasilitas ini krusial untuk memastikan bahwa informasi mengenai sarana dan prasarana yang ditawarkan kepada pengguna selalu lengkap dan akurat, mendukung kelancaran operasional dan kepuasan pengguna.



Gambar 4.51 Tampilan Halaman Pengaturan OTA Superadmin

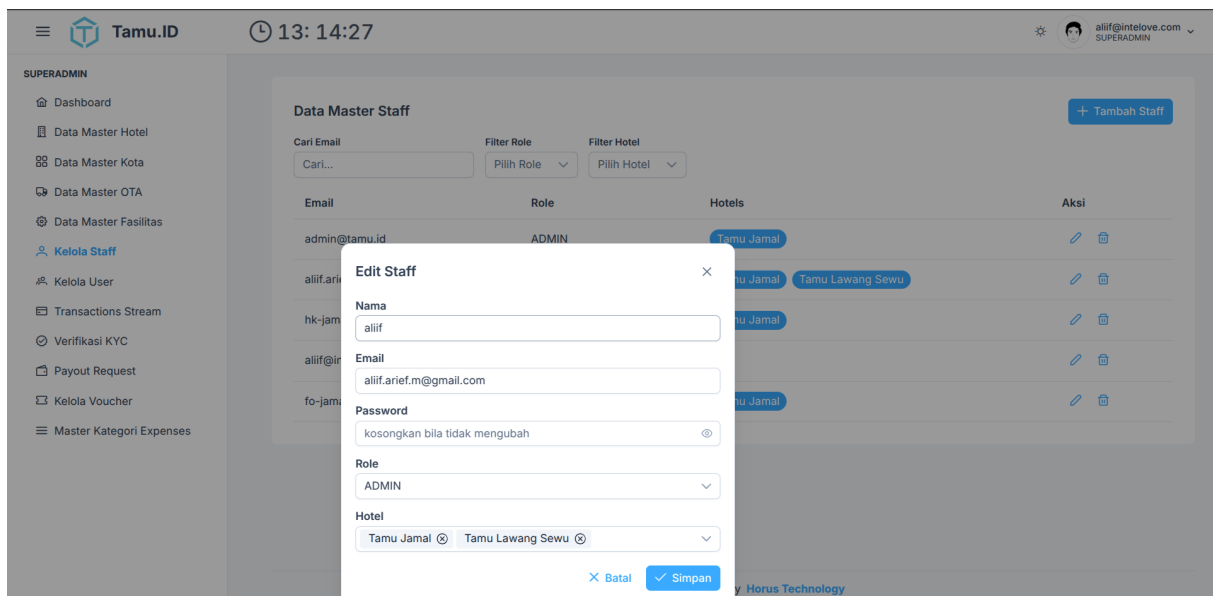
Gambar 4.51 menunjukkan tampilan halaman pengaturan OTA (Online Travel Agent) untuk Superadmin. Halaman ini memungkinkan Superadmin untuk mengelola integrasi dan konfigurasi dengan berbagai platform OTA. Pengaturan ini mencakup penambahan,

pengeditan, atau penghapusan detail OTA, memastikan bahwa sistem dapat berinteraksi secara efektif dengan mitra eksternal untuk pemasaran dan penjualan.



Gambar 4.52 Tampilan Halaman Pengaturan Cabang Superadmin

Pada Gambar 4.52 diperlihatkan halaman pengaturan cabang Superadmin. Halaman ini memberikan akses kepada Superadmin untuk mengelola informasi terkait setiap cabang operasional. Ini termasuk menambahkan cabang baru, memperbarui detail lokasi, kontak, atau informasi spesifik lainnya untuk setiap cabang. Fitur ini sangat penting untuk pengelolaan multi-lokasi yang terorganisir dan efisien.



Gambar 4.53 Tampilan Halaman Pengaturan Staff Superadmin

Gambar 4.53 menampilkan antarmuka halaman pengaturan staf untuk Superadmin. Di

halaman ini, Superadmin dapat mengelola akun dan informasi semua staf yang terdaftar dalam sistem. Ini mencakup penambahan staf baru, pengaturan peran staff, penugasan staff ke cabang, serta pembaruan atau menonaktifkan akun staf, memastikan manajemen sumber daya manusia yang terstruktur dan aman.

Manajemen Pengguna

Cari (Nama, Email, No. HP): Ketik untuk mencari... Status Verifikasi Email: Pilih Status Email Status Verifikasi KYC: Pilih Status KYC

Pengguna	No. HP	Status Akun	Verifikasi KYC	Tgl Bergabung	Aksi
D Damianus Widi damianuswidi17@gmail.com	-	Terverifikasi	Belum	11 Jun 2025	✖
N Nazil Makarim Ramadhany ramadhanynazilmakarim@gmail.com	-	Terverifikasi	Belum	11 Jun 2025	✖
M markus rr markusrr10@gmail.com	-	Terverifikasi	Belum	11 Jun 2025	✖
F fikri arjjan fikri@intellove.com	-	Terverifikasi	Belum	11 Jun 2025	✖
G Gus Cursor guscursor@gmail.com	-	Terverifikasi	Belum	11 Jun 2025	✖
E eka eka eka@intellove.com	-	Terverifikasi	Belum	10 Jun 2025	✖
J jipp java jipp.java@gmail.com	-	Terverifikasi	Belum	10 Jun 2025	✖
E Evi Nur Hidayah evi@intellove.com	-	Terverifikasi	Belum	10 Jun 2025	✖
D Daffa Haj Tsaqif nurutahaj00@gmail.com	-	Terverifikasi	Belum	10 Jun 2025	✖

Gambar 4.54 Tampilan Halaman Pengaturan User Superadmin

Pada Gambar 4.54 terlihat tampilan halaman pengaturan pengguna (user) untuk Superadmin. Halaman ini dirancang untuk memungkinkan Superadmin mengelola semua akun pengguna akhir yang terdaftar di sistem. Superadmin dapat memantau, mengaktifkan, menonaktifkan pengguna, memastikan integritas dan keamanan data pengguna serta memberikan dukungan yang diperlukan.

Manajemen Transaksi

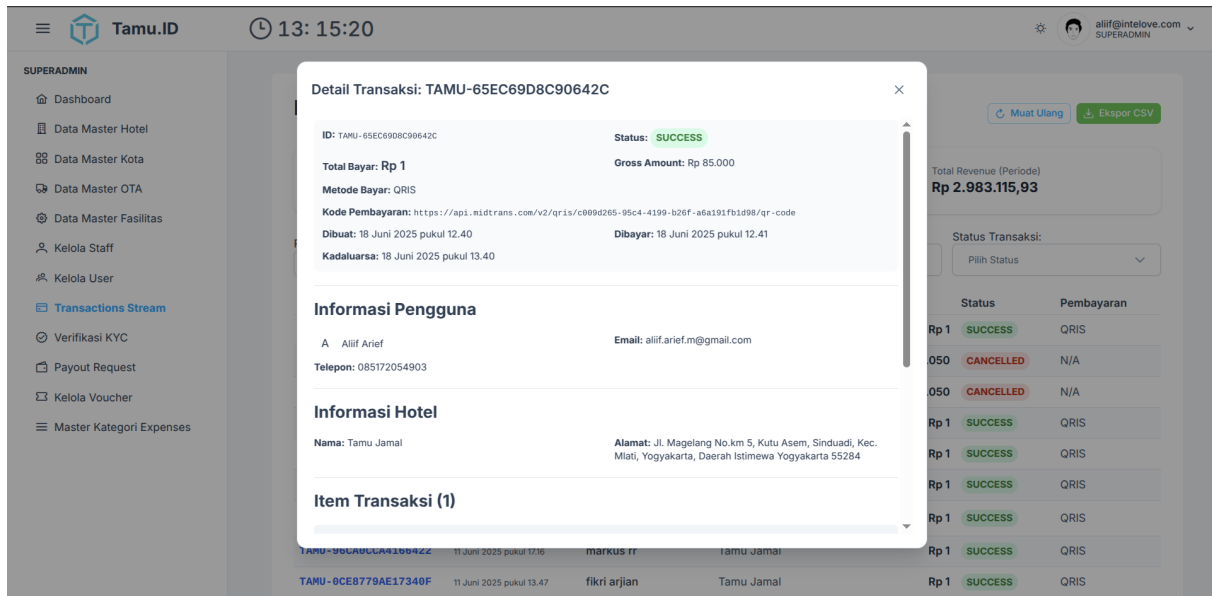
Transaksi Sukses: 38 Transaksi Gagal/Batal: 12 Total Revenue (Periode): Rp 2.983.115,93

Rentang Tanggal: July 2024 - June 2025 Cari ID Trx / Nama User: ID atau Nama... Metode Pembayaran: Pilih Metode Status Transaksi: Pilih Status

ID Transaksi	Tgl Transaksi	Pengunjung	Cabang	Total Bayar	Status	Pembayaran
TAMU - 65EC69D8C9642C	18 Juni 2025 pukul 12.40	Alif Arief	Tamu Jamal	Rp 1	SUCCESS	QRIS
TAMU - 4DB1E32403F4458	18 Juni 2025 pukul 12.39	Alif Arief	Tamu Jamal	Rp 79.050	CANCELLED	N/A
TAMU - BC6EB565632D48E	18 Juni 2025 pukul 12.38	Alif Arief	Tamu Jamal	Rp 79.050	CANCELLED	N/A
TAMU - 87CDF3D5BE35400	18 Juni 2025 pukul 12.27	Alif Arief	Tamu Jamal	Rp 1	SUCCESS	QRIS
TAMU - EC2F931BF5C44EF	17 Juni 2025 pukul 12.48	Alif Arief	Tamu Jamal	Rp 1	SUCCESS	QRIS
TAMU - A18DD1262820431	11 Juni 2025 pukul 21.08	Damianus Widi	Tamu Jamal	Rp 1	SUCCESS	QRIS
TAMU - FADC377FE9A6492	11 Juni 2025 pukul 17.25	Nazil Makarim Ramadhany	Tamu Jamal	Rp 1	SUCCESS	QRIS
TAMU - 96CA0CCA4166422	11 Juni 2025 pukul 17.16	markus rr	Tamu Jamal	Rp 1	SUCCESS	QRIS
TAMU - 0CE8779AE17340F	11 Juni 2025 pukul 13.47	fikri arjjan	Tamu Jamal	Rp 1	SUCCESS	QRIS

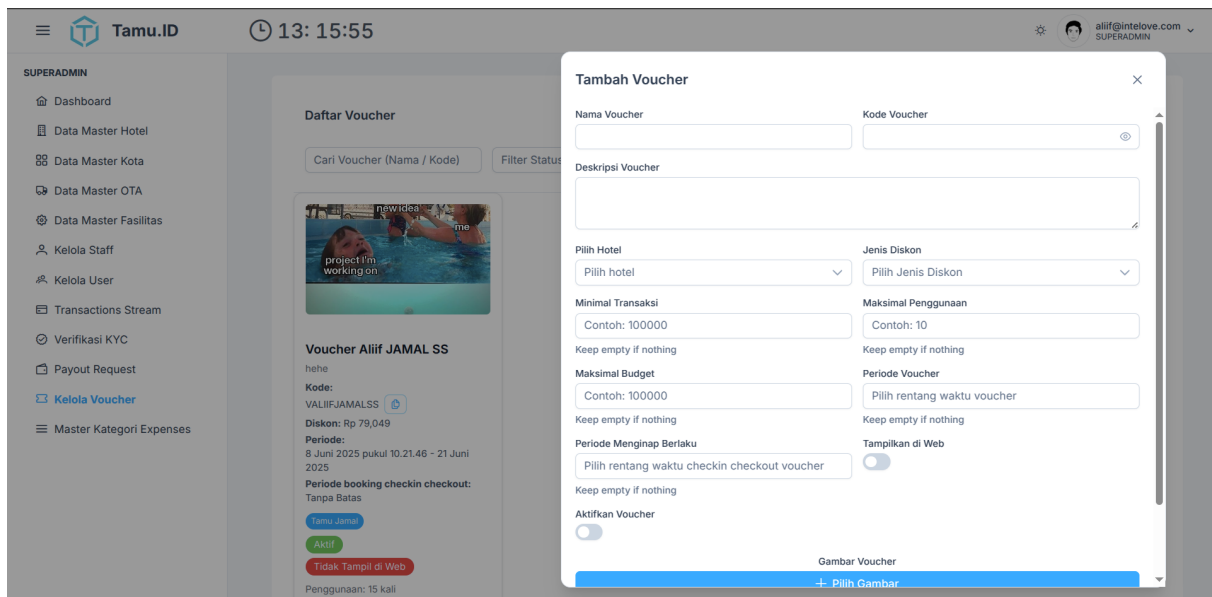
Gambar 4.55 Tampilan Halaman List Transaksi Superadmin

Gambar 4.55 menunjukkan halaman daftar transaksi untuk Superadmin. Halaman ini menyajikan ringkasan komprehensif dari semua transaksi yang telah terjadi dalam sistem. Superadmin dapat melihat daftar transaksi, memfilter berdasarkan kriteria tertentu seperti tanggal atau status, dan mendapatkan gambaran umum aktivitas finansial, yang esensial untuk pemantauan dan audit.



Gambar 4.56 Tampilan Halaman Lihat Detail Transaksi Superadmin

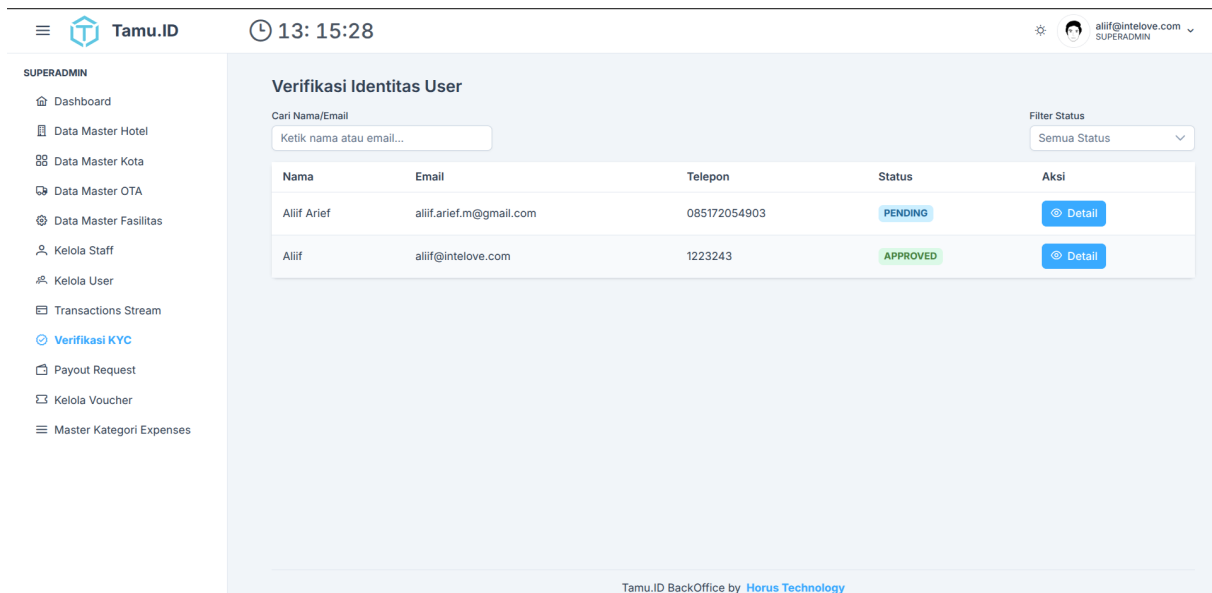
Pada Gambar 4.56 diperlihatkan tampilan halaman detail transaksi untuk Superadmin. Setelah memilih transaksi dari daftar, Superadmin dapat mengakses halaman ini untuk melihat informasi lengkap dan mendetail mengenai transaksi tersebut. Ini mencakup rincian produk/layanan, jumlah pembayaran, metode pembayaran, status transaksi, dan informasi terkait lainnya, memungkinkan investigasi atau verifikasi transaksi secara mendalam.



Gambar 4.57 Tampilan Halaman Pengaturan Voucher Superadmin

Gambar 4.57 menampilkan antarmuka halaman pengaturan voucher untuk Superadmin. Halaman ini berfungsi sebagai alat bagi Superadmin untuk membuat, mengelola, dan mendistribusikan voucher diskon atau promosi. Superadmin dapat menentukan kode voucher, nilai diskon, tanggal berlaku, dan kondisi penggunaan, yang merupakan alat penting untuk strategi pemasaran dan retensi pelanggan.

C. Proses dan Verifikasi Permintaan oleh Superadmin



Gambar 4.58 Tampilan Halaman list Permintaan KYC User di Superadmin

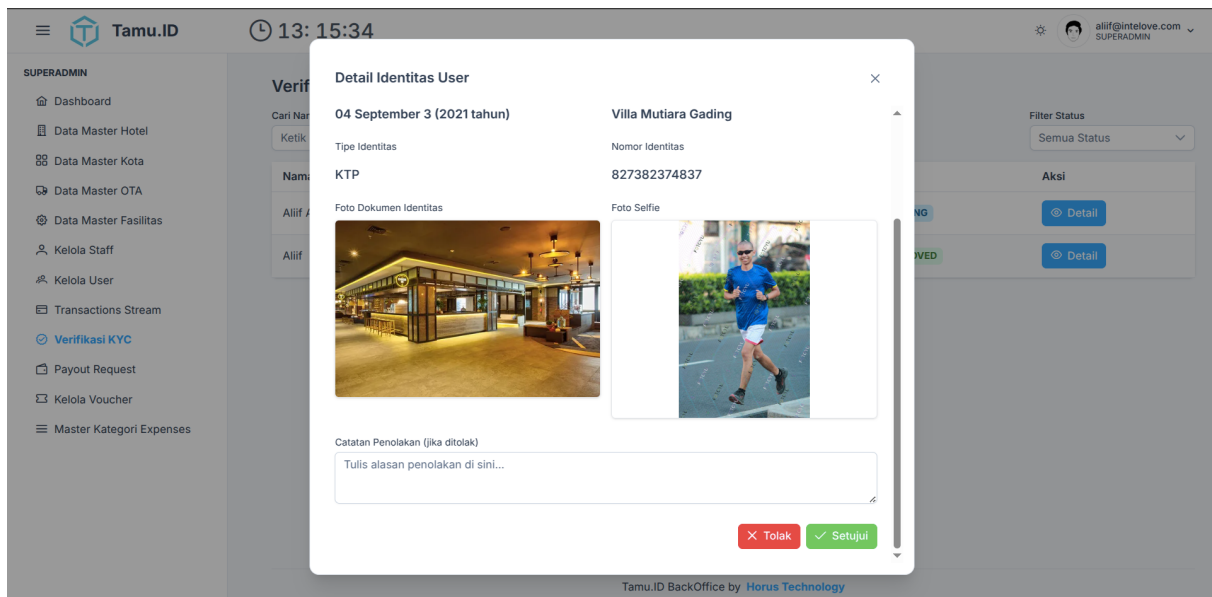
Pada Gambar 4.58 ditampilkan halaman daftar permintaan KYC (Know Your Customer) untuk pengguna di panel Superadmin. Halaman ini berfungsi sebagai pusat bagi Superadmin

untuk meninjau dan mengelola semua pengajuan verifikasi identitas dari pengguna. Di sini, Superadmin dapat melihat daftar permintaan yang masuk, statusnya, dan informasi awal yang diberikan pengguna, yang merupakan langkah krusial dalam memastikan keamanan dan kepatuhan regulasi.

```
1 def validate_referral_code(referral_code: str, transaction_id: str) -> bool:
2     try:
3         transaction = Transaction.query.filter_by(id=transaction_id).first()
4         user_referring = User.query.filter_by(username=referral_code).first()
5         referral = Referral.query.filter_by(transaction_id=transaction.id).first()
6         if referral:
7             return False
8         if user_referring.id == transaction.user_id:
9             return False
10        if not user_referring:
11            logging.info(f"Referral code {referral_code} not found.")
12            return False
13        user_referring_status = Kyc.query.filter_by(user_id=user_referring.id).first()
14        if not user_referring_status:
15            logging.info(f"User {user_referring.username} has no KYC status.")
16            return False
17        if user_referring_status.status != KYCStatusEnum.APPROVED:
18            logging.info(f"User {user_referring.username} has not been approved for KYC.")
19            return False
20        return True
21    except Exception as e:
22        logging.error(f"Error validating referral code: {e}")
23        return False
```

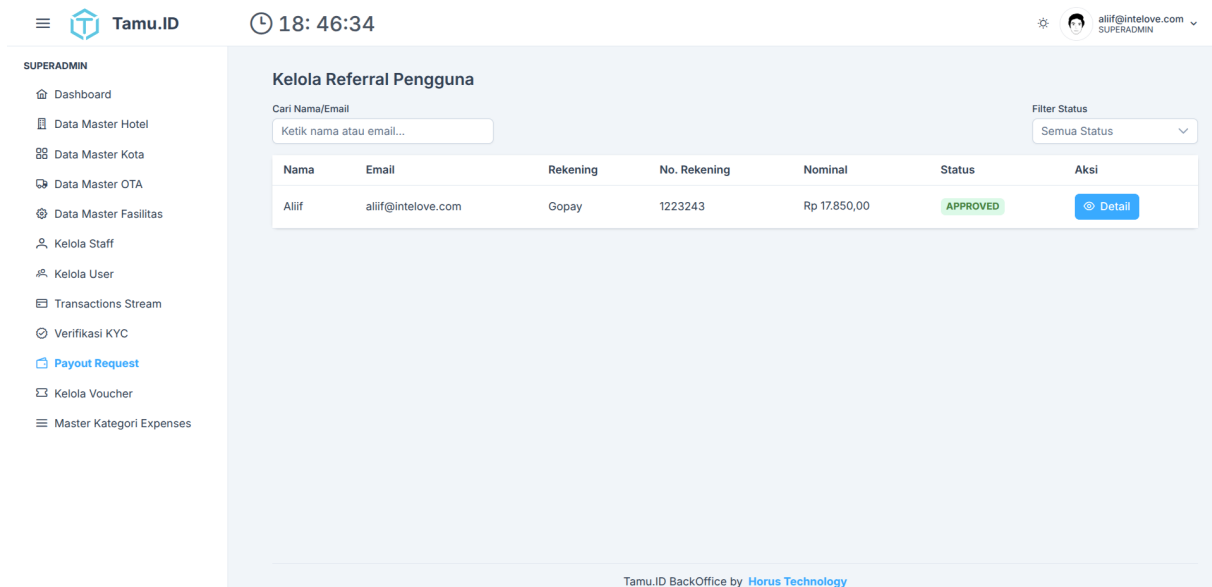
Kode 4.25: Implementasi Validasi ketika *Redeem* kode Referral

Kode 4.25 menunjukkan implementasi validasi ketika pengguna melakukan *Redeem* kode referral, dengan peraturan ini diharapkan dapat memancing pengguna untuk melakukan verifikasi identitasnya karena untuk mendapatkan komisi referral pengguna harus terverifikasi terlebih dahulu. Hal ini juga untuk memastikan bahwa pengguna yang melakukan *Redeem* kode referral adalah pengguna yang sah dan telah melalui proses *Know Your Customer* (KYC) yang benar.



Gambar 4.59 Tampilan Halaman Proses KYC User di Superadmin

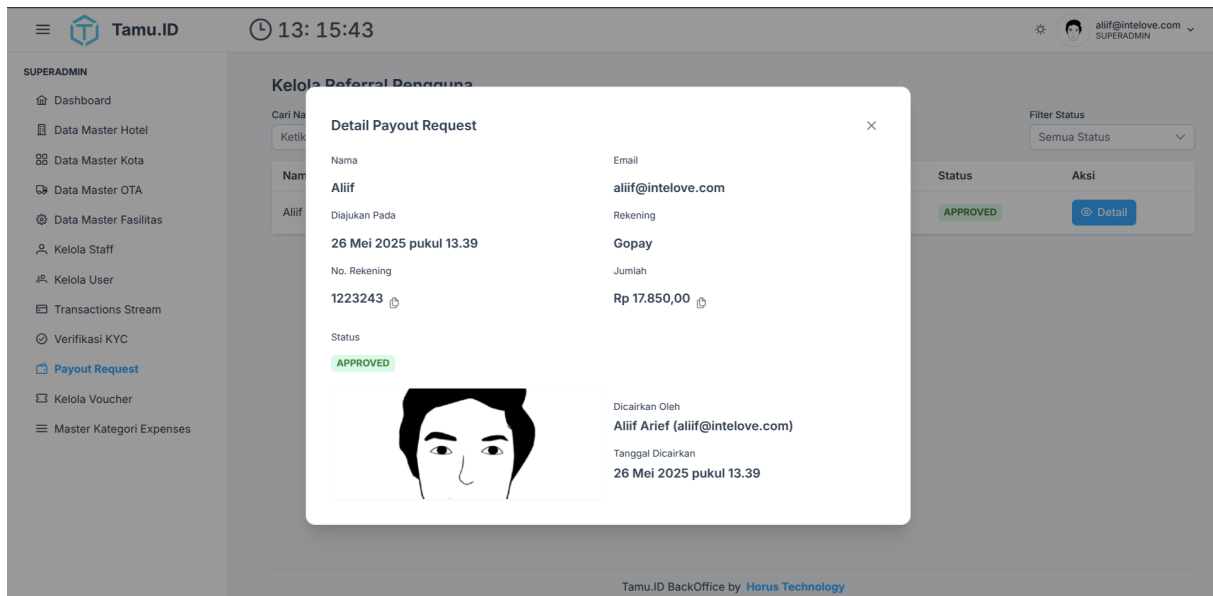
Gambar 4.59 menyajikan tampilan halaman proses KYC pengguna di Superadmin. Setelah memilih permintaan dari daftar KYC, Superadmin akan diarahkan ke halaman ini untuk melakukan verifikasi secara mendalam. Halaman ini memungkinkan Superadmin untuk melihat detail dokumen identitas yang diunggah, membandingkannya dengan data pengguna, dan kemudian memutuskan apakah pengajuan KYC diterima atau ditolak. Proses ini memastikan bahwa setiap pengguna terverifikasi sesuai standar yang ditetapkan.



Gambar 4.60 Tampilan Halaman List Permintaan Payout User di Superadmin

Pada Gambar 4.60 diperlihatkan halaman daftar permintaan payout atau penarikan dana dari pengguna di panel Superadmin. Halaman ini memberikan gambaran umum tentang semua

permintaan penarikan dana yang diajukan oleh pengguna. Superadmin dapat melihat daftar permintaan, jumlah yang diminta, status, dan informasi relevan lainnya, yang membantu dalam memantau dan memproses pembayaran secara efisien.



Gambar 4.61 Tampilan Halaman Proses Payout Referral User di Superadmin

Gambar 4.61 menampilkan tampilan halaman proses payout referral pengguna di Superadmin. Halaman ini مخصوص untuk memproses permintaan penarikan dana yang berasal dari program referral pengguna. Superadmin dapat meninjau detail komisi referral yang akan dicairkan, memverifikasi informasi sumber dananya, memutuskan untuk diterima ataupun tidak serta bila diterima maka superadmin melakukan transfer ke rekening pengguna lalu mengunggah bukti serta mengubah statusnya menjadi diterima namun bila menolaknya maka sertakan alasan penolakannya.

```

1 def redeem_referral(referral_code: str, transaction_id:str):
2     try:
3         transaction = Transaction.query.filter_by(id=transaction_id).first()
4         referrer = User.query.filter_by(username=referral_code).first()
5         referred = User.query.filter_by(id=transaction.user_id).first()
6         discount_amount = transaction.amount * 0.07
7         new_referral = Referral(
8             transaction_id=transaction.id,
9             referrer_user_id=referrer.id,
10            referred_user_id=referred.id,
11            referral_code=referral_code,
12            discount_amount=discount_amount,
13            referrer_income=discount_amount,
14        )
15        db.session.add(new_referral)
16        transaction.amount -= discount_amount
17        db.session.commit()
18        return discount_amount, referral_code
19    except Exception as e:
20        logging.error(f"Error rolling back referral: {e}")

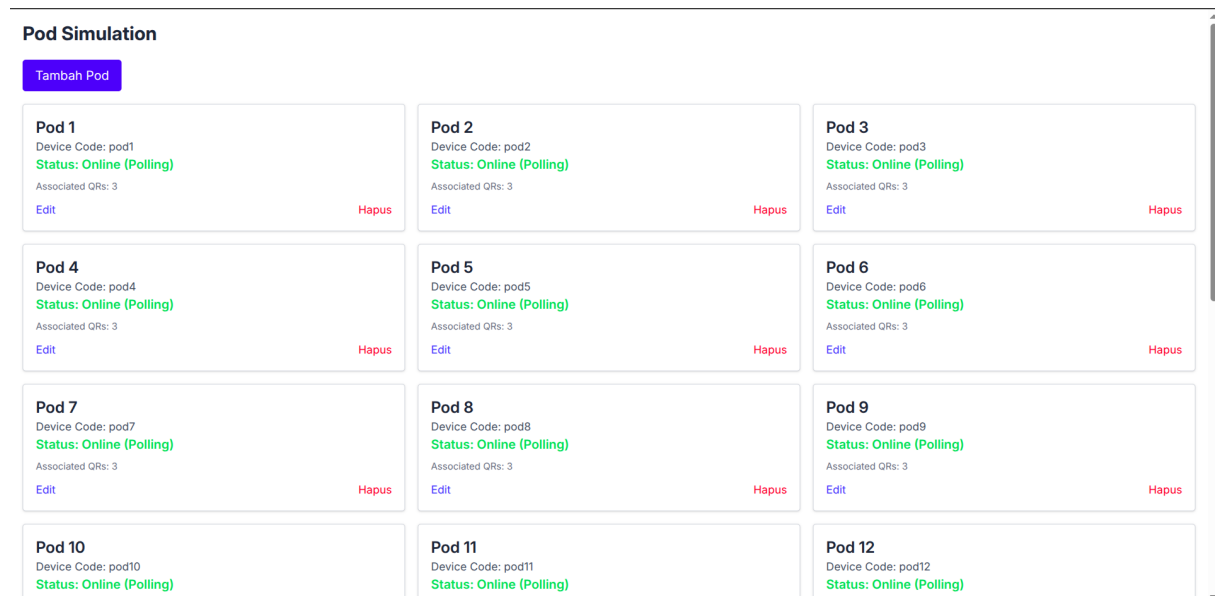
```

Kode 4.26: Implementasi Perhitungan Komisi dan Diskon Referral

Kode 4.26 menunjukkan implementasi perhitungan komisi referral yang akan diberikan kepada pengguna yang telah berhasil mengajak pengguna baru untuk melakukan reservasi, selain mendapatkan komisi referral sebesar 7% dari total transaksi yang menggunakan kodenya, transaksi yang dilakukan juga akan mendapatkan diskon sebesar 7% dari total transaksi, sehingga pengguna yang diajak merasa diuntungkan dengan adanya diskon tersebut. Hal ini diharapkan dapat meningkatkan partisipasi pengguna dalam program referral Tamu.id.

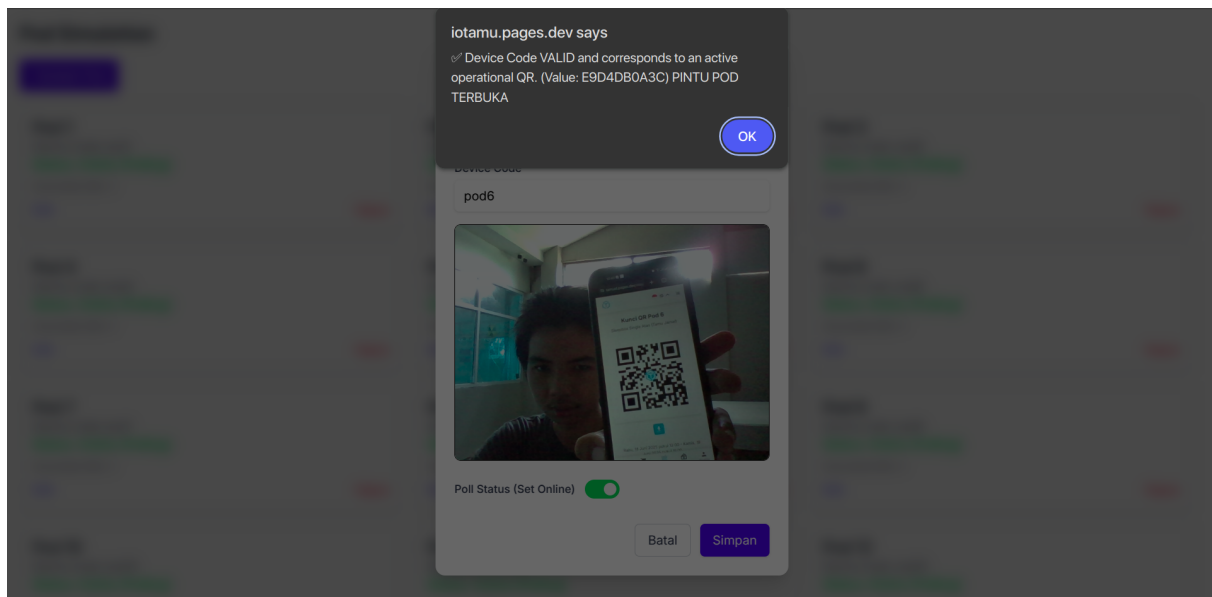
4.1.4 Halaman Simulasi Hardware

Halaman simulasi hardware adalah halaman yang ditujukan untuk mensimulasikan bagaimana integrasi antara backend dan hardware IoT dapat berjalan dengan baik.



Gambar 4.62 Tampilan List Hardware

Padaha gambar 4.62 dapat terlihat ada beberapa pod yang sudah dibuat dengan menginputkan device code untuk masing - masing pod nya sebagai identifikasi untuk pod saat terhubung dengan *Backend*.



Gambar 4.63 Tampilan QR Code Berhasil

Maka selanjutnya pada gambar 4.63 adalah tampilan setelah pengguna *Checkin* dan mendapatkan kunci berupa kode QR maka dapat discan ke pod yang ia pilih dan pod tersebut akan terbuka, namun bila pengguna salah pod maka pod tidak akan terbuka.

```

1 {
2   "qr": [
3     {
4       "until": "1748670431",
5       "value": "052322BED6"
6     },
7     {
8       "until": "",
9       "value": "A8982BC796"
10    },
11    {
12      "until": "",
13      "value": "tamumasterkey"
14    }
15  ]
16 }

```

Kode 4.27: Implementasi *Customized Linkedlist* dengan *Unix Timestamp* pada *Hardware IoT*

Pada kode 4.27 merupakan implementasi dari struktur data *Customized Linkedlist* dengan *Unix Timestamp* pada *Hardware IoT* yang dapat membuat integrasi saat pengguna melakukan *Checkin* dan langsung menggunakan *QRCode* yang ia dapatkan secara *Seamless* tanpa delay dan walaupun Pod dalam kondisi *Offline* sesaat.

```

1 def get_pod_key():
2     try:
3         device_code = request.args.get("device_code")
4         if not device_code:
5             return jsonify({"error": "device_code is required"}), 400

```

```

6     pod = Pod.query.filter_by(device_code=device_code).first()
7     if not pod:
8         return jsonify({"error": "Pod not found"}), 404
9     pod.last_online = int(datetime.now().timestamp())
10    db.session.commit()
11    qrcode = QRCode.query.filter_by(pod_id=pod.id).order_by(QRCode.current_expired.desc())
12    .first()
13    response = {
14        "qr": []
15    }
16    if qrcode:
17        if qrcode.current_code:
18            response["qr"].append({
19                "value": qrcode.current_code,
20                "until": str(qrcode.current_expired),
21            })
22        if qrcode.next_code:
23            response["qr"].append({
24                "value": qrcode.next_code,
25                "until": "",
26            })
27    master_key = os.getenv("POD_MASTER_KEY")
28    if master_key:
29        response["qr"].append({
30            "value": master_key,
31            "until": "",
32        })
33    return jsonify(response), 200
34 except Exception as e:
35     logging.error(f"Error in get_pod_key: {e}", exc_info=True)
36     return jsonify({"error": "Internal server error"}), 500

```

Kode 4.28: Implementasi Integrasi *Backend* dengan *Hardware IoT*

Kode 4.28 merupakan implementasi dari integrasi *Backend* dengan *Hardware IoT* yang dapat menghubungkan antara *Backend* dan *Hardware IoT* dengan baik, sehingga *Hardware IoT* selalu dapat terhubung dengan *Backend* dan *User* dapat masuk ke dalam Pod menggunakan kunci Pod yang telah didapatkan saat *Checkin*.

4.1.5 Analisis Implementasi Oauth2 Google dengan One Time Token (OTT)

4.1.6 Analisis Integrasi QRIS dengan Backend Sistem

4.1.7 Analisis *Offline Fault Tolerance* pada *Hardware IoT*

4.2 Hasil Pengujian

Pengujian sistem dilakukan untuk mengetahui apakah sistem yang telah dibuat dapat berjalan sesuai dengan yang diharapkan. Pengujian ini dilakukan dengan cara melakukan uji coba terhadap setiap fitur yang ada pada sistem, baik dari sisi *User* maupun dari sisi admin.

4.2.1 Pengujian Blackbox

Pengujian Blackbox adalah pengujian yang dilakukan untuk mengetahui apakah sistem yang telah dibuat dapat berjalan sesuai dengan yang diharapkan. Pengujian ini dilakukan dengan cara melakukan uji coba terhadap setiap fitur yang ada pada sistem, baik dari sisi *User*

pada tabel 4.1, *Admin* pada tabel 4.6, dan *SuperAdmin* pada tabel 4.3. Pengujian ini dilakukan tanpa melihat kode program yang ada pada sistem dan hanya berfokus pada input dan output dari interaksi antara pengguna dengan sistem.

Tabel 4.1 Hasil Pengujian Black Box (User)

Skenario Pengujian	Langkah Pengujian	Hasil yang diharapkan	Kesesuaian Hasil
Fitur Registrasi Akun			
Registrasi berhasil dengan email baru	1. Buka halaman registrasi. 2. Isi nama lengkap. 3. Masukkan email yang belum terdaftar (format valid). 4. Masukkan kata sandi kuat. 5. Konfirmasi kata sandi dengan benar. 6. (Jika ada) Centang persetujuan Syarat & Ketentuan. 7. Klik tombol "Daftar".	- Pesan sukses registrasi ditampilkan. - Pengguna diarahkan ke halaman login atau dashboard. - Email konfirmasi terkirim.	Sesuai
Registrasi gagal karena email sudah terdaftar	1. Buka halaman registrasi. 2. Isi nama lengkap. 3. Masukkan email yang sudah terdaftar. 4. Masukkan kata sandi. 5. Konfirmasi kata sandi. 6. Klik tombol "Daftar".	- Pesan error "Email sudah terdaftar" ditampilkan. - Pengguna tetap di halaman registrasi.	Sesuai
Registrasi gagal karena format email salah	1. Buka halaman registrasi. 2. Masukkan email dengan format salah (misal: user@.com, user.com). 3. Isi field lainnya. 4. Klik "Daftar".	- Pesan error "Format email tidak valid" ditampilkan.	Sesuai

Registrasi gagal karena konfirmasi kata sandi tidak cocok	1. Buka halaman registrasi. 2. Isi nama lengkap, email baru, kata sandi. 3. Masukkan konfirmasi kata sandi yang berbeda. 4. Klik tombol "Daftar".	- Pesan error "Konfirmasi kata sandi tidak cocok" ditampilkan.	Sesuai
Registrasi menggunakan akun Google (sukses, pengguna baru)	1. Buka halaman registrasi/login. 2. Klik tombol "Daftar/Masuk dengan Google". 3. Pilih akun Google yang valid dan belum terdaftar di sistem. 4. Izinkan akses jika diminta.	- Akun berhasil terbuat/terhubung. - Pengguna otomatis berhasil masuk.	Sesuai
Fitur Login Pengguna			
Login berhasil dengan email dan kata sandi valid	1. Buka halaman login. 2. Masukkan email terdaftar. 3. Masukkan kata sandi yang benar. 4. Klik tombol "Masuk".	- Login berhasil. - Pengguna berhasil masuk.	Sesuai
Login gagal (email tidak terdaftar)	1. Buka halaman login. 2. Masukkan email yang tidak terdaftar. 3. Masukkan kata sandi. 4. Klik tombol "Masuk".	- Pesan error "Email atau kata sandi salah" ditampilkan.	Sesuai
Login gagal (kata sandi salah)	1. Buka halaman login. 2. Masukkan email terdaftar. 3. Masukkan kata sandi yang salah. 4. Klik tombol "Masuk".	- Pesan error "Email atau kata sandi salah" ditampilkan.	Sesuai

Login gagal (akun belum diverifikasi email, jika ada)	1. Registrasi dengan email (misal: REG_01), jangan verifikasi email. 2. Coba login dengan email dan kata sandi tersebut.	- Pesan error "Akun belum diverifikasi. Silakan cek email Anda." atau sejenisnya.	Sesuai
Login menggunakan akun Google (sukses, akun sudah ada)	1. Buka halaman login. 2. Klik "Masuk dengan Google". 3. Pilih akun Google yang sudah terdaftar/terhubung sebelumnya.	- Login berhasil. - Pengguna diarahkan ke dashboard.	Sesuai
Fitur Lupa Kata Sandi			
Proses lupa kata sandi berhasil	1. Di halaman login, klik "Lupa Kata Sandi?". 2. Masukkan email terdaftar. 3. Klik "Kirim Instruksi". 4. Buka email, klik link reset. 5. Masukkan kata sandi baru dan konfirmasinya. 6. Klik "Reset Kata Sandi".	- Pesan sukses "Kata sandi berhasil direset". - Dapat login dengan kata sandi baru.	Sesuai
Lupa kata sandi dengan email tidak terdaftar	1. Di halaman login, klik "Lupa Kata Sandi?". 2. Masukkan email yang tidak terdaftar. 3. Klik "Kirim Instruksi".	- Pesan error "Email tidak ditemukan" atau instruksi tidak terkirim.	Sesuai
Fitur Kelola Profil Pengguna			
Mengubah semua data profil yang diizinkan (sukses)	1. Login. 2. Ke halaman "Profil Saya" lalu "Edit Profil". 3. Ubah nama, nomor telepon, alamat, gender, tanggal lahir. 4. Klik "Simpan".	- Pesan sukses "Profil berhasil diperbarui". - Semua perubahan tercermin dengan benar.	Sesuai

Mengubah data profil dengan format salah (misal: telepon)	1. Login. 2. Ke "Edit Profil". 3. Masukkan nomor telepon dengan format salah (misal: huruf). 4. Klik "Simpan".	- Pesan error "Format nomor telepon tidak valid". - Data tidak tersimpan.	Sesuai
Fitur Verifikasi Identitas (KYC)			
Mengajukan verifikasi KYC (KTP) dengan data valid	1. Login. 2. Ke halaman KYC. 3. Pilih jenis ID "KTP". 4. Masukkan nomor KTP valid. 5. Unggah foto KTP jelas. 6. Unggah foto selfie jelas. 7. Klik "Ajukan".	- Pengajuan KYC berhasil dikirim. - Status KYC menjadi "Menunggu Verifikasi".	Sesuai
Mengajukan KYC (Passport) dengan data valid	1. Login. 2. Ke halaman KYC. 3. Pilih jenis ID "Passport". 4. Masukkan nomor Passport valid. 5. Unggah foto Passport jelas. 6. Unggah foto selfie jelas. 7. Klik "Ajukan".	- Pengajuan KYC berhasil dikirim. - Status KYC menjadi "Menunggu Verifikasi".	Sesuai
Mengajukan KYC dengan file gambar terlalu besar/format salah	1. Login. 2. Ke halaman KYC. 3. Isi data. 4. Unggah file gambar dengan ukuran > batas maksimal atau format bukan JPG/PNG. 5. Klik "Ajukan".	- Pesan error "Ukuran file terlalu besar" atau "Format file tidak didukung".	Sesuai
Melihat status KYC (Pending, Approved, Rejected)	1. Login. 2. Navigasi ke halaman KYC atau Profil.	- Status KYC pengguna ditampilkan dengan benar sesuai kondisi aktual.	Sesuai
Fitur Pencarian dan Reservasi			

Mencari pod (tanggal check-out sebelum check-in)	1. Login. 2. Pilih Cabang. 3. Pilih tanggal check-in. 4. Pilih tanggal check-out lebih awal dari check-in. 5. Klik "Cari Pod".	- Pesan error "Tanggal check-out tidak valid".	Sesuai
Melakukan pemesanan pod dengan voucher kadaluarsa	1. Lakukan pencarian pod. 2. Pilih tipe pod. 3. Di halaman pembayaran, masukkan kode voucher yang sudah kadaluarsa. 4. Klik "Terapkan Voucher".	- Pesan error "Voucher tidak valid atau sudah kadaluarsa".	Sesuai
Pemesanan pod dengan jumlah melebihi kapasitas tipe pod	1. Lakukan pencarian pod. 2. Pilih tipe pod. 3. Coba pesan jumlah tamu yang melebihi kapasitas maksimal dewasa/anak tipe pod tersebut.	- Pesan error "Jumlah tamu melebihi kapasitas" atau input dibatasi.	Sesuai
Pemesanan pod, pembayaran berhasil (QRIS)	1. Lakukan pencarian pod. 2. Pilih tipe pod. 3. Lanjutkan ke pembayaran. 4. Pilih metode QRIS, pindai QR. 5. Konfirmasi pembayaran di aplikasi e-wallet.	- Pembayaran berhasil. - Tampilan menunggu dibayar berubah menjadi terbayar - Transaksi tercatat.	Sesuai
Pemesanan pod, menggunakan kode referral teman (valid, aturan disesuaikan)	1. Lakukan pencarian pod. 2. Pilih tipe pod. 3. Di halaman pembayaran, masukkan kode referral valid milik teman. 4. Klik "Terapkan". 5. Selesaikan pembayaran.	- Diskon referral berhasil diterapkan. - Harga total berkurang. - Pemesanan berhasil.	Sesuai
Fitur Sinkronisasi OTA			

Sinkronisasi reservasi OTA dengan kode valid	1. Login. 2. Ke halaman "Sinkronisasi OTA" atau "Klaim Reservasi". 3. Pilih OTA. 4. Masukkan kode booking OTA / nomor order OTA yang valid (sudah diinput Admin Cabang). 5. Klik "Sinkronisasi".	- Reservasi OTA berhasil disinkronkan/diklaim. - Muncul di daftar booking pengguna.	Sesuai
Sinkronisasi reservasi OTA dengan kode tidak ditemukan/salah	1. Login. 2. Ke halaman "Sinkronisasi OTA". 3. Masukkan kode booking OTA yang salah. 4. Klik "Sinkronisasi".	- Pesan error "Reservasi tidak ditemukan" atau "Kode booking salah".	Sesuai
Fitur Operasional Menginap			
Check-in berhasil	1. Login. 2. Buka halaman stays lalu klik tab current 3. Coba klik tombol "Check-in" 4. Pilih pod yang diinginkan 5. klik checkin	- berhasil check-in dan mendapatkan kunci pod yang diinginkan berupa QRCode	Sesuai
Check-in gagal (di luar jam operasional atau sebelum waktunya check-in)	1. Login. 2. Buka halaman stays. 3. Coba klik tombol "Check-in" sebelum jam check-in yang diizinkan atau setelah batas waktu check-in.	- Tombol "Check-in" tidak ada untuk booking yang statusnya masih upcoming	Sesuai

Check-out	1. Login. 2. Buka detail booking aktif yang sudah check-in. 3. Klik tombol "Check-out" 4. muncul modal rating (user wajib mengisi rating untuk isi review opsional) 5. klik kirim	- Check-out berhasil. - Status booking "Completed". rating dan review juga berhasil dikirim	Sesuai
Fitur Referral			
Melihat riwayat pendapatan referral	1. Login. 2. Ke halaman Referral. 3. Pilih tab "Riwayat Pendapatan".	- Menampilkan daftar transaksi yang menghasilkan pendapatan referral dengan statusnya.	Sesuai
Mengajukan pencairan referral (KYC belum disetujui)	1. Login, KYC masih "Pending" atau "Rejected". 2. Ke halaman Referral. 3. Coba klik "Cairkan Dana".	- Tombol "Cairkan Dana" tidak aktif atau pesan error "Verifikasi KYC diperlukan untuk pencairan".	Sesuai
Fitur Ulasan			
Memberikan ulasan tanpa rating bintang	1. Login, setelah checkout. 2. Ke Riwayat Booking, pilih tab oldest booking. 3. Klik "Beri Ulasan". 4. Hanya tulis komentar, tidak memilih bintang. 5. Coba Kirim.	- tombol kirim review tidak dapat di klik	Sesuai
Mencoba memberi ulasan untuk booking yang belum selesai/dibatalkan	1. Login. 2. Cari booking yang statusnya "Pending Payment" atau "Cancelled". 3. Coba cari opsi "Beri Ulasan".	- Opsi "Beri Ulasan" tidak tersedia untuk booking tersebut.	Sesuai

Tabel 4.2 Hasil Pengujian Black Box (Admin)

Skenario Pengujian	Langkah Pengujian	Hasil yang diharapkan	Kesesuaian Hasil
Fitur Login dan Profil Admin Cabang			
Login Admin Cabang berhasil	1. Buka halaman login admin. 2. Masukkan email Admin Cabang valid. 3. Masukkan kata sandi benar. 4. Klik "Masuk".	- Login berhasil. - Diarahkan ke halaman pilih cabang yang ingin dikelola Cabang.	Sesuai
Login Admin Cabang gagal (kata sandi salah)	1. Buka halaman login admin. 2. Masukkan email Admin Cabang valid. 3. Masukkan kata sandi salah. 4. Klik "Masuk".	- Pesan error "Email atau kata sandi salah". - Tetap di halaman login.	Sesuai
Mengubah profil Admin Cabang (misal: nama)	1. Login sebagai Admin Cabang. 2. Navigasi ke "Profil Saya". 3. Klik "Edit Profil". 4. Ubah nama. 5. Klik "Simpan".	- Nama Admin Cabang berhasil diperbarui.	Sesuai
Fitur Pengelolaan Operasional Cabang			
Mengedit semua informasi detail cabang yang diizinkan	1. Login. 2. Ke "Pengaturan data Cabang". 3. Ubah nama, alamat, telepon, email, deskripsi, jam check-in/out, gambar utama. 4. Klik "Simpan".	- Semua perubahan berhasil disimpan dan ditampilkan dengan benar.	Sesuai

Menambah tipe pod baru dengan semua field valid	1. Login. 2. Ke "Menu jenis pod" 3. Klik "Tambah Baru". 4. Isi nama, deskripsi, harga, kapasitas (dewasa, anak), gambar, fasilitas terkait, status breakfast. 5. Klik "Simpan".	- Tipe pod baru berhasil ditambahkan dengan semua detailnya.	Sesuai
Menghapus tipe pod yang masih memiliki pod aktif	1. Login. 2. Ke "Manajemen Pod" - > "Tipe Pod". 3. Pilih tipe pod yang memiliki pod terdaftar di bawahnya. 4. Klik "Hapus". 5. Konfirmasi.	- Pesan error "Tidak dapat menghapus tipe pod karena masih ada pod yang terhubung" atau sejenisnya. - Tipe pod tidak terhapus.	Sesuai
Menambah pod baru dengan device code IoT unik	1. Login. 2. Ke "Manajemen Pod" - > "pilih jenis pod yang pod nya ingin ditambahkan". 3. Klik "Tambah Pod". 4. Isi nama/nomor pod, pilih tipe pod, masukkan device code IoT yang unik. 5. Klik "Simpan".	- Pod baru berhasil ditambahkan. - Status awal "Tersedia".	Sesuai
Menambah pod baru dengan device code IoT yang sudah ada	1. Login. 2. Ke "Manajemen Pod" - > "Daftar Pod". 3. Klik "Tambah Pod". 4. Masukkan device code IoT yang sudah digunakan pod lain. 5. Klik "Simpan".	- Pesan error "Device code sudah digunakan". - Penambahan pod gagal.	Sesuai

Mengubah status pod menjadi "Perlu Dibersihkan" secara manual	1. Login. 2. Ke "Manajemen Pod" -> "Daftar Pod". 3. Pilih pod yang "Tersedia", klik "Edit". 4. Ubah status ke "Perlu Dibersihkan". 5. Klik "Simpan".	- Status pod berhasil diubah.	Sesuai
Membuat voucher diskon nominal untuk cabang	1. Login. 2. Ke "Manajemen Voucher Cabang". 3. Klik "Tambah Voucher". 4. Isi kode, nama, deskripsi, tipe "NOMINAL", jumlah diskon, min transaksi, periode berlaku, maks penggunaan. 5. Klik "Simpan".	- Voucher cabang berhasil dibuat.	Sesuai
Membuat voucher diskon persentase dengan batas maks diskon	1. Login. 2. Ke "Manajemen Voucher Cabang". 3. Klik "Tambah Voucher". 4. Isi kode, nama, tipe "PERSENTASE", nilai persentase, diskon maksimal, min transaksi, periode, kuota. 5. Klik "Simpan".	- Voucher cabang berhasil dibuat.	Sesuai

Menetapkan harga dinamis untuk akhir pekan	1. Login. 2. Ke "Harga Dinamis". 3. Pilih tipe pod. 4. Pilih rentang tanggal mencakup beberapa akhir pekan. 5. Masukkan harga lebih tinggi. 6. Klik "Simpan".	- Harga dinamis untuk akhir pekan berhasil disimpan.	Sesuai
Menetapkan harga dinamis yang tumpang tindih dengan aturan lain	1. Login. 2. Ke "Harga Dinamis". 3. Buat aturan harga dinamis baru yang tanggalnya tumpang tindih dengan aturan yang sudah ada untuk tipe pod yang sama.	- Sistem memberikan peringatan bahwa aturan harga ada yang tumpang tindih dan aturan harga baru tidak berhasil disimpan	Sesuai
Fitur Manajemen Reservasi dan Tamu Cabang			
Input manual transaksi OTA dengan semua field lengkap	1. Login. 2. Ke "Transaksi OTA". 3. Klik "Tambah". 4. Isi Jenis OTA, ID Order OTA, nama tamu jika ada, etanggal check-in/out, tipe pod, jumlah tamu, dan harga di OTA 5. Klik "Simpan".	- Transaksi OTA berhasil dicatat.	Sesuai
Input manual transaksi OTA dengan field wajib kosong	1. Login. 2. Ke "Transaksi OTA". 3. Klik "Tambah". 4. Kosongkan salah satu field wajib (misal: ID Order OTA). 5. Coba klik "Simpan".	- Tombol simpan tidak bisa diklik karena masih ada field wajib yang kosong	Sesuai

Memindahkan tamu ke pod lain karena ada masalah di pod asal	1. Login. 2. Cari booking tamu aktif. 3. Pilih "Pindah Pod", berikan alasan (misal: AC rusak). 4. Pilih pod lain yang kosong, tipe sama. 5. Konfirmasi.	- Tamu berhasil dipindahkan. - Pod asal mungkin statusnya berubah jadi "Maintenance".	Sesuai
Mencatat pembersihan pod dengan unggah foto bukti	1. Login. 2. Ke "Log Kebersihan" atau daftar pod. 3. Pilih pod "Perlu Dibersihkan". 4. Klik "Sudah Dibersihkan". 5. Unggah foto bukti kebersihan. 6. (Jika ada) Tambahkan catatan. 7. Simpan.	- Status pod "Tersedia". - Foto bukti dan catatan tersimpan di log.	Sesuai
Fitur Manajemen Ulasan Cabang			
Menyetujui ulasan pengguna yang konstruktif	1. Login. 2. Ke "Manajemen Ulasan". 3. Pilih ulasan "Pending" yang isinya baik. 4. Klik "Setujui/Tampilkan".	- Ulasan tampil di halaman publik.	Sesuai
Menolak ulasan pengguna yang mengandung spam/kata kasar	1. Login. 2. Ke "Manajemen Ulasan". 3. Pilih ulasan yang berisi spam. 4. Klik "Tolak/Sembunyikan".	- Ulasan tidak tampil dan statusnya diperbarui.	Sesuai

Tabel 4.3 Hasil Pengujian Black Box (SuperAdmin)

Skenario Pengujian	Langkah Pengujian	Hasil yang diharapkan	Kesesuaian Hasil
Fitur Login & Dashboard Global SuperAdmin			
Login SuperAdmin berhasil	1. Buka halaman login admin. 2. Masukkan email SuperAdmin. 3. Masukkan kata sandi yang benar. 4. Klik "Masuk".	- Login berhasil. - Diarahkan ke dashboard SuperAdmin.	Sesuai
Melihat statistik global di dashboard SuperAdmin	1. Login sebagai SuperAdmin. 2. Amati informasi di dashboard (total cabang, total pengguna, total pendapatan global, dll.).	- Menampilkan data statistik agregat dari semua cabang yang akurat.	Sesuai
Fitur Manajemen Data Master Global			
Menambah data cabang (hotel) baru	1. Login sebagai SuperAdmin. 2. Navigasi ke "Manajemen Cabang/Hotel". 3. Klik "Tambah Cabang Baru". 4. Isi semua field yang diperlukan (nama, alamat, kota, kontak, slug unik, dll.). 5. Klik "Simpan".	- Cabang baru berhasil ditambahkan ke sistem. - Cabang muncul di daftar cabang.	Sesuai

Mengedit data cabang (hotel) yang sudah ada	1. Login sebagai SuperAdmin. 2. Navigasi ke "Manajemen Cabang/Hotel". 3. Pilih cabang, klik "Edit". 4. Ubah salah satu field (misal: email kontak). 5. Klik "Simpan".	- Data cabang berhasil diperbarui.	Sesuai
Menambah data kota baru	1. Login sebagai SuperAdmin. 2. Navigasi ke "Manajemen Kota". 3. Klik "Tambah Kota". 4. Masukkan nama kota. 5. Klik "Simpan".	- Kota baru berhasil ditambahkan. - Kota dapat dipilih saat menambah/mengedit cabang.	Sesuai
Menambah akun Admin Cabang baru dan mengassign ke cabang	1. Login sebagai SuperAdmin. 2. Navigasi ke "Manajemen Staff". 3. Klik "Tambah Staff". 4. Isi detail staff (nama, email, password), pilih peran "Admin Cabang". 5. Assign ke satu atau beberapa cabang. 6. Klik "Simpan".	- Akun Admin Cabang baru berhasil dibuat. - Admin Cabang tersebut dapat login dan mengelola cabang yang di-assign.	Sesuai
Menghapus akun staff (Admin Cabang)	1. Login sebagai SuperAdmin. 2. Navigasi ke "Manajemen Staff". 3. Cari akun Admin Cabang. 4. Klik opsi "Hapus".	- Akun Admin Cabang berhasil dihapus. - Admin Cabang tersebut tidak dapat login.	Sesuai

Menambah fasilitas global baru	1. Login sebagai SuperAdmin. 2. Navigasi ke "Manajemen Fasilitas". 3. Klik "Tambah Fasilitas". 4. Masukkan nama fasilitas dan unggah ikon. 5. Klik "Simpan".	- Fasilitas baru berhasil ditambahkan. - Fasilitas dapat di-assign ke hotel atau tipe pod.	Sesuai
Membuat voucher global untuk semua cabang	1. Login. 2. Ke "Manajemen Voucher". 3. Klik "Tambah Voucher". 4. Isi detail (kode, nama, tipe, nilai, periode, kuota). 5. Pilih opsi "Berlaku di semua cabang". 6. Klik "Simpan".	- Voucher global berhasil dibuat. - Dapat digunakan pengguna di semua cabang.	Sesuai
Fitur Verifikasi & Approval Sistem			
Menyetujui permintaan verifikasi KYC pengguna yang valid	1. Login sebagai SuperAdmin. 2. Navigasi ke "Verifikasi KYC". 3. Pilih permintaan KYC yang berstatus "Pending" dengan data dan foto valid. 4. Klik "Setujui".	- Status KYC pengguna berubah menjadi "Approved". - Pengguna mendapatkan pembaharuan. - Pengguna dapat mengakses fitur yang memerlukan KYC.	Sesuai
Menolak permintaan verifikasi KYC pengguna (misal: foto buram)	1. Login sebagai SuperAdmin. 2. Navigasi ke "Verifikasi KYC". 3. Pilih permintaan KYC yang fotonya buram. 4. Klik "Tolak". 5. Masukkan alasan penolakan (misal: "Foto tidak jelas").	- Status KYC pengguna berubah menjadi "Rejected". - Pengguna mendapatkan pembaharuan beserta alasan penolakan.	Sesuai

Menyetujui permintaan pencairan saldo referral yang valid	1. Login sebagai SuperAdmin. 2. Navigasi ke "Permintaan Payout Referral". 3. Pilih permintaan yang valid (saldo pengguna cukup, data bank benar). 4. Klik "Setujui". 5. Proses transfer dana dan unggah buktinya.	- Status permintaan payout berubah menjadi "Approved". - Pengguna mendapatkan pembaharuan.	Sesuai
Menolak permintaan pencairan saldo referral (saldo kurang)	1. Login sebagai SuperAdmin. 2. Navigasi ke "Permintaan Payout Referral". 3. Pilih permintaan dimana saldo aktual pengguna kurang dari yang diajukan. 4. Klik "Tolak". 5. Masukkan alasan.	- Status permintaan payout berubah menjadi "Rejected". - Pengguna mendapatkan pembaharuan.	Sesuai

4.2.2 User Acceptance Test

User Acceptance Test (UAT) dilakukan untuk mengetahui apakah sistem yang telah dibuat dapat diterima oleh *User*. UAT dilakukan dengan cara meminta *User* untuk mencoba sistem dan memberikan feedback terhadap sistem yang telah dibuat, berikut pada tabel 4.4 adalah daftar responden yang mengikuti UAT.

Tabel 4.4 Daftar Responden *User Acceptance Test* (UAT)

No	Nama	Peran (role)	Profesi	Umur
1	Reisika Jenny	<i>SuperAdmin</i> dan <i>Admin</i>	Business Development	22
2	Asraf Rafli Daifuloh	<i>User</i>	Fullstack Programmer	24
3	Damianus Cahyo Widi Nugroho	<i>User</i>	AI Engineer	22

4	Evi Nur Hidayah	User	Mahasiwa	21
5	Moh. Eka Syafrino Nazhifan	User	Fullstack Programmer	19
6	Salsa Ainnur Muharramah	User	Mahasiswa Profesi	22
7	Daffa Haj Tsaqif	User	IoT Engineer	25
8	Markus Rabin Ronaldo	User	Mahasiswa	23
9	Bela Dwi Primasari	User	Sistem Analis	23
10	Nazil Makarim Ramadhany	User	Graphic Designer	21
11	Bagus erwanto	User	IT Support	23

Untuk mendapatkan hasil yang lebih terukur dan seimbang setelah para responden mencoba sistem, penulis memberikan kuesioner yang berisi 10 pertanyaan yang harus dijawab oleh para responden. Kuesioner ini bertujuan untuk mengetahui sejauh mana sistem yang telah dibuat dapat memenuhi kebutuhan *User*.

Tabel 4.5 Hasil Kuesioner *User Acceptance Test* - Perspektif *User*

No.	Pertanyaan	Jawaban				
		STS	TS	N	S	SS
1.	Apakah proses pendaftaran akun dan login ke aplikasi Tamu.id mudah dipahami dan dilakukan?				1	9
2.	Apakah tampilan antarmuka aplikasi Tamu.id menarik dan mudah digunakan secara umum?			1	3	6
3.	Apakah informasi mengenai cabang dan tipe pod (harga, fasilitas, ketersediaan) disajikan dengan jelas?				3	7
4.	Apakah alur proses pencarian dan pemesanan pod mudah diikuti dan efisien?				3	7
5.	Apakah proses pembayaran saat melakukan pemesanan mudah dan terasa aman?			1	1	8
6.	Apakah proses check-in dan check-out melalui aplikasi berjalan dengan lancar?					10

7.	Apakah penggunaan QR Code untuk akses pod (jika ada fitur ini) mudah dan praktis?				1	9
8.	Apakah saya mudah menemukan riwayat pemesanan dan detail transaksi saya?			1	3	6
9.	Apakah fitur untuk memberikan ulasan dan rating setelah menginap mudah digunakan?				5	5
10.	Secara keseluruhan, apakah Anda puas dengan fungsionalitas dan kemudahan penggunaan aplikasi Tamu.id?				1	9
Total Bobot				3	21	76
Skor				9	84	380
Total Skor		473				

Dari tabel 4.5 di atas, kita dapat menghitung skor *User Acceptance Test* (UAT) untuk perspektif pengguna sebagai berikut:

$$\begin{aligned}
 \text{Skor UAT User}(\%) &= \left(\frac{A}{B \times N} \right) \times 100\% \\
 &= \left(\frac{473}{50 \times 10} \right) \times 100\% \\
 &= \left(\frac{473}{500} \right) \times 100\% \\
 &= 0.946 \times 100\% \\
 &= 94.6\%
 \end{aligned}$$

Kesimpulan dari hasil UAT perspektif pengguna menunjukkan bahwa aplikasi Tamu.id mendapatkan skor 94.6%, yang berarti aplikasi ini sangat diterima oleh pengguna dan masuk dalam kategori Sangat Layak berdasarkan tabel 2.6.

Tabel 4.6 Hasil Kuesioner *User Acceptance Test* - Perspektif Admin Cabang

No.	Pertanyaan	Jawaban				
		STS	TS	N	S	SS
1.	Apakah proses login ke dashboard Admin Cabang mudah dan aman?					1

2.	Apakah tampilan dashboard Admin Cabang informatif untuk memantau operasional harian cabang?					1
3.	Apakah pengelolaan informasi umum cabang (seperti detail kontak, foto, atau deskripsi) mudah dilakukan?					1
4.	Apakah fitur untuk menambah dan mengelola tipe pod serta unit pod individual di cabang saya mudah digunakan?					1
5.	Apakah proses untuk mengatur harga pod (termasuk harga dinamis jika digunakan) cukup jelas dan mudah diterapkan?					1
6.	Apakah pengelolaan data reservasi tamu (melihat daftar booking, detail, dan status) efisien?					1
7.	Apakah fitur untuk menginput atau mengelola data reservasi yang berasal dari Online Travel Agent (OTA) mudah dioperasikan?					1
8.	Apakah proses untuk mengelola status kebersihan dan ketersediaan pod (misalnya, dari "Perlu Dibersihkan" menjadi "Tersedia") sederhana?					1
9.	Apakah fitur untuk melihat dan menanggapi (memoderasi) ulasan dari pengguna untuk cabang saya mudah digunakan?					1
10.	Secara keseluruhan, apakah dashboard Admin Cabang ini efektif membantu saya dalam mengelola operasional cabang sehari-hari?					1
Total Bobot						10
Skor						50
Total Skor		50				

Dari tabel 4.6 di atas, kita dapat menghitung skor *User Acceptance Test* (UAT) untuk perspektif Admin Cabang dengan menggunakan rumus 2.1 sebagai berikut:

$$\begin{aligned}
 \text{Skor UAT Admin}(\%) &= \left(\frac{A}{B \times N} \right) \times 100\% \\
 &= \left(\frac{50}{50 \times 1} \right) \times 100\% \\
 &= \left(\frac{50}{50} \right) \times 100\% \\
 &= 1 \times 100\% \\
 &= 100\%
 \end{aligned}$$

Kesimpulan dari hasil UAT perspektif Admin Cabang menunjukkan bahwa aplikasi Tamu.id mendapatkan skor 100%, yang berarti aplikasi ini sangat diterima oleh Admin Cabang dan masuk dalam kategori Sangat Layak berdasar tabel 2.6.

Tabel 4.7 Hasil Kuesioner *User Acceptance Test* - Perspektif *SuperAdmin*

No.	Pertanyaan	Jawaban				
		STS	TS	N	S	SS
1.	Apakah proses login ke dashboard SuperAdmin mudah dan aman?					1
2.	Apakah tampilan dashboard global SuperAdmin menyajikan informasi ringkasan sistem secara efektif dan mudah dipahami?					1
3.	Apakah pengelolaan data master sistem (seperti daftar Cabang/Hotel, Kota, OTA global) mudah dilakukan?					1
4.	Apakah fitur untuk menambah, mengedit, dan mengelola akun staf (Admin Cabang, SuperAdmin lain) serta hak aksesnya intuitif?					1
5.	Apakah proses verifikasi identitas pengguna (KYC) dari sisi SuperAdmin berjalan dengan alur yang jelas dan efisien?				1	
6.	Apakah proses approval atau penolakan permintaan pencairan saldo referral pengguna mudah dikelola dan dilacak?				1	

7.	Apakah fitur untuk membuat dan mengelola voucher yang berlaku secara global atau untuk banyak cabang mudah digunakan?					1
8.	Apakah sistem SuperAdmin menyediakan alat yang memadai untuk memantau aktivitas penting dan kesehatan sistem secara keseluruhan?					1
9.	Apakah laporan global atau data analitik yang dihasilkan sistem (jika ada) membantu dalam analisis dan pengambilan keputusan strategis?					1
10.	Secara keseluruhan, apakah antarmuka SuperAdmin efektif dalam membantu saya menjalankan tugas administrasi sistem dan pengawasan global?					1
Total Bobot					2	8
Skor					8	40
Total Skor		48				

Dari tabel 4.7 di atas, kita dapat menghitung skor *User Acceptance Test* (UAT) untuk perspektif SuperAdmin sebagai berikut:

$$\begin{aligned}
 \text{Skor UAT SuperAdmin}(\%) &= \left(\frac{A}{B \times N} \right) \times 100\% \\
 &= \left(\frac{48}{50 \times 1} \right) \times 100\% \\
 &= \left(\frac{48}{50} \right) \times 100\% \\
 &= 0.96 \times 100\% \\
 &= 96\%
 \end{aligned}$$

Kesimpulan dari hasil UAT perspektif SuperAdmin menunjukkan bahwa aplikasi Tamu.id mendapatkan skor 96%, yang berarti aplikasi ini sangat diterima oleh SuperAdmin dan masuk dalam kategori Sangat Layak berdasar tabel 2.6.

BAB V

PENUTUP

5.1 Kesimpulan

Dari rumusan masalah yang sudah diidentifikasi sebelumnya dan berdasarkan hasil penelitian yang telah dilakukan, maka dapat ditarik beberapa kesimpulan sebagai berikut:

1. Aplikasi Manajemen Hotel Kapsul multi-cabang berbasis website dengan *Frontend* Vue.js dan *Backend* Flask telah berhasil dikembangkan dan diimplementasikan dengan berbagai fitur seperti integrasi Oauth2 Google dalam sistem otentikasi, implementasi *Customized Linkedlist* dengan *Unix Timestamp* dalam rotasi kunci Pod yang berbasis QRCode untuk menangani delay dalam integrasi *Backend* dan *IoT* saat pengguna baru check-in sekaligus fitur *offline fault tolerance*, serta sistem manajemen hotel kapsul berhasil diimplementasikan dengan baik.
2. Gerbang Pembayaran Midtrans dengan metode QRIS yang diimplementasikan dengan Core API dipilih sebagai sistem pembayaran karena kemudahan integrasi dan keamanan yang ditawarkannya, serta dukungan untuk berbagai metode pembayaran digital di Indonesia.
3. Sistem yang dikembangkan telah diuji dan diterima oleh pengguna, dengan umpan balik positif mengenai kemudahan penggunaan, keamanan, dan kecepatan sistem dalam menangani transaksi dan manajemen hotel kapsul dengan tingkat penerimaan dari sisi *User* sebesar 94.6%, dari sisi *Admin* sebesar 100%, dan dari sisi *SuperAdmin* sebesar 96% dibuktikan dengan *User Acceptance Testing* yang dilakukan untuk setiap *Role* pengguna.

5.2 Saran

Berdasarkan hasil dari penelitian dan pengembangan yang sudah dilakukan maka berikut beberapa saran untuk pengembangan proyek ini kedepannya:

1. Menambahkan integrasi sistem manajemen dan pengelolaan yang lebih *Seamless* dan otomatis dengan berbagai *Online Travel Agency* (OTA) baik itu melalui Rest API dan juga webhook.
2. Menambahkan jenis pembayaran yang berlaku secara lebih luas dipasar internasional untuk saat ini seperti kartu kredit ataupun kartu debit dengan jaringan *VISA*, *Mastercard*, dan lain - lain.
3. Mengintegrasikan sistem dengan *Whatsapp* ataupun sistem *Messaging* yang lebih umum untuk kemudahan pengguna.

DAFTAR PUSTAKA

- [1] M. L. Kasavana and J. J. Cahill, *Managing technology in the hospitality industry*, 5th ed. Lansing, MI: American Hotel & Lodging Educational Institute, 2009.
- [2] Bobobox. (2024) 16 hotel kapsul di indonesia yang terbaik dan terpopuler. [Online]. Available: <https://bobobox.com/blog/hotel-kapsul-di-indonesia/>
- [3] W. G. Kim and D. J. Kim, "Impact of hotel information technology on organizational performance," *The Service Industries Journal*, vol. 29, no. 12, pp. 1635–1651, 2009.
- [4] Tazkia Royyan Hikmatiar. (2019) Qris, one qr code for all payments. [Online]. Available: https://www.bi.go.id/en/publikasi/ruang-media/news-release/Pages/SP_216219.aspx
- [5] Brick. (2024) Qris payment: Transforming indonesia's digital payment system. [Online]. Available: <https://www.linkedin.com/pulse/qris-payment-transforming-indonesias-digital-system--2zw8c>
- [6] Aris Susanto. (2025) Dynamic pricing di industri perhotelan dan cara menerapkannya. [Online]. Available: <https://hotelier.id/dynamic-pricing-di-industri-perhotelan>
- [7] Microsoft. (2025) What is oauth. [Online]. Available: <https://www.microsoft.com/id-id/security/business/security-101/what-is-oauth>
- [8] Stripe, Inc. (2025) Webhooks. [Online]. Available: <https://stripe.com/docs/webhooks>
- [9] F. Al-Turjman, H. Zahmatkesh, and R. Shahroze, "A secure and efficient qr code-based smart home access control system," *IEEE Access*, vol. 9, pp. 70 334–70 343, 2021.
- [10] M. Schuckert and H. Ye, "Motivating customers to book directly: The role of channel-specific incentives and disincentives," *International Journal of Hospitality Management*, vol. 89, p. 102555, 2020.
- [11] D. J. Anderson, *Kanban: Successful Evolutionary Change for Your Technology Business*. Blue Hole Press, 2010.
- [12] M. F. Allard and A. Voutama, "Rancang bangun sistem informasi reservasi hotel "hotel hebat" berbasis website," *Jurnal Informatika dan Teknik Elektro Terapan*, vol. 12, no. 2, Apr. 2024. [Online]. Available: <https://journal.eng.unila.ac.id/index.php/jitet/article/view/4224>
- [13] H. Iskandar, F. E. Susilawati, and A. Akramunnisa, "Rancang bangun sistem reservasi hotel pada hotel larona berbasis website," *Journal Artificial: Informatika*

- dan Sistem Informasi*, vol. 3, no. 1, pp. 21–33, Mar. 2025. [Online]. Available: <https://www.pusdig.my.id/artificial/article/view/550>
- [14] G. P. Wibowo and H. Purwanto, “Perancangan sistem informasi pemesanan tiket bus damri di bandara xyz menggunakan qr code dan web base,” *JSI (Jurnal Sistem Informasi)*, vol. 7, no. 2, pp. 69–74, 2020. [Online]. Available: <https://journal.universitassuryadarma.ac.id/index.php/jsi/article/view/449>
- [15] F. Akbar and N. S. B. Sembiring, “Perancangan aplikasi pemesanan tiket masuk pada central park zoo medan berbasis android menggunakan qr code,” *Jurnal Rekayasa Sistem (JUREKSI)*, vol. 1, no. 2, pp. 461–476, 2023. [Online]. Available: <https://www.kti.potensi-utama.org/index.php/JUREKSI/article/download/472/161>
- [16] H. J. Tan and N. A. M. Alduais, “Development of iot-based e-ticket selling and management system with qr code scanner (tickets.now),” *Applied Information Technology and Computer Science*, vol. 4, no. 2, pp. 1907–1926, Nov. 2023. [Online]. Available: <https://publisher.uthm.edu.my/periodicals/index.php/aitcs/article/view/11979>
- [17] N. H. Assidique, N. B. A. Karna, and S. Sussi, “Implementasi pembayaran dan palang otomatis pada sistem smart parking di lahan parkir menggunakan metode qr code,” *eProceedings of Engineering*, vol. 9, no. 6, 2022. [Online]. Available: <https://openlibrarypublications.telkomuniversity.ac.id/index.php/engineering/article/view/18962>
- [18] H. H. Rachman, D. S. Rusdianto, and R. C. Wihandika, “Pembangunan sistem transaksi penjualan barang dan jasa (studi kasus: Startup botanis kota),” *Jurnal Pengembangan Teknologi Informasi dan Ilmu Komputer*, vol. 7, no. 1, pp. 307–314, Feb 2023. [Online]. Available: <https://j-ptiik.ub.ac.id/index.php/j-ptiik/article/view/12171>
- [19] J. A. Alma and A. Prihanto, “Implementasi backend system untuk integrasi payment gateway pada sistem pembayaran kost menggunakan express.js,” *Journal of Informatics and Computer Science (JINACS)*, vol. 6, no. 01, p. 2, Jun 2024. [Online]. Available: <https://ejournal.unesa.ac.id/index.php/jinacs/article/view/60582>
- [20] A. P. Rahman and N. Erzed, “Sistem pengesahan dokumen dengan qr code dan memanfaatkan json web token (jwt) untuk mengurangi penyimpanan: Studi kasus universitas esa unggul kampus bekasi,” *JATI (Jurnal Mahasiswa Teknik Informatika)*, vol. 8, no. 6, pp. 1–8, Nov 2024. [Online]. Available: <https://ejournal.itn.ac.id/index.php/jati/article/view/11487>
- [21] Winarno, Wiranto, and A. Doewes, “Student mobile attendance application using qrcode and integrated with sso at universitas sebelas maret,” in *Proceedings of the 3rd International Conference on Creative Media, Design and Technology*

- (REKA 2018). Atlantis Press, 2018/11, pp. 302–305. [Online]. Available: <https://doi.org/10.2991/reka-18.2018.65>
- [22] R. M. Stair and G. Reynolds, *Principles of Information Systems*, 12th ed. Cengage Learning, 2015.
- [23] J. Mann, *Web Design for Beginners*. Packt Publishing, 2016.
- [24] D. Hardt, *OAuth 2.0*. IETF Trust, 2012. [Online]. Available: <https://oauth.net/2/>
- [25] R. Jain *et al.*, “Understanding webhooks: Real-time communication for modern applications,” *International Journal of Computer Science*, vol. 12, no. 3, pp. 45–52, 2021.
- [26] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, University of California, Irvine, 2000. [Online]. Available: <https://www.ics.uci.edu/~fielding/pubs/dissertation/fielding.dissertation.pdf>
- [27] L. Richardson and S. Ruby, *RESTful Web Services*. Sebastopol, CA: O’Reilly Media, 2007.
- [28] S. Tilkov and S. Vinoski, “Node.js and the real-time web,” *IEEE Internet Computing*, vol. 14, no. 6, pp. 83–87, 2010.
- [29] M. Burrows, *Kanban from the Inside: Understand the Kanban Method, connect it to what you already know, introduce it with confidence*. Blue Hole Press, 2014.
- [30] G. Booch, J. Rumbaugh, and I. Jacobson, *The Unified Modeling Language User Guide*. Addison-Wesley Professional, 2005.
- [31] J. Rumbaugh, I. Jacobson, and G. Booch, *The Unified Modeling Language Reference Manual*. Addison-Wesley Professional, 2005.
- [32] T. Connolly and C. Begg, *Database Systems: A Practical Approach to Design, Implementation, and Management*. Addison-Wesley, 2005.
- [33] V. Team, “Vuejs documentation,” 2023, accessed: 2025-01-19. [Online]. Available: <https://vuejs.org/guide/introduction.html>
- [34] P. S. Foundation, “Python documentation,” 2023, accessed: 2025-01-19. [Online]. Available: <https://www.python.org/>
- [35] P. Project, “Flask,” 2023, accessed: 2025-01-19. [Online]. Available: <https://flask.palletsprojects.com/en/stable/>
- [36] P. G. D. Group, “Postgresql,” 2023, accessed: 2025-01-19. [Online]. Available: <https://www.postgresql.org/>

- [37] G. SCM, “Git,” 2023, accessed: 2025-01-19. [Online]. Available: <https://git-scm.com/>
- [38] I. GitLab, “Gitlab,” 2023, accessed: 2025-01-19. [Online]. Available: <https://gitlab.com/>
- [39] Microsoft, “Visual studio code,” 2023, accessed: 2025-01-19. [Online]. Available: <https://code.visualstudio.com/>
- [40] I. Google, “Google cloud run,” 2023, accessed: 2025-01-19. [Online]. Available: <https://cloud.google.com/run>
- [41] H. Team, “Hoppscotch,” 2023, accessed: 2025-01-19. [Online]. Available: <https://hoppscotch.io/>
- [42] J. Team, “Json web tokens (jwt),” 2023, accessed: 2025-01-19. [Online]. Available: <https://jwt.io/>
- [43] SQLAlchemy, “Sqlalchemy,” 2023, accessed: 2025-01-19. [Online]. Available: <https://www.sqlalchemy.org/>
- [44] I. Google, “Google cloud storage,” 2023, accessed: 2025-01-19. [Online]. Available: <https://cloud.google.com/storage?hl=en>
- [45] I. Docker, “Docker,” 2023, accessed: 2025-01-19. [Online]. Available: <https://www.docker.com/>
- [46] L. Project, “Latex,” 2023, accessed: 2025-01-19. [Online]. Available: <https://www.latex-project.org/>
- [47] R. S. Pressman, *Software Engineering: A Practitioner’s Approach*, 8th ed. New York, NY: McGraw-Hill Education, 2014.
- [48] A. P. Mathur, *Foundations of Software Testing*, 2nd ed. Harlow, England: Pearson Education, 2017.
- [49] Statista, “Preference of using passwordless authentication compared to traditional passwords worldwide in 2023, by age group,” 2023, accessed January 12, 2025. [Online]. Available: <https://www.statista.com/statistics/1460597/preference-for-passwordless-versus-passwords-worldwide-by-age-group/>
- [50] R. Patton, *Software Testing*, 2nd ed. Indianapolis, IN: Sams Publishing, 2006.
- [51] k6 Team, “k6,” 2023, accessed: 2025-01-19. [Online]. Available: <https://k6.io/>
- [52] J. Liu and H. Wang, “Implementation of qr code in ticketing systems,” *International Journal of Computer Science and Applications*, vol. 16, no. 2, pp. 101–110, 2019.

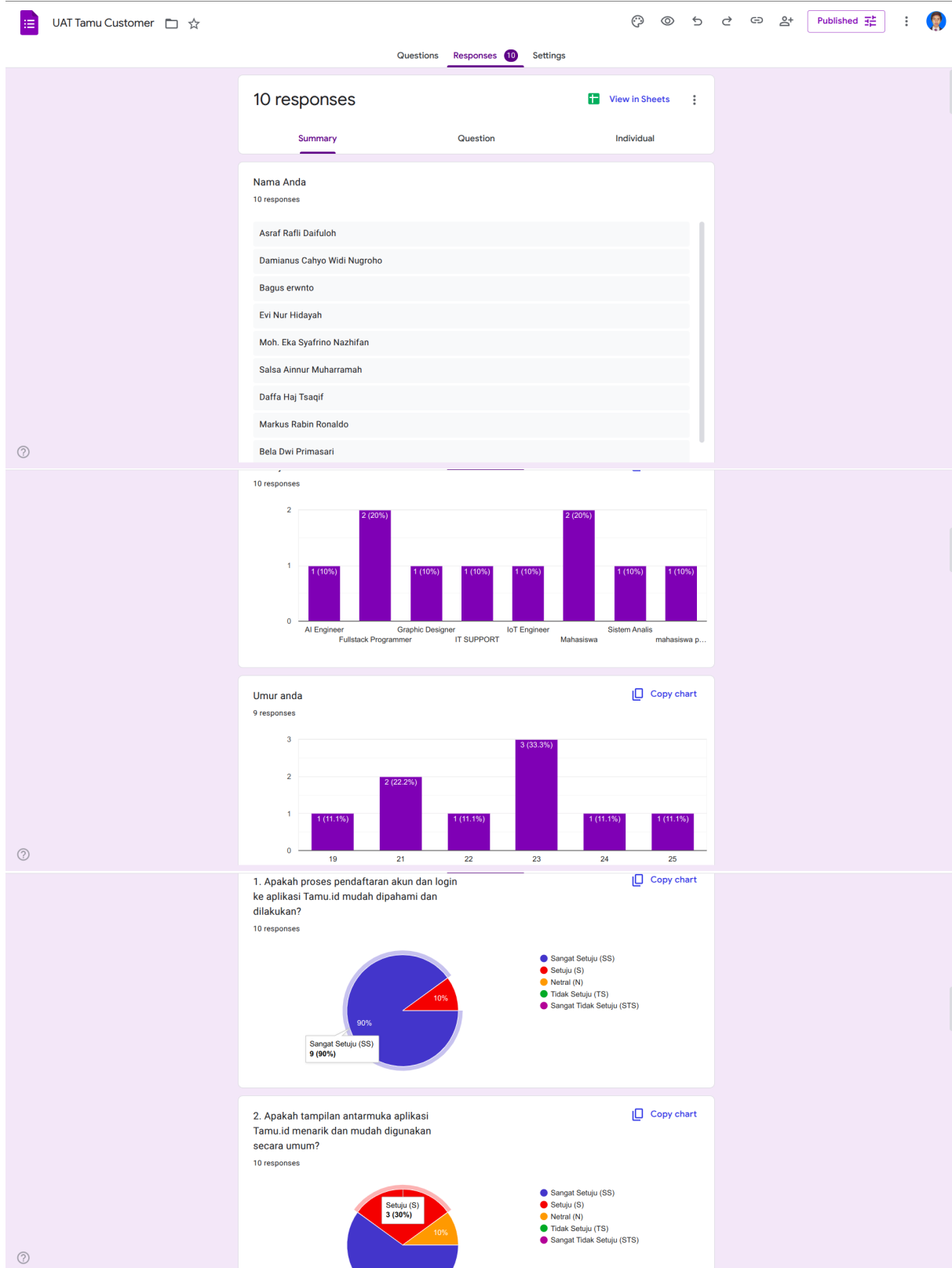
- [53] A. Johnson and R. Patel, "Webhook integration for payment notification in e-commerce systems," *Journal of Systems Integration*, vol. 12, no. 4, pp. 223–231, 2021.
- [54] K. Smith and T. Brown, "Oauth2 for passwordless authentication: An analysis of user adoption," *Journal of Web Development*, vol. 18, no. 3, pp. 45–52, 2020.
- [55] M. Lee and S. Kim, "Integrating ticketing systems with digital payment gateways," *International Journal of Software Engineering*, vol. 14, no. 6, pp. 155–163, 2020.
- [56] T. Brown and A. Johnson, "Securing modern web applications: Challenges and solutions," *Journal of Cybersecurity and Information Systems*, vol. 10, no. 2, pp. 89–97, 2022.
- [57] B. Suharto and A. Wijaya, "Pengembangan sistem ticketing konser musik berbasis web dengan validasi qr code," *Jurnal Teknologi Informasi Indonesia*, vol. 8, no. 1, pp. 23–30, 2021.
- [58] D. Hidayat and Y. Prasetyo, "Implementasi webhook pada sistem e-commerce lokal untuk integrasi pembayaran," *Jurnal Sistem Informasi*, vol. 6, no. 2, pp. 45–52, 2020.
- [59] R. Pratama and S. Kartika, "Penggunaan oauth2 untuk otentikasi pada aplikasi edukasi berbasis web di indonesia," *Jurnal Informatika dan Komputer*, vol. 14, no. 3, pp. 101–109, 2022.
- [60] F. S. Adiat Pariddudin, "Penerapan algoritma aes pada qr code untuk keamanan verifikasi tiket," *TeknoIS: Jurnal Ilmiah Teknologi Informasi dan Sains*, vol. 10, no. 2, pp. 43–52, Nov. 2020. [Online]. Available: <https://teknois.unbin.ac.id/JBS/article/view/87>
- [61] D. A. A. Nugroho and H. Supriyono, "Sistem informasi pendaftaran seminar dengan tiket berbasis qr code," *Emitor: Jurnal Teknik Elektro*, vol. 19, no. 1, pp. 36–40, 2019. [Online]. Available: <https://journals.ums.ac.id/emitor/article/view/7439>
- [62] M. F. S. Almadina, M. Martanto, A. R. Dikananda, and D. Rohman, "Implementasi teknologi quick response code dalam sistem e-ticketing pada event organizer," *Jurnal Informatika dan Teknik Elektro Terapan*, vol. 13, no. 1, 2025. [Online]. Available: <https://journal.eng.unila.ac.id/index.php/jitet/article/view/5730>
- [63] Y. A. Safikri and D. R. Prehanto, "Aplikasi payment voucher rt/rw net mikrotik berbasis android flutter dengan metode payment gateway pada dusun jomblang desa puncu kabupaten kediri," *Journal of Informatics and Computer Science (JINACS)*, vol. 3, no. 04, pp. 462–470, Jun 2022. [Online]. Available: <https://ejournal.unesa.ac.id/index.php/jinacs/article/view/46930>
- [64] M. A. Akbar and P. Nerisafitra, "Penerapan payment gateway menggunakan metode webhook dalam pengembangan bot reservasi jiot gadget solution," *Journal of Informatics*

- and Computer Science (JINACS)*, vol. 6, no. 4, pp. 932–940, Feb 2025. [Online]. Available: <https://ejournal.unesa.ac.id/index.php/jinacs/article/view/66013>
- [65] D. Hardt, “The oauth 2.0 authorization framework,” RFC 6749, 2012. [Online]. Available: <https://tools.ietf.org/html/rfc6749>
- [66] J. Bonneau, C. Herley, P. C. van Oorschot, and F. Stajano, “The quest to replace passwords: A framework for comparative evaluation of web authentication schemes,” *IEEE Symposium on Security and Privacy*, pp. 553–567, 2012.
- [67] D. Balfanz, A. Czeskis, J. Hodges, J. Jones, A. Kumar, and R. Lindemann, “Web authentication: An api for accessing public key credentials level 1,” W3C Recommendation, 2019. [Online]. Available: <https://www.w3.org/TR/webauthn/>
- [68] R. Lindemann, J. Lang, and D. Balfanz, “Fido technical white paper,” FIDO Alliance, Tech. Rep., 2015. [Online]. Available: <https://fidoalliance.org/>
- [69] R. P. M. Chan, K. A. Stol, and C. R. Halkyard, “Review of modelling and control of two-wheeled robots,” *Annual reviews in control*, vol. 37, no. 1, pp. 89–103, 2013.
- [70] P. Muhammad Rafief, A. Jaelani Sidik, and Fahmizal, “Red green blue depth (rgbd) simultaneous localization and mapping (slam) robot using kinect camera for mapping,” *Ahmad and Fahmizal, Fahmizal, Red Green Blue Depth (RGBD) Simultaneous Localization and Mapping (SLAM) Robot Using Kinect Camera for Mapping*, 2024.
- [71] Chris Ward. (2021) The pros and cons of gitops. [Online]. Available: <https://humanitec.com/blog/gitops-pros-and-cons>
- [72] R. Somani, R. Vishwakarma, R. Soni, R. Bansal, and A. Punde, “Qr based e-ticket booking system,” *International Journal of Research Publication and Reviews*, vol. 4, no. 4, pp. 3572–3574, 2023.
- [73] J. M. F. Oliveira, “Dynamic qr codes for ticketing systems,” *Master’s Thesis*, 2021.
- [74] T. Kuncara, A. S. Putra, N. Aisyah, and V. Valentino, “Effectiveness of the e-ticket system using qr codes for smart transportation systems,” *International Journal of Science, Technology & Management*, vol. 2, no. 3, pp. 900–907, 2021.
- [75] D. Pratama *et al.*, “Penggunaan oauth2 dalam aplikasi edukasi berbasis web di indonesia,” *Jurnal Pendidikan Teknologi Informasi*, vol. 10, no. 1, pp. 45–55, 2022.
- [76] R. Kumar *et al.*, “Oauth2 token validation in web applications,” *International Journal of Computer Science*, vol. 15, no. 3, pp. 112–123, 2021.

- [77] M. Nguyen *et al.*, “Webhook implementation in payment systems: A case study with stripe api integration,” *Journal of Computer Science Applications*, vol. 5, no. 4, pp. 120–130, 2021.
- [78] R. Hidayat *et al.*, “Implementasi webhook pada sistem e-commerce lokal di indonesia: Studi kasus midtrans api integration,” *Jurnal Teknologi Informasi Indonesia*, vol. 8, no. 3, pp. 70–80, 2020.
- [79] M. Brown *et al.*, “Securing webhooks in modern web applications: Best practices for payload validation and https implementation,” *Journal of Web Security Research*, vol. 7, no. 2, pp. 50–65, 2022.
- [80] B. Suharto *et al.*, “Pengembangan aplikasi ticketing berbasis web dengan integrasi qr code,” *Jurnal Teknologi dan Sistem Komputer*, vol. 9, no. 3, pp. 100–110, 2021.

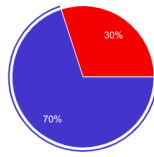
LAMPIRAN

A Lampiran Jawaban Kuesioner *User Acceptance Testing User*



3. Apakah informasi mengenai cabang dan tipe pod (harga, fasilitas, ketersediaan) disajikan dengan jelas?

10 responses

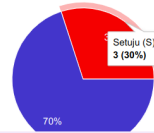


● Sangat Setuju (SS)
● Setuju (S)
● Netral (N)
● Tidak Setuju (TS)
● Sangat Tidak Setuju (STS)

[Copy chart](#)

4. Apakah alur proses pencarian dan pemesanan pod mudah diikuti dan efisien?

10 responses

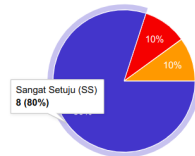


● Sangat Setuju (SS)
● Setuju (S)
● Netral (N)
● Tidak Setuju (TS)
● Sangat Tidak Setuju (STS)

[Copy chart](#)

5. Apakah proses pembayaran saat melakukan pemesanan mudah dan terasa aman?

10 responses



● Sangat Setuju (SS)
● Setuju (S)
● Netral (N)
● Tidak Setuju (TS)
● Sangat Tidak Setuju (STS)

[Copy chart](#)

6. Apakah proses check-in dan check-out melalui aplikasi berjalan dengan lancar?

10 responses

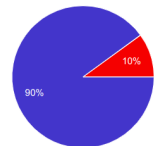


● Sangat Setuju (SS)
● Setuju (S)
● Netral (N)
● Tidak Setuju (TS)
● Sangat Tidak Setuju (STS)

[Copy chart](#)

7. Apakah penggunaan QR Code untuk akses pod (jika ada fitur ini) mudah dan praktis?

10 responses

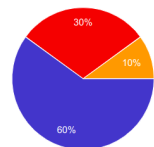


● Sangat Setuju (SS)
● Setuju (S)
● Netral (N)
● Tidak Setuju (TS)
● Sangat Tidak Setuju (STS)

[Copy chart](#)

8. Apakah saya mudah menemukan riwayat pemesanan dan detail transaksi saya?

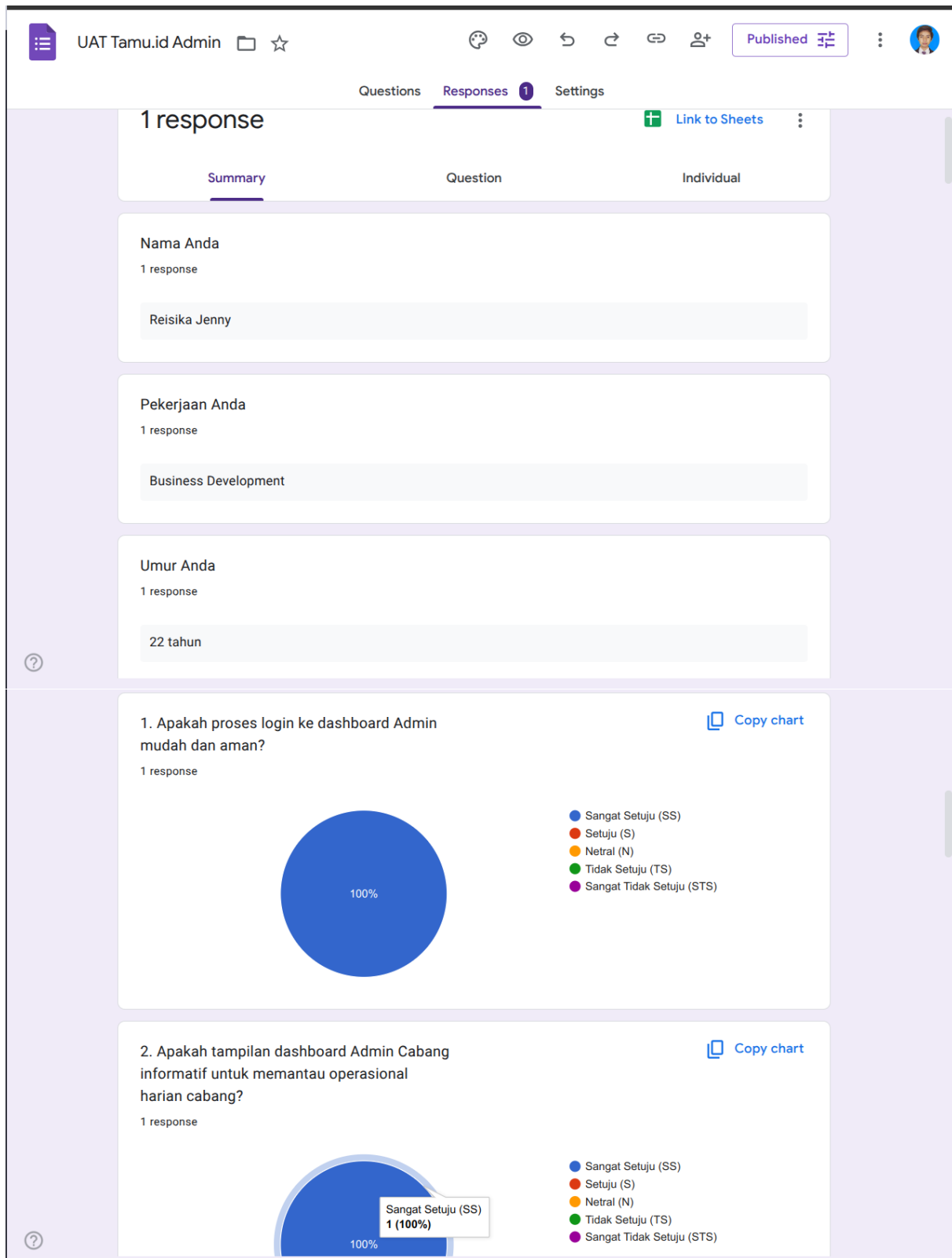
10 responses



● Sangat Setuju (SS)
● Setuju (S)
● Netral (N)
● Tidak Setuju (TS)
● Sangat Tidak Setuju (STS)

[Copy chart](#)

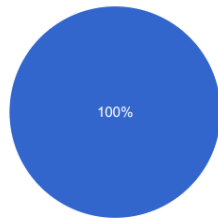
B Lampiran Jawaban Kuesioner *User Acceptance Testing Admin*



3. Apakah pengelolaan informasi umum cabang (seperti detail kontak, foto, atau deskripsi) mudah dilakukan?

[Copy chart](#)

1 response



● Sangat Setuju (SS)
● Setuju (S)
● Netral (N)
● Tidak Setuju (TS)
● Sangat Tidak Setuju (STS)

4. Apakah fitur untuk menambah dan mengelola tipe pod serta unit pod individual di cabang mudah digunakan?

[Copy chart](#)

1 response

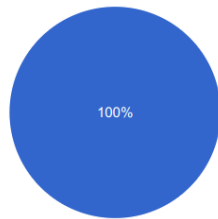


● Sangat Setuju (SS)
● Setuju (S)
● Netral (N)
● Tidak Setuju (TS)
● Sangat Tidak Setuju (STS)

5. Apakah proses untuk mengatur harga pod (termasuk harga dinamis jika digunakan) cukup jelas dan mudah diterapkan?

[Copy chart](#)

1 response



● Sangat Setuju (SS)
● Setuju (S)
● Netral (N)
● Tidak Setuju (TS)
● Sangat Tidak Setuju (STS)

6. Apakah pengelolaan data reservasi tamu (melihat daftar booking, detail, dan status) efisien?

[Copy chart](#)

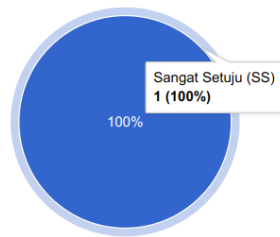
1 response



● Sangat Setuju (SS)
● Setuju (S)
● Netral (N)
● Tidak Setuju (TS)
● Sangat Tidak Setuju (STS)

dari Online Travel Agent (OTA) mudah dioperasikan?

1 response



- Sangat Setuju (SS)
- Setuju (S)
- Netral (N)
- Tidak Setuju (TS)
- Sangat Tidak Setuju (STS)

8. Apakah proses untuk mengelola status kebersihan dan ketersediaan pod (misalnya, dari "Perlu Dibersihkan" menjadi "Tersedia") sederhana?

1 response

Copy chart

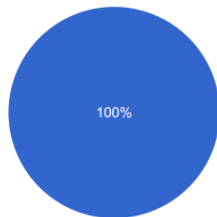


- Sangat Setuju (SS)
- Setuju (S)
- Netral (N)
- Tidak Setuju (TS)
- Sangat Tidak Setuju (STS)

9. Apakah fitur untuk menilai dan menanggapi (memoderasi) ulasan dari pengguna untuk cabang saya mudah digunakan?

1 response

Copy chart

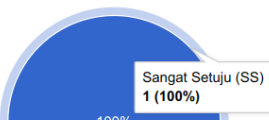


- Sangat Setuju (SS)
- Setuju (S)
- Netral (N)
- Tidak Setuju (TS)
- Sangat Tidak Setuju (STS)

10. Secara keseluruhan, apakah dashboard Admin Cabang ini efektif membantu saya dalam mengelola operasional cabang sehari-hari?

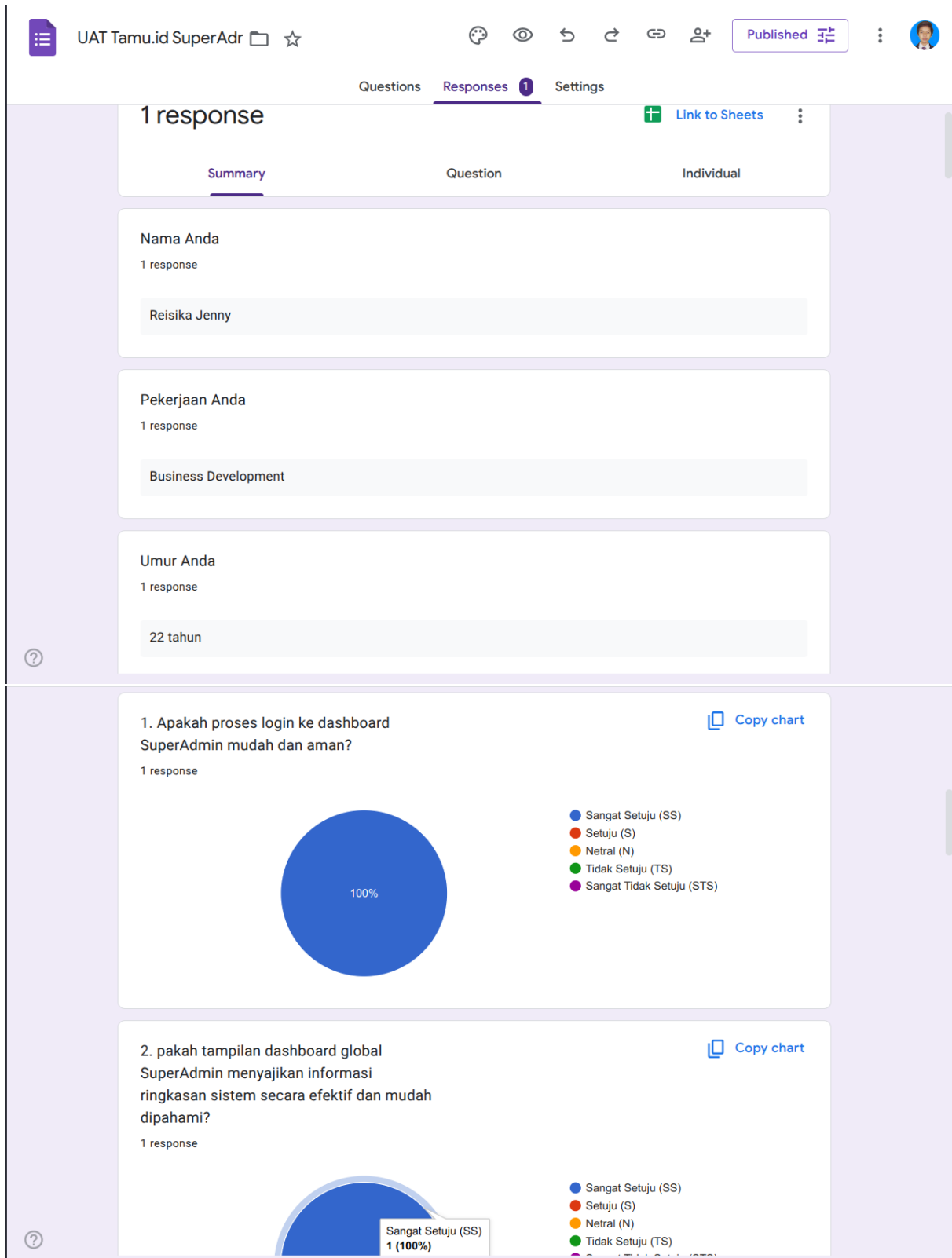
1 response

Copy chart



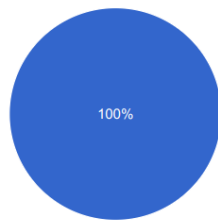
- Sangat Setuju (SS)
- Setuju (S)
- Netral (N)
- Tidak Setuju (TS)
- Sangat Tidak Setuju (STS)

C Lampiran Jawaban Kuesioner *User Acceptance Testing SuperAdmin*



(seperti daftar Cabang/Hotel, Kota, OTA global) mudah dilakukan?

1 response

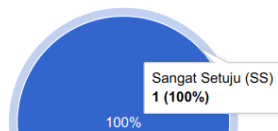


● Sangat Setuju (SS)
● Setuju (S)
● Netral (N)
● Tidak Setuju (TS)
● Sangat Tidak Setuju (STS)

4. Apakah fitur untuk menambah, mengedit, dan mengelola akun staf (Admin Cabang, SuperAdmin lain) serta hak aksesnya intuitif?

1 response

Copy chart

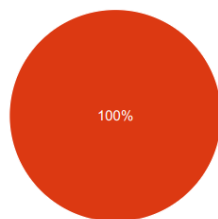


● Sangat Setuju (SS)
● Setuju (S)
● Netral (N)
● Tidak Setuju (TS)
● Sangat Tidak Setuju (STS)

5. Apakah proses verifikasi identitas pengguna (KYC) dari sisi SuperAdmin berjalan dengan alur yang jelas dan efisien?

1 response

Copy chart

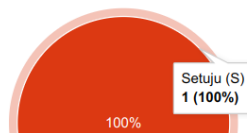


● Sangat Setuju (SS)
● Setuju (S)
● Netral (N)
● Tidak Setuju (TS)
● Sangat Tidak Setuju (STS)

6. Apakah proses approval atau penolakan permintaan pencairan saldo referral pengguna mudah dikelola dan dilacak?

1 response

Copy chart

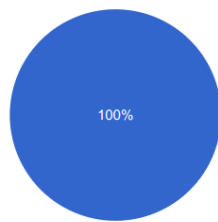


● Sangat Setuju (SS)
● Setuju (S)
● Netral (N)
● Tidak Setuju (TS)
● Sangat Tidak Setuju (STS)

7. Apakah fitur untuk membuat dan mengelola voucher yang berlaku secara global atau untuk banyak cabang mudah digunakan?

[Copy chart](#)

1 response

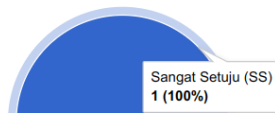


- Sangat Setuju (SS)
- Setuju (S)
- Netral (N)
- Tidak Setuju (TS)
- Sangat Tidak Setuju (STS)

8. Apakah sistem SuperAdmin menyediakan alat yang memadai untuk memantau aktivitas penting dan kesehatan sistem secara keseluruhan?

[Copy chart](#)

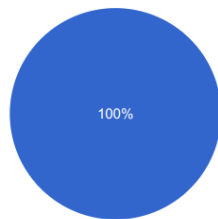
1 response



- Sangat Setuju (SS)
- Setuju (S)
- Netral (N)
- Tidak Setuju (TS)
- Sangat Tidak Setuju (STS)

yang dihasilkan sistem (jika ada) membantu dalam analisis dan pengambilan keputusan strategis?

1 response

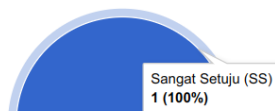


- Sangat Setuju (SS)
- Setuju (S)
- Netral (N)
- Tidak Setuju (TS)
- Sangat Tidak Setuju (STS)

10. Secara keseluruhan, apakah antarmuka SuperAdmin efektif dalam membantu saya menjalankan tugas administrasi sistem dan pengawasan global?

[Copy chart](#)

1 response



- Sangat Setuju (SS)
- Setuju (S)
- Netral (N)
- Tidak Setuju (TS)
- Sangat Tidak Setuju (STS)